Volume V

Software Architecture & The 5-Block Invariant Protocol

CreativeWorkzStudio LLC / ProfessorSeanEX

Version 0.2.0-dev, 2026-11-12

Table of Contents

1. Cornerstone & Tenebris and Lux: System Architecture Overview . . . 2
   1.1. Who This Engine Is Built For (Start Here) . . . 2
   1.2. The Big Machine and The Master Turn Gear . . . 2
   1.3. The Five Compilation Strata . . . 3
   1.4. The Universal 5-Block Protocol . . . 3
   1.5. Controlled Memory Chassis & Fall-Down Portability . . . 3
   1.6. Verifying Architectural Boundaries Yourself . . . 5
   1.7. Where to Go Next . . . 6
2. Zero-Disk Safe-Mode BIOS, Config ROM Vaults & Autonomous Probe Loop . . . 7
   2.1. Tier-0 Safe-Mode BIOS & Quarantined Fallbacks . . . 7
   2.2. Compiled-In `.rodata` Config ROM Vaults . . . 7
   2.3. Safe-Mode Boot, Floppy Floor & Downgrade Transitions . . . 8
   2.4. Flight Recorder (`log/`), Debugger (`debug/`) & Autonomous Probe Loop . . . 9
   2.5. Self-Audit & Zero-Disk Checks . . . 11
   2.6. Where to Go Next . . . 11
3. Data-Driven Architecture: Dual Data Taxonomy, Layered VFS & 5-Block JSON . . . 12
   3.1. Dual Data Taxonomy: Engine Profiles vs. Game Canon . . . 12
   3.2. Layered VFS Precedence (`path_finder`) . . . 12
   3.3. The 5-Block JSON Contract & `block_3_body` Isolation . . . 13
   3.4. Zero-Heap Scoped Accessors (`master_loader`) & Shared Span Parsers . . . 14
   3.5. Validating Data Files Before Commit . . . 16
   3.6. Where to Go Next . . . 16
4. Central Manifest Registry & Pre-Flight Loose-Bolts Inspection . . . 17
   4.1. Self-Describing Module Metadata & Registry Ontology . . . 17
   4.2. The Pre-Flight Walk (`manifest_run_preflight`) . . . 18
   4.3. Layer 3 Plug Audit Flow (`game/src/main.c` BLOCK 4) . . . 19
   4.4. Running Pre-Flight Diagnostics Yourself . . . 20
   4.5. Where to Go Next . . . 20
5. Dual-Frontend Presentation Layer: CLI, Native X11 & Movie Reel . . . 21
   5.1. One Pure Simulation, Multiple Frontends . . . 21
   5.2. Native `640x360` Viewport & The 2.5D Movie Reel Model . . . 22
   5.3. Testing Both Frontends Cleanly . . . 23
   5.4. Where to Go Next . . . 24
6. Root Strata Chronicles: Why Strengthening log/, debug/, and io/ Pays Every Future Turn . . . 25
   6.1. Lesson 1: Purging Legacy Shims Lets the Compiler Map Every Seam . . . 25
   6.2. Lesson 2: Splitting Pure Types & Measuring the Chassis Eliminates Hidden Bloat . . . 25
   6.3. Lesson 3: Building Forward for C23/C20 While Anchoring a C11/C17 Floor . . . 26
   6.4. Lesson 4: Zero-Disk Safe-Mode BIOS & Compiled `.rodata` ROMs Protect Designers . . . 27
   6.5. How Root Discipline Multiplies Velocity in `core/`, `viewport/` & `game/` . . . 27
7. Evolution of AGENTS.md: Engineering a Cognitive Compass for Human-AI Co-Creation . . . 29
   7.1. Keeping the Operator in the Director's Chair . . . 29
   7.2. Why 5-Block AsciiDoc & Automated Validators Defeat Context Drift . . . 29
   7.3. Architectural Smells `AGENTS.md` Actively Catches . . . 30
8. Layer 0 Categorized Flight Logging Subsystem Overview . . . 31

8.1. Architectural Philosophy: The Flight Recorder . . . 31
8.2. Verified Rev 5 File & Documentation Map . . . 31
8.3. Live Public Dispatch Contract . . . 32
8.4. Autonomous Probe Wiring (cog.log → cog.debug) . . . 33
8.5. Self-Audit & Quick Verification . . . 34
8.6. Companion Deep-Dive Chapters . . . 34
9. Log Module Pair: Flight Logger Router & Severity Ring (log.h + log.c) . . . 35
9.1. Configuration Loading & Data-Driven Profiles (Sub-Assembly 3.1) . . . 35
9.2. Ring Recording, Rotation & String Parsing (Sub-Assemblies 3.2–3.5) . . . 35
9.3. Validation Guards, Probes & Pre-Flight Audits (Sub-Assemblies 4.1–5.2) . . . 36
9.4. Paired Neighbors . . . 37
10. Log Leaf Contract: Canonical Types & Ring Bounds (log_types.h) . . . 39
10.1. Physical Bounds & Telemetry Value Layout (Sub-Assemblies 2.1–2.3) . . . 39
10.2. Stateless Boundary Predicates (Sub-Assembly 2.4) . . . 40
10.3. Compile-Time & Validator Checks . . . 40
10.4. Consumers of log_types.h . . . 40
11. Log Safe-Mode BIOS: Zero-Disk Fallback & ROM Vault (log_fallback.h + log_fallback.c) . . . 41
11.1. Pre-VFS Boot & Chronological Replay (Sub-Assembly 3.3 & 5.2) . . . 41
11.2. The 10-Asset Compiled Telemetry ROM Vault (Sub-Assemblies 3.1–3.2) . . . 42
11.3. BIOS Self-Audit & Emergency Diagnostic Cards . . . 42
11.4. Next Steps in log/ . . . 42
12. Log Verification Suite: 4-Tier CTest Harness & Probes . . . 43
12.1. How Each Test Executable Progresses (Crude-to-Fine Ladder) . . . 43
12.2. Live Emergence Verification (test_log_system_emergence) . . . 43
12.3. Executing the Suite & Forensic Failure Cards . . . 44
12.4. Continue Reading . . . 44
13. Layer 0 State Debugger & Diagnostic Engine Overview . . . 45
13.1. Architectural Philosophy: Observable State & Tripwires . . . 45
13.2. Verified Rev 5 File & Documentation Map . . . 45
13.3. Deterministic Capture & Time-Travel Rewind . . . 46
13.4. Autonomous Validation Probes & DEBUG_STEP_AWAIT . . . 46
13.5. Self-Audit & Quick Verification . . . 47
13.6. Companion Deep-Dive Chapters . . . 48
14. Debug Module Pair: Rewind Engine, Watchpoints & Probes (debug.h + debug.c) . . . 49
14.1. Snapshot Capture, Locking & Time-Travel Rewind . . . 49
14.2. Stepping Modes, Breakpoints & DEBUG_STEP_AWAIT . . . 50
14.3. Validation Probe Matrix & Tripwire Chain . . . 50
14.4. Paired Neighbors . . . 51
15. Debug Leaf Contract: Snapshots, GCD Ring & Tripwires (debug_types.h) . . . 52
15.1. Structural Ontology (Sub-Assembly 2.3) . . . 52
15.2. Static Bounds Verification . . . 53
15.3. Consumers of debug_types.h . . . 53
16. Debug Safe-Mode BIOS: Tripwire Emergency Floor (debug_fallback.h + debug_fallback.c) . . . 54
16.1. Tier-0 BIOS Boot & .rodata Vault (Sub-Assemblies 3.1–3.2) . . . 54
16.2. Fault Capture & Emergency AWAIT Latch (Sub-Assembly 3.3) . . . 55
16.3. BIOS Self-Audit & ROM Info (Blocks 4 & 5) . . . 55

16.4. Next Steps in debug/ . . . 55
17. Debug Verification Suite: Rewind, Probes & Tripwires Across All 4 Tiers . . . 56
17.1. Fine-Grained Invariants Tested Across debug/ . . . 56
17.2. Live Emergence Check (test_debug_unit_emergence) . . . 56
17.3. Running the Debugger Verification Targets . . . 57
17.4. Continue Reading . . . 57
18. Layer 0 Zero-Heap I/O Pipeline Subsystem Overview . . . 58
18.1. Architectural Philosophy: Assembly-Line Ingestion Without Heap . . . 58
18.2. Verified Rev 5 File & Documentation Map . . . 58
18.3. Live VFS Routes & Shared JSON Span Resolution . . . 59
18.4. Zero-Duplication Parser Bridge & Verification . . . 60
18.5. Companion Deep-Dive Chapters . . . 61
19. I/O Umbrella Facade & 4-Pillar Boot Pipeline (io.h) . . . 62
19.1. Downward Re-Export Order (BLOCK 2 of io.h) . . . 62
19.2. Configuration Anchor (engine/config/io/) . . . 62
19.3. Coordinated Validation & Lifecycle Gates . . . 63
19.4. Explore the Five Child I/O Cogs . . . 63
20. I/O Module Pair: Bounded File & Memory Reader (fs_utils.h + fs_utils.c) . . . 64
20.1. Bounded Reads, Atomic-Minded Writes & Directory Creation (Sub-Assemblies 3.1–3.2) . . . 64
20.2. Telemetry Counters, Probes & Self-Audit (Blocks 4 & 5) . . . 65
20.3. Continue in io/ . . . 65
21. I/O Module Pair: Layered VFS Domain Router (path_finder.h + path_finder.c) . . . 66
21.1. How Multi-Tier Resolution Works (vfs_routes.json) . . . 66
21.2. Public Resolution API (Sub-Assemblies 3.1–3.2) . . . 67
21.3. Validation, Probes & Inspection (Sub-Assemblies 4.1–5.2) . . . 68
21.4. Continue in io/ . . . 68
22. I/O Module Pair: Zero-Heap Single-Pass JSON Scanner (json_utils.h + json_utils.c) . . . 69
22.1. Single-Pass Token Locators & Typed Extractors (Sub-Assemblies 3.1–3.4) . . . 69
22.2. Structural Guards & Validation Probes (Sub-Assemblies 4.1–5.2) . . . 70
22.3. Continue in io/ . . . 70
23. I/O Module Pair: 5-Block Contract Hydrator (master_loader.h + master_loader.c) . . . 71
23.1. Dual-Path Hydration: Disk, VFS, rom:// & Memory . . . 71
23.2. Strictly Scoped block_3_body Getters . . . 72
23.3. Runtime Loader Policy (loader_contracts.json) . . . 73
23.4. Error Taxonomy & Diagnostic Probes (Block 4) . . . 73
23.5. Inspection Commands & Lifecycle Ownership . . . 73
24. I/O Safe-Mode BIOS: Zero-Disk Fallback & ROM Vault (io_fallback.h + io_fallback.c) . . . 74
24.1. Shared ROM-Safe JSON Policy Parsers . . . 74
24.2. Compiled .rodata Config ROM Vault (Sub-Assembly 3.2) . . . 75
24.3. Unified 4-Pillar Emergency Cogs (Sub-Assembly 3.3) . . . 75
24.4. Autonomous Probes & Safe-Mode Inspection (Block 4) . . . 75
24.5. BIOS Boot, Floppy Mode & Deterministic Teardown (Block 5) . . . 75
25. I/O Verification Suite: Bounded I/O, VFS Routing & Downgrades Across All 4 Tiers . . . 77
25.1. Crude-to-Fine Band Design . . . 77
25.2. System-Tier Emergence & Downgrade Snapshot . . . 78
25.3. Running & Auditing the io/ Test Suite . . . 79

26. Layer 0 Core Utilities Code Tour . . . 80
26.1. Overview . . . 80
26.2. Architectural Philosophy & Mission . . . 80
26.3. Subsystem Layout & File Manifest . . . 80
26.4. Core Engine Cogs & API Tour . . . 81
26.5. Verification & Testing Matrix . . . 83
27. Layer 1 Simulation Engine Code Tour . . . 84
27.1. Overview . . . 84
27.2. State Structure & Memory Layout (`player_state_t`) . . . 84
27.3. Universal Action Dispatcher (`game_process_command`) . . . 84
27.4. Diagnostic Self-Auditing (`game_audit_state`) . . . 85
28. Layer 2 Presentation Viewport Code Tour . . . 86
28.1. Overview . . . 86
28.2. The Movie Reel Mental Model & Subfolder Strata . . . 86
28.3. The Movie Reel: Horizontal Motion & Vertical Depth . . . 87
28.4. The 2.5D Tactical Isometric Engine . . . 87
28.5. Non-Anonymous Type Invariant Covenant . . . 88
28.6. Verification Telemetry . . . 88
29. Layer 0/2 Audio Event Bus & Mixer Subsystem . . . 89
29.1. Overview . . . 89
29.2. Architectural Philosophy: The Decoupled Audio Machine . . . 89
29.3. Subsystem File Layout & Header Structure . . . 89
29.4. Multi-Channel Mixer Architecture . . . 90
29.5. Non-Anonymous Ontological Types . . . 90
29.6. Verification Telemetry . . . 91
30. Layer 4 Verification & Test Suite Code Tour . . . 92
30.1. Overview . . . 92
30.2. The Four-Tier Testing Hierarchy . . . 92
30.3. The Four Ontological Testing Sections . . . 93
30.4. CMake & CTest Test Targets . . . 94
30.5. Execution & Verification Commands . . . 94
Appendix A: Wholesale Source Code Reference Plates . . . 95
A.1. Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17) . . . 95
A.2. Layer 3 Plug Entry & Manifest Wiring (init_engine_manifest + main) . . . 96
A.3. Build-Time Compiled .rodata Config ROM Generator . . . 100
A.4. Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch . . . 104
A.5. Autonomous Validation Probes & Tripwire Trap Chain . . . 105
A.6. Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1) . . . 109
A.7. Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json) . . . 111
A.8. JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json) . . . 113
A.9. Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2) . . . 114
A.10. Shared Zero-Heap JSON Block Resolver & VFS Parser Group . . . 116
A.11. Central Manifest Metadata & Registry Ontology (manifest.h Sub-Assembly 2.3) . . . 118
A.12. Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight) . . . 121
A.13. Application Audit Integration (game/src/main.c BLOCK 4) . . . 122
A.14. Layer 1 Deterministic Simulation State Audit (sim_state_audit) . . . 124

A.15. Layer 2 640x360 Viewport & 2.5D Movie Reel Data Structures (gui_types.h Sub-Assembly 2.3) . . . 125
A.16. Flight Log Leaf Vocabulary & Entry Layout (log_types.h Sub-Assembly 2.3) . . . 129
A.17. Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3) . . . 131
A.18. Tier-0 Flight Log BIOS Boot & Early-Entry Replay Drain . . . 133
A.19. Flight Log System Emergence Verification (test_log_system_emergence) . . . 134
A.20. State Debugger Leaf Vocabulary & Snapshot Structs (debug_types.h Sub-Assembly 2.3) . . . 136
A.21. Deterministic State Capture & Time-Travel Rewind (debug.c) . . . 140
A.22. Tier-0 Debugger BIOS Boot & ROM Profile Hydration (debug_fallback.c) . . . 142
A.23. Debugger Unit Emergence & Tripwire Verification (test_debug_unit_emergence) . . . 145
A.24. Bounded POSIX Filesystem & Read-Only Guard API (fs_utils.h Sub-Assembly 3.2) . . . 146
A.25. Multi-Tier VFS Path & Asset Resolution API (path_finder.h Sub-Assembly 3.2) . . . 147
A.26. Zero-Heap JSON Token Span Locator API (json_utils.h Sub-Assembly 3.1) . . . 149
A.27. I/O Pipeline System Emergence & ROM Downgrade Verification (test_io_system_emergence) . . . 151



Volume V Synopsis

Cornerstone Big Machine vs. Tenebris and Lux Master Turn Gear, 5 strata, Rev 5 5-block protocol, Safe-Mode BIOS & .rodata ROM vaults, VFS data pipelines, pre-flight audits, and dual-frontend presentation.

Chapter 1. Cornerstone & Tenebris and Lux: System Architecture Overview

Source File: `docs/architecture/ARCHITECTURE.adoc`

This guide explains how the **Cornerstone Game Engine** (`engine/`) and **Tenebris and Lux** (`game/`) work together—written so a designer tuning battle balance, a writer editing dialogue, a student learning systems architecture, or an engineer hardening C23 code can build a shared mental model in minutes.



Textbook Code Snapshots Enabled: Code blocks marked as excerpts in this chapter are pulled live from the repository during documentation builds. Clickable wholesale cross-references jump directly to complete listings in the Appendix when viewing compiled PDF or book builds.

1.1. Who This Engine Is Built For (Start Here)

A game engine should feel like a well-labeled workshop, not a locked vault. You do not need senior systems experience to use **Cornerstone Game Engine** safely:

- **Creators & Designers** edit declarative 5-block JSON files (`engine/config/` and `game/data/`). If a syntax mistake slips into a data file, built-in ROM vaults keep the engine running in Safe-Mode and pinpoint the exact line to fix.
- **Playtesters & Narrative Leads** test every tactical choice using the lightweight headless terminal (`--cli`) or diagnostic audit (`--audit`) without needing heavy graphics setups.
- **C & Rust Developers** gain zero-heap determinism, strict compile-time compiler walls (`-Wall -Wextra -Wpedantic`), four test tiers (`65` CTest suites + `30` Rust unit tests), and uniform 5-block structure across source, JSON, CMake, and documentation.

1.2. The Big Machine and The Master Turn Gear

The architecture rests on a clean mechanical metaphor:

- `engine/` — **The Big Machine (Cornerstone Game Engine):** Reusable infrastructure needed by **any** tactical or adventure game: math, clocks, string parsing, virtual filesystems, flight logging, replay debugging, audio mixing, and a `640x360` X11 viewport.
- `game/` — **The Master Turn Gear (Tenebris and Lux):** Pure domain rules of Sacyra and Libira: tactical grids, Elijah & Katrina's vocations, Twin Relics, combat stances, moral sway toward **Lux** or **Tenebris**, and the multi-ending state machine.
- `game/src/main.c` — **Layer 3 Plug:** The single mechanical coupling joining the Big Machine to the Master Turn Gear at boot.

Cornerstone vs. Tenebris and Lux Structural Decoupling

Anti-Domain-Bleed Rule: Nothing under `engine/` is ever allowed to include `game/` headers or mention story canon (`Elijah`, `Katrina`, `Sacyra`, `Usuru`). Conversely, `CMakeLists.txt` enforces separate include fences (`ENGINE_PUBLIC_INCLUDES` vs. `COMMON_INCLUDES`) so `engine/` builds cleanly completely on its own.

Cog Elevation Rule: Whenever a helper inside `game/` solves a general systems problem (screen wipes, easing, audio cue scheduling, JSON parsing) with zero story coupling, it elevates into `engine/` so future games inherit it automatically.

1.3. The Five Compilation Strata

Code dependencies flow strictly **downward**—lower layers never call upward into higher layers:

Layer	Directory	Memory Contract	Responsibility
Layer 4	`tests/` (`functional`, `unit`, `integrated`, `system`)	Test Harness (`ALLOC_ZERO_HEAP` sim)	Proves mathematical, structural, integration, and full CLI parity across 65 CTest checks.
Layer 3	`game/src/main.c`, `tools/`	Startup Plug / Rust Tooling	Parses CLI flags (`--cli`, `--gui`, `--debug`, `--audit`), builds manifest, compiles data/guides.
Layer 2	`engine/*/viewport/`	`ALLOC_ZERO_HEAP` (`.bss`)	`640x360` logical software rasterizer, Movie Reel camera, native X11 display window.
Layer 1	`game/` (`hero`, `combat`, `relics`, `world`, `ending`, `dispatch`)	`ALLOC_ZERO_HEAP`	Deterministic tactical RPG rules completely independent of console vs. desktop rendering.
Layer 0	`engine/` (`core`, `log`, `debug`, `io`, `audio`)	`ALLOC_ZERO_HEAP`	`44` header ecosystem (`31` Layer 0 + `13` Layer 2) providing math, VFS, flight logs, replay debug, 7-bus audio.

1.4. The Universal 5-Block Protocol

Whether you open a `.h` contract, `.c` engine file, `.json` config, `CMakeLists.txt`, Rust tool, or `.adoc` chapter, everything shares the exact same mental geography:

Block	Code Role (C / Rust)	Data Role (JSON)	Doc Role (AsciiDoc `////`)
Block 1	Metadata header, tags, invariants	`block_1_metadata` identity & rules	Document provenance, target audience, snapshot bindings
Block 2	Includes, types, macros, `.bss` (`2.1`–`2.5`)	`block_2_types` / `block_2_opening` limits	Title, attributes, reader orientation & quick TOC
Block 3	Core algorithms & API logic (`3.1`+)	`block_3_body` live runtime payload	Technical narrative, diagrams, tables, snapshot plates
Block 4	Probes, tripwires, self-audits (`4.1`–`4.2`)	`block_4_audit` tripwires & metrics	Verification commands & reader Q&A checkpoints
Block 5	Diagnostic dump cards, boot & shutdown (`5.1`–`5.2`)	`block_5_closing` handshakes & links	Cross-volume navigation & next readings

1.5. Controlled Memory Chassis & Fall-Down Portability

Avoiding `malloc`/`calloc`/`free` inside Layers 0–2 (`ALLOC_ZERO_HEAP`) creates a **Controlled Memory Chassis**: every byte of memory is visible in `.text`, `.rodata`, `.data`, `.bss`, and bounded stack frames. When RPG tables grow, capacity pressure shows up immediately at compile time instead of hiding inside unpredictable heap leaks.

At the same time, Cornerstone uses an **Engine-as-a-Java "Fall-Down Stairs"** portability pattern. Rather than adding legacy compatibility clutter, we write forward-first for **ISO C23** (`-std=c23`) and **C++20** headers (`extern "C"`) anchored

through engine/include/core/manifest.h so older compilers step down gracefully to **C17** or **C11** (CORNERSTONE_C_STANDARD=AUTO|23|17|11).

Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17) (engine/include/core/manifest.h#L82-L143 · Mode: Subassembly)

```
 82 /*
 83 ------------------------------------------------------------------------------
 84 Sub-Assembly 2.1: Pre-Conditions, Progressive Standard Macros & Buffer Limits
 85 ------------------------------------------------------------------------------
 86 Compile-time sizing limits for registry module nodes, dependencies, rules, and
 87 Cornerstone's "Fall-Down" ISO C23 / C17 / C11 + C++20 / C++17 capability macros.
 88
 89 For Designers:
 90 Allows Cornerstone to take advantage of modern compiler safety checks on current
 91 hardware while stepping cleanly onto older systems without legacy clutter. Also
 92 bounds the engine registry to up to 32 major systems, 8 core rules, and 4 links.
 93
 94 For Programmers:
 95 - Guarantees 100% zero-heap allocation (ALLOC_ZERO_HEAP) in .rodata and .bss.
 96 - Exposes CS_C_STD_TIER (23/17/11/0) and CS_CXX_STD_TIER (20/17/11/0) plus
 97   graceful ISO attribute fall-down macros (CS_NODISCARD, CS_MAYBE_UNUSED,
 98   CS_FALLTHROUGH, CS_UNREACHABLE, CS_ALIGNOF, CS_NORETURN, CS_STATIC_ASSERT,
 99   CS_HAS_EMBED) compiled cleanly without extensions under -Wpedantic.
100 ------------------------------------------------------------------------------
101 */
102
103 /** Active ISO C standard tier compiled for this translation unit (23, 17, 11, or 0 in C++). */
104 #if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202311L
105 #    define CS_C_STD_TIER 23
106 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201710L
107 #    define CS_C_STD_TIER 17
108 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201112L
109 #    define CS_C_STD_TIER 11
110 #else
111 #    define CS_C_STD_TIER 0
112 #endif
113
114 /** Active ISO C++ standard tier when included from C++ (20, 17, 11, or 0 in C). */
115 #if defined(__cplusplus) && __cplusplus >= 202002L
116 #    define CS_CXX_STD_TIER 20
117 #elif defined(__cplusplus) && __cplusplus >= 201703L
118 #    define CS_CXX_STD_TIER 17
119 #elif defined(__cplusplus)
120 #    define CS_CXX_STD_TIER 11
121 #else
122 #    define CS_CXX_STD_TIER 0
123 #endif
124
125 /** Marks return values that callers MUST inspect (C23/C++17+ [[nodiscard]], C11/17 floor empty). */
126 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
127 #    define CS_NODISCARD [[nodiscard]]
128 #else
129 #    define CS_NODISCARD
130 #endif
131
132 /** Silences unused-entity diagnostics cleanly on intentional dormant/platform helpers. */
133 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
134 #    define CS_MAYBE_UNUSED [[maybe_unused]]
135 #else
136 #    define CS_MAYBE_UNUSED
137 #endif
138
139 /** Annotates explicit switch-case fall-through cleanly under -Wextra -Wpedantic. */
140 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
141 #    define CS_FALLTHROUGH [[fallthrough]]
142 #elif defined(__GNUC__) && (__GNUC__ >= 7)
143 #    define CS_FALLTHROUGH __attribute__((fallthrough))
```

Compile-time standard detection establishing C23/C17/C11 and C20/C17 parity without legacy shims.

Wholesale Listing: Complete verbatim code (#L82-L194) → Appendix: Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17).

To see how Layer 3 connects this portability bedrock to the game world at startup, examine the boot entry group extracted from game/src/main.c:

Layer 3 Plug Entry & Manifest Wiring (init_engine_manifest + main) (game/src/main.c#L71-L116 · Mode: Group)

```c
/**
 * @brief Registers all core engine subsystem manifests into central registry.
 *
 * ### Machine Role & Mission
 * Populates the central manifest registry with all Layer 0 utilities (manifest,
 * fs, json, time, math, geom, string, log, debug), Layer 1 simulation engine (state,
 * combat, world, dispatcher, game), and Layer 2 presentation viewport (gui).
 * Enables boot-time loose-bolts invariant checks and dependency graph resolution
 * before any game loop execution begins.
 *
 * ### Ontological Classification
 * - Stratum: Layer 3 (Application Entry & Presentation Router)
 * - Machine: app.entry.main (Executable Entry & Presentation Router)
 * - Assembly: Pre-Flight Manifest Registry Initialization
 * - Cog: Module Manifest Registration Primitive
 *
 * ### Execution & Resource Contract
 * - Memory Model: ALLOC_ZERO_HEAP (Stack-allocated fixed manifest tables).
 * - Concurrency: CONCURRENCY_MAIN_THREAD (Called at startup on main thread).
 * - Latency Budget: < 50 microseconds.
 *
 * @param[out] registry Target manifest registry to initialize and populate.
 */
static void init_engine_manifest(manifest_registry_t *registry)
{
    /* Step 1: Zero-initialize registry table */
    manifest_registry_init(registry);

    /* Step 2: Register Layer 0 Core Utilities */
    manifest_register(registry, &manifest_utils_manifest);
    manifest_register(registry, &fs_utils_manifest);
    manifest_register(registry, &json_utils_manifest);
    manifest_register(registry, &path_finder_manifest);
    manifest_register(registry, &master_loader_manifest);
    manifest_register(registry, &time_utils_manifest);
    manifest_register(registry, &math_utils_manifest);
    manifest_register(registry, &geom_utils_manifest);
    manifest_register(registry, &string_utils_manifest);
    manifest_register(registry, &clock_pacer_manifest);
    manifest_register(registry, &formula_eval_manifest);
    manifest_register(registry, &log_manifest);
    manifest_register(registry, &debug_manifest);

    /* Step 3: Register Layer 1 Simulation Engine Subsystems */
    manifest_register(registry, &sim_state_manifest);
    manifest_register(registry, &combat_manifest);
```

Demonstrates how Layer 3 game/src/main.c wires engine/ and game/ strata together via the central manifest registry.

Wholesale Listing: Complete verbatim code (#L71-L522) → Appendix: Layer 3 Plug Entry & Manifest Wiring (init_engine_manifest + main).

1.6. Verifying Architectural Boundaries Yourself

```
cmake -S . -B build && cmake --build build -j4
./tools/target/debug/tl_tools validate all
ctest --test-dir build --output-on-failure
./build/tenebris-lux --audit
```

1. *Why are there no* `malloc()` *calls in gameplay or engine code?*

Fixed `.rodata` and `.bss` buffers guarantee deterministic replay hashes and keep memory consumption measurable down to the byte.

2. *What enforces separation between* `engine/` *and* `game/`*?*

   CMake object library scoping (`ENGINE_PUBLIC_INCLUDES` vs. `COMMON_INCLUDES`) prevents `engine/` targets from seeing any `game/` header at compile time.

3. *Where do code changes break first if an interface changes?*

   No shims are kept behind—strict `-Wall -Wextra -Wpedantic` compiler checks and `65` CTest suites map every affected call site immediately.

1.7. Where to Go Next

- Continue to Chapter 2: Zero-Disk Safe-Mode BIOS, Config ROM Vaults & Autonomous Probe Loop.
- Review how data configs stay isolated in Data-Driven Architecture.
- Browse the live extracted code plate catalog in Snapshots Index.

Chapter 2. Zero-Disk Safe-Mode BIOS, Config ROM Vaults & Autonomous Probe Loop

Source File: `docs/architecture/resilience_bios_and_probes.adoc`

Software fails in the real world when external files go missing, break during an edit, or lose write permissions. In Rev 5, the **Cornerstone Game Engine** never crashes blindly on startup: it boots through **Tier-0 Safe-Mode BIOS vaults**, falls back to **compiled-in `.rodata` Config ROMs**, and pauses transparently on bad assumptions using an **Autonomous Probe → Safe-Mode Downgrade →** `DEBUG_STEP_AWAIT` safety loop.

2.1. Tier-0 Safe-Mode BIOS & Quarantined Fallbacks

Think of `*_fallback.[ch]` files as a motherboard BIOS. Even if your hard drive configuration folder is wiped clean, the engine must still initialize telemetry, mount virtual read-only defaults, mix safe audio cues, and open a window showing what went wrong.

Every modernized subsystem follows a **3-file quarantine layout**:

File Pattern	Responsibility	Why Quarantine Matters
`<mod>_types.h`	Pure enums, structs, limits, callbacks	Breaks circular header dependencies between main code, BIOS fallback, and tests.
`<mod>_fallback.[ch]`	Minimal Tier-0 Safe-Mode boot & ROM reader	Zero dependencies on higher layers or disk parsers; boots even on an empty disk.
`<mod>.h` / `<mod>.c`	Full runtime subsystem + probes	Hydrates disk overrides on top of BIOS defaults and registers Block 4 probes.



> **Honest Telemetry Flag (`loaded_from_data`):** Every hydrated policy carries an explicit boolean `loaded_from_data`. It is set to `false` under Safe-Mode ROM/floppy defaults and flips `true` **only** after an authentic external JSON file parses and validates cleanly.

2.2. Compiled-In `.rodata` Config ROM Vaults

Rather than hardcoding duplicate default tables inside C source files, CMake runs `cmake/GenerateConfigRom.cmake` during build configuration (`generate_telemetry_rom`, `generate_diagnostics_rom`, `generate_io_rom`). Every `.json` file inside those `engine/config/` subtrees is packed directly into immutable read-only `.rodata` C byte arrays under `build/generated/`.

Build-Time Compiled .rodata Config ROM Generator (`cmake/GenerateConfigRom.cmake#L113-L156` · Mode: `Whole`)

```
113 # ================================================================================
114 # BLOCK 3: BODY BLOCK
115 # ( Deterministic Discovery, Hex-to-C Encoding & Table Construction )
116 # ================================================================================
117 # Engine:        Cornerstone Game Engine
118 # Stratum:       Layer 0 Infrastructure (Build Tooling)
119 # Module:        build.rom.config
120 # Mission:       Traverse sorted configuration domains and encode every JSON
121 #                asset into null-terminated .rodata C arrays and table rows.
122 # ================================================================================
123
124 # --------------------------------------------------------------------------------
125 # Sub-Assembly 3.1: Sorted Domain Traversal & Identifier Mapping
126 # --------------------------------------------------------------------------------
```

```
set(_sorted_subdirs ${SUBDIRS})
list(SORT _sorted_subdirs) ①

foreach(_subdir IN LISTS _sorted_subdirs)
    set(_abs_dir "${CONFIG_ROOT}/${_subdir}")
    if(NOT IS_DIRECTORY "${_abs_dir}")
        message(FATAL_ERROR
            "GenerateConfigRom.cmake: Subdirectory does not exist: '${_abs_dir}'")
    endif()

    file(GLOB _dir_jsons "${_abs_dir}/*.json") ②
    list(SORT _dir_jsons)
    list(LENGTH _dir_jsons _dir_count)

    if(_dir_count EQUAL 0)
        message(FATAL_ERROR
            "GenerateConfigRom.cmake: No JSON files found in '${_abs_dir}'")
    endif()

    string(APPEND _subdirs_doc "  engine/config/${_subdir}/ (${_dir_count} assets)\n")
    get_filename_component(_subdir_leaf "${_subdir}" NAME)

    # ------------------------------------------------------------------------
    # Sub-Assembly 3.2: Deterministic Byte-Array Encoding & Table Row Emission
    # ------------------------------------------------------------------------
    foreach(_json_path IN LISTS _dir_jsons)
        get_filename_component(_stem "${_json_path}" NAME_WE)
        get_filename_component(_filename "${_json_path}" NAME)

        if(ID_MODE STREQUAL "STEM")
```

① Sorts configuration subdirectories so generated .rodata asset tables remain deterministic across hosts.

② Discovers all JSON configs in each domain directory before sorting and byte-encoding them.

Transforms engine/config//.json directly into immutable ELF .rodata C byte tables at configure/build time.*

Wholesale Listing: Complete verbatim code (#L1-L302) → Appendix: Build-Time Compiled .rodata Config ROM Generator.

That single build step eliminates drift between engine JSON schemas and emergency defaults: when a disk config is missing or corrupted, Tier-0 reads the **exact same canonical JSON bytes** straight out of executable read-only memory!

2.3. Safe-Mode Boot, Floppy Floor & Downgrade Transitions

When io_fallback encounters missing files, path traversal (..), or read-only disk conditions, it executes structured transitions between full boot (io_fallback_boot), stripped-down emergency floor (io_fallback_boot_floppy), and read-only lock (io_fallback_downgrade_safe_mode):

Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch (engine/src/io/io_fallback.c#L3309-L3332 · Mode: Group)

```
bool io_fallback_boot_floppy(void)
{
    io_fallback_config_t floppy = s_io_floppy_template;
    if (io_fallback_load_rom_config("floppy_safe", &floppy) ||
        io_fallback_load_rom_config("default", &floppy)) {
        s_io_fb.rom_hydrated = true;
    } else {
        s_io_fb.rom_hydrated = false;
    }
    floppy.read_only_mode = true;
    floppy.loaded_from_data = false;
    if (floppy.profile_name[0] == '\0' || strcmp(floppy.profile_name, "default") == 0) {
        str_safe_copy(floppy.profile_name, IO_FALLBACK_FLOPPY_PROFILE_NAME,
                      sizeof(floppy.profile_name));
    }

```

```
    s_io_fb.config = floppy;
    io_fb_hydrate_subpolicies();
    s_io_fb.active_sink = IO_FB_SINK_FLOPPY_RO;
    str_safe_copy(s_io_fb.active_root, IO_FB_DEFAULT_ROOT, sizeof(s_io_fb.active_root));
    s_io_fb.booted = true;
    io_fallback_register_debug_probes();
    return true;
}
```

Zero-dependency Safe-Mode transitions ensuring deterministic boot even when disk configs are missing or corrupt.

Wholesale Listing: Complete verbatim code (#L2908-L3332) → Appendix: Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch.

2.4. Flight Recorder (log/), Debugger (debug/) & Autonomous Probe Loop

Two foundational Layer 0 systems watch every transition:

- **Flight Recorder (cog.log):** Zero-heap categorized circular ring buffer preserving telemetry before filesystem logs even open. Sub-Assembly 4.1 wires health probes straight into the debugger:

Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1) (engine/src/log/log.c#L1934-L1977 · Mode: Subassembly)

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 4.1: Contract, Input & Cog Verification Cogs
Mission: Verify received parameters, configuration contracts, and data provenance;
verify that declared subsystem cogs run properly as standalone units.
--------------------------------------------------------------------------------
*/

/**
 * @brief Returns whether the active logging subsystem was configured from data sources.
 *
 * ### For Game Designers & Non-Programmers:
 * Checks if settings were loaded from actual JSON files rather than emergency defaults.
 *
 * ### For Systems Programmers & Engine Architects:
 * Queries provenance boolean s_log.config.loaded_from_data.
 *
 * @return true if loaded from JSON data; false if using fallback code page.
 */
bool log_is_data_driven(void)
{
    return s_log.config.loaded_from_data;
}

/**
 * @brief Queries whether the logging engine is currently operating in Safe Mode.
 *
 * ### For Game Designers & Non-Programmers:
 * Returns true if the engine booted using compiled-in ROM backups or emergency
 * defaults instead of live external JSON directories on disk.
 *
 * ### For Systems Programmers & Engine Architects:
 * Checks initialization state alongside s_log.safe_mode (!loaded_from_data).
 *
 * @return true if active and running in Safe Mode; false otherwise.
 */
bool log_is_safe_mode(void)
{
    return s_log.initialized && s_log.safe_mode;
}

/**
 * @brief Returns the current state of the logging data pipeline tripwire.
 *
```

Shows how cog.log verifies its own static ring limits and binds probes into cog.debug.

Wholesale Listing: Complete verbatim code (#L1934-L2028) → Appendix: Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1).

- **Time-Machine Debugger (cog.debug):** Captures deterministic FNV-1a hashed state snapshots, evaluates active invariants, and coordinates the **Autonomous Probe → Safe-Mode →** DEBUG_STEP_AWAIT loop:

Autonomous Validation Probes & Tripwire Trap Chain (engine/src/debug/debug.c#L2367-L2411 · Mode: Group)

```
2367 /**
2368  * @brief Legacy compatibility helper that latches a state-corruption tripwire trap.
2369  *
2370  * @deprecated Prefer `debug_trigger_probe()` or `DEBUG_TRIGGER_PROBE()` so every trap
2371  *             carries a structured probe ID, tier classification, and forensic report.
2372  *
2373  *   ### For Game Designers & Non-Programmers:
2374  *   Preserved safety brake for older modules that report state errors without
2375  *   naming a specific modern probe identifier.
2376  *
2377  *   ### For Systems Programmers & Engine Architects:
2378  *   Latches `DEBUG_TRIPWIRE_STATE_CORRUPT`, increments `traps_triggered`, notifies
2379  *   BIOS via `debug_fallback_record_fault()`, performs guarded snapshot lock and
2380  *   `DEBUG_STEP_AWAIT` entry when active and non-audit, and emits `LOG_WARN`.
2381  *
2382  * @param[in] subsystem Subsystem tag reporting the fault, or NULL for `"unknown"`.
2383  * @param[in] condition Description of the violated condition, or NULL for `"anomaly"`.
2384  * @param[in] file      Originating source file (`__FILE__`).
2385  * @param[in] line      Originating source line (`__LINE__`).
2386  * @triggers Latches tripwire state, records BIOS fault, and warns via guarded log output.
2387  */
2388 void debug_trap_tripwire(const char *subsystem, const char *condition,
2389                          const char *file, int line)
2390 {
2391     /* Step 1: Latch classic STATE_CORRUPT code and forensic detail string verbatim */
2392     s_debug.tripwire_status = DEBUG_TRIPWIRE_STATE_CORRUPT;
2393     s_debug.traps_triggered++;
2394 
2395     snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
2396              "[%s] %s (%s:%d)",
2397              (subsystem != NULL) ? subsystem : "unknown",
2398              (condition != NULL) ? condition : "anomaly",
2399              (file != NULL) ? file : "unknown", line);
2400 
2401     /* Step 2: Record fault in Tier-0 BIOS and complete via Safe-Mode fallback */
2402     (void)debug_fallback_record_fault(
2403         subsystem,
2404         (condition != NULL) ? condition : "anomaly",
2405         file,
2406         line,
2407         "debug_trap_tripwire"
2408     );
2409 
2410     if (s_debug_active && !s_debug.in_probe && !s_debug.in_audit) {
2411         (void)debug_fallback_finish_with_fallback(
```

Realizes the Probe → Safe-Mode Downgrade → DEBUG_STEP_AWAIT loop inside Layer 0 debug infrastructure.

Wholesale Listing: Complete verbatim code (#L1918-L2438) → Appendix: Autonomous Validation Probes & Tripwire Trap Chain.

Anatomy of a Resilient Failure Recovery

1. A subsystem detects invalid config bounds or corrupt JSON in its Block 4 validation probe.
2. Instead of calling `abort()` or continuing silently with corrupted math, the probe traps into `debug_trap_tripwire()` / Safe-Mode downgrade (`loaded_from_data = false`).

3. Execution enters `DEBUG_STEP_AWAIT`, holding the frame intact with a human-readable reason, fault ring trace, and snapshot state so developers or tests can inspect **why** right away.

2.5. Self-Audit & Zero-Disk Checks

```
# Run Layer 0 log, debug, and io integration tests through CTest
ctest --test-dir build -R "log|debug|io|fallback|rom" --output-on-failure

# Boot headless audit cleanly even against an empty data root
./build/tenebris-lux --audit --data-root /tmp/empty_safe_mode_test
```

1. *If a designer accidentally deletes a JSON file, does the engine lose all defaults?*

   No. `GenerateConfigRom.cmake` embeds verified JSON defaults in `.rodata`, so Tier-0 BIOS boots cleanly with `loaded_from_data == false`.

2. *Does Safe-Mode silently pretend nothing went wrong?*

   No. Telemetry logs a clear downgrade event, locks unsafe disk mutations if damaged, and fires validation probes toward `DEBUG_STEP_AWAIT`.

2.6. Where to Go Next

- Previous: Chapter 1 — System Architecture Overview.
- Next: Chapter 3 — Data-Driven Architecture: Dual Data Taxonomy, Layered VFS & 5-Block JSON.
- Code Tours: See Part 2 chapters for Flight Logging, State Debugger, and Zero-Heap I/O Pipeline.

Chapter 3. Data-Driven Architecture: Dual Data Taxonomy, Layered VFS & 5-Block JSON

Source File: `docs/architecture/data_driven_architecture.adoc`

You should never need to recompile C source code just to adjust a damage curve, swap a font theme, remap controls, or tweak a debug threshold. **Cornerstone Game Engine** is engineered so creators work in structured **5-Block JSON**, while compiled Rust tooling (`tl_tools`) and C23 zero-heap loaders keep every setting trustworthy at runtime.

3.1. Dual Data Taxonomy: Engine Profiles vs. Game Canon

Cornerstone strictly separates **how the machine runs** from **what story universe it runs**:

Data Domain	Canonical Root	Example Subtrees & Contents
Machine Profiles (Engine)	`engine/config/`	• `viewport/`: window layout, `640x360` scalers, themes, typography, assets. • `audio/`, `input/`: mixer channels, default keyboard & gamepad bindings. • `runtime/`, `io/`: math formulas, VFS routes, filesystem buffer policies. • `diagnostics/`, `standards/`: telemetry rules, probe matrix, 5-block specs.
Domain Canon (Game)	`game/data/`	• `entities/`: vocations, roster templates, protagonist stats. • `mechanics/`: Twin Relics, combat skills, tactical terrain, dispatch scripts. • `world/`, `presentation/`: Libira & Sacyra provinces, UI data overrides.



> **Why split these trees?** A new game built on **Cornerstone Game Engine** replaces `game/data/` completely while keeping `engine/config/` intact. Meanwhile, **Tenebris and Lux** can override individual engine defaults cleanly through VFS precedence without mutating upstream engine baselines.

3.2. Layered VFS Precedence (`path_finder`)

Instead of scattering raw string paths like `../../data/foo.json` across C files, every subsystem queries logical domains through `path_finder` (`engine/include/io/path_finder.h`). For the `"config"` domain, `vfs_routes.json` defines an explicit override cascade:

1. `game/config/<rel_path>` — Game-specific or user mod override (highest priority).
2. `engine/config/<rel_path>` — Standalone engine baseline default.
3. `game/data/presentation/config/<rel_path>` — Symbolic presentation linkers.

Examine the live **`vfs_routes.json`** configuration (`whole` file tracked via Appendix A, with `block_3_body` highlighted inline):

Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json) (engine/config/io/vfs_routes.json#L55-L99 · Mode: Whole)

```
55  "block_3_body": {
56    "default_data_root": "game/data",
57    "env_override_var": "TL_DATA_ROOT",
58    "block_path_traversal": true,
59    "max_symlink_depth": 4,
60    "allow_mod_mounts": true,
61    "rom_fallback_enabled": true,
62    "probe_await_on_corrupt": true,
63    "config_precedence": [
64      "game/config/",
65      "engine/config/",
66      "game/data/presentation/config/"
67    ],
68    "fallback_candidate_roots": [
69      "game/data",
70      "../game/data",
71      "engine/config",
72      "../engine/config",
73      "data",
74      "../data"
75    ],
76    "asset_roots": [
77      "game/assets",
78      "../game/assets",
79      "assets",
80      "../assets"
81    ],
82    "domain_search_paths": [
83      "",
84      "runtime/",
85      "runtime/math/",
86      "runtime/string/",
87      "runtime/time/",
88      "diagnostics/",
89      "diagnostics/telemetry/",
90      "diagnostics/debugger/",
91      "diagnostics/taxonomy/",
92      "diagnostics/formats/",
93      "io/",
94      "audio/",
95      "viewport/",
96      "input/",
97      "standards/",
98      "standards/blocks/",
99      "mechanics/",
```

Canonical 5-block engine config defining VFS root search hierarchy, alias tables, and fail-closed safety flags.

Wholesale Listing: Complete verbatim code (#L1-L159) → Appendix: Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json).

3.3. The 5-Block JSON Contract & block_3_body Isolation

Just like source files and documentation chapters, every data config shares the exact 5-block skeleton (block_1_metadata, block_2_types/block_2_opening, block_3_body, block_4_audit, block_5_closing).

To prevent accidental leaks where an authoring invariant string in metadata could be misparsed as a live setting, master_loader **never** reads top-level metadata during runtime config queries. It isolates block_3_body first—shown directly by this block-granularity JSON snapshot:

JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json) (engine/config/io/vfs_routes.json#L55-L102 · Mode: Block)

```
55  "block_3_body": {
```

```
    "default_data_root": "game/data",
    "env_override_var": "TL_DATA_ROOT",
    "block_path_traversal": true,
    "max_symlink_depth": 4,
    "allow_mod_mounts": true,
    "rom_fallback_enabled": true,
    "probe_await_on_corrupt": true,
    "config_precedence": [
      "game/config/",
      "engine/config/",
      "game/data/presentation/config/"
    ],
    "fallback_candidate_roots": [
      "game/data",
      "../game/data",
      "engine/config",
      "../engine/config",
      "data",
      "../data"
    ],
    "asset_roots": [
      "game/assets",
      "../game/assets",
      "assets",
      "../assets"
    ],
    "domain_search_paths": [
      "",
      "runtime/",
      "runtime/math/",
      "runtime/string/",
      "runtime/time/",
      "diagnostics/",
      "diagnostics/telemetry/",
      "diagnostics/debugger/",
      "diagnostics/taxonomy/",
      "diagnostics/formats/",
      "io/",
      "audio/",
      "viewport/",
      "input/",
      "standards/",
      "standards/blocks/",
      "mechanics/",
      "mechanics/items/",
      "mechanics/combat/",
      "mechanics/tactics/",
```

Demonstrates scoped block_3_body extraction—matching master_loader's strict rule against reading Block 1/2 metadata as live gameplay config.

Wholesale Listing: Complete verbatim code (#L55-L125) → Appendix: JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json).

3.4. Zero-Heap Scoped Accessors (`master_loader`) & Shared Span Parsers

In Layer 0 (ALLOC_ZERO_HEAP), parsing JSON must allocate **zero heap memory**. Caller buffers (fs_utils), Tier-0 span scanners (io_fallback), and typed scoped accessors (master_loader) collaborate over caller-owned buffers:

Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2)
(engine/include/io/master_loader.h#L436-L479 · Mode: Subassembly)

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 3.2: Scoped `block_3_body` Accessors
Mission: Restrict all payload getters and array locators strictly inside the
         verified `[body_start, body_start + body_len)` span (or named `sub_obj`).

  ### For Game Designers & Non-Programmers:
```

```
 Reads gameplay properties cleanly from inside `block_3_body` only, preventing
 file headers or audit sections from ever colliding with gameplay field names.

 ### For Systems Programmers & Engine Architects:
 Delegates scope isolation and typed extraction exclusively to span-bounded
 `json_*_span` APIs with unconditional NUL termination on extracted strings.
--------------------------------------------------------------------------------
*/

/**
 * @brief Extracts a string value strictly within `block_3_body` (or named sub-object).
 *
 * ### For Game Designers & Non-Programmers:
 * Safely reads text properties (character names, formulas, IDs) from the data body
 * without accidentally matching header text outside Block 3.
 *
 * ### For Systems Programmers & Engine Architects:
 * Uses span-bounded `json_find_object_span()` + `json_get_string_span()` bounded to
 * `loader->body_len`. Unconditionally NUL-terminates `out_val` when `out_size > 0`.
 * @triggers `io.json.depth_bounds` if scan breaches `max_json_depth`.
 *
 * @param[in]  loader   Ready `master_loader_t` instance.
 * @param[in]  sub_obj  Optional sub-object key inside `block_3_body` (or NULL for root body).
 * @param[in]  key      Target property key inside resolved body/sub-object span.
 * @param[out] out_val  Caller string buffer.
 * @param[in]  out_size Capacity of `out_val` in bytes.
 * @return true on clean extraction; false if missing, wrong type, or invalid params.
 */
bool master_loader_get_string(const master_loader_t *loader,
                              const char *sub_obj,
                              const char *key,
                              char *out_val,
                              size_t out_size);

/**
 * @brief Extracts a signed integer strictly within `block_3_body` (or named sub-object).
 *
```

Zero-heap typed getters that scan directly inside a validated block_3_body span.

Wholesale Listing: Complete verbatim code (#L436-L608) → Appendix: Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2).

Under the hood, both BIOS boot routines and master_loader reuse the same shared span-resolution cogs inside engine/src/io/io_fallback.c:

Shared Zero-Heap JSON Block Resolver & VFS Parser Group (engine/src/io/io_fallback.c#L779-L806 · Mode: Group)

```c
static bool io_fb_resolve_body_span(const char *json_text,
                                    size_t json_len,
                                    bool allow_strlen,
                                    const char **out_body_start,
                                    const char **out_body_end)
{
    if (json_text == NULL || out_body_start == NULL || out_body_end == NULL) {
        return false;
    }

    size_t effective_len = (json_len > 0U) ? json_len : (allow_strlen ? strlen(json_text) : 0U);
    if (effective_len == 0U || effective_len > IO_FB_JSON_MAX_BYTES) {
        return false;
    }

    const char *json_end = json_text + effective_len;
    const char *body_start = NULL;
    size_t body_len = 0U;
    if (!io_fb_find_block_span(json_text, json_end, "block_3_body", '{', '}',
                               &body_start, &body_len) ||
        body_start == NULL || body_len == 0U) {
        return false;
    }

```

```
803     *out_body_start = body_start;
804     *out_body_end = body_start + body_len;
805     return true;
806 }
```

Tier-0 in-place span scanning reused seamlessly between BIOS fallback and master_loader.

Wholesale Listing: Complete verbatim code (#L779-L1194) → Appendix: Shared Zero-Heap JSON Block Resolver & VFS Parser Group.

Where Publication & Theme Configs Plug In

Even this textbook manual follows the exact same rule: docs/master_guide_manifest.json, docs/guide_theme_config.json, docs/code_snapshots_manifest.json, and docs/adoc_5block_standard.json drive typography, PDF margins, code extraction modes, and chapter assembly purely from JSON data rather than Rust constants.

3.5. Validating Data Files Before Commit

```
# Audit all engine/config/ and game/data/ 5-block manifests & JSON schemas
./tools/target/debug/tl_tools validate all

# Launch with a custom mod/testing mount via CLI override
./build/tenebris-lux --audit --data-root game/data
```

1. *Can* master_loader *accidentally read a description string from* block_1_metadata*?*

   No. Every typed lookup in master_loader first narrows the scan window strictly inside "block_3_body".

2. *Who checks that 5-block JSON configs remain well-formed?*

   tl_tools validate all (tools/src/validator/json_data.rs) verifies every config file under engine/config/ and game/data/ before compilation or commits.

3.6. Where to Go Next

- Previous: Chapter 2 — Zero-Disk Safe-Mode BIOS, Config ROM Vaults & Autonomous Probe Loop.
- Next: Chapter 4 — Central Manifest Registry & Pre-Flight Loose-Bolts Inspection.
- Code Tour: Explore the I/O module details in Layer 0 Zero-Heap I/O Pipeline.

Chapter 4. Central Manifest Registry & Pre-Flight Loose-Bolts Inspection

Source File: docs/architecture/preflight_and_manifest_spec.adoc

Before an aircraft leaves the runway, technicians inspect critical fasteners, sensor buses, and fuel lines in a strict checklist order. In the **Cornerstone Game Engine**, **cog.manifest** (engine/include/core/manifest.h) executes that exact pre-flight walk at startup: verifying that every engine subsystem exists, obeys its dependency DAG, honors ALLOC_ZERO_HEAP, and passes internal self-checks before a single turn of gameplay executes.

4.1. Self-Describing Module Metadata & Registry Ontology

Every subsystem exposes a module_metadata_t descriptor (covering up to 32 modules in a static .bss manifest_registry_t) declaring its identity, layer, memory contract, upstream dependencies, and preflight callback.

Examine the canonical structs extracted from engine/include/core/manifest.h Sub-Assembly 2.3:

Central Manifest Metadata & Registry Ontology (manifest.h Sub-Assembly 2.3)
(engine/include/core/manifest.h#L207-L254 · Mode: Subassembly)

```
207 /*
208 ------------------------------------------------------------------------------
209 Sub-Assembly 2.3: Structural Entity & Object Definitions
210 ------------------------------------------------------------------------------
211 Ontological types for architectural strata, memory models, and diagnostic audits.
212 ------------------------------------------------------------------------------
213 */
214
215 /**
216  * @enum module_layer
217  * @brief Architectural layer strata within the hierarchical engine topology.
218  *
219  * For Designers: The 5 vertical tiers of the game engine, from deep bedrock
220  * utilities (Layer 0) up to the game rules (Layer 1) and graphical display (Layer 2).
221  *
222  * For Programmers: Strict downward dependency hierarchy enforced at compile time.
223  */
224 typedef enum module_layer {
225     LAYER_0_CORE_UTIL = 0, /**< Layer 0: Foundational zero-dependency utils. */
226     LAYER_1_SIMULATION,    /**< Layer 1: Headless game simulation engine. */
227     LAYER_2_PRESENTATION,  /**< Layer 2: Graphical and text viewports. */
228     LAYER_3_APPLICATION,   /**< Layer 3: Application entry and routing. */
229     LAYER_4_TESTS,         /**< Layer 4: Verification test harness suites. */
230     LAYER_COUNT
231 } module_layer_t;
232
233 /**
234  * @enum alloc_policy
235  * @brief Memory allocation contract governing a module's execution model.
236  *
237  * For Designers: Rules defining how a system manages memory, ensuring bedrock
238  * systems never allocate heap memory to prevent crashes and stutter.
239  *
240  * For Programmers: Invariant memory policy verified during code audit.
241  */
242 typedef enum alloc_policy {
243     ALLOC_ZERO_HEAP = 0, /**< 100% stack or static layout; zero dynamic heap. */
244     ALLOC_FIXED_POOL,    /**< Pre-allocated fixed-capacity buffer pools. */
245     ALLOC_ARENA,         /**< Frame/scene temporary arena allocation. */
246     ALLOC_DYNAMIC        /**< Managed dynamic heap allocation. */
247 } alloc_policy_t;
248
249 /**
250  * @enum concurrency_policy
251  * @brief Concurrency and reentrancy guarantees declared by a module.
252  *
```

```
 * For Designers: Rules specifying whether a system can run simultaneously across
 * multiple CPU threads or must remain tied to the main display loop.
```

Self-describing module contracts pairing engine layers, allocation policy, thread safety, and preflight hooks.

Wholesale Listing: Complete verbatim code (#L207-L370) → Appendix: Central Manifest Metadata & Registry Ontology (manifest.h Sub-Assembly 2.3).

Preflight Outcome	Severity Classification	Engine Behavior at Startup
PREFLIGHT_STATUS_OK	SEVERITY_PASS	Nominal green boot; full data-driven subsystems active.
PREFLIGHT_STATUS_WARNINGS	SEVERITY_WARNING	Boot continues with telemetry warnings (e.g., optional enhancement missing).
PREFLIGHT_STATUS_DOWNGRADED	SEVERITY_CRITICAL	Safe-Mode fallback engaged; game runs smoothly on .rodata ROM defaults (loaded_from_data = false).
PREFLIGHT_STATUS_HALT_FATAL	SEVERITY_FATAL	Immediate preflight halt (--audit returns non-zero); triggers tripwire report before corrupted state spreads.

4.2. The Pre-Flight Walk (manifest_run_preflight)

At startup or under ./tenebris-lux --audit, manifest_run_preflight() performs two deterministic passes: first verifying every declared upstream dependencies[] entry exists in the registry (preventing hidden layer inversions), then executing each subsystem's registered preflight_check() hook:

Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight) (engine/src/core/manifest.c#L412-L455 · Mode: Function)

```c
/**
 * @brief Executes comprehensive pre-flight diagnostics across all registered modules.
 */
preflight_status_t manifest_run_preflight(manifest_registry_t *registry)
{
    if (registry == NULL) {
        debug_trap_tripwire("manifest", "NULL registry passed to manifest_run_preflight", __FILE__, __LINE__);
        return PREFLIGHT_STATUS_HALT_FATAL;
    }

    registry->pass_count = 0;
    registry->warn_count = 0;
    registry->critical_count = 0;
    registry->fatal_count = 0;

    /* Step 1: Active DAG Dependency Graph Verification */
    for (size_t i = 0; i < registry->count; i++) {
        const module_metadata_t *mod = registry->modules[i];
        if (mod == NULL) continue;

        for (size_t d = 0; d < mod->dependency_count; d++) {
            const char *req_dep = mod->dependencies[d];
            if (req_dep == NULL) continue;

            bool found = false;
            for (size_t j = 0; j < registry->count; j++) {
                if (registry->modules[j] != NULL &&
                    strcmp(registry->modules[j]->module_id, req_dep) == 0) {
                    found = true;
                    break;
                }
            }

            if (!found) {
                diag_result_t dep_res;
```

```
                dep_res.passed = false;
                dep_res.severity = SEVERITY_FATAL;
                dep_res.subsystem = mod->module_id;
                dep_res.check_name = "Dependency DAG";
                dep_res.details =
                    "Fatal: Unregistered upstream dependency declared.";
                registry->results[i] = dep_res;
                registry->fatal_count++;
                registry->status = PREFLIGHT_STATUS_HALT_FATAL;
```

Iterates every registered module sequentially, classifying OK, WARNINGS, DOWNGRADED, and HALT_FATAL states.

Wholesale Listing: Complete verbatim code (#L412-L509) → Appendix: Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight).

4.3. Layer 3 Plug Audit Flow (game/src/main.c BLOCK 4)

The Layer 3 application plug bridges engine preflight checks, Layer 1 gameplay invariant audits (game_run_self_audit()), and Layer 2 viewport bounds audits (gui_audit_layout()) inside one inspectable audit routine:

Application Audit Integration (game/src/main.c BLOCK 4) (game/src/main.c#L193-L238 · Mode: Block)

```
// ============================================================================
// BLOCK 4: AUDIT PROCEDURES BLOCK
// Stratum: Layer 3 (Application Entry) | Machine: Presentation Router
// ============================================================================

// ----------------------------------------------------------------------------
// [ SUB-ASSEMBLY 4.1: COMPREHENSIVE SUBSYSTEM SELF-AUDIT SUITE ]
// Mission: Execute pre-flight manifest checks, sim invariants, and GUI bounds.
// ----------------------------------------------------------------------------

/**
 * @brief Executes the comprehensive engine self-audit suite and manifests.
 *
 * ### Machine Role & Mission
 * Serves as the `--audit` diagnostic tripwire. Executes:
 * 1. Pre-flight verification across all registered modules with DAG checks.
 * 2. Layer 1 simulation invariant audit (`game_run_self_audit()`).
 * 3. Layer 2 GUI viewport layout and collision audit (`gui_audit_layout()`).
 * Returns non-zero if any invariant check or pre-flight test fails.
 *
 * ### Ontological Classification
 * - Stratum: Layer 3 (Application Entry & Presentation Router)
 * - Machine: app.entry.main (Executable Entry & Presentation Router)
 * - Assembly: Diagnostic Self-Audit & Pre-Flight Verification
 * - Cog: Full Engine Diagnostics & Invariant Self-Audit Suite
 *
 * ### Execution & Resource Contract
 * - Memory Model: ALLOC_ZERO_HEAP (Stack allocation only).
 * - Concurrency: CONCURRENCY_MAIN_THREAD (Synchronous diagnostic sweep).
 * - Latency Budget: < 5000 microseconds execution time.
 *
 * @return 0 if all invariant checks pass; non-zero if any check fails.
 */
static int run_full_subsystem_audit(void)
{
    log_init(NULL);
    LOG_INFO(LOG_CAT_LOG, "Audit", 0.0f, "Starting comprehensive diagnostic self-audit");

    /* Step 1: Initialize central manifest and run pre-flight diagnostics */
    manifest_registry_t registry;
    init_engine_manifest(&registry);
    manifest_run_preflight(&registry);
    manifest_print_report(&registry);

    /* Step 2: Evaluate pre-flight outcome for fatal loose-bolts */
    if (registry.status == PREFLIGHT_STATUS_HALT_FATAL) {
```

Shows Layer 3 executing run_full_subsystem_audit() across Engine + Game + Test invariants from --audit.

Wholesale Listing: Complete verbatim code (#L193-L334) → Appendix: Application Audit Integration (game/src/main.c BLOCK 4).



Zero Silent Failures: Notice how `manifest_execute_dump_card()` lets operators inspect live memory budgets, subsystem statuses, and dependency trees interactively in `--debug` or terminal output—turning engine boot state into a readable inspection card rather than opaque startup noise.

4.4. Running Pre-Flight Diagnostics Yourself

```
# Execute full Layer 3 + Engine + Sim + GUI diagnostic sweep
./build/tenebris-lux --audit

# Validate C header manifest metadata & DAG registrations across all 44 engine headers
./tools/target/debug/tl_tools validate headers
```

1. *What happens if a module declares a dependency that was never registered?*

   Step 1 of `manifest_run_preflight()` catches the missing upstream node before any subsystem initialization runs, traps a tripwire, and returns `PREFLIGHT_STATUS_HALT_FATAL`.

2. *Does `tl_tools validate headers` cross-check these C manifest rules too?*

   Yes. `tools/src/validator/c_header.rs` checks all 44 Cornerstone engine headers against `engine/config/standards/subsystems.json` and the 5-block standard.

4.5. Where to Go Next

- Previous: Chapter 3 — Data-Driven Architecture: Dual Data Taxonomy, Layered VFS & 5-Block JSON.
- Next: Chapter 5 — Dual-Frontend Presentation Layer: CLI, Native X11 & Movie Reel.
- Deep Dive: Read Layer 0 Core Utilities Code Tour.

Chapter 5. Dual-Frontend Presentation Layer: CLI, Native X11 & Movie Reel

Source File: `docs/architecture/dual_frontend_architecture.adoc`

Many RPG engines tangle combat formulas directly into UI button handlers or draw loops—making automated testing brittle and porting nearly impossible. The **Cornerstone Game Engine** splits **Simulation Truth (Layer 1)** cleanly from **Presentation Viewports (Layer 2)** so **Tenebris and Lux** plays identically in a lightning-fast headless CLI session, inside the `--debug` time-machine REPL, or on a pixel-crisp `640x360` native X11 desktop window.

5.1. One Pure Simulation, Multiple Frontends

Every gameplay rule lives inside `game/` (Layer 1) operating strictly over `player_state_t`, pure integer coordinates, and deterministic combat/morality dispatch. Neither headless text commands nor graphical mouse clicks bypass Layer 1 validation.

Consider `sim_state_audit()` in `game/src/hero/sim_state.c`—a complete pure-C simulation audit `function` that operates cleanly without opening any X11 display or GUI window:

Layer 1 Deterministic Simulation State Audit (sim_state_audit) (`game/src/hero/sim_state.c#L315-L376` · Mode: `Function`)

```
315 game_audit_report_t sim_state_audit(const player_state_t *state)
316 {
317     game_audit_report_t report;
318     report.is_valid = true;
319     report.error_count = 0;
320     report.error_log[0] = '\0';
321 
322     if (state == NULL) {
323         report.is_valid = false;
324         report.error_count = 1;
325         snprintf(report.error_log, sizeof(report.error_log),
326                  "[ERROR] Player state pointer is NULL.");
327         return report;
328     }
329 
330     if (state->shield_power < 0 || state->shield_power > MAX_RESONANCE) {
331         report.is_valid = false;
332         report.error_count++;
333         snprintf(report.error_log + strlen(report.error_log),
334             sizeof(report.error_log) - strlen(report.error_log),
335             "[ERROR] Shield power (%d) outside bounds [0, %d]. ",
336             state->shield_power, MAX_RESONANCE);
337     }
338 
339     if (state->sword_edge < 0 || state->sword_edge > MAX_RESONANCE) {
340         report.is_valid = false;
341         report.error_count++;
342         snprintf(report.error_log + strlen(report.error_log),
343             sizeof(report.error_log) - strlen(report.error_log),
344             "[ERROR] Sword edge (%d) outside bounds [0, %d]. ",
345             state->sword_edge, MAX_RESONANCE);
346     }
347 
348     if (state->moral_sway < -100 || state->moral_sway > 100) {
349         report.is_valid = false;
350         report.error_count++;
351         snprintf(report.error_log + strlen(report.error_log),
352             sizeof(report.error_log) - strlen(report.error_log),
353             "[ERROR] Moral sway (%d) outside bounds [-100, 100]. ",
354             state->moral_sway);
355     }
356 
357     if (state->stance < 0 || state->stance >= STANCE_COUNT) {
```

```
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Stance enum (%d) outside valid range [0, %d). ",
            state->stance, STANCE_COUNT);
    }

    if (state->location < 0 || state->location >= LOC_COUNT) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Location enum (%d) outside valid range [0, %d). ",
            state->location, LOC_COUNT);
    }

    return report;
}
```

Pure simulation integrity audit verifying hero stats, inventory bounds, morality sway, and world state with zero frontend dependencies.

Wholesale Listing: Complete verbatim code (#L315-L376) → Appendix: Layer 1 Deterministic Simulation State Audit (sim_state_audit).

Launch Flag (tenebris-lux)	Frontend Mode	Ideal Audience & Workflow
-g, --gui	Native X11 640x360 Viewport	Players, pixel artists, and UI designers reviewing animations, palettes, and 5-screen menus.
-c, --cli	Headless Terminal REPL	Playtesters, writers, and automated scripts stepping through choices and endings at CPU speed.
-d, --debug	Interactive Time-Machine Debugger	Engineers and QA inspecting FNV-1a hashes, rewinding snapshots, and checking DEBUG_STEP_AWAIT traps.
-a, --audit	Pre-Flight + Invariant Sweep	CI pipelines and local commits verifying boot health, zero-heap limits, and GUI panel bounds in <5ms.

5.2. Native 640x360 Viewport & The 2.5D Movie Reel Model

Rather than pulling in external third-party UI frameworks or bloat, Layer 2 (engine/include/viewport/) implements a self-contained software rasterizer rendering directly to a native 640x360 logical canvas backed by <X11/Xlib.h> in Linux desktop builds.

Visually, the viewport works like a **Classic Cinema Movie Reel**:

- **Horizontal Strip Navigation:** Screens glide left and right cleanly across canonical game views (Title, Gameplay, Party/Inventory, Map/Journal, System Settings).
- **Vertical Depth Projection:** Integer tactical grid coordinates (gui_grid_coord_t) project cleanly through dimetric isometric transforms (gui_iso_point_t) and painter's-algorithm depth queues (gui_depth_entity_t).
- **Integer/Letterbox Scaling (geom_utils):** A fixed 640x360 logical frame scales cleanly onto any desktop monitor without stretching pixel art or leaking screen coordinates back into Layer 1 game mechanics.

Inspect the canonical Layer 2 data contracts extracted from engine/include/viewport/raster/gui_types.h Sub-Assembly 2.3:

Layer 2 640x360 Viewport & 2.5D Movie Reel Data Structures (gui_types.h Sub-Assembly 2.3)
(engine/include/viewport/raster/gui_types.h#L138-L185 · Mode: Subassembly)

```c
/*
--------------------------------------------------------------------------------
Sub-Assembly 2.3: Structural Entity & Object Definitions
--------------------------------------------------------------------------------
Core types, layout bounding rectangles, color models, view modes, and widget
structures for the desktop presentation engine.
--------------------------------------------------------------------------------
*/

/**
 * @enum gui_screen_mode
 * @brief Presentation screen views and modal navigation states.
 *
 * For Designers:
 * Represents the 5 primary game screens accessible via the top navigation tabs:
 * Tactical Grid, Hero Dossier, Inventory, Provincial Map, and Chronicle Codex.
 *
 * For Programmers:
 * Bounded enumerator indexed [0, 4] matching tab descriptors.
 */
typedef enum gui_screen_mode {
    GUI_SCREEN_TACTICAL = 0,    /**< 2.5D / 2D Tactical battlefield & combat */
    GUI_SCREEN_STATUS,          /**< Full protagonist dossier & attributes */
    GUI_SCREEN_INVENTORY,       /**< Bag, key items, Grandma's Watch */
    GUI_SCREEN_WORLD_MAP,       /**< Provincial realm topology & travel */
    GUI_SCREEN_CHRONICLE        /**< Complete lore codex & history of deeds */
} gui_screen_mode_t;

/**
 * @struct gui_tab
 * @brief Top navigation tab descriptor.
 *
 * For Designers:
 * A clickable header tab bar button linking directly to a primary screen.
 *
 * For Programmers:
 * Bounded rectangular button descriptor with single hotkey accelerator.
 */
typedef struct gui_tab {
    int x;
    int y;
    int width;
    int height;
    const char *label;
    gui_screen_mode_t screen;
    char hotkey;
} gui_tab_t;
```

Canonical non-anonymous structs driving screens, palettes, layouts, and dimetric depth sorting.

Wholesale Listing: Complete verbatim code (#L138-L448) → Appendix: Layer 2 640x360 Viewport & 2.5D Movie Reel Data Structures (gui_types.h Sub-Assembly 2.3).

Future-Proof 3D Horizon Without Simulation Rewrites

Because player_state_t in Layer 1 tracks logical integers rather than X11 pixels or sprites, Cornerstone's Layer 2 viewport could evolve into hardware-accelerated 2D or full 3D rendering in the future without changing a single combat formula, dialogue branch, or save state inside game/.

5.3. Testing Both Frontends Cleanly

```
# Verify headless CLI, debugger, GUI layout audit, and X11 smoke checks via CTest
ctest --test-dir build --output-on-failure
```

```
# Run interactive headless gameplay or debugger directly in terminal
./build/tenebris-lux --cli
./build/tenebris-lux --debug
```

1. *Does testing the game require an active X11 monitor or GPU?*

   No. Headless `--cli`, `--debug`, `--audit`, and all simulation tests run cleanly in pure terminal/CI environments without opening an X11 window.

2. *How does the `640x360` GUI avoid getting out of sync with CLI gameplay?*

   Both frontends dispatch actions into the exact same Layer 1 `game/` state machine and share deterministic FNV-1a snapshot verification.

5.4. Where to Go Next

- Previous: Chapter 4 — Central Manifest Registry & Pre-Flight Loose-Bolts Inspection.
- Continue into **Volume V, Part 2** subsystem code tours starting at Layer 0 Core Utilities Code Tour and Layer 2 Presentation Viewport Code Tour.
- Review all extracted textbook code plates in Snapshots Index (or **Appendix A** when reading the compiled PDF book).

Chapter 6. Root Strata Chronicles: Why Strengthening log/, debug/, and io/ Pays Every Future Turn

Source File: `docs/architecture/chronicles/01_root_stratum_story_and_game_payoff.adoc`

For Game Designers & Creators: It is tempting to ask why months of careful baseline work go into flight logs, diagnostic tripwires, and bounded file loaders before drawing a single new combat animation or writing a new dialogue scene in **Tenebris and Lux**. The answer is simple: **a game engine is a living tree, and `log/`, `debug/`, and `io/` are its roots**. When the roots rot—or silently allocate unmanaged memory, crash on a missing JSON comma, or swallow errors—every branch above them (vocations, twin relics, grid combat, X11 frames) becomes fragile and twice as slow to build.

For Systems Architects: During the Governance Revision 5 (`Rev 5`) baseline sprint of **Cornerstone Engine** (`engine/`), modernizing `log/` → `debug/` → `io/` ahead of `core/` Block 3 work, `audio/`, `viewport/`, and `game/` surfaced structural lessons that fundamentally de-risk the rest of the project.

Stratum / Root	Without Rev 5 Hardening	With Rev 5 Root Baseline
`log/` (Flight Recorder)	Silent early-boot drops; interleaved disk crashes; ad-hoc `printf`.	Zero-disk `log_fallback` boot, typed severity rings, ROM defaults, and deterministic test resets.
`debug/` (Nervous System)	Bugs manifest five systems later as cryptic segfaults with zero state trail.	Probes catch bad invariants at the boundary and pause cleanly inside `DEBUG_STEP_AWAIT`.
`io/` (Supply Line)	Duplicate JSON parsers, hidden `static char[32768]` scratch bloat, fragile file paths.	3-tier VFS (`game/config` → `engine/config` → `presentation`), `rom://io`, and `block_3_body` isolation.

6.1. Lesson 1: Purging Legacy Shims Lets the Compiler Map Every Seam

Early in engine refactors, keeping "temporary compatibility wrappers" feels polite—nothing breaks immediately. In practice, shims hide stale callers across multiple modules until a subtle behavioral mismatch blows up weeks later inside gameplay code.

Cornerstone's **Active Iteration Feedback Covenant** flips that habit on its head:

1. **Delete retired shims immediately** from headers and sources.
2. **Compile strictly** (`-Wall -Wextra -Wpedantic`, zero suppressed warnings) and run the 4-tier CTest suite so the compiler itself points to every single outdated call site.
3. **Upgrade every caller cleanly** to the true contract before advancing.

By paying that small upfront cleanup cost across `log/` and `debug/`, `io/` inherited clean caller patterns from day one—and future `core/`, `audio/`, and `game/` refactors will never trip over ghost wrappers left behind in `engine/`.

6.2. Lesson 2: Splitting Pure Types & Measuring the Chassis Eliminates Hidden Bloat

Two physical changes transformed dependency flow and memory visibility across the root strata:

- **Quarantined Leaf Type Headers (`*_types.h`)**: Separating plain data definitions (`log_types.h`, `debug_types.h`) from behavioral headers broke circular header includes between full engines and Safe-Mode BIOS fallbacks (`*_fallback.[ch]`), keeping the dependency graph strictly acyclic.
- **Controlled Memory Chassis (`ALLOC_ZERO_HEAP`)**: Because Layers 0, 1, and 2 allow zero dynamic heap allocation, every single byte lives visibly in ELF `.rodata`, `.data`, `.bss`, and bounded stack frames. Auditing `io/` revealed how easily forgotten function-local `static char[]` scratch buffers quietly inflate `.bss`; moving temporary scans onto tight caller slices shrank engine chassis overhead so remaining fixed capacity serves genuine RPG growth (units, vocations, skill catalogs, turn history) rather than dead engine scratch space.

6.3. Lesson 3: Building Forward for C23/C20 While Anchoring a C11/C17 Floor

Cornerstone treats portability like a **Java-style capability ladder** built directly inside bedrock `engine/include/core/manifest.h`:

Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17) (`engine/include/core/manifest.h#L82-L143` · Mode: `Subassembly`)

```
 82 /*
 83 ------------------------------------------------------------------------------
 84 Sub-Assembly 2.1: Pre-Conditions, Progressive Standard Macros & Buffer Limits
 85 ------------------------------------------------------------------------------
 86 Compile-time sizing limits for registry module nodes, dependencies, rules, and
 87 Cornerstone's "Fall-Down" ISO C23 / C17 / C11 + C++20 / C++17 capability macros.
 88
 89 For Designers:
 90 Allows Cornerstone to take advantage of modern compiler safety checks on current
 91 hardware while stepping cleanly onto older systems without legacy clutter. Also
 92 bounds the engine registry to up to 32 major systems, 8 core rules, and 4 links.
 93
 94 For Programmers:
 95 - Guarantees 100% zero-heap allocation (ALLOC_ZERO_HEAP) in .rodata and .bss.
 96 - Exposes CS_C_STD_TIER (23/17/11/0) and CS_CXX_STD_TIER (20/17/11/0) plus
 97   graceful ISO attribute fall-down macros (CS_NODISCARD, CS_MAYBE_UNUSED,
 98   CS_FALLTHROUGH, CS_UNREACHABLE, CS_ALIGNOF, CS_NORETURN, CS_STATIC_ASSERT,
 99   CS_HAS_EMBED) compiled cleanly without extensions under -Wpedantic.
100 ------------------------------------------------------------------------------
101 */
102
103 /** Active ISO C standard tier compiled for this translation unit (23, 17, 11, or 0 in C++). */
104 #if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202311L
105 #    define CS_C_STD_TIER 23
106 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201710L
107 #    define CS_C_STD_TIER 17
108 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201112L
109 #    define CS_C_STD_TIER 11
110 #else
111 #    define CS_C_STD_TIER 0
112 #endif
113
114 /** Active ISO C++ standard tier when included from C++ (20, 17, 11, or 0 in C). */
115 #if defined(__cplusplus) && __cplusplus >= 202002L
116 #    define CS_CXX_STD_TIER 20
117 #elif defined(__cplusplus) && __cplusplus >= 201703L
118 #    define CS_CXX_STD_TIER 17
119 #elif defined(__cplusplus)
120 #    define CS_CXX_STD_TIER 11
121 #else
122 #    define CS_CXX_STD_TIER 0
123 #endif
124
125 /** Marks return values that callers MUST inspect (C23/C++17+ [[nodiscard]], C11/17 floor empty). */
126 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
127 #    define CS_NODISCARD [[nodiscard]]
128 #else
129 #    define CS_NODISCARD
130 #endif
```

```
131
132 /** Silences unused-entity diagnostics cleanly on intentional dormant/platform helpers. */
133 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
134 #    define CS_MAYBE_UNUSED [[maybe_unused]]
135 #else
136 #    define CS_MAYBE_UNUSED
137 #endif
138
139 /** Annotates explicit switch-case fall-through cleanly under -Wextra -Wpedantic. */
140 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
141 #    define CS_FALLTHROUGH [[fallthrough]]
142 #elif defined(__GNUC__) && (__GNUC__ >= 7)
143 #    define CS_FALLTHROUGH __attribute__((fallthrough))
```

Compile-time standard detection establishing C23/C17/C11 and C20/C17 parity without legacy shims.

Wholesale Listing: Complete verbatim code (#L82-L194) → Appendix: Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17).

Instead of writing lowest-common-denominator code **or** breaking older build environments, **the engine builds forward-first for ISO C23 and C20** (`CORNERSTONE_C_STANDARD=AUTO|23|17|11`) and steps gracefully down the stairs (`CS_NODISCARD`, `CS_MAYBE_UNUSED`, `CS_FALLTHROUGH`, `CS_UNREACHABLE`) onto C17/C11 and C17 floors without compatibility clutter. Modern compilers enforce stricter safety diagnostics automatically today, while older hardware still compiles the exact same tree without feature hacks.

6.4. Lesson 4: Zero-Disk Safe-Mode BIOS & Compiled .rodata ROMs Protect Designers

Previously, broken or missing config JSON files meant fragile dummy defaults hardcoded separately inside C structs—guaranteeing drift between real configs and fallbacks. Under Rev 5:

1. Authors edit one authoritative 5-block JSON config in engine/config/ (or game overrides in game/config/).
2. At build time, cmake/GenerateConfigRom.cmake compiles those exact JSON files into read-only ELF .rodata headers (*_config_rom.h).
3. If disk files vanish or fail syntax/schema validation, **_fallback** mounts the embedded .rodata ROM (rom://log, rom://debug, rom://io), sets provenance (loaded_from_data = false / loaded_from_rom = true), and lets the engine boot all the way to DEBUG_STEP_AWAIT so the developer sees *why without crashing.

The Game Payoff for Tenebris and Lux: Designers can iterate rapidly on game/data/ combat tables, terrain rules, or UI themes with total confidence that a malformed file triggers a clean, readable diagnostic card instead of silent data corruption or a dead startup window.

6.5. How Root Discipline Multiplies Velocity in core/, viewport/ & game/

Upcoming Subsystem	Direct Dividend Paid by the log/ → debug/ → io/ Roots
core/ Block 3 & Math/Time	Can wire formulas.json and manifest.json immediately through master_loader with pre-built rom:// fallback and typed spans.
audio/ & viewport/ (640x360 X11)	Inherit pre-wired probe namespaces, bounded .bss accounting, and deterministic CLI/GUI diagnostics from boot frame zero.
game/ (The Master Turn Gear)	Stays purely focused on RPG rules (Twin Relics, Stances, Moral Sway, Sacyra/Libira topology) without writing a single line of file I/O or log plumbing.

Upcoming Subsystem	Direct Dividend Paid by the log/ → debug/ → io/ Roots
Future Cornerstone Titles	Drop cleanly onto the same Layer 3 plug (main.c) because zero domain lore leaked into engine/.

- Continue to Chronicle 2: AGENTS.md Evolution & AI-Assisted Architecture.
- Return to Architecture Index or Codebase Implementation Index.

Chapter 7. Evolution of AGENTS.md: Engineering a Cognitive Compass for Human-AI Co-Creation

Source File: `docs/architecture/chronicles/02_agents_md_evolution_and_ai_architecture.adoc`

Long-running software projects involving C11/C23 systems programming, Rust verification tools, JSON schema contracts, and deep RPG lore break predictably under standard AI workflows for one reason: **context drift**. Left unanchored across multi-hour sprints, autonomous agents invent plausible-sounding lore names, re-introduce heap allocations in deterministic code, tuck quick shims over broken contracts, or forget how earlier subsystems were verified.

Across Iterations leading up to Governance Revision 5 (`Rev 5`), `AGENTS.md` evolved away from a loose checklist into a **5-Block Cognitive Compass** encoding **why** Cornerstone works the way it does alongside **how** every boundary is mechanically verified.

Governance Evolution	Engineering Shift Achieved
Passive Prose → 21 Explicit Invariants	Turns soft preferences into binary rules (`ALLOC_ZERO_HEAP`, strict ISO C, anti-domain-bleed, authentic canon: Sole Antagonist **The Dark Lord Usuru**).
File-Path IDs → Movement-Resilient `@id`	Allows cogs elevated from `game/` → `engine/` to keep logical identity (`cog.<module>`) while `@file` and `@layer` update cleanly.
Arbitrary Style → Universal 5-Block Protocol	Forces every C header, C source, JSON payload, CMake script, and AsciiDoc chapter into predictable structural slots (`BLOCK 1..5`).

7.1. Keeping the Operator in the Director's Chair

Automated systems fail when they strip architectural judgment away from the human creator. Cornerstone's `Rev 5` ruleset intentionally splits responsibility between **human creative direction** and **automated structural discipline**:

1. **Waterfall Bedrock vs. Agile Exploration (`AGENTS.md` §0.1)**: The agent never compromises fixed physical laws (zero heap in Layers 0–2, downward dependencies, zero domain bleed inside `engine/`), freeing the human director to iterate boldly on gameplay mechanics, tone, documentation depth, and visual presentation without worrying that an experimental sprint will rot the foundation.
2. **Dual-Audience Documentation Covenant (`Invariant 17`)**: Requiring every header, operational API, and architecture document to speak clearly both **For Game Designers & Non-Programmers** and **For Systems Programmers & Engine Architects** forces the AI to translate every low-level optimization into plain-English creative impact.
3. **Active Proposal Over Blind Guessing**: When toolchains step between C23, C17, and C11 (`CORNERSTONE_C_STANDARD=AUTO`) or telemetry uncovers `.bss` bloat, the architecture surfaces explicit options and seams so the operator steers design priorities consciously.

7.2. Why 5-Block AsciiDoc & Automated Validators Defeat Context Drift

Treating documentation files with the same **Universal 5-Block Comment Protocol** (`//// BLOCK 1..5 ////` and `// --- Sub-Assembly X.Y`) used in C and JSON closes a gap that normally ruins project documentation: docs going stale quietly after code moves.

Feedback Loop	What It Guarantees Automatically
`master_guide --snapshots`	Extracts verbatim source spans (`Whole`, `Block`, `Subassembly`, `Function`, `Group`) directly from live `engine/`, `game/`, `tests/`, and `cmake/` files into textbook plates—eliminating hand-copied drift.
`master_guide --validate-layout`	Reads data-driven PDF geometry constraints (`docs/adoc_pdf_layout_rules.json`), catches mid-file title resets, broken `include::` targets, unbalanced code fences, and over-tall/over-wide code listings before `asciidoctor-pdf` wastes pages.
`tl_tools validate all` + CTest	Audits Block 1–5 conformance, compiler warning cleanliness (`-Wall -Wextra -Wpedantic`), Rust `-D warnings`, and all 4-tier tests after edits land.

Because layout budgets, snapshot bindings (`@snapshots uses: ⋯`), chassis telemetry (`@chassis`), and probe routes (`@seams`) live explicitly in repository files rather than transient chat history, **every new session inherits exact structural truth immediately**.

7.3. Architectural Smells `AGENTS.md` Actively Catches

- **Fake Fallbacks & Dead Stubs**: Explicitly banned in favor of compiled `.rodata` ROM vaults (`cmake/GenerateConfigRom.cmake`) and quarantined `*_fallback.[ch]` engines sharing verified JSON parsers.
- **Hidden Static Scratch Bloat**: Caught directly by `@chassis` measurement (`size -A`, `nm --size-sort -S -r`), turning silent `.bss` waste into transparent numbers.
- **Non-Canonical Lore Drift**: Blocked at Block 1 governance and `game/data/` contract checks so **Tenebris and Lux** never acquires hallucinated characters, relics, or factions.
- **Unreadable Auto-Generated PDFs**: Solved by separating human narrative into structured 5-block chapters, verifying exact printable page dimensions mathematically, and compiling live textbook plates + wholesale appendices cleanly.

- Previous chronicle: Chronicle 1: Why Strengthening `log/`, `debug/`, and `io/` Pays Every Future Turn.
- Explore root governance directly in `AGENTS.md`, or return to Architecture Index.

Chapter 8. Layer 0 Categorized Flight Logging Subsystem Overview

Source File: docs/code/logging_system.adoc

Think of the **Layer 0 Categorized Flight Logging Subsystem** (engine/include/log/ & engine/src/log/) as an aircraft's black-box flight recorder: long before windows open or disk folders are found, it quietly records every subsystem heartbeat into fixed memory without ever asking the operating system for dynamic heap (ALLOC_ZERO_HEAP).



> **How this chapter fits with the log/ folder**: Read this overview for the big picture, then dive into docs/code/log/ for paired walkthroughs of log.h + log.c, log_types.h, log_fallback.h + log_fallback.c, and the 4-tier log verification suite.

8.1. Architectural Philosophy: The Flight Recorder

Printing raw debug text straight to the terminal breaks down the moment a real game hits frame budgets, file-permission errors, or multi-step bug reproduction. Cornerstone replaces ad-hoc printf calls with a two-stage deterministic flight recorder:

1. **Pre-VFS Tier-0 BIOS Capture (log_fallback)**: Before io/ mounts disk routes, log_fallback_boot() arms a 64-entry static BIOS ring buffer (12.3 KiB .bss) backed by 10 compiled-in .rodata telemetry ROMs (57.9 KiB). Early boot messages are never lost.
2. **Early-Entry Replay Drain (log_fallback_drain_entries)**: Once log_init() comes online, every entry captured during Tier-0 boot is drained cleanly into the primary circular ring buffer so the chronological timeline has zero gaps.
3. **Categorized Primary Ring & Rolling Disk Stream (cog.log)**: Messages are formatted on fixed stack frames (⇐ 1 KiB), stored in a bounded circular ring, tagged by severity (TRACE..FATAL) and channel mask (engine vs. game), and automatically rotated on disk when size ceilings are reached.
4. **Autonomous Probe Hooks & Safe-Mode Downgrade**: Sub-Assembly 4.1 binds log.* health probes directly into cog.debug, allowing damaged JSON telemetry configs to degrade safely onto compiled ROM profiles and optionally pause at DEBUG_STEP_AWAIT.

8.2. Verified Rev 5 File & Documentation Map

Companion Doc	Engine Files Covered	Role & Chassis Highlights
01: Router & Ring Engine	engine/include/log/log.h + engine/src/log/log.c (cog.log)	Primary circular ring recorder, channel masks, disk rotation, telemetry config loader, and log.* debug probes (~24.7 KiB .data, 16.4 KiB .bss VFS scratch, ~8.6 KiB .rodata).
02: Vocabulary Leaf Types	engine/include/log/log_types.h (cog.log.types)	Pure POD enums, bounds, log_entry_t, log_config_t, and log_stats_t. Zero .bss/.data state; breaks circular dependency between log and log_fallback.

Companion Doc	Engine Files Covered	Role & Chassis Highlights
03: Tier-0 Fallback BIOS	engine/include/log/log_fallback.h + engine/src/log/log_fallback.c (cog.log.fallback)	Quarantined emergency BIOS (12.3 KiB .bss) and 10 compiled .rodata telemetry ROMs (57.9 KiB), shared JSON config parser, and replay drain into primary logging.
04: Logging Verification Suite	5 CTest targets across tests/{unit,functional,integrated,system}/	Progressive crude-to-fine test coverage across happy path, hostile NULL/bounds, invariant checks, cross-tier replay emergence, and machine self-audit.

8.2.1. Telemetry Profiles in engine/config/diagnostics/telemetry/

Behavior is controlled without recompiling C code through JSON manifests in engine/config/diagnostics/telemetry/:

- manifest.json & default.json — baseline catalog and default profile binding.
- development.json, production.json, quiet.json, combat_focus.json — operational severity/channel presets.
- engine_channels.json, game_channels.json, severity_weights.json, forensic_telemetry.json — channel taxonomies, severity weights, and forensic export rules (all 10 mirrored into log_telemetry_rom.h at build time).

8.3. Live Public Dispatch Contract

log.h exposes clean categorized dispatch routines so engine subsystems and gameplay modules share one unified timeline without allocating memory:

Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3) (engine/include/log/log.h#L406-L445 · Mode: Subassembly)

```
406 /*
407 ------------------------------------------------------------------------------
408 Sub-Assembly 3.3: Formatted Entry Dispatch Cogs
409 Mission: Format, serialize, and emit structured flight recorder entries.
410 @triggers
411   - Tripwires      : LOG_TRIPWIRE_SETTING_INVALID / BOUNDS_VIOLATION -> probe "log.write_telemetry"
412   - Block 4 Audits : log_validate_telemetry_integrity(), log_preflight()
413   - Layer 4 Tests  :
414       * tests/unit/log/test_log_unit.c
415         [Section 3: INVARIANTS & CONSTRAINTS / test_log_unit_assertions_and_guarantees]
416         [Section 4: RING BUFFER FIFO OVERWRITE & FILE ROTATION / test_log_unit_emergence]
417       * tests/functional/test_log_functional.c
418         [Section 3: INVARIANTS & SATURATION / test_log_functional_assertions_and_guarantees]
419       * tests/integrated/test_log_integrated.c
420         [Section 4: CATEGORICAL DISK ROUTING & HUD RING BUFFER / test_log_integrated_emergence]
421       * tests/system/test_log_system.c
422         [Section 1: NOMINAL HIGH-THROUGHPUT BURST / test_log_system_always_pass]
423         [Section 3: FIXED-WIDTH REGEX DETERMINISM / test_log_system_assertions_and_guarantees]
424 ------------------------------------------------------------------------------
425 */
426
427 /**
428  * @brief Emits a structured log entry into the ring buffer, file, and console.
429  *
430  * ### For Game Designers & Non-Programmers:
431  * Writes an event to the game's flight recorder journal, capturing the character involved,
432  * their current moral balance percentage, and what action took place.
433  *
434  * ### For Systems Programmers & Engine Architects:
435  * Formats a fixed-column log record with microsecond monotonic timestamp and ISO 8601 UTC time,
436  * pushes it to the static circular ring buffer, and writes to disk archives with zero heap allocation.
437  *
438  * @param[in] severity Graded severity level.
```

```
439  * @param[in] category Target functional category.
440  * @param[in] entity   Entity/Actor label (e.g. "Player", "Companion", "System").
441  * @param[in] grade    Normalized grade [-1.0f .. +1.0f] (e.g. alignment sway or vitals).
442  * @param[in] file     Source file name (__FILE__).
443  * @param[in] line     Source line number (__LINE__).
444  * @param[in] func     Source function name (__func__).
445  * @param[in] fmt      Printf-style format string.
```

Public categorized log macros and fixed-stack formatting interface for engine and game channels.

Wholesale Listing: Complete verbatim code (#L406-L484) → Appendix: Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3).

8.4. Autonomous Probe Wiring (cog.log → cog.debug)

Rather than silently swallowing ring overflows or missing telemetry files, log.c registers live probes inside Sub-Assembly 4.1:

Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1) (engine/src/log/log.c#L1934-L1977 · *Mode:* Subassembly)

```
1934 /*
1935 ------------------------------------------------------------------------------
1936 Sub-Assembly 4.1: Contract, Input & Cog Verification Cogs
1937 Mission: Verify received parameters, configuration contracts, and data provenance;
1938 verify that declared subsystem cogs run properly as standalone units.
1939 ------------------------------------------------------------------------------
1940 */
1941
1942 /**
1943  * @brief Returns whether the active logging subsystem was configured from data sources.
1944  *
1945  * ### For Game Designers & Non-Programmers:
1946  * Checks if settings were loaded from actual JSON files rather than emergency defaults.
1947  *
1948  * ### For Systems Programmers & Engine Architects:
1949  * Queries provenance boolean s_log.config.loaded_from_data.
1950  *
1951  * @return true if loaded from JSON data; false if using fallback code page.
1952  */
1953 bool log_is_data_driven(void)
1954 {
1955     return s_log.config.loaded_from_data;
1956 }
1957
1958 /**
1959  * @brief Queries whether the logging engine is currently operating in Safe Mode.
1960  *
1961  * ### For Game Designers & Non-Programmers:
1962  * Returns true if the engine booted using compiled-in ROM backups or emergency
1963  * defaults instead of live external JSON directories on disk.
1964  *
1965  * ### For Systems Programmers & Engine Architects:
1966  * Checks initialization state alongside s_log.safe_mode (!loaded_from_data).
1967  *
1968  * @return true if active and running in Safe Mode; false otherwise.
1969  */
1970 bool log_is_safe_mode(void)
1971 {
1972     return s_log.initialized && s_log.safe_mode;
1973 }
1974
1975 /**
1976  * @brief Returns the current state of the logging data pipeline tripwire.
1977  *
```

Shows how cog.log verifies its own static ring limits and binds probes into cog.debug.

Wholesale Listing: Complete verbatim code (#L1934-L2028) → Appendix: Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1).

8.5. Self-Audit & Quick Verification

```
ctest --test-dir build -R "_log_" --output-on-failure
./build/tenebris-lux --audit
```

1. *Why split* `log_types.h` *out of* `log.h`*?*

   Both primary `log.c` and quarantined `log_fallback.c` need `log_entry_t` and `log_config_t`. Keeping types in a stateless leaf header prevents circular includes while letting `log_fallback` boot cleanly even if the full logging layer is down.

2. *What happens if telemetry files on disk are missing or corrupt?*

   `log_fallback_load_rom_profile()` hydrates the matching profile directly from the compiled `.rodata` ROM vault, preserves `loaded_from_data = false`, and reports the downgrade cleanly through debugger probes.

8.6. Companion Deep-Dive Chapters

1. Open Paired `log.h` + `log.c` Walkthrough for the ring buffer and disk rotation internals.
2. Read `log_types.h` Leaf Vocabulary and Paired `log_fallback.h` + `log_fallback.c` BIOS.
3. Continue to State Debugger Overview or I/O Pipeline Overview.

Chapter 9. Log Module Pair: Flight Logger Router & Severity Ring (log.h + log.c)

Source File: docs/code/log/01_log_router_and_ring.adoc

Attribute	Verified Contract (cog.log)
Public Contract (.h)	engine/include/log/log.h (954 lines) — operational API, convenience log macros, audit & inspection gates.
Runtime Engine (.c)	engine/src/log/log.c (2,833 lines) — circular ring storage, JSON config loader, file rotation, and debug probe wiring.
Chassis Footprint	.data ~24.7 KiB (s_log + tables) + .bss 16.4 KiB (s_log_vfs_scratch) + .rodata ~8.6 KiB; stack framing ⇐ 1 KiB.
Inputs & Fallbacks	Reads engine/config/diagnostics/telemetry/<profile>.json; degrades cleanly to rom://telemetry via cog.log.fallback.

log.h defines the public switchboard used by every engine subsystem and gameplay turn loop, while log.c houses the circular recording tape (256 recent messages), optional rolling .log file sinks, and live health probes. Designers control verbosity with JSON telemetry profiles without touching C code; programmers get O(1) append latency and reproducible memory bounds.

9.1. Configuration Loading & Data-Driven Profiles (Sub-Assembly 3.1)

When log_init(const log_config_t *config) runs—or when log_load_config_profile() loads "default", "development", "production", "quiet", or "combat_focus"—log.c resolves configuration data through path_finder / master_loader before falling back to compiled ROMs if disk assets are absent or invalid:

- log_load_config_from_file() parses a specific profile payload into log_config_t.
- log_load_config_from_manifest() reads engine/config/diagnostics/telemetry/manifest.json to discover the active startup profile.
- log_load_config_profile() looks up a named preset, tagging loaded_from_data = true only when hydrated from disk JSON (never pretending ROM data came from disk).

9.2. Ring Recording, Rotation & String Parsing (Sub-Assemblies 3.2–3.5)

Every log emission flows through log_write_entry() into fixed stack buffers (LOG_MAX_MESSAGE_LEN bounded, zero malloc):

Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3) (engine/include/log/log.h#L406-L445 · *Mode:* Subassembly)

```
406 /*
407 ------------------------------------------------------------------------------
408 Sub-Assembly 3.3: Formatted Entry Dispatch Cogs
409 Mission: Format, serialize, and emit structured flight recorder entries.
410 @triggers
411   - Tripwires      : LOG_TRIPWIRE_SETTING_INVALID / BOUNDS_VIOLATION -> probe "log.write_telemetry"
412   - Block 4 Audits : log_validate_telemetry_integrity(), log_preflight()
413   - Layer 4 Tests  :
414       * tests/unit/log/test_log_unit.c
415         [Section 3: INVARIANTS & CONSTRAINTS / test_log_unit_assertions_and_guarantees]
416         [Section 4: RING BUFFER FIFO OVERWRITE & FILE ROTATION / test_log_unit_emergence]
```

```
417       * tests/functional/test_log_functional.c
418         [Section 3: INVARIANTS & SATURATION / test_log_functional_assertions_and_guarantees]
419       * tests/integrated/test_log_integrated.c
420         [Section 4: CATEGORICAL DISK ROUTING & HUD RING BUFFER / test_log_integrated_emergence]
421       * tests/system/test_log_system.c
422         [Section 1: NOMINAL HIGH-THROUGHPUT BURST / test_log_system_always_pass]
423         [Section 3: FIXED-WIDTH REGEX DETERMINISM / test_log_system_assertions_and_guarantees]
424 ------------------------------------------------------------------------------
425 */
426
427 /**
428  * @brief Emits a structured log entry into the ring buffer, file, and console.
429  *
430  * ### For Game Designers & Non-Programmers:
431  * Writes an event to the game's flight recorder journal, capturing the character involved,
432  * their current moral balance percentage, and what action took place.
433  *
434  * ### For Systems Programmers & Engine Architects:
435  * Formats a fixed-column log record with microsecond monotonic timestamp and ISO 8601 UTC time,
436  * pushes it to the static circular ring buffer, and writes to disk archives with zero heap allocation.
437  *
438  * @param[in] severity Graded severity level.
439  * @param[in] category Target functional category.
440  * @param[in] entity   Entity/Actor label (e.g. "Player", "Companion", "System").
441  * @param[in] grade    Normalized grade [-1.0f .. +1.0f] (e.g. alignment sway or vitals).
442  * @param[in] file     Source file name (__FILE__).
443  * @param[in] line     Source line number (__LINE__).
444  * @param[in] func     Source function name (__func__).
445  * @param[in] fmt      Printf-style format string.
```

Public categorized log macros and fixed-stack formatting interface for engine and game channels.

Wholesale Listing: Complete verbatim code (#L406-L484) → Appendix: Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3).

Cog Group	Public API & Behavior
File Sink & Rotation (3.2)	`log_is_active()` reports engine readiness; `log_rotate_if_needed()` rolls active log files (`rotate_archive_stream` internal helper) before exceeding configured file-size thresholds.
Ring Retrieval (3.4)	`log_get_total_entries()` returns lifetime sequence count; `log_get_recent_entries()` and `log_get_top_entries()` copy bounded snapshots out of the ring without exposing internal static pointers.
Enum Parsers (3.5)	`log_severity_from_string()` and `log_category_from_string()` map human-readable JSON tokens (`"INFO"`, `"WARN"`, `"COMBAT"`, `"IO"`) back to strongly typed enums.

Separation Between Machine Noise and Story Events

Engine infrastructure channels (`CORE`, `IO`, `LOG`, `DEBUG`, `VIEWPORT`) can run at terse `WARN`/`INFO` levels while gameplay testers turn up `COMBAT`, `HERO`, or `RELICS` via `combat_focus.json`—keeping diagnostic logs focused and readable.

9.3. Validation Guards, Probes & Pre-Flight Audits (Sub-Assemblies 4.1–5.2)

Inside Block 4 of `log.c`, `log_register_debug_probes()` registers telemetry invariants directly with `cog.debug`, while `log_preflight()` and `log_audit()` validate manifest contracts, rotation limits, and ring consistency:

Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1) (engine/src/log/log.c#L1934-L1977 · Mode: Subassembly)

```
1934 /*
1935 ------------------------------------------------------------------------------
1936 Sub-Assembly 4.1: Contract, Input & Cog Verification Cogs
1937 Mission: Verify received parameters, configuration contracts, and data provenance;
1938 verify that declared subsystem cogs run properly as standalone units.
1939 ------------------------------------------------------------------------------
1940 */
1941
1942 /**
1943  * @brief Returns whether the active logging subsystem was configured from data sources.
1944  *
1945  * ### For Game Designers & Non-Programmers:
1946  * Checks if settings were loaded from actual JSON files rather than emergency defaults.
1947  *
1948  * ### For Systems Programmers & Engine Architects:
1949  * Queries provenance boolean s_log.config.loaded_from_data.
1950  *
1951  * @return true if loaded from JSON data; false if using fallback code page.
1952  */
1953 bool log_is_data_driven(void)
1954 {
1955     return s_log.config.loaded_from_data;
1956 }
1957
1958 /**
1959  * @brief Queries whether the logging engine is currently operating in Safe Mode.
1960  *
1961  * ### For Game Designers & Non-Programmers:
1962  * Returns true if the engine booted using compiled-in ROM backups or emergency
1963  * defaults instead of live external JSON directories on disk.
1964  *
1965  * ### For Systems Programmers & Engine Architects:
1966  * Checks initialization state alongside s_log.safe_mode (!loaded_from_data).
1967  *
1968  * @return true if active and running in Safe Mode; false otherwise.
1969  */
1970 bool log_is_safe_mode(void)
1971 {
1972     return s_log.initialized && s_log.safe_mode;
1973 }
1974
1975 /**
1976  * @brief Returns the current state of the logging data pipeline tripwire.
1977  *
```

Shows how cog.log verifies its own static ring limits and binds probes into cog.debug.

Wholesale Listing: Complete verbatim code (#L1934-L2028) → Appendix: Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1).

- **Runtime Mode Predicates (4.1)**: `log_is_data_driven()`, `log_is_safe_mode()`, and `log_reset_tripwire()`.
- **Contract Validators (4.2)**: `log_validate_manifest_contract()`, `log_validate_setting_contract()`, `log_validate_rotation_bounds()`, and `log_validate_telemetry_integrity()`, aggregated by `log_preflight()` / `log_audit(bool verbose)`.
- **Lifecycle & Cards (5.1–5.2)**: `log_execute_inspect()` populates caller-owned `log_stats_t`; `log_execute_dump_card()` formats a printable status card; `log_init()` drains any early `log_fallback` BIOS entries into the main ring; `log_shutdown()` flushes and closes streams cleanly.

9.4. Paired Neighbors

- Vocabulary types consumed here: `log_types.h`

- Early-boot ROM vault & replay source: `log_fallback.h` + `log_fallback.c`
- Automated verification ladder: Logging Testing Suite

Chapter 10. Log Leaf Contract: Canonical Types & Ring Bounds (log_types.h)

Source File: docs/code/log/02_log_types_and_contracts.adoc

Attribute	Verified Contract (cog.log.types)
Header File	engine/include/log/log_types.h (421 lines) — header-only pure vocabulary (archetype: pure_types | stateless).
Runtime Source	None intentionally. Holds zero mutable global storage (0 B .bss, 0 B .data).
Physical Bounds	LOG_MAX_ENTITY_LEN = 16, LOG_MAX_MESSAGE_LEN = 256, BIOS ring capacity 64.
Consumers	Included downward by both log_fallback.h and log.h plus extern "C" C++20/17 tooling.

If log_fallback.h had to include log.h just to learn what a log_entry_t looks like, a Safe-Mode fallback would depend on the primary logging module it is meant to rescue! Extracting shared enums, byte limits, and plain-old-data (POD) structs into log_types.h gives both modules an identical binary vocabulary with zero dependency tangle.

10.1. Physical Bounds & Telemetry Value Layout (Sub-Assemblies 2.1–2.3)

In Block 2 (Sub-Assembly 2.1), log_types.h locks string sizes (LOG_MAX_ENTITY_LEN, LOG_MAX_MESSAGE_LEN) and uses _Static_assert / static_assert parity via core/manifest.h so silent struct growth cannot overrun stack budgets. In Sub-Assembly 2.3, it defines the exact records carried across ring buffers and disk logs:

Flight Log Leaf Vocabulary & Entry Layout (log_types.h Sub-Assembly 2.3) (engine/include/log/log_types.h#L153-L194 · Mode: Subassembly)

```
153 /*
154 ------------------------------------------------------------------------------
155 Sub-Assembly 2.3: Shared Structural Entity & Telemetry Value Definitions
156 ------------------------------------------------------------------------------
157 Canonical non-anonymous enums and POD structures shared across operational and
158 quarantined BIOS telemetry cogs.
159 ------------------------------------------------------------------------------
160 */
161
162 /**
163  * @enum log_severity
164  * @brief Graded severity taxonomy for engine diagnostics and telemetry events.
165  *
166  * Ordered from highest severity (FATAL) down to finest detail (TRACE).
167  *
168  * ### For Game Designers & Non-Programmers:
169  * Defines how critical an event is:
170  *   - FATAL/CRITICAL: Showstoppers that threaten gameplay continuity.
171  *   - ERROR/WARNING: Problems handled gracefully with automatic fallbacks.
172  *   - INFO: Normal story, quest, and milestone announcements.
173  *   - DEBUG/TRACE: Granular technical values for combat balance and timing.
174  *
175  * ### For Systems Programmers & Engine Architects:
176  * Strict integer ordering enables threshold filtering and weight-table lookup.
177  */
178 typedef enum log_severity {
179     LOG_SEV_FATAL = 0,    /**< Unrecoverable failure or fatal state corruption. */
180     LOG_SEV_CRITICAL,     /**< Severe failure threatening gameplay loop. */
181     LOG_SEV_ERROR,        /**< Discrete operation failure with fallback. */
182     LOG_SEV_WARNING,      /**< Suspicious condition or degraded state. */
183     LOG_SEV_INFO,         /**< Major milestone or narrative state shift. */
184     LOG_SEV_DEBUG,        /**< Granular spatial, coordinate, or stance telemetry. */
185     LOG_SEV_TRACE,        /**< High-frequency per-tick calculation telemetry. */
186     LOG_SEV_COUNT         /**< Enumerator bound sentinel (not a valid severity). */
```

```
} log_severity_t;

/**
 * @enum log_category
 * @brief Functional categorization of logging events for focused channel filtering.
 *
 * ### For Game Designers & Non-Programmers:
 * Divides events into distinct thematic channels (`CHARACTER`, `MESSAGE`,
```

Zero-.bss leaf types shared cleanly by cog.log and quarantined cog.log.fallback.

Wholesale Listing: Complete verbatim code (#L153-L303) → Appendix: Flight Log Leaf Vocabulary & Entry Layout (log_types.h Sub-Assembly 2.3).

Core Type / Enum	Architectural Responsibility
log_severity_t	Ordered verbosity levels (TRACE, DEBUG, INFO, WARN, ERROR, FATAL) shared consistently by JSON filters and C macros.
log_category_t	Channel taxonomy separating Core/IO/Viewport machine diagnostics from Game Simulation channels (COMBAT, HERO, RELICS, WORLD).
log_entry_t	Fixed-width immutable snapshot of a single event: sequence number, timestamp, category, severity, source line/file tag, and formatted payload.
log_config_t & log_stats_t	Caller-owned runtime policy settings (including provenance flag loaded_from_data) and telemetry counter snapshots returned by log_execute_inspect().

10.2. Stateless Boundary Predicates (Sub-Assembly 2.4)

Because range checking a log_severity_t enum or validating whether a config string length fits within stack limits needs zero global state, log_types.h provides lightweight static inline predicates directly in Sub-Assembly 2.4, compiled into caller call sites cleanly under both C23/C17/C11 and C20/C17.

10.3. Compile-Time & Validator Checks

```
cargo run --manifest-path tools/Cargo.toml --bin tl_tools -- validate headers
```

1. *Why does* log_types.h *disallow mutable global variables?*

   A pure_types leaf header must act like a shared dictionary, never an active machine. Any global ring or state counter belongs strictly inside log.c or log_fallback.c.

10.4. Consumers of log_types.h

- Paired log.h + log.c Router & Ring Engine
- Paired log_fallback.h + log_fallback.c BIOS

Chapter 11. Log Safe-Mode BIOS: Zero-Disk Fallback & ROM Vault (log_fallback.h + log_fallback.c)

Source File: docs/code/log/03_log_fallback_bios.adoc

Attribute	Verified Contract (cog.log.fallback)
Fallback Contract (.h)	engine/include/log/log_fallback.h (811 lines) — Tier-0 BIOS boot, ROM asset lookup, replay drain, and emergency card dump APIs.
BIOS Implementation (.c)	engine/src/log/log_fallback.c (1,865 lines) — self-contained Safe-Mode recorder (s_fb) and generated telemetry ROM bridge.
Chassis Footprint	.bss 12.3 KiB (s_fb ring + state) + .rodata 57.9 KiB (10 embedded telemetry JSON ROMs) + stack ⇐ 1 KiB.
Quarantined Boundary	archetype: quarantined_fallback | stateful; consumes log_telemetry_rom.h & cog.log.types.

In real game development, crashes most often strike **during** startup—when asset folders move, a user edits a JSON setting incorrectly, or disk permissions turn read-only. If the logger itself refuses to boot without healthy JSON files on disk, developers go completely blind right when telemetry matters most. log_fallback.[ch] solves that by booting **before** io/ even initializes.

11.1. Pre-VFS Boot & Chronological Replay (Sub-Assembly 3.3 & 5.2)

log_fallback_boot() arms a lightweight 64-entry ring buffer right at process start. Any early warning from io_fallback or manifest lands cleanly in log_fallback_record() (and optionally an emergency fallback stream if disk writing works). When disk storage fails or goes read-only, disk_fault latches without crashing the game. Later, when primary log_init() comes online, log_fallback_drain_entries() transfers every buffered early entry cleanly into the full 256-entry ring:

Tier-0 Flight Log BIOS Boot & Early-Entry Replay Drain (engine/src/log/log_fallback.c#L1701-L1740 · Mode: Group)

```
1701 /**
1702  * @brief Arms the Tier-0 BIOS Safe-Mode logger before CLI flags or disk VFS boot.
1703  *
1704  * ### For Game Designers & Non-Programmers:
1705  * Turns on emergency logging immediately when the program starts so early boot
1706  * bugs or missing folders are never lost, while leaving repo folders clean.
1707  *
1708  * ### For Systems Programmers & Engine Architects:
1709  * When opt_cfg is NULL, arms purely in .bss ring + stderr alert mode with zero
1710  * filesystem I/O. When opt_cfg is supplied, opens bounded Safe-Mode disk output
1711  * (applying build/logs sandbox protection) and degrades gracefully to RAM Ring +
1712  * stderr if mkdir/fopen fails on read-only or full media.
1713  *
1714  * @param[in] opt_cfg Optional Safe-Mode config (NULL for clean RAM BIOS boot).
1715  * @return true on clean BIOS Safe-Mode arming (survives disk failures).
1716  */
1717 bool log_fallback_boot(const log_config_t *opt_cfg)
1718 {
1719     if (s_fb.file != NULL) {
1720         fflush(s_fb.file);
1721         fclose(s_fb.file);
1722         s_fb.file = NULL;
1723     }
1724
1725     s_fb.disk_fault = false;
1726     s_fb.current_bytes = 0;
```

```
1727
1728    if (opt_cfg == NULL) {
1729        s_fb.config = s_log_floppy_template;
1730        s_fb.config.loaded_from_data = false;
1731        if (io_fallback_is_disk_read_only()) {
1732            s_fb.disk_fault = true;
1733        }
1734        s_fb.active_sink = LOG_FALLBACK_SINK_RING_AND_STDERR;
1735        s_fb.booted = true;
1736        return true;
1737    }
1738
1739    s_fb.config = *opt_cfg;
1740    s_fb.config.loaded_from_data = false;
```

Arms the 64-entry pre-VFS BIOS ring before disk mount and replays captured records once primary cog.log boots.

Wholesale Listing: Complete verbatim code (#L1256-L1801) → Appendix: Tier-0 Flight Log BIOS Boot & Early-Entry Replay Drain.

11.2. The 10-Asset Compiled Telemetry ROM Vault (Sub-Assemblies 3.1–3.2)

At CMake build time, cmake/GenerateConfigRom.cmake compiles every JSON file inside engine/config/diagnostics/telemetry/ into read-only C byte tables inside log_telemetry_rom.h:

- log_fallback_get_rom_info() reports embedded asset count (10) and total .rodata byte footprint (57.9 KiB runtime measured chassis).
- log_fallback_get_rom_asset() returns non-owning read-only pointers directly to .rodata bytes with zero copying or heap allocation.
- log_fallback_load_rom_profile() and log_fallback_parse_config_span() share one boundary-safe JSON key scanner across fallback and normal loading, guaranteeing that Safe Mode boots with genuine structured settings while flagging loaded_from_data = false.

11.3. BIOS Self-Audit & Emergency Diagnostic Cards

- log_fallback_validate_boot_contract() and log_fallback_audit() inspect ROM vault alignment, BIOS ring bounds, and sink modes non-destructively.
- log_fallback_get_stats() copies log_fallback_stats_t (booted, disk_fault, rom_hydrated, total_recorded, total_replayed, ring_count, ROM metrics).
- log_fallback_execute_inspect() and log_fallback_execute_dump_card() let test suites and CLI recovery tools print human-readable Safe-Mode status even in zero-disk environments.

11.4. Next Steps in log/

- Review the tests verifying BIOS capture and drain: Logging Testing & Probe Verification
- Inspect the shared vocabulary header: log_types.h

Chapter 12. Log Verification Suite: 4-Tier CTest Harness & Probes

Source File: docs/code/log/04_log_testing_and_probes.adoc

Cornerstone validates cog.log and cog.log.fallback from fine isolated unit checks outward to crude multi-module system boot runs. Every test executable follows five consistent bands (always_pass, never_fail, assertions_and_guarantees, emergence, and preflight/audit) anchored by tests/common/cornerstone_test.h.

Tier	CTest Target	Source File	Primary Verification Focus
Unit	unit_log_engine_test	tests/unit/log/test_log_unit.c	Ring rollover, channel masks, severity filtering, disk rotation bounds, and log_preflight() / log_audit().
Unit (BIOS)	unit_log_fallback_test	tests/unit/log/test_log_fallback_unit.c	log_fallback_boot(), 10-asset ROM lookups, read-only disk-fault latch, and early-entry drain behavior.
Functional	functional_log_test	tests/functional/test_log_functional.c	End-to-end logging workflows using public headers cleanly as an external caller.
Integrated	integrated_log_test	tests/integrated/test_log_integrated.c	Cross-module coordination: log + log_fallback drain + debug probe registration + io/ JSON telemetry loading.
System	system_log_test	tests/system/test_log_system.c	Full system boot, corrupt/missing disk fallback to Safe-Mode telemetry ROMs, and recovery verification.

12.1. How Each Test Executable Progresses (Crude-to-Fine Ladder)

Rather than mixing trivial sanity checks with catastrophic failure injection inside one disorganized file, every log test binary separates concerns cleanly into five progressive stages:

1. always_pass **(Baseline Contract)**: Confirms normal initialization (log_init, log_fallback_boot), entry dispatch, and expected clean stats under ideal inputs.
2. never_fail **(Hostile Boundary Defense)**: Passes NULL pointers, zero-length buffers, oversized format strings, and invalid severity tokens to confirm graceful false/safe returns with zero crashes.
3. assertions_and_guarantees **(Mathematical & Static Invariants)**: Verifies ring sequence monotonicity, bounded truncation guarantees (LOG_MAX_MESSAGE_LEN), and exact loaded_from_data provenance tracking.
4. emergence **(Multi-Component System Behavior)**: Tests interactions that cannot exist in single functions—such as early BIOS capture draining chronologically into log_init() and Safe-Mode downgrade under corrupt telemetry manifests.
5. preflight / post_audit **(Machine Self-Audit)**: Executes log_preflight(), log_audit(), and log_fallback_audit() automatically inside the verification loop.

12.2. Live Emergence Verification (test_log_system_emergence)

Below is an excerpt from the Layer 4 system suite demonstrating end-to-end telemetry verification:

Flight Log System Emergence Verification (test_log_system_emergence) (tests/system/test_log_system.c#L283-L320 · Mode: Function)

```
283 /**
284  * @brief Executes 10,000-entry chaos load, floppy BIOS endurance burst, and probe checks.
285  */
286 static void test_log_system_emergence(void)
287 {
288     printf("  [PASS-4] Running Section 4: Emergence (10,000-Entry Chaos, Floppy BIOS & Probes)...\n");
289
290     TEST_ASSERT(debug_init(NULL), "Debugger online to monitor log system endurance");
291
292     log_config_t cfg = log_get_default_config();
293     cfg.echo_to_console = false;
294     cfg.min_severity = LOG_SEV_TRACE;
295     cfg.save_by_category = true;
296     cfg.max_file_size_bytes = 16384U;
297     cfg.max_rotated_files = 10U;
298     str_safe_copy(cfg.log_dir, "build/logs/system_chaos", sizeof(cfg.log_dir));
299     str_safe_copy(cfg.base_filename, "chaos.log", sizeof(cfg.base_filename));
300
301     TEST_ASSERT(log_init(&cfg), "Initialized for chaos stress");
302
303     for (int i = 1; i <= TEST_LOG_SYS_CHAOS_COUNT; i++) {
304         log_severity_t sev = (log_severity_t)(i % LOG_SEV_COUNT);
305         log_category_t cat = (log_category_t)(i % LOG_CAT_COUNT);
306         float grade = ((float)(i % 200) - 100.0f) / 100.0f;
307
308         LOG_RECORD(sev, cat, "ChaosAgent", grade, "High-stress event step #%06d", i);
309
310         if (i % 2000 == 0) {
311             log_entry_t sample[10];
312             size_t fetched = log_get_top_entries(sample, 10U);
313             TEST_ASSERT_INT_EQ((int)fetched, 10,
314                                "Ring buffer top entries query succeeds during live write");
315         }
316     }
317
318     log_stats_t final_stats;
319     TEST_ASSERT(log_execute_inspect(&final_stats), "Inspected final chaos stats");
320     TEST_ASSERT(final_stats.total_entries >= (uint64_t)TEST_LOG_SYS_CHAOS_COUNT,
```

Exercises full boot, rotation, Safe-Mode telemetry profile fallback, and recovery across Layer 4.

Wholesale Listing: Complete verbatim code (#L283-L385) → Appendix: Flight Log System Emergence Verification (test_log_system_emergence).

12.3. Executing the Suite & Forensic Failure Cards

Because every executable includes tests/common/cornerstone_test.h (test.common.harness), any failing assertion queries debug / debug_fallback tripwire cards before exiting so testers see which contract tripped and why:

```
ctest --test-dir build -L log --output-on-failure
ctest --test-dir build -R "^(unit_log_|functional_log_|integrated_log_|system_log_)" --output-on-failure
```

12.4. Continue Reading

- Inspect the files verified by these suites: log.h + log.c and log_fallback.h + log_fallback.c.
- Advance to the next root subsystem: Layer 0 State Debugger Overview.

Chapter 13. Layer 0 State Debugger & Diagnostic Engine Overview

Source File: `docs/code/debug_system.adoc`

If `log/` is the black-box voice recorder of Cornerstone, **`debug/`** (`engine/include/debug/ & engine/src/debug/`) is the cockpit telemetry computer and circuit-breaker panel. Instead of chasing ghost bugs by peppering code with disposable print statements, Cornerstone captures deterministic turn snapshots, hashes state fields cleanly, hosts up to 32 active validation probes, and halts cleanly on `DEBUG_STEP_AWAIT` whenever an invariant breaks.



> **Navigate the `debug/` folder**: Pair this overview with docs/code/debug/ for detailed guides on `debug.h` + `debug.c`, `debug_types.h`, `debug_fallback.h` + `debug_fallback.c`, and all 5 `debug` CTest suites.

13.1. Architectural Philosophy: Observable State & Tripwires

Why build an engine-level debugger into Layer 0 before high-level gameplay even finishes? Because deterministic tactical RPGs fail silently when moral sway drifts, an equipment bonus overflows, or a corrupted config quietly loads half a table. Cornerstone turns hidden failures into explicit diagnostics:

- **Standalone Integer/Char Snapshot Contract (`debug_types.h`)**: Layer 1 gameplay projects its live simulation fields into neutral integers inside `debug_state_fields_t`—preserving downward dependency flow with zero `void *` type erasure.
- **Zero-Heap Snapshot Ring (`DEBUG_MAX_SNAPSHOTS = 64`)**: Captures state frames and 32-bit FNV-1a checksums, supports snapshot locking with O(cap) GCD wrap rotation, and rewinds backward cleanly via `debug_rewind_to_snapshot()`.
- **Autonomous Probe Matrix & `DEBUG_STEP_AWAIT`**: Subsystems register up to 32 named validation probes (`debug_register_validation_probe()`). When a fault hits, `debug_trigger_probe()` and `debug_trap_tripwire()` drop cleanly to a Safe-Mode fallback and hold `DEBUG_STEP_AWAIT` until resolved or cleared.
- **Tier-0 BIOS Resilience (`debug_fallback.[ch]`)**: Backed by compiled `.rodata` ROMs (`debug_diagnostics_rom.h`) over `engine/config/diagnostics/{debugger,taxonomy}/`, error codes and inspection cards stay available even with zero disk files.

13.2. Verified Rev 5 File & Documentation Map

Companion Doc	Engine Files Covered	Role & Chassis Highlights
01: Debugger Engine & Probes	`engine/include/debug/debug.h` + `engine/src/debug/debug.c` (`cog.debug`)	Core 64-slot snapshot ring (`s_debug` + `s_audit_backup`, `70.5 KiB .bss`), FNV-1a hashing, breakpoints, probe registry, rewind, and inspect/card dumps.
02: Debugger Leaf Vocabulary	`engine/include/debug/debug_types.h` (`cog.debug.types`)	Pure POD enums, state field packets, snapshots, breakpoints, tripwires, and config records (`0 B .bss`, zero `void *` erasure).

Companion Doc	Engine Files Covered	Role & Chassis Highlights
03: Debugger Tier-0 BIOS	engine/include/debug/debug_fallback.h + engine/src/debug/debug_fallback.c (cog.debug.fallback)	Standalone BIOS templates, emergency fault recorder, .rodata ROM vault (76,136 B) spanning debugger & taxonomy configs (256 B .bss, 464 B .data).
04: Debugger Testing Suite	5 CTest targets across tests/{unit,functional,integrated,system}/	Crude-to-fine verification of snapshot rollover, GCD wrap preservation, probe firing, DEBUG_STEP_AWAIT, and fallback recovery.

13.3. Deterministic Capture & Time-Travel Rewind

Deterministic State Capture & Time-Travel Rewind (debug.c) (engine/src/debug/debug.c#L1503-L1540 · Mode: Group)

```
1503 /**
1504  * @brief Rewinds state fields back to snapshot `index` after cryptographic checksum check.
1505  *
1506  * ### For Game Designers & Non-Programmers:
1507  * Restores actor stats from an earlier saved frame, pausing safely in AWAIT if memory
1508  * corruption altered the snapshot's stored fingerprint.
1509  *
1510  * ### For Systems Programmers & Engine Architects:
1511  * Fetches snapshot via `debug_get_snapshot()`, verifies `debug_compute_state_hash(&snap.state)`
1512  * matches `snap.state_hash` (engaging `DEBUG_TRIPWIRE_HASH_MISMATCH` + Safe-Mode AWAIT +
1513  * `LOG_ERROR` on mismatch), and emits `LOG_INFO` on verified restore.
1514  *
1515  * @triggers
1516  * - Layer 4 Tests : `tests/unit/debug/test_debug_unit.c:Section1`
1517  *
1518  * @param[in]  index      Chronological target snapshot offset.
1519  * @param[out] out_fields Restored simulation state fields on verified success.
1520  * @return true if checksum passed and `out_fields` populated; false on error or mismatch.
1521  */
1522 bool debug_rewind_to_snapshot(size_t index, debug_state_fields_t *out_fields)
1523 {
1524     /* Step 1: Validate preconditions and fetch chronological snapshot copy */
1525     if (!s_debug_active || out_fields == NULL) {
1526         return false;
1527     }
1528
1529     debug_snapshot_t snap;
1530     if (!debug_get_snapshot(index, &snap)) {
1531         return false;
1532     }
1533
1534     /* Step 2: Verify determinism checksum against tampering before restoring */
1535     uint32_t verify_hash = debug_compute_state_hash(&snap.state);
1536     if (verify_hash != snap.state_hash) {
1537         s_debug.tripwire_status = DEBUG_TRIPWIRE_HASH_MISMATCH;
1538         snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
1539                  "Rewind checksum mismatch on snap #%lu (0x%08X != 0x%08X)",
1540                  (unsigned long)snap.snapshot_id, verify_hash, snap.state_hash);
```

Captures FNV-1a hashed simulation states in a 64-slot ring and restores bounded prior frames cleanly.

Wholesale Listing: Complete verbatim code (#L1264-L1567) → Appendix: Deterministic State Capture & Time-Travel Rewind (debug.c).

13.4. Autonomous Validation Probes & DEBUG_STEP_AWAIT

Autonomous Validation Probes & Tripwire Trap Chain (engine/src/debug/debug.c#L2367-L2411 · Mode: Group)

```
2367 /**
```

```
2368  * @brief Legacy compatibility helper that latches a state-corruption tripwire trap.
2369  *
2370  * @deprecated Prefer `debug_trigger_probe()` or `DEBUG_TRIGGER_PROBE()` so every trap
2371  *             carries a structured probe ID, tier classification, and forensic report.
2372  *
2373  *   ### For Game Designers & Non-Programmers:
2374  *   Preserved safety brake for older modules that report state errors without
2375  *   naming a specific modern probe identifier.
2376  *
2377  *   ### For Systems Programmers & Engine Architects:
2378  *   Latches `DEBUG_TRIPWIRE_STATE_CORRUPT`, increments `traps_triggered`, notifies
2379  *   BIOS via `debug_fallback_record_fault()`, performs guarded snapshot lock and
2380  *   `DEBUG_STEP_AWAIT` entry when active and non-audit, and emits `LOG_WARN`.
2381  *
2382  * @param[in] subsystem Subsystem tag reporting the fault, or NULL for `"unknown"`.
2383  * @param[in] condition Description of the violated condition, or NULL for `"anomaly"`.
2384  * @param[in] file      Originating source file (`__FILE__`).
2385  * @param[in] line      Originating source line (`__LINE__`).
2386  * @triggers Latches tripwire state, records BIOS fault, and warns via guarded log output.
2387  */
2388 void debug_trap_tripwire(const char *subsystem, const char *condition,
2389                          const char *file, int line)
2390 {
2391     /* Step 1: Latch classic STATE_CORRUPT code and forensic detail string verbatim */
2392     s_debug.tripwire_status = DEBUG_TRIPWIRE_STATE_CORRUPT;
2393     s_debug.traps_triggered++;
2394 
2395     snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
2396              "[%s] %s (%s:%d)",
2397              (subsystem != NULL) ? subsystem : "unknown",
2398              (condition != NULL) ? condition : "anomaly",
2399              (file != NULL) ? file : "unknown", line);
2400 
2401     /* Step 2: Record fault in Tier-0 BIOS and complete via Safe-Mode fallback */
2402     (void)debug_fallback_record_fault(
2403         subsystem,
2404         (condition != NULL) ? condition : "anomaly",
2405         file,
2406         line,
2407         "debug_trap_tripwire"
2408     );
2409 
2410     if (s_debug_active && !s_debug.in_probe && !s_debug.in_audit) {
2411         (void)debug_fallback_finish_with_fallback(
```

Realizes the Probe → Safe-Mode Downgrade → DEBUG_STEP_AWAIT loop inside Layer 0 debug infrastructure.

Wholesale Listing: Complete verbatim code (#L1918-L2438) → Appendix: Autonomous Validation Probes & Tripwire Trap Chain.

13.5. Self-Audit & Quick Verification

```
ctest --test-dir build -R "_debug_" --output-on-failure
./build/tenebris-lux --audit
```

1. *Why keep a separate* `s_audit_backup` *inside* `debug.c`*?*

   Running destructive rewind, tripwire, or wrap drills during `debug_audit()` without clobbering live gameplay telemetry requires isolating self-test state safely in static `.bss` (`35,232 B` live + `35,232 B` backup).

2. *Can* `cog.debug` *detect bugs even if a game UI hasn't rendered a single pixel yet?*

   Yes. Every Layer 0 cog registers health probes (`io.`, `log.`, etc.) during boot, catching config corruption or structural contract drift before Layer 1 simulation starts.

13.6. Companion Deep-Dive Chapters

1. Open Paired `debug.h` + `debug.c` Walkthrough for ring rewind and probe internals.
2. Read `debug_types.h` Leaf Vocabulary and Paired `debug_fallback.h` + `debug_fallback.c` BIOS.
3. Continue onward to Layer 0 Zero-Heap I/O Pipeline Overview.

Chapter 14. Debug Module Pair: Rewind Engine, Watchpoints & Probes (debug.h + debug.c)

Source File: docs/code/debug/01_debug_engine_and_probes.adoc

Attribute	Verified Contract (cog.debug)
Public Contract (.h)	engine/include/debug/debug.h (1,207 lines) — stepping control, snapshot rewind, watchpoints, 32-slot probe registry, tripwires, cards, and audit gates.
Runtime Engine (.c)	engine/src/debug/debug.c (3,221 lines) — FNV-1a snapshot ring, GCD in-place wrap rotation, interactive AWAIT state machine, and audit isolation.
Chassis Footprint	.bss 70.5 KiB (s_debug 35,232 B + s_audit_backup 35,232 B) + .rodata 9.7 KiB + single-slot normalize stack frame.
Inputs & Seams	Reads engine/config/diagnostics/{debugger,taxonomy}/*.json; pairs downward with cog.debug.types, cog.debug.fallback, and cog.log.

debug.h + debug.c turn "something went wrong during frame 412" into inspectable facts: every snapshot records simulation numbers, hashes them with 32-bit FNV-1a, preserves explicitly locked milestones when the circular buffer wraps, and hosts live health probes across the rest of the engine.

14.1. Snapshot Capture, Locking & Time-Travel Rewind

Calling debug_record_snapshot() freezes a copy of debug_state_fields_t alongside a 32-bit FNV-1a checksum and the synchronized log_sequence_id from cog.log. Calling debug_lock_snapshot() protects milestone frames from overwrite; when the ring wraps, debug.c uses O(cap) GCD wrap rotation to evict unlocked slots while preserving chronological order. debug_rewind_to_snapshot() and debug_diff_snapshots() restore or compare any retained frame:

Deterministic State Capture & Time-Travel Rewind (debug.c) (engine/src/debug/debug.c#L1503-L1540 · Mode: Group)

```
1503 /**
1504  * @brief Rewinds state fields back to snapshot `index` after cryptographic checksum check.
1505  *
1506  * ### For Game Designers & Non-Programmers:
1507  * Restores actor stats from an earlier saved frame, pausing safely in AWAIT if memory
1508  * corruption altered the snapshot's stored fingerprint.
1509  *
1510  * ### For Systems Programmers & Engine Architects:
1511  * Fetches snapshot via `debug_get_snapshot()`, verifies `debug_compute_state_hash(&snap.state)`
1512  * matches `snap.state_hash` (engaging `DEBUG_TRIPWIRE_HASH_MISMATCH` + Safe-Mode AWAIT +
1513  * `LOG_ERROR` on mismatch), and emits `LOG_INFO` on verified restore.
1514  *
1515  * @triggers
1516  * - Layer 4 Tests : `tests/unit/debug/test_debug_unit.c:Section1`
1517  *
1518  * @param[in]  index      Chronological target snapshot offset.
1519  * @param[out] out_fields Restored simulation state fields on verified success.
1520  * @return true if checksum passed and `out_fields` populated; false on error or mismatch.
1521  */
1522 bool debug_rewind_to_snapshot(size_t index, debug_state_fields_t *out_fields)
1523 {
1524     /* Step 1: Validate preconditions and fetch chronological snapshot copy */
1525     if (!s_debug_active || out_fields == NULL) {
1526         return false;
1527     }
1528
1529     debug_snapshot_t snap;
1530     if (!debug_get_snapshot(index, &snap)) {
1531         return false;
1532     }
```

```
1533
1534     /* Step 2: Verify determinism checksum against tampering before restoring */
1535     uint32_t verify_hash = debug_compute_state_hash(&snap.state);
1536     if (verify_hash != snap.state_hash) {
1537         s_debug.tripwire_status = DEBUG_TRIPWIRE_HASH_MISMATCH;
1538         snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
1539                  "Rewind checksum mismatch on snap #%lu (0x%08X != 0x%08X)",
1540                  (unsigned long)snap.snapshot_id, verify_hash, snap.state_hash);
```

Captures FNV-1a hashed simulation states in a 64-slot ring and restores bounded prior frames cleanly.

Wholesale Listing: Complete verbatim code (#L1264-L1567) → Appendix: Deterministic State Capture & Time-Travel Rewind (debug.c).

14.2. Stepping Modes, Breakpoints & DEBUG_STEP_AWAIT

Capability	Public Contract in debug.h
Execution Pacing & Await	debug_set_step_mode(), debug_is_awaiting(), debug_can_step(), and debug_resolve_await() pause execution deterministically after a fault or watchpoint hit until resolved.
Simulation Watchpoints	debug_add_breakpoint(), debug_remove_breakpoint(), debug_clear_breakpoints(), and debug_check_breakpoints() watch up to 16 numeric thresholds (hp == 0, stance shift, alignment breach, etc.) in O(n).
Provenance Queries	debug_is_data_driven() and debug_is_safe_mode() distinguish disk JSON profiles from compiled .rodata ROM fallback mode cleanly.

14.3. Validation Probe Matrix & Tripwire Chain

Child modules bind self-checks directly into debug.c□s `32-slot probe array:

Autonomous Validation Probes & Tripwire Trap Chain (engine/src/debug/debug.c#L2367-L2411 · Mode: Group)

```
2367 /**
2368  * @brief Legacy compatibility helper that latches a state-corruption tripwire trap.
2369  *
2370  * @deprecated Prefer `debug_trigger_probe()` or `DEBUG_TRIGGER_PROBE()` so every trap
2371  *             carries a structured probe ID, tier classification, and forensic report.
2372  *
2373  *   ### For Game Designers & Non-Programmers:
2374  *   Preserved safety brake for older modules that report state errors without
2375  *   naming a specific modern probe identifier.
2376  *
2377  *   ### For Systems Programmers & Engine Architects:
2378  *   Latches `DEBUG_TRIPWIRE_STATE_CORRUPT`, increments `traps_triggered`, notifies
2379  *   BIOS via `debug_fallback_record_fault()`, performs guarded snapshot lock and
2380  *   `DEBUG_STEP_AWAIT` entry when active and non-audit, and emits `LOG_WARN`.
2381  *
2382  * @param[in] subsystem Subsystem tag reporting the fault, or NULL for `"unknown"`.
2383  * @param[in] condition Description of the violated condition, or NULL for `"anomaly"`.
2384  * @param[in] file      Originating source file (`__FILE__`).
2385  * @param[in] line      Originating source line (`__LINE__`).
2386  * @triggers Latches tripwire state, records BIOS fault, and warns via guarded log output.
2387  */
2388 void debug_trap_tripwire(const char *subsystem, const char *condition,
2389                          const char *file, int line)
2390 {
2391     /* Step 1: Latch classic STATE_CORRUPT code and forensic detail string verbatim */
2392     s_debug.tripwire_status = DEBUG_TRIPWIRE_STATE_CORRUPT;
2393     s_debug.traps_triggered++;
2394
2395     snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
2396              "[%s] %s (%s:%d)",
```

```
            (subsystem != NULL) ? subsystem : "unknown",
            (condition != NULL) ? condition : "anomaly",
            (file != NULL) ? file : "unknown", line);

    /* Step 2: Record fault in Tier-0 BIOS and complete via Safe-Mode fallback */
    (void)debug_fallback_record_fault(
        subsystem,
        (condition != NULL) ? condition : "anomaly",
        file,
        line,
        "debug_trap_tripwire"
    );

    if (s_debug_active && !s_debug.in_probe && !s_debug.in_audit) {
        (void)debug_fallback_finish_with_fallback(
```

Realizes the Probe → Safe-Mode Downgrade → DEBUG_STEP_AWAIT loop inside Layer 0 debug infrastructure.

Wholesale Listing: Complete verbatim code (#L1918-L2438) → Appendix: Autonomous Validation Probes & Tripwire Trap Chain.

- **Probe Lifecycle**: `debug_register_validation_probe()`, `debug_find_probe()`, `debug_get_probe_count()`, `debug_get_probe_at()`, `debug_evaluate_probe()`, and `debug_clear_validation_probes()`.
- **Tripwire Matrix & Cards**: `debug_check_tripwires()`, `debug_get_tripwire_report()`, `debug_dump_tripwire_report()`, `debug_get_tripwire_status()`, `debug_reset_tripwire()`, `debug_execute_inspect()`, and `debug_execute_dump_card()`.
- **Safe Self-Audit (`debug_preflight` / `debug_audit`)**: Swaps live state cleanly against `s_audit_backup` in `.bss`, runs destructive probe/rewind drills safely, and restores pre-audit telemetry without touching heap memory.

14.4. Paired Neighbors

- Pure leaf vocabulary types: `debug_types.h`
- Tier-0 debugger BIOS & ROM vault: `debug_fallback.h` + `debug_fallback.c`
- Verification test ladder: Debugger Testing & Tripwires

Chapter 15. Debug Leaf Contract: Snapshots, GCD Ring & Tripwires (debug_types.h)

Source File: docs/code/debug/02_debug_types_and_telemetry.adoc

Attribute	Verified Contract (cog.debug.types)
Header File	engine/include/debug/debug_types.h (652 lines) — standalone leaf POD vocabulary (archetype: pure_types | stateless).
Runtime Source	None intentionally (0 B .bss, 0 B .data).
Fixed Capacities	DEBUG_MAX_SNAPSHOTS = 64, 16 breakpoints, 32 validation probes (Sub-Assembly 2.1).
Architectural Fix	Eliminated legacy struct debug_config; forward declarations and unsafe void * pointer erasure.

Before debug_types.h was isolated as a pure leaf vocabulary, debug.h and debug_fallback.h had to dance around forward declarations and void * parameters to avoid circular header includes. By giving both files direct compile-time knowledge of debug_state_fields_t, debug_snapshot_t, and debug_config_t, every mismatch becomes an instant compiler error instead of a hidden runtime bug.

15.1. Structural Ontology (Sub-Assembly 2.3)

debug_types.h groups enums (debug_tripwire_t, debug_tripwire_reaction_t, debug_test_tier_t, debug_step_mode_t, debug_breakpoint_type_t) alongside caller-owned POD structs:

State Debugger Leaf Vocabulary & Snapshot Structs (debug_types.h Sub-Assembly 2.3)
(engine/include/debug/debug_types.h#L147-L190 · Mode: Subassembly)

```
147 /*
148 ---------------------------------------------------------------------------
149 Sub-Assembly 2.3: Shared Structural Ontology & POD Debugger Types
150 ---------------------------------------------------------------------------
151 Canonical non-anonymous enums and structures shared identically between
152 `cog.debug` and quarantined `cog.debug.fallback`.
153 ---------------------------------------------------------------------------
154 */
155
156 /**
157  * @enum debug_tripwire
158  * @brief Latched diagnostic tripwire states for the debugger and validation pipeline.
159  *
160  * ### For Game Designers & Non-Programmers:
161  * Alarm switches indicating whether configuration data was missing or damaged,
162  * an automated safety test fired, or Safe Mode engaged after a fault.
163  *
164  * ### For Systems Programmers & Engine Architects:
165  * Latching error enum preserving `0..4` legacy discriminants and `5..7` active
166  * validation probe, audit breach, and Safe-Mode downgrade discriminants.
167  */
168 typedef enum debug_tripwire {
169     DEBUG_TRIPWIRE_CLEAR = 0,         /**< All tripwires clear; simulation healthy. */
170     DEBUG_TRIPWIRE_MANIFEST_MISSING,  /**< Debugger/subsystem manifest unavailable. */
171     DEBUG_TRIPWIRE_SETTING_MISSING,   /**< Designated JSON setting profile missing. */
172     DEBUG_TRIPWIRE_STATE_CORRUPT,     /**< Simulation state breached domain invariants. */
173     DEBUG_TRIPWIRE_HASH_MISMATCH,     /**< Snapshot checksum diverged from recorded state. */
174     DEBUG_TRIPWIRE_TEST_FAILED,       /**< Registered validation probe / fail-test fired. */
175     DEBUG_TRIPWIRE_AUDIT_BREACH,      /**< Block 4 preflight or audit invariant failed. */
176     DEBUG_TRIPWIRE_FALLBACK_ENGAGED   /**< Downgraded to Safe Mode to complete execution. */
177 } debug_tripwire_t;
178
179 /**
```

```
180  * @enum debug_tripwire_reaction
181  * @brief Recovery action taxonomy ingested from taxonomy/tripwire_matrix.json.
182  *
183  * ### For Game Designers & Non-Programmers:
184  * Selects how the engine reacts when an alarm trips: logging warnings, clamping
185  * bad numbers, dropping into Safe Mode, rewinding time, or halting in AWAIT.
186  *
187  * ### For Systems Programmers & Engine Architects:
188  * Mapped 1:1 to `tripwire_matrix.json` canonical reaction identifier strings.
189  */
190 typedef enum debug_tripwire_reaction {
```

Standalone integer/char POD snapshots eliminating void type erasure between Layer 0 and Layer 1.*

Wholesale Listing: Complete verbatim code (#L147-L453) → Appendix: State Debugger Leaf Vocabulary & Snapshot Structs (debug_types.h Sub-Assembly 2.3).

Type Name	What It Captures Without game/ Coupling
debug_state_fields_t	Pure integer snapshot packet (turn, hp, max_hp, mp, tp, shield_power, sword_edge, stance, moral_sway, location, status counters, foes_vanquished) linking Layer 1 into Layer 0 safely.
debug_snapshot_t	Timestamped ring slot binding snapshot_id, state_hash (FNV-1a), correlated log_sequence_id, and eviction locked flag.
debug_breakpoint_t	Numeric trigger descriptor (type, target_value, is_active, 64-byte label).
debug_config_t	Runtime toggles (break_on_tripwire, break_on_sway_breach, interactive_repl, loaded_from_data provenance marker).

15.2. Static Bounds Verification

Sub-Assembly 2.1 enforces static_assert proofs ensuring snapshot ring and breakpoint capacities remain strictly positive across C23/C17/C11 and C20/C17 compilation modes. Verify header compliance via:

```
cargo run --manifest-path tools/Cargo.toml --bin tl_tools -- validate headers
```

15.3. Consumers of debug_types.h

- Paired debug.h + debug.c Engine
- Paired debug_fallback.h + debug_fallback.c BIOS

Chapter 16. Debug Safe-Mode BIOS: Tripwire Emergency Floor (debug_fallback.h + debug_fallback.c)

Source File: docs/code/debug/03_debug_fallback_bios.adoc

Attribute	Verified Contract (cog.debug.fallback)
Fallback Contract (.h)	engine/include/debug/debug_fallback.h (900 lines) — BIOS boot, JSON span parsers, ROM vault loaders, fault capture, and AWAIT latch.
BIOS Implementation (.c)	engine/src/debug/debug_fallback.c (1,852 lines) — static fault templates, .rodata ROM bridge, and emergency state verification.
Chassis Footprint	.bss 256 B + .data 464 B + .rodata 76,136 B (debug_diagnostics_rom.h vault) + stack ⇐ 1 KiB.
Inputs & Pairing	Embedded ROM mirrors of engine/config/diagnostics/debugger/.json & taxonomy/.json; pairs downward with cog.log.fallback.

When disk diagnostics catalogs vanish or a debugger JSON config gets mangled during rapid iteration, debug_fallback.[ch] keeps every error code, snapshot limit template, and DEBUG_STEP_AWAIT safety latch intact directly out of read-only ELF .rodata.

16.1. Tier-0 BIOS Boot & .rodata Vault (Sub-Assemblies 3.1–3.2)

debug_fallback_boot() initializes Safe-Mode inspection policies before full I/O mounts, loading profiles cleanly out of debug_diagnostics_rom.h:

Tier-0 Debugger BIOS Boot & ROM Profile Hydration (debug_fallback.c) (engine/src/debug/debug_fallback.c#L1715-L1752 · Mode: Group)

```
1715 /**
1716  * @brief Arms Tier-0 Safe-Mode BIOS state and seeds active ROM profile telemetry.
1717  *
1718  * ### For Game Designers & Non-Programmers:
1719  * Turns on emergency fault recording right at startup so any failure before external
1720  * configs finish loading is still captured and recoverable.
1721  *
1722  * ### For Systems Programmers & Engine Architects:
1723  * Copies caller profile name into `s_fb.active_rom_profile`, or hydrates `"default"`
1724  * via `debug_fallback_load_rom_profile()` with monolith fallback, sets `s_fb.booted = true`,
1725  * and logs an isolated `LOG_SEV_INFO` boot record into `cog.log.fallback`.
1726  *
1727  * @triggers
1728  * - Layer 4 Tests : `tests/unit/debug/test_debug_fallback_unit.c:Section4`
1729  *
1730  * @param[in] opt_config  Optional pre-populated `debug_config_t` (or NULL for ROM default).
1731  * @param[in] config_size Must equal `sizeof(debug_config_t)` when `opt_config != NULL`.
1732  * @return true once Tier-0 Safe-Mode debugger BIOS is booted.
1733  */
1734 bool debug_fallback_boot(const debug_config_t *opt_config, size_t config_size)
1735 {
1736     /* Step 1: Resolve active Safe-Mode profile string from caller, ROM, or monolith */
1737     debug_config_t scratch;
1738     if (opt_config != NULL && config_size == sizeof(debug_config_t)) {
1739         str_safe_copy(s_fb.active_rom_profile, opt_config->profile_name,
1740                       sizeof(s_fb.active_rom_profile));
1741     } else if (!debug_fallback_load_rom_profile("default", &scratch, sizeof(scratch))) {
1742         debug_get_monolith_fallback_config_raw(&scratch, sizeof(scratch));
1743         str_safe_copy(s_fb.active_rom_profile, scratch.profile_name,
1744                       sizeof(s_fb.active_rom_profile));
1745         fb_emit_bios_log(
```

```
1746            LOG_SEV_WARNING,
1747            __FILE__,
1748            __LINE__,
1749            __func__,
1750            MODULE_ID,
1751            "ROM default profile unavailable; seeded Safe-Mode BIOS from monolith template."
1752        );
```

Hydrates Safe-Mode debugger policies directly from embedded .rodata ROMs when disk files are unavailable.

Wholesale Listing: Complete verbatim code (#L1008-L1766) → Appendix: Tier-0 Debugger BIOS Boot & ROM Profile Hydration (debug_fallback.c).

ROM & Parser Function	Verified Behavior
debug_fallback_load_rom_profile()	Loads compiled "default", "strict", or "trace" debugger presets while keeping loaded_from_data = false.
debug_fallback_load_rom_snapshot_policy() & debug_fallback_load_rom_tripwire_matrix()	Hydrates ring limits and tripwire severity matrices directly from embedded .rodata payloads (76,136 B total diagnostics ROM vault).
debug_fallback_parse_config_json() / parse_snapshot_policy_json()	Shared zero-heap JSON parsers reused symmetrically by disk loaders and ROM hydration (Sub-Assembly 3.1).

16.2. Fault Capture & Emergency AWAIT Latch (Sub-Assembly 3.3)

If a critical subsystem faults during Safe Mode, debug_fallback_record_fault() logs the anomaly cleanly, debug_fallback_finish_with_fallback() downgrades execution onto known-good ROM boundaries, and debug_fallback_is_awaiting() / debug_fallback_clear_await() expose the held state to CLI/test operators cleanly.

16.3. BIOS Self-Audit & ROM Info (Blocks 4 & 5)

- debug_fallback_audit() verifies embedded ROM accessibility and static templates without altering live debug sessions.
- debug_fallback_get_rom_info() reports exact ROM counts and bytes (debug_fallback_rom_info_t).
- debug_fallback_get_stats() and debug_fallback_execute_inspect() populate caller structs deterministically without dynamic allocation.

16.4. Next Steps in debug/

- Check the 4-tier tests covering debugger fallback and probe escalation: Debugger Testing & Tripwires
- Trace how primary debug.c hosts live probes: debug.h + debug.c

Chapter 17. Debug Verification Suite: Rewind, Probes & Tripwires Across All 4 Tiers

Source File: docs/code/debug/04_debug_testing_and_tripwires.adoc

Because cog.debug serves as Cornerstone's self-verification backbone (tests/common/cornerstone_test.h), testing debug/ requires both proving the debugger's own state math in isolation **and** proving that child modules trigger probes and DEBUG_STEP_AWAIT cleanly when integrated.

Tier	CTest Target	Source File	Primary Verification Focus
Unit	unit_debug_engine_test	tests/unit/debug/test_debug_unit.c	Snapshot capture/hash determinism, GCD wrap with locked slots, watchpoints, time-travel rewind, and probe registration.
Unit (BIOS)	unit_debug_fallback_test	tests/unit/debug/test_debug_fallback_unit.c	Emergency ROM hydration, debug_fallback_boot(), fault capture, and fallback AWAIT clear transitions.
Functional	functional_debug_test	tests/functional/test_debug_functional.c	External caller simulation: stepping mode, inspection cards, and tripwire dumps via public headers.
Integrated	integrated_debug_test	tests/integrated/test_debug_integrated.c	Cross-module coordination tying debug + debug_fallback + log + io/ JSON debugger profiles cleanly.
System	system_debug_test	tests/system/test_debug_system.c	End-to-end tripwire escalation, missing-disk downgrade drills, and debug_audit() recovery checks.

17.1. Fine-Grained Invariants Tested Across debug/

- **always_pass & never_fail**: Confirms clean debug_init() / debug_shutdown(), null-safe snapshot getters, and zero-crash clamping when callers pass out-of-range snapshot indices.
- **assertions_and_guarantees**: Verifies deterministic 32-bit FNV-1a checksums across identical debug_state_fields_t structs and strict protection of locked milestone snapshots during circular buffer wrap.
- **emergence & audit_gates**: Exercises full probe traps (debug_trigger_probe → debug_trap_tripwire → DEBUG_STEP_AWAIT → debug_resolve_await) alongside debug_audit() isolation using s_audit_backup.

17.2. Live Emergence Check (test_debug_unit_emergence)

Debugger Unit Emergence & Tripwire Verification (test_debug_unit_emergence)
(tests/unit/debug/test_debug_unit.c#L410-L447 · Mode: Function)

```
410 static void test_debug_unit_emergence(void)
411 {
412     cornerstone_test_section(4, "Emergence (1000-Turn Wrap, 4-Slot Lock Quota & AWAIT Recovery)");
413 
414     TEST_ASSERT(debug_init(NULL), "debug_init succeeds for emergence section");
415 
416     /* 4.1 1000-turn ring wrap-around */
417     for (int t = 1; t <= 1000; t++) {
```

```
        debug_state_fields_t s = cornerstone_test_make_state(
            t, "Elijah", 500 - (t % 200), t % 100, (t * 2) % 100, t % 4, (t % 200) - 100);
        uint64_t sid = debug_record_snapshot(&s, "sim_turn", "step");
        TEST_ASSERT_U64_EQ(sid, (uint64_t)t, "Monotonic ID strictly tracked across 1000 turns");
    }

    TEST_ASSERT_U64_EQ(debug_get_snapshot_count(), (uint64_t)DEBUG_MAX_SNAPSHOTS,
                       "Ring capacity capped at 64");

    debug_snapshot_t oldest;
    TEST_ASSERT(debug_get_snapshot(0U, &oldest) && (oldest.state.turn == 937),
                "Oldest turn after 1000 pushes is 937");
    const debug_snapshot_t *latest = debug_get_latest_snapshot();
    TEST_ASSERT((latest != NULL) && (latest->state.turn == 1000), "Latest turn is 1000");

    debug_state_fields_t restored;
    TEST_ASSERT(debug_rewind_to_snapshot(30U, &restored) &&
                    (restored.turn == 967) &&
                    (restored.hp == (500 - (967 % 200))),
                "Rewind index 30 restores turn 967 and exact HP formula");

    debug_shutdown();

    /* 4.2 Locked critical snapshot quota (4) + DEBUG_TRIGGER_PROBE -> AWAIT recovery */
    TEST_ASSERT(debug_init(NULL), "Re-init debugger for lock quota & AWAIT check");
    TEST_ASSERT(debug_can_step() && !debug_is_awaiting(), "Stepping armed initially");

    uint64_t lock_ids[5] = { 0U };
    for (int t = 1; t <= 6; t++) {
        debug_state_fields_t s = cornerstone_test_make_state(t, "Elijah", 450 + t, 60, 40, t % 4, t);
```

Verifies locked snapshot preservation, GCD ring rotation, probe traps, and Safe-Mode AWAIT clearing.

Wholesale Listing: Complete verbatim code (#L410-L510) → Appendix: Debugger Unit Emergence & Tripwire Verification (test_debug_unit_emergence).

17.3. Running the Debugger Verification Targets

```
ctest --test-dir build -L debug --output-on-failure
ctest --test-dir build \
  -R "^(unit_debug_|functional_debug_|integrated_debug_|system_debug_)" --output-on-failure
```

1. *Why does* `system_debug_test` *take slightly longer (~0.58 s) than pure arithmetic unit tests?*

   `system_debug_test` exercises full snapshot dumps, forensic cards, multi-cycle ring rotations, probe evaluations, and disk/ROM profile transitions across the integrated stack cleanly inside CTest.

17.4. Continue Reading

- Revisit debugger implementation: `debug.h` + `debug.c`
- Advance to the next root pillar: Layer 0 Zero-Heap I/O Pipeline Overview

Chapter 18. Layer 0 Zero-Heap I/O Pipeline Subsystem Overview

Source File: `docs/code/io_pipeline.adoc`

Most engines leak memory or crash unpredictably when loading files: missing folders, oversized JSON payloads, or half-edited game mods turn into hard-to-trace bugs. Cornerstone's **Layer 0 Zero-Heap I/O Pipeline** (`engine/include/io/ & engine/src/io/`) solves this like an assembly line with built-in quality gates: locate the file across layered VFS tiers (or fall back to a compiled `.rodata` ROM), read bounded bytes into caller-owned memory, scan JSON in a single stack pass, and refuse to hand gameplay anything that fails 5-block schema checks.



Navigate the `io/` folder: Pair this overview with docs/code/io/ for detailed chapters on `io.h`, `fs_utils`, `path_finder`, `json_utils`, `master_loader`, `io_fallback`, and all 8 `io` CTest suites.

18.1. Architectural Philosophy: Assembly-Line Ingestion Without Heap

Rather than forcing four separate fallback files (`*_fallback`), Rev 5 unified emergency I/O services inside a single Tier-0 BIOS module (`io_fallback.[ch]`) backed by `io_config_rom.h` (8 compiled `.rodata` JSONs covering `engine/config/io/`), sitting beneath four focused operational engines:

1. **`path_finder` (`cog.path_finder`)**: Anchors repository root dynamically and resolves `"config"` domain files across 3 tiers: `game/config/` → `engine/config/` → `game/data/presentation/config/`.
2. **`fs_utils` (`cog.fs_utils`)**: Performs POSIX stat, mkdir, bounded file reads/writes (⇐ `64 KiB` policy ceiling), read-only Safe-Mode write locks, and `rom://` fallback reads.
3. **`json_utils` (`cog.json_utils`)**: Executes zero-heap single-pass JSON token/value scanning capped at bracket nesting depth ⇐ `32` on caller stack—zero hidden static scratch buffers.
4. **`master_loader` (`cog.master_loader`)**: Verifies the 5-block JSON envelope (`block_1_metadata`..`block_5_closing`), slices non-owning pointers strictly to `block_3_body`, checks schema IDs/versions, and downgrades corrupt configs to ROM Safe Mode.
5. **`io.h` Umbrella (`cog.io`)**: Stateless compile-time facade coordinating includes and unified subsystem audits cleanly across all child cogs.

18.2. Verified Rev 5 File & Documentation Map

Companion Doc	Engine Files Covered	Role & Chassis Highlights
01: I/O Umbrella Facade	`engine/include/io/io.h` (`cog.io`)	Header-only stateless aggregator (`0 B .bss/.data/.text`); unified audit and lifecycle convenience seam.
02: Bounded Filesystem I/O	`engine/include/io/fs_utils.h` + `engine/src/io/fs_utils.c` (`cog.fs_utils`)	Bounded reads/writes (⇐ `64 KiB`), `read_only_safe_mode` latch, `rom://` reads (`232 B .bss`, `4,064 B .rodata`).
03: 3-Tier VFS Path Finder	`engine/include/io/path_finder.h` + `engine/src/io/path_finder.c` (`cog.path_finder`)	Upward root resolution & `vfs_routes.json` hierarchy across up to 36 routes (`288 B .bss`, `5,300 B .rodata`).

Companion Doc	Engine Files Covered	Role & Chassis Highlights
04: Zero-Heap JSON Scanner	`engine/include/io/json_utils.h` + `engine/src/io/json_utils.c` (`cog.json_utils`)	Single-pass token locator & typed span extractors capped at depth ⇐ `32` (`144 B .bss`, `5,740 B .rodata`).
05: Master Manifest Loader	`engine/include/io/master_loader.h` + `engine/src/io/master_loader.c` (`cog.master_loader`)	5-block envelope validator & non-owning `block_3_body` slicer (`112 B .bss`, `5,916 B .rodata`).
06: Tier-0 I/O BIOS & ROM	`engine/include/io/io_fallback.h` + `engine/src/io/io_fallback.c` (`cog.io.fallback`)	Unified BIOS across all 4 I/O pillars, floppy boot mode, and 8 compiled `.rodata` ROMs (`8,312 B .bss`, `40,256 B .rodata`).
07: I/O Testing Suite	8 CTest targets across `tests/{unit,functional,integrated,system}/`	Exhaustive verification of bounds clamping, 3-tier precedence, floppy BIOS, and corrupt JSON downgrade.

18.3. Live VFS Routes & Shared JSON Span Resolution

`engine/config/io/vfs_routes.json` governs repository root discovery and 3-tier precedence cleanly in data:

Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json) (`engine/config/io/vfs_routes.json#L55-L99` · Mode: `Whole`)

```
55  "block_3_body": {
56    "default_data_root": "game/data",
57    "env_override_var": "TL_DATA_ROOT",
58    "block_path_traversal": true,
59    "max_symlink_depth": 4,
60    "allow_mod_mounts": true,
61    "rom_fallback_enabled": true,
62    "probe_await_on_corrupt": true,
63    "config_precedence": [
64      "game/config/",
65      "engine/config/",
66      "game/data/presentation/config/"
67    ],
68    "fallback_candidate_roots": [
69      "game/data",
70      "../game/data",
71      "engine/config",
72      "../engine/config",
73      "data",
74      "../data"
75    ],
76    "asset_roots": [
77      "game/assets",
78      "../game/assets",
79      "assets",
80      "../assets"
81    ],
82    "domain_search_paths": [
83      "",
84      "runtime/",
85      "runtime/math/",
86      "runtime/string/",
87      "runtime/time/",
88      "diagnostics/",
89      "diagnostics/telemetry/",
90      "diagnostics/debugger/",
91      "diagnostics/taxonomy/",
92      "diagnostics/formats/",
```

```
93      "io/",
94      "audio/",
95      "viewport/",
96      "input/",
97      "standards/",
98      "standards/blocks/",
99      "mechanics/",
```

Canonical 5-block engine config defining VFS root search hierarchy, alias tables, and fail-closed safety flags.

Wholesale Listing: Complete verbatim code (#L1-L159) → Appendix: Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json).

18.4. Zero-Duplication Parser Bridge & Verification

Notice in io_fallback.c how BIOS fallback and master_loader share the exact same boundary-safe block_3_body span scanner (io_fb_resolve_body_span):

Shared Zero-Heap JSON Block Resolver & VFS Parser Group (engine/src/io/io_fallback.c#L779-L806 · Mode: Group)

```
779 static bool io_fb_resolve_body_span(const char *json_text,
780                                       size_t json_len,
781                                       bool allow_strlen,
782                                       const char **out_body_start,
783                                       const char **out_body_end)
784 {
785     if (json_text == NULL || out_body_start == NULL || out_body_end == NULL) {
786         return false;
787     }
788 
789     size_t effective_len = (json_len > 0U) ? json_len : (allow_strlen ? strlen(json_text) : 0U);
790     if (effective_len == 0U || effective_len > IO_FB_JSON_MAX_BYTES) {
791         return false;
792     }
793 
794     const char *json_end = json_text + effective_len;
795     const char *body_start = NULL;
796     size_t body_len = 0U;
797     if (!io_fb_find_block_span(json_text, json_end, "block_3_body", '{', '}',
798                                &body_start, &body_len) ||
799         body_start == NULL || body_len == 0U) {
800         return false;
801     }
802 
803     *out_body_start = body_start;
804     *out_body_end = body_start + body_len;
805     return true;
806 }
```

Tier-0 in-place span scanning reused seamlessly between BIOS fallback and master_loader.

Wholesale Listing: Complete verbatim code (#L779-L1194) → Appendix: Shared Zero-Heap JSON Block Resolver & VFS Parser Group.

```
ctest --test-dir build -R "(fs_utils|path_finder|json_utils|master_loader|io_)" --output-on-failure
cargo run --manifest-path tools/Cargo.toml --bin tl_tools -- validate data
```

1. *Why prohibit static 32 KiB scratch arrays inside I/O C functions?*

   A forgotten function-local static char buf[32768] permanently pins idle bytes inside .bss. Passing caller-owned slices keeps fs_utils (232 B .bss), path_finder (288 B .bss), json_utils (144 B .bss), and master_loader (112 B .bss) razor-lean!

18.5. Companion Deep-Dive Chapters

1. Inspect `io.h` Umbrella Facade and `fs_utils.h` + `fs_utils.c`.
2. Trace VFS & parsing via `path_finder`, `json_utils`, and `master_loader`.
3. Understand zero-disk survival in `io_fallback` and I/O Testing.

Chapter 19. I/O Umbrella Facade & 4-Pillar Boot Pipeline (io.h)

Source File: docs/code/io/01_io_umbrella_and_pipeline.adoc

Attribute	Verified Contract (cog.io)
Header File	engine/include/io/io.h (301 lines) — pure compile-time facade (archetype: facade_umbrella | stateless).
Runtime Source	None intentionally (0 B .bss, 0 B .data, 0 B .text); delegates runtime execution cleanly to its 5 child cogs.
Constituent Cogs	io_fallback.h → fs_utils.h → json_utils.h → path_finder.h → master_loader.h (below core/manifest.h).
Manifest Anchor	engine/config/io/manifest.json (probes: io_umbrella.*).

Think of io.h as a clean electrical junction box, never an extra processor. Higher layers (game/src/main.c or test harnesses) can include "io/io.h" to bring all five I/O pillars online in verified topological order—while internal log/ or lower core/ cogs can still include just the fine-grained header they need without dragging unnecessary dependencies upward.

19.1. Downward Re-Export Order (BLOCK 2 of io.h)

io.h opens cleanly with standard ISO C11 headers (<stdbool.h>, <stddef.h>, <stdint.h>), binds "core/manifest.h", and re-exports child cogs in strict topological sequence:

1. "io/io_fallback.h" — Tier-0 BIOS & .rodata ROM vault (cog.io.fallback).
2. "io/fs_utils.h" — bounded POSIX file/stream primitives (cog.fs_utils).
3. "io/json_utils.h" — zero-heap JSON span scanner (cog.json_utils).
4. "io/path_finder.h" — 3-tier VFS path resolution (cog.path_finder).
5. "io/master_loader.h" — 5-block envelope validator & scoped body slicer (cog.master_loader).

Sub-Assembly 2.1 aligns system-wide ceilings across the suite—such as FS_PATH_MAX = 512 bytes and 64 KiB configuration payload limits—ensuring static stack and .bss buffers across layers agree at compile time.

19.2. Configuration Anchor (engine/config/io/)

All operational policies hydrated through io.h live in data files under engine/config/io/, anchored by vfs_routes.json and manifest.json:

Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json) (engine/config/io/vfs_routes.json#L55-L99 · Mode: Whole)

```
55  "block_3_body": {
56    "default_data_root": "game/data",
57    "env_override_var": "TL_DATA_ROOT",
58    "block_path_traversal": true,
59    "max_symlink_depth": 4,
60    "allow_mod_mounts": true,
61    "rom_fallback_enabled": true,
62    "probe_await_on_corrupt": true,
63    "config_precedence": [
64      "game/config/",
65      "engine/config/",
66      "game/data/presentation/config/"
```

```
    ],
    "fallback_candidate_roots": [
      "game/data",
      "../game/data",
      "engine/config",
      "../engine/config",
      "data",
      "../data"
    ],
    "asset_roots": [
      "game/assets",
      "../game/assets",
      "assets",
      "../assets"
    ],
    "domain_search_paths": [
      "",
      "runtime/",
      "runtime/math/",
      "runtime/string/",
      "runtime/time/",
      "diagnostics/",
      "diagnostics/telemetry/",
      "diagnostics/debugger/",
      "diagnostics/taxonomy/",
      "diagnostics/formats/",
      "io/",
      "audio/",
      "viewport/",
      "input/",
      "standards/",
      "standards/blocks/",
      "mechanics/",
```

Canonical 5-block engine config defining VFS root search hierarchy, alias tables, and fail-closed safety flags.

Wholesale Listing: Complete verbatim code (`#L1-L159`) → Appendix: Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json).

19.3. Coordinated Validation & Lifecycle Gates

`Sub-Assemblies 4.1` through `5.2` inside `io.h` do **not** invent separate state flags or bypass child cogs; instead they coordinate cross-cog initialization, probe verification (`io_umbrella.*`), and unified pre-flight checks across all five underlying modules cleanly.

1. *Why enforce an explicit non-owning stateless facade invariant on `io.h`?*

   If someone sneaks parsing helpers or static buffers directly into `io.h`, callers importing only `fs_utils.h` or `path_finder.h` silently diverge from full builds. Keeping `io.h` strictly stateless preserves explicit include hygiene (`Invariant 3`).

19.4. Explore the Five Child I/O Cogs

1. `fs_utils.h` + `fs_utils.c`
2. `path_finder.h` + `path_finder.c`
3. `json_utils.h` + `json_utils.c`
4. `master_loader.h` + `master_loader.c`
5. `io_fallback.h` + `io_fallback.c`

Chapter 20. I/O Module Pair: Bounded File & Memory Reader (fs_utils.h + fs_utils.c)

Source File: docs/code/io/02_fs_utils_bounded_io.adoc

Attribute	Verified Contract (cog.fs_utils)
Public Contract (.h)	engine/include/io/fs_utils.h (838 lines) — stat inspection, bounded reads/writes, policy accessors, probes, and lifecycle hooks.
Runtime Engine (.c)	engine/src/io/fs_utils.c (1,462 lines) — POSIX file operations, read_only_safe_mode enforcement, and rom:// bridge.
Chassis Footprint	.bss 232 B (s_fs_state) + .rodata 4,064 B + 8 KiB transient stack on policy load; zero idle scratch bloat.
Policy Config	engine/config/io/fs_buffers.json & engine/config/io/default.json (probes: io.fs.* → DEBUG_STEP_AWAIT).

Reading a file in plain C often hides two subtle hazards: reading a huge asset into a fixed buffer while silently throwing away the end of the file, or accidentally scribbling logs onto a locked/read-only cartridge or install directory. fs_utils.[ch] makes both hazards explicit and measurable.

20.1. Bounded Reads, Atomic-Minded Writes & Directory Creation (Sub-Assemblies 3.1–3.2)

Callers pass explicitly sized buffers (char *buf, size_t buf_size) cleanly bounded by policy ceilings up to 64 KiB (max_config_bytes), ensuring oversized payloads trigger an explicit truncation_rejections increment instead of partial garbage:

Bounded POSIX Filesystem & Read-Only Guard API (fs_utils.h Sub-Assembly 3.2)
(engine/include/io/fs_utils.h#L359-L400 · Mode: Subassembly)

```
359 /*
360 ---------------------------------------------------------------------------
361 Sub-Assembly 3.2: Bounded Buffered I/O & Directory Management Cogs
362 Mission: Bounded text reading, safe file writing, and parent directory synthesis.
363
364   ### For Game Designers & Non-Programmers:
365   Loads configuration and story files safely within memory caps and saves outputs
366   atomically while blocking mutations during Read-Only Safe Mode.
367
368   ### For Systems Programmers & Engine Architects:
369   Enforces active policy ceilings (`max_config_bytes`, `path_max_bytes`), NUL
370   termination, Safe-Mode RO protection, and shared Tier-0 BIOS atomic write/mkdir.
371 ---------------------------------------------------------------------------
372 */
373
374 /**
375  * @brief Reads entire text file contents into a bounded, caller-supplied buffer.
376  *
377  * Guarantees unconditional null termination and prevents buffer overruns.
378  * If the file exceeds `buf_size - 1` or `max_config_bytes`, the read is safely rejected.
379  *
380  * ### For Game Designers & Non-Programmers:
381  * Loads an entire text file safely into memory, guaranteeing it never overflows or crashes the engine.
382  *
383  * ### For Systems Programmers & Engine Architects:
384  * Convenience wrapper delegating cleanly into `fs_read_file_bounded()` without length output.
385  *
386  * @param[in]  filepath Null-terminated path or `"rom://..."` URI to target file.
```

```
387  * @param[out] buf      Caller-supplied memory destination buffer.
388  * @param[in]  buf_size Byte capacity of destination buffer (including null terminator).
389  * @return true if file opened and completely read within buffer bounds; false otherwise.
390  * @triggers Inherits truncation fault and warning log behavior from `fs_read_file_bounded()`.
391  */
392 bool fs_read_file_contents(const char *filepath, char *buf, size_t buf_size);
393
394 /**
395  * @brief Reads file or `"rom://..."` text and reports exact count of bytes read.
396  *
397  * Similar to `fs_read_file_contents()`, and writes the exact number of bytes read
398  * (excluding NUL terminator) into optional `out_bytes_read`.
399  *
400  * ### For Game Designers & Non-Programmers:
```

Caller-owned bounded file transfers with truncation protection and Safe-Mode write locks.

Wholesale Listing: Complete verbatim code (#L359-L455) → Appendix: Bounded POSIX Filesystem & Read-Only Guard API (fs_utils.h Sub-Assembly 3.2).

- **Probing (3.1)**: `fs_file_exists()`, `fs_is_regular_file()`, and `fs_inspect_file()` (`fs_file_info_t`).
- **Mounting & `rom://` Fallback (3.3)**: `fs_set_data_root()` and `fs_resolve_data_path()` cooperate cleanly with `rom://` URIs when disk storage degrades.
- **Policy Hydration (3.4)**: `fs_utils_load_policy()`, `fs_utils_set_policy()`, and `fs_utils_get_policy()` (`io_fallback_config_t`) enforce runtime ceilings and `read_only_safe_mode` latches (`read_only_blocks` counter increments when writes or `fs_ensure_directory()` are stopped under read-only safety lock).

20.2. Telemetry Counters, Probes & Self-Audit (Blocks 4 & 5)

`fs_utils_stats_t` exposes O(1) non-allocating inspection across lifetime `read_ops`, `write_ops`, `truncation_rejections`, `read_only_blocks`, and `rom_fallback_reads`:

- **Probe Hooks (4.1–4.2)**: `fs_utils_validate_policy_contract()`, `fs_utils_register_debug_probes()`, `fs_utils_trigger_policy_probe()`, and `fs_utils_audit()`.
- **Operational Cards (5.1–5.2)**: `fs_execute_read()`, `fs_execute_write()`, `fs_execute_dump_card()`, `fs_utils_get_stats()`, `fs_utils_init()`, and `fs_utils_shutdown()`.

20.3. Continue in io/

- Next cog up the pipeline: `path_finder.h` + `path_finder.c`
- Emergency `rom://` provider: `io_fallback.h` + `io_fallback.c`

Chapter 21. I/O Module Pair: Layered VFS Domain Router (path_finder.h + path_finder.c)

Source File: docs/code/io/03_path_finder_vfs.adoc

Attribute	Verified Contract (cog.path_finder)
Public Contract (.h)	engine/include/io/path_finder.h (719 lines) — root discovery, route loading, multi-tier VFS resolution, probes, and audit gates.
Runtime Engine (.c)	engine/src/io/path_finder.c (1,992 lines) — POSIX lstat/readlink root anchoring, internal vfs_hydrate_routes_internal(), and route search.
Chassis Footprint	.bss 288 B (s_vfs_state) + .rodata 5,300 B; max path PATH_FINDER_MAX_PATH = 512, MAX_ROOT = 256.
Route Config	engine/config/io/vfs_routes.json (⇐ 3 config tiers, ⇐ 36 routes; probes: io.vfs.* → DEBUG_STEP_AWAIT).

Whether you launch the compiled binary from repository root, from inside build/, or from a nested CTest build folder, path_finder walks upward deterministically to anchor the workspace root. Even better: designers can drop an override JSON inside game/config/ to tweak **Tenebris and Lux** without ever mutating Cornerstone's baseline defaults inside engine/config/.

21.1. How Multi-Tier Resolution Works (vfs_routes.json)

When a subsystem asks for domain "config" with relative path diagnostics/telemetry/default.json, path_finder consults the hydrated route table (io_fb_vfs_routes_t) in fixed precedence:

JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json) (engine/config/io/vfs_routes.json#L55-L102 · Mode: Block)

```
55  "block_3_body": {
56    "default_data_root": "game/data",
57    "env_override_var": "TL_DATA_ROOT",
58    "block_path_traversal": true,
59    "max_symlink_depth": 4,
60    "allow_mod_mounts": true,
61    "rom_fallback_enabled": true,
62    "probe_await_on_corrupt": true,
63    "config_precedence": [
64      "game/config/",
65      "engine/config/",
66      "game/data/presentation/config/"
67    ],
68    "fallback_candidate_roots": [
69      "game/data",
70      "../game/data",
71      "engine/config",
72      "../engine/config",
73      "data",
74      "../data"
75    ],
76    "asset_roots": [
77      "game/assets",
78      "../game/assets",
79      "assets",
80      "../assets"
81    ],
82    "domain_search_paths": [
83      "",
```

```
      "runtime/",
      "runtime/math/",
      "runtime/string/",
      "runtime/time/",
      "diagnostics/",
      "diagnostics/telemetry/",
      "diagnostics/debugger/",
      "diagnostics/taxonomy/",
      "diagnostics/formats/",
      "io/",
      "audio/",
      "viewport/",
      "input/",
      "standards/",
      "standards/blocks/",
      "mechanics/",
      "mechanics/items/",
      "mechanics/combat/",
      "mechanics/tactics/",
```

Demonstrates scoped block_3_body extraction—matching master_loader's strict rule against reading Block 1/2 metadata as live gameplay config.

Wholesale Listing: Complete verbatim code (#L55-L125) → Appendix: JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json).

21.2. Public Resolution API (Sub-Assemblies 3.1–3.2)

Multi-Tier VFS Path & Asset Resolution API (path_finder.h Sub-Assembly 3.2)
(engine/include/io/path_finder.h#L396-L435 · Mode: Subassembly)

```
/*
-------------------------------------------------------------------------------
Sub-Assembly 3.2: Multi-Tier VFS Path & Asset Resolution Cogs
Mission: Execute layered precedence search, diagnostic alias expansion, symlink
         depth bounds, traversal rejection, and `.rodata` ROM fallback cleanly.

  ### For Game Designers & Non-Programmers:
  Finds configuration manifests, game data files, fonts, and artwork across game
  overrides, engine baselines, asset folders, and zero-disk ROM fallbacks.

  ### For Systems Programmers & Engine Architects:
  Bounded-stack candidate resolution enforcing `max_symlink_depth`, `..` traversal
  containment, destination `out_size` overflow guards, and `io.vfs.traversal_guard`.
-------------------------------------------------------------------------------
*/

/**
 * @brief Resolves a logical domain and relative path to a concrete disk file or `rom://` URI.
 *
 * Search sequence (all tables driven by `io/vfs_routes.json`):
 *  1. `..` parent traversal sandbox guard (`block_path_traversal`) & explicit `rom://` check.
 *  2. Direct accessible file probe (enforcing `max_symlink_depth`).
 *  3. Mounted `active_root` (`<root>/<domain>/<clean>`, `<root>/<clean>`) and `env_override_var`.
 *  4. Layered `config_precedence` tiers (`game/config/`, `engine/config/`, linkers) + `diagnostic_aliases`.
 *  5. Hydrated `fallback_candidate_roots` x `domain_search_paths` matrix.
 *  6. Compiled `.rodata` ROM vault fallback (`rom://<clean>`) when disk targets are absent.
 *
 * ### For Game Designers & Non-Programmers:
 * Finds any configuration file, story file, or manifest automatically—checking game overrides
 * first, engine baselines second, and built-in ROM backups last.
 *
 * ### For Systems Programmers & Engine Architects:
 * Bounded stack probing (zero heap), strict `out_size` overflow rejection, `lstat`/`readlink`
 * symlink depth verification, and `LOG_CAT_IO` telemetry tracking.
 * @triggers `io.vfs.traversal_guard` (on `..`/symlink rejection), `LOG_CAT_IO`.
 *
 * @param[in]  domain   Optional logical domain hint (`"config"`, `"mechanics"`, `"io"`, etc.) or NULL.
 * @param[in]  rel_path Relative asset/config path or explicit `"rom://..."` URI.
 * @param[out] out_path Caller-supplied output path buffer (`out_path[0] = '\0'` on failure).
```

```
435  * @param[in]  out_size Capacity of `out_path` in bytes.
```

Resolves game/config → engine/config → game/data/presentation/config deterministically without heap allocation.

Wholesale Listing: Complete verbatim code (#L396-L510) → Appendix: Multi-Tier VFS Path & Asset Resolution API (path_finder.h Sub-Assembly 3.2).

- **Root & Route Setup (3.1)**: path_finder_set_data_root(), path_finder_load_routes(), path_finder_set_routes(), and path_finder_get_routes().
- **Domain Lookups (3.2)**: path_finder_resolve(), path_finder_find_manifest(), path_finder_resolve_asset(), and path_finder_resolve_with_hint(). When no disk file matches any tier, callers smoothly receive a deterministic rom:// URI or Safe-Mode fallback notice cleanly.

21.3. Validation, Probes & Inspection (Sub-Assemblies 4.1–5.2)

- **Validation (4.1–4.2)**: path_finder_validate_routes_contract(), path_finder_register_debug_probes(), path_finder_trigger_routes_probe(), path_finder_preflight(), and path_finder_audit().
- **Inspection & Lifecycle (5.1–5.2)**: path_finder_execute_resolve(), path_finder_execute_dump_card(), path_finder_get_stats(), path_finder_init(), and path_finder_shutdown().

21.4. Continue in io/

- Next stage (tokenizing found files): json_utils.h + json_utils.c
- Pre-flight manifest loader: master_loader.h + master_loader.c

Chapter 22. I/O Module Pair: Zero-Heap Single-Pass JSON Scanner (json_utils.h + json_utils.c)

Source File: docs/code/io/04_json_utils_parser.adoc

Attribute	Verified Contract (cog.json_utils)
Public Contract (.h)	engine/include/io/json_utils.h (1,205 lines) — token/object/array span locators, typed getters, policy contracts, probes, and lifecycle hooks.
Runtime Engine (.c)	engine/src/io/json_utils.c (3,092 lines) — string-escape-aware single-pass parser, depth guards, and zero-static-buffer scanning.
Chassis Footprint	.bss 144 B (s_json_state) + .rodata 5,740 B + zero function-local static scratch buffers; stack depth capped at ⇐ 32.
Policy Config	engine/config/io/loader_contracts.json (probes: io.json.* → DEBUG_STEP_AWAIT).

Standard JSON libraries allocate dozens or hundreds of tiny heap nodes per file—making fragmentation unavoidable and deterministic memory budgeting nearly impossible. json_utils.[ch] skips building a heap DOM altogether: it walks raw JSON text in a single pass over caller memory (O(N)), tracks nesting safely up to depth 32, and returns exact (ptr, len) spans ready for zero-copy slicing.

22.1. Single-Pass Token Locators & Typed Extractors (Sub-Assemblies 3.1–3.4)

Sub-Assembly 3.1 locates bounded child { ⋯ } and [⋯] byte ranges cleanly inside any slice without mutating the source text (essential when scanning immutable .rodata ROMs!):

Zero-Heap JSON Token Span Locator API (json_utils.h Sub-Assembly 3.1) (engine/include/io/json_utils.h#L357-L396 · *Mode:* Subassembly)

```
357 /*
358 ------------------------------------------------------------------------------
359 Sub-Assembly 3.1: JSON Token Locator Cogs
360 Mission: Scan flat or span-bounded JSON payloads for matching keys, objects, and
361          arrays while respecting escape sequences and nesting limits cleanly.
362
363   ### For Game Designers & Non-Programmers:
364   Finds named sections, tables, and lists inside JSON configuration files cleanly
365   without mistaking words inside text descriptions for structural headings.
366
367   ### For Systems Programmers & Engine Architects:
368   Exposes zero-copy, zero-heap token and balanced-container locators constrained
369   to `[json_start, json_start + json_len)` and active `max_json_depth` policy.
370 ------------------------------------------------------------------------------
371 */
372
373 /**
374  * @brief Scans a JSON buffer for a specific top-level or scoped key token.
375  *
376  * Core search primitive that locates occurrences of "key" followed by a colon
377  * (:), ignoring intervening whitespace. Prevents false positive matches on
378  * substring prefixes or keys embedded inside quoted strings.
379  *
380  * ### For Game Designers & Non-Programmers:
381  * Searches through a data file to locate where a property begins (e.g. finding "attack_power:").
382  *
383  * ### For Systems Programmers & Engine Architects:
384  * Reentrant zero-heap token locator respecting quote escapes and whitespace.
385  * @triggers On nesting limit exceeded -> `IO_ERR_JSON_DEPTH_EXCEEDED` via `io.json.depth_bounds`.
386  *
387  * @param[in] json Null-terminated JSON buffer.
```

```
388  * @param[in] key  Null-terminated key name to search for (without quotes).
389  * @return Pointer to value position immediately following the colon (:),
390  *         or NULL if key is not found or parameters are invalid.
391  */
392 const char *json_find_key(const char *json, const char *key);
393
394 /**
395  * @brief Span-bounded key locator preventing sibling overrun across sliced buffers.
396  *
```

Single-pass bounded span slicing capped at bracket nesting depth ⇐ 32 on caller stack.

Wholesale Listing: Complete verbatim code (#L357-L461) → Appendix: Zero-Heap JSON Token Span Locator API (json_utils.h Sub-Assembly 3.1).

Sub-Assembly	Core Capabilities
Nested Value Getters (3.2)	`json_get_string()`, `json_get_int()`, `json_get_size_t()`, `json_get_float()`, `json_get_bool()`, plus scoped `json_find_object()` and `json_get_object_*()` helpers.
Array Scanning (3.3)	`json_find_array()`, `json_array_get_count()`, `json_array_get_string()`, and `json_array_get_object()` for indexed table iteration.
Span-Bounded Scanners (3.4)	`json_get_{string,int,size_t,float,bool}_span()` scan strictly inside a `[start, start+len)` boundary—forming the backbone of `master_loader`s `block_3_body` isolation.
Runtime Policy (3.4)	`json_utils_load_policy()`, `json_utils_get_policy()`, and `json_utils_set_policy()` synchronize token/depth limits from `loader_contracts.json`.

22.2. Structural Guards & Validation Probes (Sub-Assemblies 4.1–5.2)

- `json_validate_span()` and `json_utils_enforce_valid_span()` reject truncated strings, unbalanced braces, or excessive nesting before domain extractors read bad keys.
- `json_utils_validate_policy_contract()`, `json_utils_register_debug_probes()` (3 probes in `s_json_probes[3]`), `json_preflight()`, and `json_utils_audit()` wire syntax/depth breaches cleanly into `cog.debug`.

22.3. Continue in io/

- Next layer (5-block contract enforcer): `master_loader.h` + `master_loader.c`
- Shared ROM span resolver: `io_fallback.h` + `io_fallback.c`

Chapter 23. I/O Module Pair: 5-Block Contract Hydrator (master_loader.h + master_loader.c)

Source File: docs/code/io/05_master_loader_hydration.adoc

Attribute	Verified Contract (cog.master_loader)
Public Contract (.h)	engine/include/io/master_loader.h (987 lines) — loader_status_t, loader_header_t, master_loader_t, master_loader_stats_t, block_3_body getters, schema/version validators, probes, and audit controls.
Runtime Engine (.c)	engine/src/io/master_loader.c (2,051 lines) — orchestrator wiring path_finder VFS lookup, fs_utils file/ROM reads, json_utils span queries, cog.log, and cog.debug probes.
Controlled Chassis	.bss 112 B (s_loader_state) + .rodata 5,916 B + transient preflight stack frame; caller-owned master_loader_t slices capped at max_payload_bytes ⇐ 65536, max_json_depth ⇐ 32, max_json_tokens ⇐ 1024.
Seams & Probes	in: engine/config/io/loader_contracts.json, rom://io | uses: cog.path_finder, cog.fs_utils, cog.json_utils, cog.log, cog.debug | probes: io.loader.* → DEBUG_STEP_AWAIT | dag: core.io.master_loader

For Game Designers & Data Authors: master_loader is the gatekeeper every JSON file in engine/config/ and game/data/ must pass through. It checks file headers (block_1_metadata and block_2_opening), enforces schema compatibility, and lets gameplay code read **only** inside block_3_body so a typo in file notes or metadata never hijacks a stat formula. If disk data is missing or corrupted, it falls back seamlessly to rom://io compiled defaults without crashing.

For Engine Programmers: Instead of duplicating JSON parsing or path lookup per game system, master_loader.[ch] binds all lower io/ primitives under one deterministic state machine backed by runtime policy (master_loader_load_contract()), strict error counters (syntax_errors 0x0204, schema_mismatches 0x0203, version_mismatches 0x0207, body_missing_errors 0x0206), and DEBUG_STEP_AWAIT traps.

23.1. Dual-Path Hydration: Disk, VFS, rom:// & Memory

Entry Point	Execution Path & Provenance Tracking
master_loader_load()	Resolves (domain, rel_path) via path_finder_resolve(). Sets loaded_from_data = true and loaded_from_rom = false for disk payloads, or loaded_from_rom = true and loaded_from_data = false when hydrated from rom://io.
master_loader_load_from_path()	Reads an explicit file path via bounded fs_utils, skipping domain search while still enforcing full 5-block header validation and size bounds.
master_loader_load_from_memory()	Parses an already-resident byte buffer (.rodata ROM image or test fixture) in place with zero filesystem calls.

During envelope parsing, loader_header_t inspects is_5block, has_block_1, and has_block_2, then isolates [body_start, body_start + body_len) over raw_len before returning master_loader_t to the caller. Missing block_3_body records body_missing_errors (0x0206) and trips Safe-Mode diagnostics rather than silently reading stale or adjacent fields.

23.2. Strictly Scoped `block_3_body` Getters

All runtime property queries are pinned strictly inside the verified `block_3_body` window (or an optional named `sub_obj` child inside `block_3_body`):

Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2)
(engine/include/io/master_loader.h#L436-L479 · Mode: Subassembly)

```
436 /*
437 ------------------------------------------------------------------------------
438 Sub-Assembly 3.2: Scoped `block_3_body` Accessors
439 Mission: Restrict all payload getters and array locators strictly inside the
440          verified `[body_start, body_start + body_len)` span (or named `sub_obj`).
441
442   ### For Game Designers & Non-Programmers:
443   Reads gameplay properties cleanly from inside `block_3_body` only, preventing
444   file headers or audit sections from ever colliding with gameplay field names.
445
446   ### For Systems Programmers & Engine Architects:
447   Delegates scope isolation and typed extraction exclusively to span-bounded
448   `json_*_span` APIs with unconditional NUL termination on extracted strings.
449 ------------------------------------------------------------------------------
450 */
451
452 /**
453  * @brief Extracts a string value strictly within `block_3_body` (or named sub-object).
454  *
455  * ### For Game Designers & Non-Programmers:
456  * Safely reads text properties (character names, formulas, IDs) from the data body
457  * without accidentally matching header text outside Block 3.
458  *
459  * ### For Systems Programmers & Engine Architects:
460  * Uses span-bounded `json_find_object_span()` + `json_get_string_span()` bounded to
461  * `loader->body_len`. Unconditionally NUL-terminates `out_val` when `out_size > 0`.
462  * @triggers `io.json.depth_bounds` if scan breaches `max_json_depth`.
463  *
464  * @param[in]  loader   Ready `master_loader_t` instance.
465  * @param[in]  sub_obj  Optional sub-object key inside `block_3_body` (or NULL for root body).
466  * @param[in]  key      Target property key inside resolved body/sub-object span.
467  * @param[out] out_val  Caller string buffer.
468  * @param[in]  out_size Capacity of `out_val` in bytes.
469  * @return true on clean extraction; false if missing, wrong type, or invalid params.
470  */
471 bool master_loader_get_string(const master_loader_t *loader,
472                               const char *sub_obj,
473                               const char *key,
474                               char *out_val,
475                               size_t out_size);
476
477 /**
478  * @brief Extracts a signed integer strictly within `block_3_body` (or named sub-object).
479  *
```

Zero-heap typed getters that scan directly inside a validated block_3_body span.

Wholesale Listing: Complete verbatim code (#L436-L608) → Appendix: Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2).

Typed extractors cover the full primitive and structured vocabulary without dynamic memory allocation:

- `master_loader_get_string()` — copies and unconditionally NUL-terminates string values.
- `master_loader_get_int()`, `master_loader_get_size_t()`, `master_loader_get_float()`, `master_loader_get_bool()` — strict typed extraction with fallback signaling on wrong types.
- `master_loader_find_array()` and `master_loader_get_array_object()` — zero-copy array cursor iteration over object elements inside `block_3_body`.

23.3. Runtime Loader Policy (loader_contracts.json)

Rather than baking ceilings permanently into code, master_loader_init() hydrates engine/config/io/loader_contracts.json (with rom://io safe fallback on corruption, incrementing corrupt_contract_downgrades):

- master_loader_load_contract(), master_loader_get_contract(), master_loader_set_contract() — inspect or override byte/depth/token limits and strict schema rules in controlled tests.
- master_loader_validate_schema(), master_loader_validate_version(), master_loader_validate_envelope(), master_loader_validate_contract() — granular contract gates executed prior to trusting any loaded JSON table.

23.4. Error Taxonomy & Diagnostic Probes (Block 4)

Every failure increments machine-inspectable counters in master_loader_stats_t (readable via master_loader_get_stats()) alongside categorized hex codes registered under io.loader.*:

Telemetry Counter (master_loader_stats_t)	Code	Fault Condition & Escalation
not_found_errors	0x0201	Neither disk domains nor rom:// fallback satisfied the target relative path.
overflow_errors	0x0202	File or payload exceeded caller slice or max_payload_bytes (⇐ 65536).
schema_mismatches	0x0203	block_2_opening declared an unexpected schema_id.
syntax_errors	0x0204	Malformed JSON braces/escapes or nesting exceeding max_json_depth (⇐ 32).
body_missing_errors	0x0206	Required block_3_body object absent from a 5-block envelope.
version_mismatches	0x0207	Manifest contract version incompatible with the active engine target.

master_loader_register_debug_probes() and master_loader_trigger_probe() wire these thresholds directly into cog.debug so violations halt execution in DEBUG_STEP_AWAIT during audit/debug runs instead of failing silently downstream.

- master_loader_preflight() verifies s_loader_state.initialized, contract_from_data, and probe registration before main() advances past io/.
- master_loader_audit() runs deep structural self-checks on cached policy limits and live load counters (total_loads = disk_loads + rom_loads + memory_loads, body_queries).

23.5. Inspection Commands & Lifecycle Ownership

Operational tooling and Layer 3 startup call master_loader_execute_inspect(), master_loader_execute_dump_card(), and master_loader_execute_load_verified() to print human-readable status cards and assert strict load contracts before accepting gameplay assets. master_loader_shutdown() clears s_loader_state back to dormant .bss zeroes deterministically between sessions or test suites.

- Back to Layer 0 I/O Folder Index or Layer 0 I/O Subsystem Overview.
- Next module: Paired Module: io_fallback.h + io_fallback.c.

Chapter 24. I/O Safe-Mode BIOS: Zero-Disk Fallback & ROM Vault (io_fallback.h + io_fallback.c)

Source File: docs/code/io/06_io_fallback_bios_and_rom.adoc

Attribute	Verified Contract (cog.io.fallback)
Public Contract (.h)	engine/include/io/io_fallback.h (1,519 lines) — IO_ERR_* codes, io_fallback_config_t, io_fb_vfs_routes_t, shared safe JSON parsers, ROM vault lookup, unified emergency cogs, and BIOS boot/downgrade controls.
Runtime Engine (.c)	engine/src/io/io_fallback.c (3,355 lines) — standalone floppy-grade I/O engine operating cleanly when fs_utils, path_finder, json_utils, or master_loader fail or disk assets are absent.
Controlled Chassis	.bss 8,312 B (s_io_fb) + .rodata 40,256 B (8 embedded I/O ROMs generated into io_config_rom.h) + stack ⇐ 1 KiB.
Seams & Archetype	quarantined_fallback | stateful | in: io_config_rom.h | uses: cog.log.fallback, cog.debug.fallback | probes: io_fallback.* → DEBUG_STEP_AWAIT | dag: io.fallback_bios

For Designers & Playtesters: Deleting or breaking engine/config/io/ on disk does not brick Cornerstone. Instead, io_fallback acts like a motherboard BIOS stored directly inside read-only binary memory (.rodata): it still knows every VFS search route, still extracts fields from block_3_body, logs the missing disk file via log_fallback, and hands diagnostics straight to debug_fallback / DEBUG_STEP_AWAIT.

For Systems Programmers: io_fallback.[ch] obeys strict quarantine boundaries (*_fallback.[ch] convention). Notice that its shared JSON body parser (io_fb_resolve_body_span(), io_fallback_parse_vfs_routes_json()) also feeds the full-featured I/O stack when booting from ROM, eliminating duplicate parser logic while preventing circular dependencies with master_loader or path_finder.

24.1. Shared ROM-Safe JSON Policy Parsers

Because disk configuration might be unreachable or unreadable during boot, Sub-Assembly 3.1 parses JSON byte spans directly inside read-only .rodata memory without scratch arrays on the heap:

Shared Zero-Heap JSON Block Resolver & VFS Parser Group (engine/src/io/io_fallback.c#L779-L806 · Mode: Group)

```
779 static bool io_fb_resolve_body_span(const char *json_text,
780                                       size_t json_len,
781                                       bool allow_strlen,
782                                       const char **out_body_start,
783                                       const char **out_body_end)
784 {
785     if (json_text == NULL || out_body_start == NULL || out_body_end == NULL) {
786         return false;
787     }
788
789     size_t effective_len = (json_len > 0U) ? json_len : (allow_strlen ? strlen(json_text) : 0U);
790     if (effective_len == 0U || effective_len > IO_FB_JSON_MAX_BYTES) {
791         return false;
792     }
793
794     const char *json_end = json_text + effective_len;
795     const char *body_start = NULL;
796     size_t body_len = 0U;
797     if (!io_fb_find_block_span(json_text, json_end, "block_3_body", '{', '}',
798                                &body_start, &body_len) ||
799         body_start == NULL || body_len == 0U) {
```

```
800         return false;
801     }
802
803     *out_body_start = body_start;
804     *out_body_end = body_start + body_len;
805     return true;
806 }
```

Tier-0 in-place span scanning reused seamlessly between BIOS fallback and master_loader.

Wholesale Listing: Complete verbatim code (#L779-L1194) → Appendix: Shared Zero-Heap JSON Block Resolver & VFS Parser Group.

- io_fb_resolve_body_span() isolates block_3_body bounds cleanly inside immutable ROM text.
- io_fallback_parse_vfs_routes_json() extracts io_fb_vfs_routes_t search domains (config, data, assets) deterministically into .bss so even floppy Safe-Mode resolves logical routes accurately.

24.2. Compiled .rodata Config ROM Vault (Sub-Assembly 3.2)

At build time, cmake/GenerateConfigRom.cmake compiles **8 canonical engine/config/io/*.json files** into io_config_rom.h totaling 40,256 B of read-only .rodata. Sub-Assembly 3.2 exposes O(1)/bounded name resolution over rom://io, allowing path_finder and master_loader to hydrate identical JSON contracts whether loaded from disk or emergency binary memory.

24.3. Unified 4-Pillar Emergency Cogs (Sub-Assembly 3.3)

Sub-Assembly 3.3 consolidates minimalist emergency fallbacks for **all four I/O operational pillars** inside one ring-buffered state block (s_io_fb, 8,312 B):

1. **Bounded Read Fallback**: Safe bounds-checked file/memory slice copies rejecting overflow (IO_ERR_*).
2. **Route Fallback**: Resolves canonical domain priorities using io_fb_vfs_routes_t hydrated from io_config_rom.h.
3. **Token/Span Fallback**: Minimal key lookup inside validated block_3_body slices.
4. **Fault Ring Buffer**: Captures failure provenance (loaded_from_data = false), error codes, and path breadcrumbs without losing early faults before cog.log is even online.

24.4. Autonomous Probes & Safe-Mode Inspection (Block 4)

Sub-Assembly 4.1 maintains s_io_fb_probes[2] bound to cog.debug.fallback under the io_fallback.* namespace. When disk configuration corruption triggers emergency takeover, io_fallback_downgrade_safe_mode() emits structured fault cards across both log_fallback and debug_fallback so DEBUG_STEP_AWAIT pauses inspection cleanly on the root cause rather than masking missing files.

Sub-Assembly 4.2 verifies ROM vault integrity, io_fallback_config_t sanity, and active fault counts before signaling readiness up to io_audit().

24.5. BIOS Boot, Floppy Mode & Deterministic Teardown (Block 5)

*Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch (*engine/src/io/io_fallback.c#L3309-L3332 · *Mode:* Group*)*

```
3309 bool io_fallback_boot_floppy(void)
3310 {
3311     io_fallback_config_t floppy = s_io_floppy_template;
3312     if (io_fallback_load_rom_config("floppy_safe", &floppy) ||
```

```
3313         io_fallback_load_rom_config("default", &floppy)) {
3314         s_io_fb.rom_hydrated = true;
3315     } else {
3316         s_io_fb.rom_hydrated = false;
3317     }
3318     floppy.read_only_mode = true;
3319     floppy.loaded_from_data = false;
3320     if (floppy.profile_name[0] == '\0' || strcmp(floppy.profile_name, "default") == 0) {
3321         str_safe_copy(floppy.profile_name, IO_FALLBACK_FLOPPY_PROFILE_NAME,
3322                       sizeof(floppy.profile_name));
3323     }
3324
3325     s_io_fb.config = floppy;
3326     io_fb_hydrate_subpolicies();
3327     s_io_fb.active_sink = IO_FB_SINK_FLOPPY_RO;
3328     str_safe_copy(s_io_fb.active_root, IO_FB_DEFAULT_ROOT, sizeof(s_io_fb.active_root));
3329     s_io_fb.booted = true;
3330     io_fallback_register_debug_probes();
3331     return true;
3332 }
```

Zero-dependency Safe-Mode transitions ensuring deterministic boot even when disk configs are missing or corrupt.

Wholesale Listing: Complete verbatim code (#L2908-L3332) → Appendix: Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch.

Control Hook	Operational Role
io_fallback_boot()	Standard BIOS startup hydrating io_config_rom.h tables into s_io_fb.
io_fallback_boot_floppy()	Explicit zero-disk minimal boot mode testing zero external filesystem reliance.
io_fallback_downgrade_safe_mode()	Transitions active failure into read-only Safe-Mode ROM service and trips io_fallback.* debug probes.
io_fallback_teardown() / io_fallback_is_booted()	Shuts down the fallback chassis cleanly or queries live BIOS readiness state.
io_fallback_reset_for_testing()	Deterministic Layer 4 test reset scrubbing s_io_fb cleanly between unit/system test cases.

- Back to Layer 0 I/O Folder Index or Previous Module: master_loader.h + master_loader.c.
- Next document: Layer 4 I/O Testing & Downgrade Drills.

Chapter 25. I/O Verification Suite: Bounded I/O, VFS Routing & Downgrades Across All 4 Tiers

Source File: `docs/code/io/07_io_testing_and_downgrades.adoc`

The `io/` subsystem carries **8 dedicated CTest targets** stepping cleanly from isolated per-cog unit tests (`Unit`) up through public API simulation (`Functional`), cross-module orchestration (`Integrated`), and full missing-disk downgrade recovery (`System`).

Tier	CTest Target	Source File	Primary Verification Focus
Unit	`unit_fs_utils_test`	`tests/unit/io/test_fs_utils_unit.c`	`O(1)` byte-limit truncation checks, zero-NUL safety, and policy ceiling overrides.
Unit	`unit_path_finder_test`	`tests/unit/io/test_path_finder_unit.c`	3-tier domain precedence (`game/config` → `engine/config` → `presentation`), path traversal rejection, and `rom://` routing.
Unit	`unit_json_utils_test`	`tests/unit/io/test_json_utils_unit.c`	Single-pass span lookup, escape-sequence parsing, depth ⇐ `32` bounds, and token limits.
Unit	`unit_master_loader_test`	`tests/unit/io/test_master_loader_unit.c`	5-block header validation, strict `block_3_body` query isolation, and `0x0201`–`0x0207` error taxonomy.
Unit (BIOS)	`unit_io_fallback_test`	`tests/unit/io/test_io_fallback_unit.c`	`io_config_rom.h` ROM vault reads, `io_fallback_boot_floppy()`, shared route JSON parser, and `io_fallback_reset_for_testing()`.
Functional	`functional_io_test`	`tests/functional/test_io_functional.c`	External caller simulation driving `io.h` umbrella contracts and `master_loader_load()` end-to-end without touching internals.
Integrated	`integrated_io_test`	`tests/integrated/test_io_integrated.c`	Cross-subsystem plumbing uniting `io/` + `log/` + `debug/` probe callbacks + ROM config hydration cleanly.
System	`system_io_test`	`tests/system/test_io_system.c`	Full system boot/audit loops, malformed payload downgrades, `DEBUG_STEP_AWAIT` capture, and recovery determinism.

25.1. Crude-to-Fine Band Design

Every CTest suite in `io/` executes through the shared `tests/common/cornerstone_test.h` harness (`test.common.harness`), partitioning assertions into progressively tighter assurance bands:

Band	Verification Question Answered
always_pass	Can io_init() / io_fallback_boot() initialize safely from clean .bss without crashing or touching the heap?
never_fail	Do oversized payloads (> 65536 B), NULL buffers, .. traversal attempts, and invalid block_3_body spans return safe typed errors instead of faulting?
assertions_and_guarantees	Do disk overrides (game/config) outrank engine/config, and do missing files increment rom_loads with loaded_from_rom = true identically?
emergence	When chained across real subsystem boundaries, do VFS lookup, 5-block JSON parsing, logging, and probe triggers compose cleanly without state leaks?
preflight_audit / audit	Do io_preflight(), io_audit(), and master_loader_audit() pass cleanly with deterministic post-run counters and clean teardown?

25.2. System-Tier Emergence & Downgrade Snapshot

tests/system/test_io_system.c exercises complete cross-subsystem initialization, missing/corrupt contract escalation through io.loader.* probes, and deterministic verification via cornerstone_test.h:

I/O Pipeline System Emergence & ROM Downgrade Verification (test_io_system_emergence)
(tests/system/test_io_system.c#L328-L367 · Mode: Function)

```c
/**
 * @brief Verifies sandboxed mount redirection and live v1 -> v2 JSON hot-patch reflection.
 */
static void test_io_system_emergence(void)
{
    printf("  [EMERGE-4] Running Section 4: Emergence (Dynamic Mount & Live Hot-Patching)...\n");

    const char *hotpatch_dir = "build/system_test_patch/system";
    const char *hotpatch_file = "build/system_test_patch/system/runtime_patch.json";

    TEST_ASSERT(fs_ensure_directory(hotpatch_dir), "Synthesized hot-patch directory");

    static const char patch_v1[] =
        "{\n"
        "  \"block_1_metadata\": {\"project\": \"Tenebris and Lux\", \"file\": \"runtime_patch.json\"},\n"
        "  \"block_2_types\": {\"schema_id\": \"tenlux.data.patch.v1\"},\n"
        "  \"block_3_body\": {\"difficulty_mult\": 1.0, \"active_boss\": \"Usuru\"}\n"
        "}\n";
    TEST_ASSERT(fs_write_file_contents(hotpatch_file, patch_v1, strlen(patch_v1)),
                "Wrote patch v1");

    path_finder_init();
    master_loader_init();
    path_finder_set_data_root("build/system_test_patch");

    master_loader_t loader;
    bool l1 = master_loader_execute_load_verified("system", "runtime_patch.json",
                                                  "tenlux.data.patch.v1",
                                                  s_io_sys_load_buf,
                                                  sizeof(s_io_sys_load_buf),
                                                  &loader);
    TEST_ASSERT(l1 && loader.status == LOADER_STATUS_OK, "Loaded patch v1");
    float diff1 = 0.0f;
    char boss1[64];
    TEST_ASSERT(master_loader_get_float(&loader, NULL, "difficulty_mult", &diff1) &&
                diff1 >= 0.999f && diff1 <= 1.001f,
                "Patch v1 difficulty_mult == 1.0");
    TEST_ASSERT(master_loader_get_string(&loader, NULL, "active_boss", boss1, sizeof(boss1)),
                "Patch v1 boss extracted");
```

```
367     TEST_ASSERT_STR_EQ(boss1, "Usuru", "Boss is authentic Dark Lord Usuru");
```

Drills end-to-end VFS resolution, corrupt JSON downgrade to ROM Safe Mode, and coordinated umbrella audit.

Wholesale Listing: Complete verbatim code (#L328-L399) → Appendix: I/O Pipeline System Emergence & ROM Downgrade Verification (test_io_system_emergence).

25.3. Running & Auditing the io/ Test Suite

```
cmake -B build && cmake --build build -j
ctest --test-dir build \
  -R "(fs_utils|path_finder|json_utils|master_loader|io_fallback|_io_)" --output-on-failure
```

If any io/ probe fires unexpectedly during a run, tests/common/cornerstone_test.h prints the forensic debug / debug_fallback ring snapshot and pinpoints the exact test file, band, and DEBUG_STEP_AWAIT code.

- Up one level: Layer 0 I/O Folder Index | Layer 0 I/O Subsystem Overview.
- Peer testing suites: log/ 4-Tier Testing | debug/ 4-Tier Testing.

Chapter 26. Layer 0 Core Utilities Code Tour

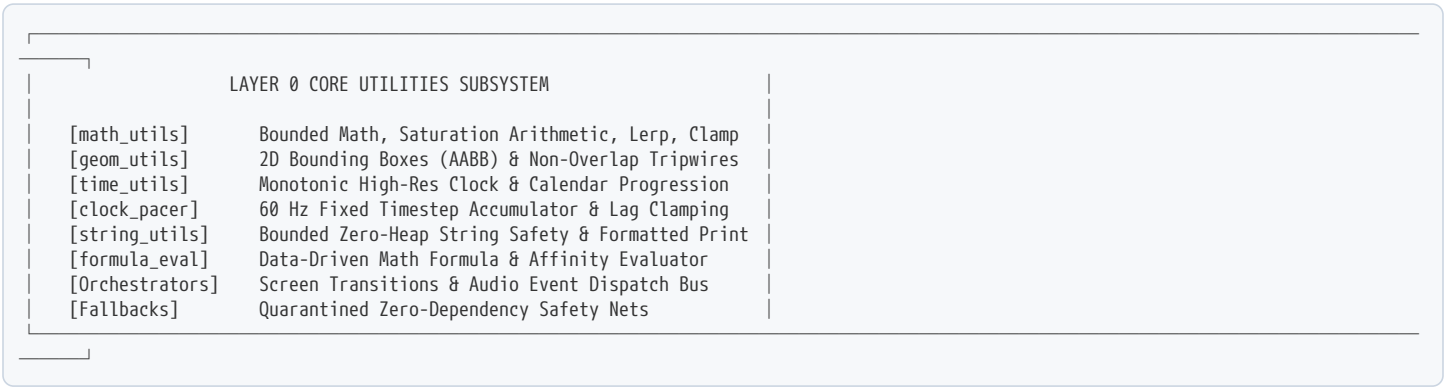
Source File: `docs/code/core_utilities.adoc`

26.1. Overview

This document provides a technical walkthrough and architectural manual for the **Layer 0 Core Utilities Subsystem** (`engine/include/core/` and `engine/src/core/`). Representing the mathematical, geometric, temporal, and string foundations of **The Big Machine**, the Core Utilities stratum provides bounded arithmetic, 2D AABB geometry, monotonic clock pacing, fixed-timestep simulation accumulators, memory-bounded string manipulation, and data-driven formula evaluation. Strict adherence to `ALLOC_ZERO_HEAP`, reentrancy, sub-microsecond latency budgets, and quarantined emergency fallbacks ensure that foundational operations never wrap around integer bounds, leak memory, or introduce non-deterministic time dilation.

26.2. Architectural Philosophy & Mission

At the very bedrock of the engine frame lies Layer 0 Core Utilities:



26.2.1. Key Architectural Pillars

1. **Zero Dynamic Heap Allocations (`ALLOC_ZERO_HEAP`)**: Pure reentrant C11 functions operating solely on CPU registers, primitive values, and caller-provided stack buffers.
2. **Saturation Arithmetic & Overflow Clamping**: Mathematical utilities enforce saturation clamping; calculations never wrap around `INT32_MAX` or underflow `INT32_MIN`.
3. **Decoupled Simulation Timestep**: `clock_pacer` decouples fixed simulation updates (exactly 60 Hz / 16.666ms) from variable physical render frame rates, calculating fractional interpolation alphas (`render_alpha`).
4. **Data-Driven Formula Evaluation**: Formulas for damage, mitigations, and affinity interactions are evaluated through `formula_eval` driven by JSON manifests in `game/data/system/runtime/math/` rather than hardcoding arithmetic in C sources.
5. **Quarantined Fallback Isolation**: Every core utility module is paired with a standalone, zero-dependency `*_fallback.[ch]` module ensuring zero-configuration compilation and hermetic test isolation.

26.3. Subsystem Layout & File Manifest

```
engine/include/core/
 ├─── math_utils.h             // Bounded math, clamping, interpolation, rounding
```

```
├──── math_utils_fallback.h     // Standalone minimal math fallback
├──── geom_utils.h              // 2D AABB geometry, point-in-rect, intersection
├──── geom_utils_fallback.h     // Standalone geometric fallback
├──── time_utils.h              // Monotonic clock querying & calendar time
├──── time_utils_fallback.h     // Standalone monotonic time fallback
├──── clock_pacer.h             // Fixed-timestep simulation pacer & lag defense
├──── clock_pacer_fallback.h    // Minimal fixed-tick accumulator fallback
├──── string_utils.h            // Safe bounded string copying, format & trimming
├──── string_utils_fallback.h   // Minimal string safety fallback
├──── formula_eval.h            // Data-driven formula evaluator
├──── formula_eval_fallback.h   // Static arithmetic evaluation fallback
├──── audio_bus.h               // Universal audio event dispatch bus
└──── screen_transition.h       // Screen fading & transition orchestrator

engine/src/core/
├──── math_utils.c              // Mathematical implementations & saturation clamps
├──── math_utils_fallback.c     // Quarantined math fallback implementation
├──── geom_utils.c              // AABB collision and bounds validation
├──── geom_utils_fallback.c     // Quarantined geometry fallback implementation
├──── time_utils.c              // POSIX clock_gettime monotonic queries
├──── time_utils_fallback.c     // Quarantined time query fallback
├──── clock_pacer.c             // Timestep accumulation & render interpolation
├──── clock_pacer_fallback.c    // Quarantined clock pacer fallback
├──── string_utils.c            // Safe bounded string functions
├──── string_utils_fallback.c   // Quarantined string fallback implementation
├──── formula_eval.c            // Expression evaluation and parameter binding
├──── formula_eval_fallback.c   // Quarantined formula evaluator fallback
├──── audio_bus.c               // Audio cue dispatch orchestrator
└──── screen_transition.c       // Transition easing curve evaluator
```

26.4. Core Engine Cogs & API Tour

26.4.1. Bounded Math & Saturation Arithmetic (`math_utils.[ch]`)

Guarantees that mathematical calculations remain strictly bounded within defined intervals:

```c
/* Clamping primitives */
int32_t math_clamp_i32(int32_t val, int32_t min, int32_t max);
float   math_clamp_float(float val, float min, float max);
double  math_clamp_double(double val, double min, double max);

/* Saturation arithmetic preventing integer overflow / underflow */
int32_t math_sadd_i32(int32_t a, int32_t b);
int32_t math_ssub_i32(int32_t a, int32_t b);

/* Linear interpolation and percentage scaling */
float   math_lerp_float(float a, float b, float t);
int32_t math_scale_pct(int32_t base, int32_t pct);
```

26.4.2. 3.2 2D AABB Geometry & Layout Tripwires (`geom_utils.[ch]`)

Defines 2D axis-aligned bounding boxes (AABB) for desktop HUD panels, button hits, and tactical grid coordinates:

```c
typedef struct {
    int32_t x;
    int32_t y;
    int32_t width;
    int32_t height;
} geom_rect_t;

/* Collision and containment queries */
bool geom_rect_contains_point(const geom_rect_t *r, int32_t px, int32_t py);
bool geom_aabb_intersects(const geom_rect_t *a, const geom_rect_t *b);
bool geom_rect_fits_inside(const geom_rect_t *inner, const geom_rect_t *outer);
```

Invariants Enforced: * Rect widths and heights must be non-negative. * HUD panel manifests are audited against

`geom_aabb_intersects` to prevent layout overlaps.

26.4.3. Monotonic Time & Clock Pacing (`time_utils.[ch]`, `clock_pacer.[ch]`)

Provides nanosecond-precision monotonic timing and decouples simulation updates from render FPS:

```
/* High-resolution monotonic clock query */
uint64_t time_monotonic_now_ns(void);
double   time_delta_ms(uint64_t start_ns, uint64_t end_ns);

/* Fixed-timestep clock pacer */
typedef struct {
    double fixed_step_ms;       /* Standard: 16.666667 ms (60 Hz) */
    double accumulator_ms;     /* Accumulated unprocessed frame time */
    double max_delta_clamp_ms; /* Maximum lag spike clamp (100.0 ms) */
    uint32_t max_substeps;     /* Max simulation steps per render (3) */
} clock_pacer_t;

void clock_pacer_init(clock_pacer_t *pacer, double step_ms, double clamp_ms, uint32_t max_substeps);
bool clock_pacer_tick(clock_pacer_t *pacer, double frame_delta_ms, double *out_render_alpha);
```

Lag Spike Defense: * If an OS hitch stalls execution (> 100ms), `clock_pacer_tick` clamps the delta to `max_delta_clamp_ms` and executes at most `max_substeps` (3 ticks). This prevents the dreaded "spiral of death" where the CPU stalls trying to catch up with accumulated time debt.

26.4.4. Bounded String Safety (`string_utils.[ch]`)

Eliminates buffer overflow vulnerabilities (`strcpy`, `strcat`, `sprintf` are strictly forbidden):

```
/* Guaranteed null-terminated bounded copy */
size_t str_copy_safe(char *dest, size_t dest_size, const char *src);

/* Bounded string concatenation */
size_t str_concat_safe(char *dest, size_t dest_size, const char *src);

/* In-place whitespace trimming and case conversion */
void str_trim(char *str);
void str_to_lower(char *str);
void str_to_upper(char *str);
```

26.4.5. Data-Driven Formula Evaluator (`formula_eval.[ch]`)

Parses and computes bounded mathematical formulas loaded from `formulas.json` (e.g. physical attack, magical mitigation, poise damage):

```
typedef struct {
    int32_t attacker_atk;
    int32_t defender_def;
    int32_t weapon_might;
    int32_t stance_mult_pct;
    int32_t affinity_mult_pct;
} formula_combat_params_t;

/* Computes final damage ensuring result is clamped to [1, 9999] */
int32_t formula_eval_physical_damage(const formula_combat_params_t *params);
```

26.4.6. Universal Engine Orchestrators

- **Audio Event Bus (`audio_bus.[ch]`)**: Zero-heap queue routing audio cue events (`sfx_ward_lux_barrio`, `sfx_strike_cuttatio_tenebris`) to mixing channels without directly binding simulation to hardware sound drivers.
- **Screen Transition Orchestrator (`screen_transition.[ch]`)**: Manages screen fading, wipes, iris dissolves, and easing curves (`EASE_LINEAR`, `EASE_QUAD_IN`, `EASE_CUBIC_OUT`) between scene and screen changes.

26.5. Verification & Testing Matrix

The Core Utilities subsystem is continuously verified across 4 ontological test tiers: 1. **Functional Tests (`tests/functional/test_core_functional.c`)**: Verifies saturation arithmetic, bounding rect containment, and string truncation. 2. **Unit Tests (`tests/unit/core/test_*_unit.c` and `*_fallback_test.c`)**: Individual unit test binaries for each of the 6 core modules and their fallback companions. 3. **Integrated Tests (`tests/integrated/test_core_integrated.c`)**: Verifies multi-cog cooperation (pacer driving formula evaluator and geometry updates). 4. **System Tests (`tests/system/test_core_system.c`)**: Stress tests clock pacing across 60 FPS, 144 FPS, and simulated 100ms hitch drops, confirming strict determinism and zero dynamic memory leaks.

Chapter 27. Layer 1 Simulation Engine Code Tour

Source File: `docs/code/simulation_engine.adoc`

27.1. Overview

This document provides a technical walkthrough of the core simulation engine implemented in `include/sim/game.h` and `src/sim/game.c`. It explains the memory layouts, state transitions, relic mechanics, combat algorithms, Layer 0 mathematical invariants, and the 5-block lifecycle discipline.

27.2. State Structure & Memory Layout (`player_state_t`)

The heart of the simulation is defined in `include/sim/game.h`:

```
typedef struct {
    int shield_power;           // [0..100] Barrier integrity & Lux charge
    int sword_edge;             // [0..100] Blade sharpness & Tenebris charge
    int moral_sway;             // [-100..+100] Moral alignment axis
    int threat_hp;              // [0..100] Threat vigor of active foe
    combat_stance_t stance;     // Active stance: warding, striking, equilibrium
    location_id_t location;     // Current Kingdom province
    bool is_running;            // Game loop active flag
    int turns_taken;            // Total turns / actions taken
    int foes_vanquished;        // Number of vanguard defeated
    int innocents_warded;       // Number of allies protected
} player_state_t;
```

All state variables are strictly bounded integers with zero dynamic allocations on the heap. This ensures deterministic cache locality, zero memory leaks, and instant serialization.

27.3. Universal Action Dispatcher (`game_process_command`)

Located in `src/sim/game.c`, `game_process_command()` acts as the singular entry point for all player intentions, whether emitted by the terminal CLI or the native GUI:

```
void game_process_command(player_state_t *state, const char *raw_input,
    char *out_msg, size_t msg_len);
```

27.3.1. Execution Pipeline:

1. **Input Sanitization**: Copies `raw_input` safely via `str_safe_copy()` and executes in-place whitespace trimming (`str_trim_inplace()`).
2. **Verb Tokenization**: Splits input into `command` and trailing `arg`.
3. **Action Dispatch**:
   - `ward`: Calls `handle_ward()`, modifying `shield_power` via `math_saturating_add()`, staggering active threats with `math_saturating_sub()`, and elevating moral virtue with `math_clamp_int()`.
   - `strike`: Calls `handle_strike()`, spending `sword_edge` via `math_saturating_sub()` to inflict heavy severance damage.
   - `balance`: Calls `handle_balance()`, simultaneously recovering both relics, parrying blows, and nudging moral sway toward 0 via `math_nudge_toward()`.
   - `stance`: Calls `handle_stance()`, shifting the tactical state machine.
   - `move`: Calls `handle_move()`, validating directional topology and spawning province encounters.
4. **Invariant Logging**: Writes a human-readable telemetry outcome into `out_msg` for presentation.

27.4. Diagnostic Self-Auditing (game_audit_state)

Following Block 4 of the OmniCode protocol, src/sim/game.c includes built-in diagnostic audit routines:

```
game_audit_report_t game_audit_state(const player_state_t *state);
int game_run_self_audit(void);
```

Every state boundary (shield power, sword edge, moral sway, stance, location) is checked against compile-time limits. If an invariant is violated, an audit report is generated with exact error messages, enabling automated continuous integration testing without relying on crude process crashes.

Chapter 28. Layer 2 Presentation Viewport Code Tour

Source File: docs/code/presentation_gui.adoc

28.1. Overview

This document provides a comprehensive architectural and code walkthrough of the modular Layer 2 Presentation engine implemented in engine/include/viewport/ and engine/src/viewport/. Decomposed into four specialized sub-assemblies (viewport/raster/, viewport/hud/, viewport/reel/, viewport/styling/), the system implements the **Movie Reel Architecture**: a continuous cinematic scene film strip traversed via horizontal Push/Pull transitions, with vertical Pop-Out/Sink-In depth kinematics, high fantasy palette blending, and a 60 FPS double-buffered X11 presentation pump.

28.2. The Movie Reel Mental Model & Subfolder Strata

The viewport subsystem is structured into four cohesive sub-assemblies matching its mechanical mental model:

```
engine/
├──── include/viewport/
│    ├──── viewport.h             <-- Master Viewport Umbrella Header
│    ├──── raster/                <-- Low-level double-buffered X11 rasterization & 2.5D math
│    │    ├──── gui.h
│    │    ├──── gui_types.h
│    │    └──── gui_view2d.h
│    ├──── hud/                   <-- Desktop HUD panels, layout bounding boxes, and action dock
│    │    ├──── gui_hud.h
│    │    ├──── gui_layout.h
│    │    └──── gui_widgets.h
│    ├──── reel/                  <-- The Movie Reel engine & canonical screen frames
│    │    ├──── reel.h
│    │    └──── gui_screens.h
│    └──── styling/               <-- Visual palettes, TrueType fonts, and asset abstraction
│         ├──── gui_palette.h
│         ├──── gui_fonts.h
│         └──── gui_assets.h
└──── src/viewport/
     ├──── raster/
     ├──── hud/
     ├──── reel/
     └──── styling/
```

Sub-Assembly	Source Stratum	Architectural Mission
raster/	gui.c, gui_view2d.c	Double-buffered X11 context, 2.5D diamond isometric projection, letterbox/pillarbox scalers.
hud/	gui_hud.c, gui_layout.c, gui_widgets.c	Desktop HUD, 5 disjoint panels, non-overlapping AABB audits, interactive action dock.
reel/	reel.c, gui_screens.c	The Movie Reel orchestrator: horizontal scene strip (push/pull) and vertical modal depth (pop out/sink in).
styling/	gui_palette.c, gui_fonts.c, gui_assets.c	Radiant hope colormaps, TrueType typography (Cinzel, MedievalSharp), procedural vector art abstraction.

28.3. The Movie Reel: Horizontal Motion & Vertical Depth

The core conceptual paradigm of the Rev 4 presentation engine is the **Movie Reel**:

```
                          POP OUT (Z-Depth / Up)
                          ▲ [Modals, Inspection Cards, Dialogues]
                          |
[Scene 0: TACTICAL] <==== PUSH / PULL ====> [Scene 1: STATUS] <=====> [Scene 2: INVENTORY] ...
(Film Cell Frame)         (Left / Right)     (Film Cell Frame)        (Film Cell Frame)
                          |
                          ▼ [Terrain Floor, Fog, Sunken Canvas]
                          SINK IN (Z-Depth / Down)
```

28.3.1. Horizontal Axis: Push and Pull (Left and Right)

Scenes and screens are treated as consecutive frames along a physical celluloid reel strip: * **Push Right (`REEL_DIR_PUSH_RIGHT`)**: Advancing forward along the reel to a subsequent screen (e.g., from `GUI_SCREEN_TACTICAL` [F1] to `GUI_SCREEN_STATUS` [F2]). The active scene slides leftward as the new scene arrives from the right. * **Pull Left (`REEL_DIR_PULL_LEFT`)**: Rewinding backward along the reel to a prior screen (e.g., returning from `GUI_SCREEN_STATUS` [F2] back to `GUI_SCREEN_TACTICAL` [F1]). The active scene slides rightward as the prior scene is pulled back in. * **Smooth-Step Kinematics**: Horizontal transition progress is evaluated using normalized Hermite interpolation ($S(t) = 3t^2 - 2t^3$) scaled to the viewport width, guaranteeing fluid motion without visual tearing.

28.3.2. Vertical Axis: Pop Out and Sink In (Up and Down)

Within any active scene on the reel, visual elements possess continuous Z-elevation depth: * **Pop Out (`REEL_DEPTH_POPPED = +1`)**: Elements elevate upward toward the viewer ($\Delta Y = -16\text{ px}$, scale $1.05\times$, full 1.0 opacity, drop shadow). Used for dialogue boxes, relic inspection modals, victory fanfares, and action tooltips. * **Sink In (`REEL_DEPTH_SUNKEN = -1`)**: Elements submerge downward into the background canvas ($\Delta Y = +12\text{ px}$, scale $0.95\times$, attenuated 0.60 opacity, ambient recess). Used for background terrain tiles, ambient fog layers, and sunken meters when a modal pops out. * **Surface (`REEL_DEPTH_SURFACE = 0`)**: The neutral baseline scene plane ($\Delta Y = 0$, scale 1.0, alpha 1.0).

28.4. The 2.5D Tactical Isometric Engine

The tactical battlefield features dual visual projection modes toggleable via `[M]`: * `GUI_VIEW_2D_FLAT`: Classic orthographic top-down grid (38×38 px cells) for overworld travel. * `GUI_VIEW_25D_ISOMETRIC`: Diamond dimetric projection (48×24 px tiles) with vertical Z-axis elevation prisms (8 px per elevation level) for tactical skirmishes.

28.4.1. Isometric Projection & Unprojection Reciprocity:

$$\text{screen_x} = X_0 + (gx - gy) \cdot \frac{W}{2}$$

$$\text{screen_y} = Y_0 + (gx + gy) \cdot \frac{H}{2} - gz \cdot E$$

Unprojecting mouse clicks (sx, sy) on ground plane gz back to discrete grid (gx, gy):

$$gx = \left\lfloor \frac{sx - X_0}{W} + \frac{sy + gz \cdot E - Y_0}{H} + 0.5 \right\rfloor$$

$$gy = \left\lfloor \frac{sy + gz \cdot E - Y_0}{H} - \frac{sx - X_0}{W} + 0.5 \right\rfloor$$

28.5. Non-Anonymous Type Invariant Covenant

In strict compliance with engine architectural invariants, **all structs and enums** throughout the viewport and reel strata are explicitly tagged: * `typedef enum gui_screen_mode { ⋯ } gui_screen_mode_t;` * `typedef enum reel_direction { ⋯ } reel_direction_t;` * `typedef enum reel_state { ⋯ } reel_state_t;` * `typedef enum reel_depth { ⋯ } reel_depth_t;` * `typedef struct reel_depth_element { ⋯ } reel_depth_element_t;` * `typedef struct reel { ⋯ } reel_t;` * `typedef struct gui_palette { ⋯ } gui_palette_t;` * `typedef struct gui_layout { ⋯ } gui_layout_t;` * `typedef struct gui_button { ⋯ } gui_button_t;`

28.6. Verification Telemetry

The presentation stratum is verified by `tests/unit/test_gui_unit.c` and `tests/unit/viewport/test_reel_unit.c` across the four ontological testing tiers: 1. **Section 1 (Always Pass)**: Baseline layout bounds, reel default state, and 9-button command dock. 2. **Section 2 (Never Fail)**: Out-of-bounds click safety, negative duration clamping, and element capacity limits. 3. **Section 3 (Axioms & Tripwires)**: AABB panel disjointness, projection reciprocity, push/pull kinematics, and pop-out/sink-in elevation vectors. 4. **Section 4 (Emergence)**: Full `gui_audit_layout()` and `reel_audit()` preflight execution.

Chapter 29. Layer 0/2 Audio Event Bus & Mixer Subsystem

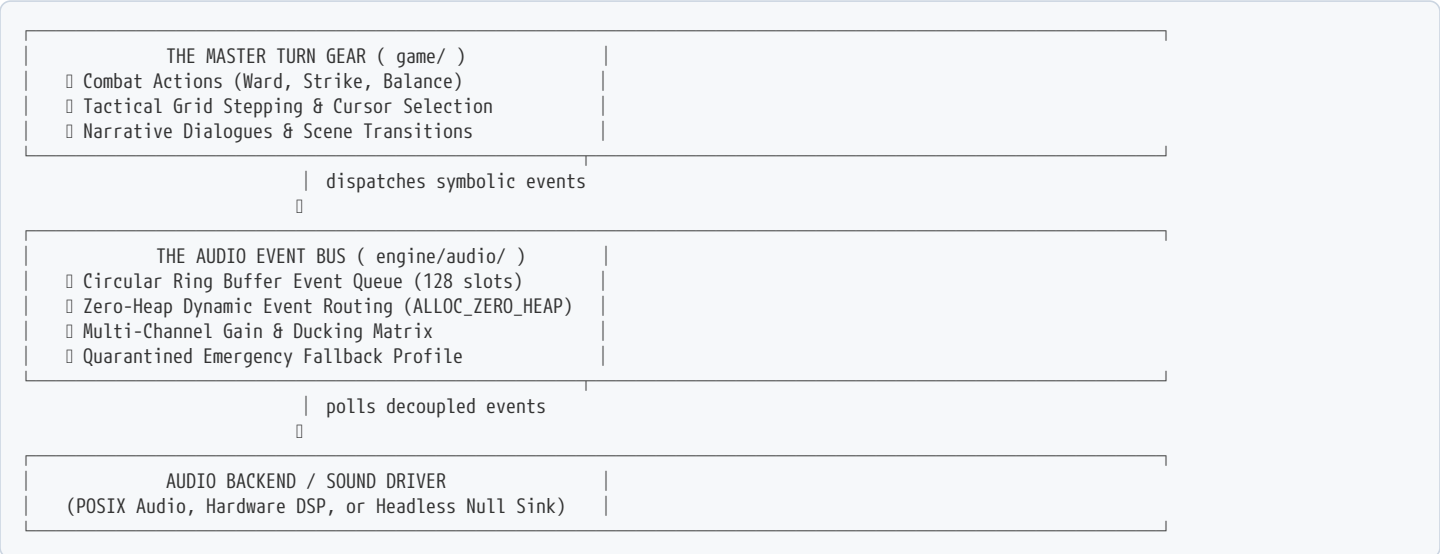
Source File: docs/code/audio_subsystem.adoc

29.1. Overview

This document provides a comprehensive architectural and code walkthrough of the dedicated Audio subsystem implemented in engine/include/audio/ and engine/src/audio/. Operating as an essential stratum of **The Big Machine**, the audio subsystem provides a multi-channel mixing bus (Master, BGM, Ambience, Combat SFX, Tactical SFX, UI, Stingers), a zero-heap circular ring buffer event queue, bounded volume attenuation, high-priority ducking matrices, and a quarantined monolithic fallback profile.

29.2. Architectural Philosophy: The Decoupled Audio Machine

In traditional game engines, audio drivers are often tightly coupled to simulation loops or low-level OS sound APIs (ALSA, PulseAudio, WASAPI). In **Tenebris and Lux**, the audio architecture strictly obeys the **Machine vs. Master Turn Gear Separation**:



29.2.1. Axiomatic Invariants:

1. **Zero Dynamic Heap Allocations (ALLOC_ZERO_HEAP)**: The event bus allocates zero bytes on the heap; all queues, channels, and cue manifests reside in static, reentrant memory.
2. **Read-Only Simulation Influence**: Audio strictly follows simulation events. Audio playback never gates, blocks, or alters simulation state transitions or turn resolution.
3. **Hardware Limiter & Bounded Gain**: All channel volume levels are strictly clamped to $[0.0, 1.0]$.
4. **All Structs and Enums Explicitly Named**: Zero anonymous structs or enums.

29.3. Subsystem File Layout & Header Structure

The audio subsystem is cleanly partitioned into public interfaces, implementations, and fallback:

```
engine/
├─── include/audio/
|    ├─── audio.h             <-- Master Audio Umbrella Header
|    ├─── audio_types.h       <-- Channel enums, event packets, cue manifests, stats
|    ├─── audio_bus.h         <-- Public event posting, polling, channel volume API
```

```
│   └──── audio_fallback.h     <-- Quarantined emergency fallback profile header
└──── src/audio/
    ├──── audio_bus.c          <-- Circular ring buffer, channel gain, event routing
    └──── audio_fallback.c     <-- Monolithic emergency fallback profile implementation
```

29.4. Multi-Channel Mixer Architecture

The audio bus implements seven canonical mixing channels matching `game/data/presentation/audio/channels.json`:

Channel Enum	Canonical Channel Name	Default Gain	Role & Attenuation Behavior
`AUDIO_CHANNEL_MASTER`	Master Output Bus	100%	Controls overall presentation master gain.
`AUDIO_CHANNEL_BGM`	Background Music	75%	Loopable orchestral streams; ducked under stingers.
`AUDIO_CHANNEL_AMBIENCE`	Soundscapes & Ambience	60%	Environmental biome loops (wind, meadow hum, dunes).
`AUDIO_CHANNEL_SFX_COMBAT`	Combat Sound Effects	90%	Strikes, spell chants, Lux Barrio barrier wards.
`AUDIO_CHANNEL_SFX_TACTICAL`	Tactical Grid Audio	80%	Footsteps, tile selection, zone-of-control pings.
`AUDIO_CHANNEL_UI`	User Interface	85%	Button clicks, tab switches, stance toggle cues.
`AUDIO_CHANNEL_STINGER`	Fanfares & Stingers	95%	Victory stingers, battle intro fanfares (ducking source).

29.5. Non-Anonymous Ontological Types

In strict conformance with project governance standards, all data structures and enumerations are explicitly named:

```c
typedef enum audio_channel {
    AUDIO_CHANNEL_MASTER = 0,
    AUDIO_CHANNEL_BGM,
    AUDIO_CHANNEL_AMBIENCE,
    AUDIO_CHANNEL_SFX_COMBAT,
    AUDIO_CHANNEL_SFX_TACTICAL,
    AUDIO_CHANNEL_UI,
    AUDIO_CHANNEL_STINGER,
    AUDIO_CHANNEL_COUNT
} audio_channel_t;

typedef enum audio_event_type {
    AUDIO_EVENT_NONE = 0,
    AUDIO_EVENT_PLAY_SFX,
    AUDIO_EVENT_STOP_SFX,
    AUDIO_EVENT_PLAY_BGM,
    AUDIO_EVENT_STOP_BGM,
    AUDIO_EVENT_CROSSFADE_BGM,
    AUDIO_EVENT_PLAY_AMBIENCE,
    AUDIO_EVENT_STOP_AMBIENCE,
    AUDIO_EVENT_CROSSFADE_AMBIENCE,
    AUDIO_EVENT_PLAY_STINGER,
    AUDIO_EVENT_SET_VOLUME,
    AUDIO_EVENT_MUTE_CHANNEL,
    AUDIO_EVENT_STOP_ALL
} audio_event_type_t;
```

```
typedef struct audio_event {
    audio_event_type_t type;
    audio_channel_t channel;
    char cue_id[AUDIO_MAX_CUE_ID_LEN];
    float volume;
    float pan;
    float pitch;
    float fade_duration_sec;
    bool loop;
    uint64_t timestamp_us;
} audio_event_t;
```

29.6. Verification Telemetry

The audio subsystem is validated by two dedicated unit test harnesses: * `tests/unit/audio/test_audio_bus_unit.c`: Validates channel default gains, null guards, volume clamping, 128-event ring buffer capacity, FIFO queue order preservation, and high-watermark telemetry. * `tests/unit/audio/test_audio_fallback_unit.c`: Validates the quarantined emergency fallback profile, provenance tagging (`loaded_from_data == false`), and fallback profile application to the live bus.

Chapter 30. Layer 4 Verification & Test Suite Code Tour

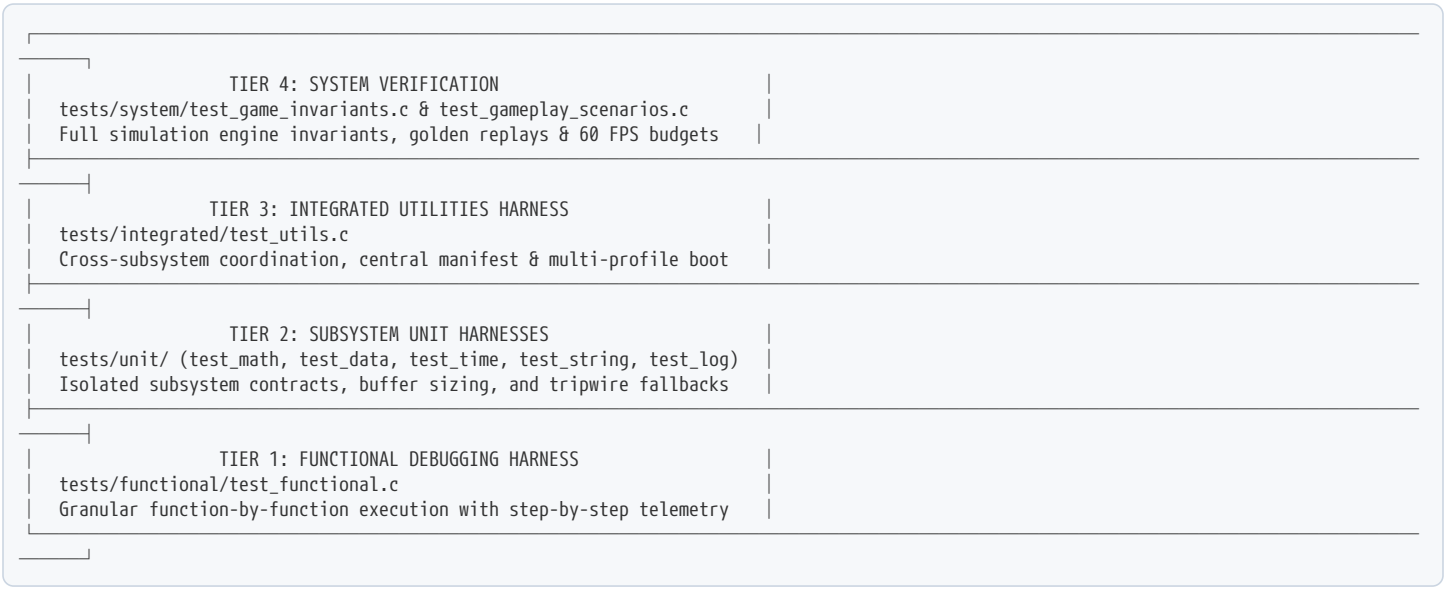
Source File: `docs/code/test_suite.adoc`

30.1. Overview

This document provides a comprehensive technical walkthrough of the automated game development verification architecture in **Tenebris and Lux**. Organized into strict hierarchical strata across `tests/`, the test suite mirrors the Layer 0-4 engine ontology and enforces four distinct ontological verification sections within every test harness:

1. **Always Pass (Nominal Golden Baseline)**: Verifies nominal parameters, default asset loading, and canonical execution paths.
2. **Never Fail (Fault-Tolerant Tripwires)**: Injects boundary breaches, NULL pointers, missing files, and corrupt states to prove that the engine never crashes or invokes undefined behavior.
3. **Assertion & Guarantee (Axiomatic Invariants)**: Mathematical guardrails, energy conservation, monotonic clocks, and DAG dependencies guaranteed by engine contracts.
4. **Emergence (Stochastic Stress & Chained Pipelines)**: Emergent combinatorial state spaces, 1,000 chaotic command fuzzing, bitwise golden replay determinism, and sub-50-microsecond simulation latency budgets.

30.2. The Four-Tier Testing Hierarchy



30.2.1. Tier 1: Functional Debugging (`tests/functional/`)

Designed specifically for interactive debugging and pinpointing anomalies in isolated functions. Emits explicit `[DEBUG-TRACE]` entries showing arguments before and after execution for each individual cog (e.g. `math_clamp_int`, `math_saturating_add`, `str_safe_copy`, `str_trim_inplace`, `json_get_string`, `combat_get_stance_name`).

30.2.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`)

Provides 1:1 test coverage for each dedicated Layer 0 utility subsystem folder: * **`test_math.c`**: Bounded arithmetic, saturating additions/subtractions, step-bounded equilibrium nudging, proportional ratio scaling, 2D discrete AABB hit-testing, and `data/math/default.json` profile loading. * **`test_data.c`**: Zero-heap filesystem I/O (`fs_utils.c`), canonical buffer sizing (`FS_BUFFER_SMALL_BYTES` = 1024 to `FS_BUFFER_MAX_CONFIG` = 65536), bounded file reading/writing, scoped JSON

object querying, and array iteration. * `test_time.c`: Monotonic millisecond/microsecond clocks, POSIX nanosleep frame pacing, ISO 8601 timestamp generation ("YYYY-MM-DDTHH:MM:SSZ"), and 60 FPS / 120 FPS profile loading. * `test_string.c`: Buffer-safe bounded copying, in-place whitespace trimming, case-insensitive equality/substring matching, parent directory extraction, and duplicate-slash immune path joining. * `test_log.c`: Forensic telemetry, graded severity filtering, categorized logging channels, in-memory static ring buffer, and emergency monolithic code page fallbacks. * `test_debug.c`: Interactive state debugger, frozen snapshot ring buffer, FNV-1a checksum determinism, time-travel state rewind, state diffing, dynamic breakpoints, and emergent chaotic stress.

30.2.3. Tier 3: Integrated Utilities Harness (`tests/integrated/`)

- `test_utils.c`: Validates cross-subsystem coordination of all Layer 0 utilities. Exercises the central manifest registry (`manifest.c`), validates upstream DAG dependency resolution, enforces diagnostic escalation policies (WARNING → `PREFLIGHT_STATUS_WARNINGS`, CRITICAL → `PREFLIGHT_STATUS_DOWNGRADED`, FATAL → `PREFLIGHT_STATUS_HALT_FATAL`), and benchmarks multi-profile synchronous boot latency (< 5 milliseconds).

30.2.4. Tier 4: System Verification (`tests/system/`)

- `test_game_invariants.c`: Validates Layer 1 simulation engine state invariants: twin relic power bounds ($[0, 100]$), moral sway continuum boundaries ($[-100, +100]$), protagonist name customization, and bidirectional provincial map topology.
- `test_gameplay_scenarios.c`: Simulates end-to-end tactical combat routes (Righteous Warden and Umbral Striker paths), 15-step golden deterministic replay parity, 1,000 chaotic command fuzzing for softlock immunity, and 10,000-turn simulation latency benchmarking (< 50 microseconds per turn).

30.3. The Four Ontological Testing Sections

Every test harness in the repository implements the four-section ontological model:

Section	Category	Mission & Verification Scope
Section 1	**Always Pass**	Valid inputs, nominal parameters, default configuration loading, standard state initialization, well-formed JSON, normal gameplay actions.
Section 2	**Never Fail**	Inviolability under error. Injects NULL pointers, zero capacities, missing files, corrupted JSON, and unknown stances. Proves tripwire activation and safe fallback without OS traps.
Section 3	**Assertion & Guarantee**	Axiomatic mathematical laws and structural contracts: bounded clamping, non-decreasing monotonic clocks, strict null-termination under truncation, and deterministic stance completeness.
Section 4	**Emergence**	Higher-order systemic behaviors: 1,000 pseudo-random fuzzing iterations, spatial grid overlap emergence, golden replay bitwise equality across runs, and < 50 us latency budgets.

30.4. CMake & CTest Test Targets

The build configuration registers twelve automated test targets within `CMakeLists.txt`:

```
# CLI Interface Contract Verification Tests
add_test(NAME cli_help_test COMMAND tenebris-lux --help)
add_test(NAME cli_version_test COMMAND tenebris-lux --version)
add_test(NAME subsystem_audit_test COMMAND tenebris-lux --audit)

# Tier 1: Functional Debugging Harness
add_test(NAME functional_cogs_test COMMAND test_functional)

# Tier 2: Subsystem Unit Harnesses (Isolated Subsystems)
add_test(NAME unit_math_test COMMAND test_math)
add_test(NAME unit_data_test COMMAND test_data)
add_test(NAME unit_time_test COMMAND test_time)
add_test(NAME unit_string_test COMMAND test_string)
add_test(NAME unit_log_test COMMAND test_log)
add_test(NAME unit_debug_test COMMAND test_debug)

# Tier 3: Integrated Utilities Harness (Multi-Subsystem Coordination)
add_test(NAME integrated_utils_test COMMAND test_utils)

# Tier 4: System Verification Harnesses (Full Game Simulation)
add_test(NAME system_game_invariants_test COMMAND test_game_invariants)
add_test(NAME system_gameplay_scenarios_test COMMAND test_gameplay_scenarios)
```

30.5. Execution & Verification Commands

Developers and CI/CD pipelines can invoke the complete suite or isolate specific tiers:

```
# Compile all binaries and test targets under strict zero-warning policy
cmake --build build --config Debug

# Execute all 13 automated verification suites
ctest --test-dir build -C Debug --output-on-failure

# Run individual tier harnesses directly for step-by-step debugging
./build/test_functional         # Granular cog trace output
./build/test_math               # Math & geometry unit verification
./build/test_data               # Filesystem & JSON unit verification
./build/test_time               # Timing & clock unit verification
./build/test_string             # Safe string unit verification
./build/test_log                # Forensic telemetry unit verification
./build/test_debug              # State debugger & rewind unit verification
./build/test_utils              # Integrated utilities verification
./build/test_game_invariants    # Simulation state invariants
./build/test_gameplay_scenarios # Gameplay scenarios & latency benchmark
```

Appendix A: Wholesale Source Code Reference Plates

Complete verbatim reference plates for source files, blocks, sub-assemblies, and function groups excerpted in preceding chapters.

A.1. Cornerstone Fall-Down Stairs Portability Ladder (C23/C17/C11 & C++20/17)

Snapshot ID: snap_manifest_ladder_sub2_1 | **Mode**: Subassembly | **File**: engine/include/core/manifest.h (L82–L194 of 707 lines) | Jump back to chapter text

```
 82 /*
 83 ------------------------------------------------------------------------------
 84 Sub-Assembly 2.1: Pre-Conditions, Progressive Standard Macros & Buffer Limits
 85 ------------------------------------------------------------------------------
 86 Compile-time sizing limits for registry module nodes, dependencies, rules, and
 87 Cornerstone's "Fall-Down" ISO C23 / C17 / C11 + C++20 / C++17 capability macros.
 88
 89 For Designers:
 90 Allows Cornerstone to take advantage of modern compiler safety checks on current
 91 hardware while stepping cleanly onto older systems without legacy clutter. Also
 92 bounds the engine registry to up to 32 major systems, 8 core rules, and 4 links.
 93
 94 For Programmers:
 95 - Guarantees 100% zero-heap allocation (ALLOC_ZERO_HEAP) in .rodata and .bss.
 96 - Exposes CS_C_STD_TIER (23/17/11/0) and CS_CXX_STD_TIER (20/17/11/0) plus
 97   graceful ISO attribute fall-down macros (CS_NODISCARD, CS_MAYBE_UNUSED,
 98   CS_FALLTHROUGH, CS_UNREACHABLE, CS_ALIGNOF, CS_NORETURN, CS_STATIC_ASSERT,
 99   CS_HAS_EMBED) compiled cleanly without extensions under -Wpedantic.
100 ------------------------------------------------------------------------------
101 */
102
103 /** Active ISO C standard tier compiled for this translation unit (23, 17, 11, or 0 in C++). */
104 #if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202311L
105 #    define CS_C_STD_TIER 23
106 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201710L
107 #    define CS_C_STD_TIER 17
108 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201112L
109 #    define CS_C_STD_TIER 11
110 #else
111 #    define CS_C_STD_TIER 0
112 #endif
113
114 /** Active ISO C++ standard tier when included from C++ (20, 17, 11, or 0 in C). */
115 #if defined(__cplusplus) && __cplusplus >= 202002L
116 #    define CS_CXX_STD_TIER 20
117 #elif defined(__cplusplus) && __cplusplus >= 201703L
118 #    define CS_CXX_STD_TIER 17
119 #elif defined(__cplusplus)
120 #    define CS_CXX_STD_TIER 11
121 #else
122 #    define CS_CXX_STD_TIER 0
123 #endif
124
125 /** Marks return values that callers MUST inspect (C23/C++17+ [[nodiscard]], C11/17 floor empty). */
126 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
127 #    define CS_NODISCARD [[nodiscard]]
128 #else
129 #    define CS_NODISCARD
130 #endif
131
132 /** Silences unused-entity diagnostics cleanly on intentional dormant/platform helpers. */
133 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
134 #    define CS_MAYBE_UNUSED [[maybe_unused]]
135 #else
136 #    define CS_MAYBE_UNUSED
137 #endif
138
139 /** Annotates explicit switch-case fall-through cleanly under -Wextra -Wpedantic. */
140 #if (CS_CXX_STD_TIER >= 17) || (CS_C_STD_TIER >= 23)
141 #    define CS_FALLTHROUGH [[fallthrough]]
```

```
#elif defined(__GNUC__) && (__GNUC__ >= 7)
#    define CS_FALLTHROUGH __attribute__((fallthrough))
#elif defined(__clang__) && defined(__has_attribute)
#    if __has_attribute(fallthrough)
#        define CS_FALLTHROUGH __attribute__((fallthrough))
#    else
#        define CS_FALLTHROUGH ((void)0)
#    endif
#else
#    define CS_FALLTHROUGH ((void)0)
#endif

/** Marks statically unreachable execution branches cleanly across C23/C17/C11/C++. */
#if (CS_C_STD_TIER >= 23) && defined(unreachable)
#    define CS_UNREACHABLE() unreachable()
#elif defined(__GNUC__) || defined(__clang__)
#    define CS_UNREACHABLE() __builtin_unreachable()
#else
#    define CS_UNREACHABLE() ((void)0)
#endif

/** Portable alignment query resolving to C23/C++ alignof vs C11/C17 _Alignof without <stdalign.h>. */
#if defined(__cplusplus) || (CS_C_STD_TIER >= 23)
#    define CS_ALIGNOF(type) alignof(type)
#else
#    define CS_ALIGNOF(type) _Alignof(type)
#endif

/** Portable non-returning function specifier (C23/C++ [[noreturn]] vs C11/C17 _Noreturn). */
#if defined(__cplusplus) || (CS_C_STD_TIER >= 23)
#    define CS_NORETURN [[noreturn]]
#else
#    define CS_NORETURN _Noreturn
#endif

/** Portable compile-time assertion (C23/C++ keyword vs C11/C17 <assert.h> macro). */
#define CS_STATIC_ASSERT(cond, msg) static_assert((cond), msg)

/** Non-zero when C23 #embed directive is natively available for binary/text ROM embedding. */
#ifdef __has_embed
#    define CS_HAS_EMBED 1
#else
#    define CS_HAS_EMBED 0
#endif

/** Maximum number of registered engine/game subsystem metadata nodes. */
#define MANIFEST_MAX_MODULES      32

/** Maximum documented invariant rules declared per module node. */
#define MANIFEST_MAX_INVARIANTS   8

/** Maximum upstream prerequisite dependencies declared per module node. */
#define MANIFEST_MAX_DEPENDENCIES 4
```

A.2. Layer 3 Plug Entry & Manifest Wiring (init_engine_manifest + main)

Snapshot ID: snap_main_boot_group | **Mode**: Group | **File**: game/src/main.c (L71–L522 of 524 lines) | Jump back to chapter text

```
/**
 * @brief Registers all core engine subsystem manifests into central registry.
 *
 * ### Machine Role & Mission
 * Populates the central manifest registry with all Layer 0 utilities (manifest,
 * fs, json, time, math, geom, string, log, debug), Layer 1 simulation engine (state,
 * combat, world, dispatcher, game), and Layer 2 presentation viewport (gui).
 * Enables boot-time loose-bolts invariant checks and dependency graph resolution
 * before any game loop execution begins.
 *
 * ### Ontological Classification
 * - Stratum: Layer 3 (Application Entry & Presentation Router)
 * - Machine: app.entry.main (Executable Entry & Presentation Router)
```

```
 * - Assembly: Pre-Flight Manifest Registry Initialization
 * - Cog: Module Manifest Registration Primitive
 *
 * ### Execution & Resource Contract
 * - Memory Model: ALLOC_ZERO_HEAP (Stack-allocated fixed manifest tables).
 * - Concurrency: CONCURRENCY_MAIN_THREAD (Called at startup on main thread).
 * - Latency Budget: < 50 microseconds.
 *
 * @param[out] registry Target manifest registry to initialize and populate.
 */
static void init_engine_manifest(manifest_registry_t *registry)
{
    /* Step 1: Zero-initialize registry table */
    manifest_registry_init(registry);

    /* Step 2: Register Layer 0 Core Utilities */
    manifest_register(registry, &manifest_utils_manifest);
    manifest_register(registry, &fs_utils_manifest);
    manifest_register(registry, &json_utils_manifest);
    manifest_register(registry, &path_finder_manifest);
    manifest_register(registry, &master_loader_manifest);
    manifest_register(registry, &time_utils_manifest);
    manifest_register(registry, &math_utils_manifest);
    manifest_register(registry, &geom_utils_manifest);
    manifest_register(registry, &string_utils_manifest);
    manifest_register(registry, &clock_pacer_manifest);
    manifest_register(registry, &formula_eval_manifest);
    manifest_register(registry, &log_manifest);
    manifest_register(registry, &debug_manifest);

    /* Step 3: Register Layer 1 Simulation Engine Subsystems */
    manifest_register(registry, &sim_state_manifest);
    manifest_register(registry, &combat_manifest);
    manifest_register(registry, &items_manifest);
    manifest_register(registry, &world_manifest);
    manifest_register(registry, &dispatcher_manifest);
    manifest_register(registry, &game_manifest);

    /* Step 4: Register Layer 2 Presentation & Audio Viewport */
    manifest_register(registry, &audio_bus_manifest);
    manifest_register(registry, &gui_manifest);
}
```

```
/**
 * @brief Application entry point and presentation frontend router.
 *
 * ### Machine Role & Mission
 * Serves as the operating system process entry point. Parses command-line
 * arguments, executes pre-flight loose-bolts machine checks, initializes the
 * player simulation state, and dynamically routes to either native X11 GUI
 * or terminal CLI mode based on user flags and hardware display availability.
 *
 * ### Ontological Classification
 * - Stratum: Layer 3 (Application Entry & Presentation Router)
 * - Machine: app.entry.main (Executable Entry & Presentation Router)
 * - Assembly: Application Lifecycle & Frontend Router
 * - Cog: Process Entry Point & Presentation Mode Dispatcher
 *
 * ### Execution & Resource Contract
 * - Memory Model: ALLOC_ZERO_HEAP (Zero dynamic heap allocations in router).
 * - Concurrency: CONCURRENCY_MAIN_THREAD (Main thread process lifecycle).
 * - Latency Budget: Lifespan of game process.
 *
 * @param[in] argc Number of command-line arguments.
 * @param[in] argv Array of command-line argument strings.
 * @return Process exit code (0 on success, non-zero on failure).
 */
int main(int argc, char *argv[])
{
    /* Step 0: Arm Tier-0 BIOS Safe-Mode telemetry immediately (RAM Ring + stderr alerts) */
    log_fallback_boot(NULL);
    LOG_INFO(LOG_CAT_LOG, "BootBIOS", 0.0f, "Tier-0 BIOS Safe-Mode telemetry armed");

    /* Step 1: Initialize presentation mode flags */
```

```
    bool force_cli = false;
    bool force_gui = false;
    bool force_debug = false;

    /* Step 2: Parse command-line arguments */
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0) {
            print_usage();
            return 0;
        }
        if (strcmp(argv[i], "--version") == 0 || strcmp(argv[i], "-v") == 0) {
            print_version();
            return 0;
        }
        if (strcmp(argv[i], "--audit") == 0 || strcmp(argv[i], "-a") == 0) {
            return run_full_subsystem_audit();
        }
        if (strcmp(argv[i], "--debug") == 0 || strcmp(argv[i], "-d") == 0) {
            force_debug = true;
            force_cli = true;
        } else if (strcmp(argv[i], "--cli") == 0 || strcmp(argv[i], "-c") == 0) {
            force_cli = true;
        } else if (strcmp(argv[i], "--gui") == 0 ||
                   strcmp(argv[i], "-g") == 0) {
            force_gui = true;
        } else if (strcmp(argv[i], "--data-root") == 0 || strcmp(argv[i], "-r") == 0) {
            if (i + 1 < argc) {
                path_finder_set_data_root(argv[++i]);
            } else {
                log_fallback_record(
                    LOG_SEV_ERROR,
                    LOG_CAT_LOG,
                    "BootCLI",
                    -0.5f,
                    __FILE__,
                    __LINE__,
                    __func__,
                    "CLI argument --data-root missing directory parameter"
                );
                return 1;
            }
        } else if (strncmp(argv[i], "--data-root=", 12) == 0) {
            path_finder_set_data_root(argv[i] + 12);
        } else {
            log_fallback_record(
                LOG_SEV_ERROR,
                LOG_CAT_LOG,
                "BootCLI",
                -0.25f,
                __FILE__,
                __LINE__,
                __func__,
                "Unknown CLI option '%s'; use --help for usage",
                argv[i]
            );
            return 1;
        }
    }

    /* Step 3: Initialize forensic logging & debugging (auto-escalate to Safe Mode if needed) */
    if (!log_init(NULL)) {
        log_init_emergency_monolith();
    }
    debug_init(NULL);
    LOG_INFO(LOG_CAT_LOG, "Engine", 0.0f, "Tenebris and Lux boot initialized (mode: %s%s)",
             force_cli ? "CLI" : (force_gui ? "GUI" : "Auto"),
             force_debug ? " [DEBUG-ACTIVE]" : "");

    /* Step 4: Run Pre-Flight ("Don't start machine with loose bolts") */
    manifest_registry_t registry;
    init_engine_manifest(&registry);
    preflight_status_t preflight = manifest_run_preflight(&registry);

    if (preflight == PREFLIGHT_STATUS_HALT_FATAL) {
        LOG_FATAL(LOG_CAT_LOG, "Engine", -1.0f, "Engine startup halted due to loose bolts!");
```

```
        fprintf(stderr,
                "\n[FATAL ERROR] Engine startup halted due to loose bolts!\n");
        manifest_print_report(&registry);
        debug_shutdown();
        log_shutdown();
        return 1;
    }

    /* Step 5: Initialize simulation player state */
    player_state_t state;
    game_init(&state);

    /* Record initial genesis snapshot */
    game_record_debug_snapshot(&state, "init", "genesis");

    /* Step 6: Route explicit Debug / CLI launch request */
    if (force_debug) {
        printf("============================================================\n");
        printf("   TENEBRIS AND LUX - INTERACTIVE DEBUGGER ACTIVE\n");
        printf("   Type 'debug help' or 'inspect' during gameplay\n");
        printf("============================================================\n\n");
        LOG_INFO(LOG_CAT_DEBUG, "Engine", 0.0f, "Entering interactive debug mode in CLI");
        game_loop_cli(&state);
        debug_shutdown();
        log_shutdown();
        return 0;
    }

    if (force_cli) {
        LOG_INFO(LOG_CAT_LOG, "Engine", 0.0f, "Entering terminal CLI game loop");
        game_loop_cli(&state);
        debug_shutdown();
        log_shutdown();
        return 0;
    }

    /* Step 7: Route explicit GUI launch request */
    if (force_gui) {
        if (!gui_is_available()) {
            LOG_ERROR(LOG_CAT_LOG, "Engine", -0.5f, "No graphical X11 display available for --gui");
            fprintf(stderr,
                    "Error: No graphical X11 display available for --gui.\n");
            debug_shutdown();
            log_shutdown();
            return 1;
        }
        LOG_INFO(LOG_CAT_LOG, "Engine", 0.0f, "Launching native X11 GUI");
        int rc = gui_run(&state);
        debug_shutdown();
        log_shutdown();
        return rc;
    }

    /* Step 8: Graded policy fallback: downgrade to CLI if display missing */
    if (preflight == PREFLIGHT_STATUS_DOWNGRADED || !gui_is_available()) {
        LOG_WARN(LOG_CAT_LOG, "Engine", -0.2f, "Graphical display unavailable, downgrading to CLI");
        printf("[Notice] Graphical display unavailable. "
               "Downgrading to CLI safe mode.\n\n");
        game_loop_cli(&state);
        debug_shutdown();
        log_shutdown();
        return 0;
    }

    /* Step 9: Default presentation: launch native GUI */
    LOG_INFO(LOG_CAT_LOG, "Engine", 0.0f, "Launching native X11 GUI default presentation");
    int rc = gui_run(&state);
    debug_shutdown();
    log_shutdown();
    return rc;
}
```

A.3. Build-Time Compiled .rodata Config ROM Generator

Snapshot ID: snap_cmake_rom_whole | **Mode**: Whole | **File**: cmake/GenerateConfigRom.cmake (L1–L302 of 302 lines) | Jump back to chapter text

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Identity, Provenance, Scaling Telemetry & Architectural Boundary Contracts )
# ============================================================================
# @project       Tenebris and Lux
# @file          cmake/GenerateConfigRom.cmake
# @id            build.rom.config
# @layer         Layer 0 Infrastructure (Build Tooling)
# @archetype     operational | stateless
# @version       0.1.1-dev
# @interface_rev Rev 5
# @date          2026-11-11
# @author        CreativeWorkzStudio LLC / ProfessorSeanEX
# @repository    https://github.com/Novaking34/tenebris-lux
# @system        Universal Cornerstone Configuration ROM Embedder
# @role          Build-Time Script: Encodes canonical JSON manifests and presets
#                under engine/config/<subdir> into immutable ISO C11 .rodata
#                byte-array headers for zero-disk Safe-Mode BIOS boot.
# @maturity      Rev 5 Baseline | Generalized multi-subdir byte-array ROM compiler.
# @chassis       zero-heap | Emits pure static const .rodata C tables; zero disk I/O at runtime.
# @scale         O(F * B) compile-time hex conversion over F sorted JSON configs and B bytes.
# @seams         in: engine/config/**/*.json | uses: CMake file(READ HEX) | probes: Block 4 script guards
# @invariants    1. Pure ISO C11/C++17 byte arrays (avoids -Woverlength-strings).
#                2. Strictly ISO C-style /* ... */ comments only in emitted headers.
#                3. Deterministic alphabetical ordering across subdirectories and JSON stems.
#                4. Collision-free C symbol identifiers across multi-subdir vaults.
#                5. Content-preserving emission via cmake -E copy_if_different.
#
# PURPOSE & ARCHITECTURAL ROLE:
#   Generates compiled-in `.rodata` ROM vault headers directly from canonical
#   engine JSON configurations so `log_fallback`, `debug_fallback`, and
#   `io_fallback` retain structured Safe-Mode presets when disk files are absent.
#
#   ### For Game Designers & Non-Programmers:
#   Automatically bakes emergency read-only copies of JSON presets inside the
#   compiled engine binary every time `engine/config/` presets change, keeping
#   Safe-Mode fallbacks aligned without manual editing.
#
#   ### For Systems Programmers & Engine Architects:
#   Script-mode CMake generator (`cmake -P`) converting raw hex nibbles into
#   null-terminated 10-byte-per-line `static const unsigned char` arrays paired
#   with indexed lookup metadata structs.
# ============================================================================

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ( Pre-Conditions, Default Contracts, Patterns & Script Accumulators )
# ============================================================================
# Engine:        Cornerstone Game Engine
# Stratum:       Layer 0 Infrastructure (Build Tooling)
# Module:        build.rom.config
# Mission:       Validate input arguments, establish symbol defaults, and
#                initialize deterministic ROM generation accumulators.
# ============================================================================

# ----------------------------------------------------------------------------
# Sub-Assembly 2.1: Parameter Pre-Conditions & Caller Contract Validation
# ----------------------------------------------------------------------------
if(NOT DEFINED CONFIG_ROOT OR NOT DEFINED SUBDIRS OR NOT DEFINED OUTPUT_FILE)
    message(FATAL_ERROR
        "GenerateConfigRom.cmake requires -DCONFIG_ROOT=<dir>, -DSUBDIRS=<list>, and -DOUTPUT_FILE=<file>")
endif()

if(NOT IS_DIRECTORY "${CONFIG_ROOT}")
    message(FATAL_ERROR
        "GenerateConfigRom.cmake: CONFIG_ROOT does not exist: '${CONFIG_ROOT}'")
endif()

```

```
# ------------------------------------------------------------------------------
# Sub-Assembly 2.2: Default Symbol Contract Bindings
# ------------------------------------------------------------------------------
if(NOT DEFINED HEADER_GUARD)
    set(HEADER_GUARD "CORNERSTONE_CONFIG_ROM_H")
endif()
if(NOT DEFINED SYMBOL_PREFIX)
    set(SYMBOL_PREFIX "CONFIG_ROM")
endif()
if(NOT DEFINED STRUCT_TAG)
    set(STRUCT_TAG "config_rom_asset")
endif()
if(NOT DEFINED TYPE_NAME)
    set(TYPE_NAME "config_rom_asset_t")
endif()
if(NOT DEFINED TABLE_NAME)
    set(TABLE_NAME "s_config_rom_table")
endif()
if(NOT DEFINED VAR_PREFIX)
    set(VAR_PREFIX "s_rom_")
endif()
if(NOT DEFINED BANNER_TITLE)
    set(BANNER_TITLE "AUTO-GENERATED ENGINE CONFIG ROM IMAGE (DO NOT EDIT MANUALLY)")
endif()
if(NOT DEFINED ID_MODE)
    set(ID_MODE "SUBDIR_STEM")
endif()

# ------------------------------------------------------------------------------
# Sub-Assembly 2.3: Hex Formatting Constants & Accumulation Buffers
# ------------------------------------------------------------------------------
set(_byte_pat "0x[0-9a-f][0-9a-f], ")
set(_ten_pat
    "(${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat}${_byte_pat})"
)

set(_arrays_code "")
set(_table_entries "")
set(_total_raw_bytes 0)
set(_rom_file_count 0)
set(_subdirs_doc "")
set(_seen_c_idents "")
set(_seen_asset_ids "")

# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Deterministic Discovery, Hex-to-C Encoding & Table Construction )
# ==============================================================================
# Engine:        Cornerstone Game Engine
# Stratum:       Layer 0 Infrastructure (Build Tooling)
# Module:        build.rom.config
# Mission:       Traverse sorted configuration domains and encode every JSON
#                asset into null-terminated .rodata C arrays and table rows.
# ==============================================================================

# ------------------------------------------------------------------------------
# Sub-Assembly 3.1: Sorted Domain Traversal & Identifier Mapping
# ------------------------------------------------------------------------------
set(_sorted_subdirs ${SUBDIRS})
list(SORT _sorted_subdirs)

foreach(_subdir IN LISTS _sorted_subdirs)
    set(_abs_dir "${CONFIG_ROOT}/${_subdir}")
    if(NOT IS_DIRECTORY "${_abs_dir}")
        message(FATAL_ERROR
            "GenerateConfigRom.cmake: Subdirectory does not exist: '${_abs_dir}'")
    endif()

    file(GLOB _dir_jsons "${_abs_dir}/*.json")
    list(SORT _dir_jsons)
    list(LENGTH _dir_jsons _dir_count)

    if(_dir_count EQUAL 0)
        message(FATAL_ERROR
            "GenerateConfigRom.cmake: No JSON files found in '${_abs_dir}'")
```

```
    endif()

    string(APPEND _subdirs_doc "  engine/config/${_subdir}/ (${_dir_count} assets)\n")
    get_filename_component(_subdir_leaf "${_subdir}" NAME)

    # ------------------------------------------------------------------------
    # Sub-Assembly 3.2: Deterministic Byte-Array Encoding & Table Row Emission
    # ------------------------------------------------------------------------
    foreach(_json_path IN LISTS _dir_jsons)
        get_filename_component(_stem "${_json_path}" NAME_WE)
        get_filename_component(_filename "${_json_path}" NAME)

        if(ID_MODE STREQUAL "STEM")
            set(_asset_id "${_stem}")
            string(REGEX REPLACE "[^A-Za-z0-9_]" "_" _c_ident "${_stem}")
        else()
            set(_asset_id "${_subdir_leaf}/${_stem}")
            string(REGEX REPLACE "[^A-Za-z0-9_]" "_" _c_ident "${_subdir_leaf}_${_stem}")
        endif()

        list(FIND _seen_c_idents "${_c_ident}" _dup_c_idx)
        if(NOT _dup_c_idx EQUAL -1)
            message(FATAL_ERROR
                "GenerateConfigRom.cmake: Duplicate C identifier '${_c_ident}' from '${_json_path}'")
        endif()
        list(APPEND _seen_c_idents "${_c_ident}")

        list(FIND _seen_asset_ids "${_asset_id}" _dup_id_idx)
        if(NOT _dup_id_idx EQUAL -1)
            message(FATAL_ERROR
                "GenerateConfigRom.cmake: Duplicate ROM asset ID '${_asset_id}' from '${_json_path}'")
        endif()
        list(APPEND _seen_asset_ids "${_asset_id}")

        file(READ "${_json_path}" _hex_content HEX)
        string(LENGTH "${_hex_content}" _hex_len)
        math(EXPR _hex_mod "${_hex_len} % 2")
        if(NOT _hex_mod EQUAL 0)
            message(FATAL_ERROR
                "GenerateConfigRom.cmake: Odd hex length (${_hex_len}) reading '${_json_path}'")
        endif()

        math(EXPR _byte_len "${_hex_len} / 2")
        math(EXPR _total_raw_bytes "${_total_raw_bytes} + ${_byte_len}")
        math(EXPR _rom_file_count "${_rom_file_count} + 1")

        if(_byte_len GREATER 0)
            string(REGEX REPLACE "([0-9a-f][0-9a-f])" "0x\\1, " _c_hex "${_hex_content}")
            set(_c_hex "${_c_hex}0x00")
            string(REGEX REPLACE "${_ten_pat}" "    \\1\n" _c_lines "${_c_hex}")
            string(REGEX REPLACE "\n(0x)" "\n    \\1" _c_lines "${_c_lines}")
            string(REGEX REPLACE " +\n" "\n" _c_lines "${_c_lines}")
            if(NOT "${_c_lines}" MATCHES "^    ")
                set(_c_lines "    ${_c_lines}")
            endif()
        else()
            set(_c_lines "    0x00")
        endif()

        string(APPEND _arrays_code
            "/* Embedded ROM Asset: ${_subdir}/${_filename} (${_byte_len} bytes) */\n"
            "static const unsigned char ${VAR_PREFIX}${_c_ident}[] = {\n"
            "${_c_lines}\n"
            "};\n\n"
        )

        string(APPEND _table_entries
            "    {\n"
            "        \"${_asset_id}\",\n"
            "        \"${_subdir}/${_filename}\",\n"
            "        ${VAR_PREFIX}${_c_ident},\n"
            "        ${_byte_len}U\n"
            "    },\n"
        )
    endforeach()
```

```
endforeach()

# ============================================================================
# BLOCK 4: AUDIT PROCEDURES BLOCK
# ( Build-Time Integrity Gates, Asset Parity & C++ Compatibility Synthesis )
# ============================================================================
# Engine:        Cornerstone Game Engine
# Stratum:       Layer 0 Infrastructure (Build Tooling)
# Module:        build.rom.config
# Mission:       Verify non-empty ROM image construction and assemble a clean
#                C11/C++17-compatible generated header unit.
# ============================================================================

# ----------------------------------------------------------------------------
# Sub-Assembly 4.1: Pre-Flight Asset Count & Header Synthesis Checks
# ----------------------------------------------------------------------------
if(_rom_file_count LESS_EQUAL 0)
    message(FATAL_ERROR
        "GenerateConfigRom.cmake: Produced 0 ROM assets across subdirs '${SUBDIRS}'")
endif()

set(_out_header "")
string(APPEND _out_header
    "/*\n"
    "================================================================================\n"
    "${BANNER_TITLE}\n"
    "Generated by cmake/GenerateConfigRom.cmake from:\n"
    "${_subdirs_doc}"
    "Assets: ${_rom_file_count} | Raw JSON Bytes: ${_total_raw_bytes}\n"
    "================================================================================\n"
    "*/\n"
    "#ifndef ${HEADER_GUARD}\n"
    "#define ${HEADER_GUARD}\n\n"
    "#include <stddef.h>\n\n"
    "#ifdef __cplusplus\n"
    "extern \"C\" {\n"
    "#endif\n\n"
    "#define ${SYMBOL_PREFIX}_ASSET_COUNT ${_rom_file_count}U\n"
    "#define ${SYMBOL_PREFIX}_TOTAL_BYTES ${_total_raw_bytes}U\n\n"
    "typedef struct ${STRUCT_TAG} {\n"
    "    const char          *id;\n"
    "    const char          *rel_path;\n"
    "    const unsigned char *data;\n"
    "    size_t               size;\n"
    "} ${TYPE_NAME};\n\n"
    "${_arrays_code}"
    "static const ${TYPE_NAME} ${TABLE_NAME}[${_rom_file_count}] = {\n"
    "${_table_entries}"
    "};\n\n"
    "#ifdef __cplusplus\n"
    "}\n"
    "#endif\n\n"
    "#endif /* ${HEADER_GUARD} */\n"
)

# ============================================================================
# BLOCK 5: CLOSING BLOCK
# ( Atomic Header Emission, Timestamp Preservation & Workspace Cleanup )
# ============================================================================
# Engine:        Cornerstone Game Engine
# Stratum:       Layer 0 Infrastructure (Build Tooling)
# Module:        build.rom.config
# Mission:       Write generated header atomically via copy_if_different to avoid
#                spurious incremental rebuilds when JSON bytes are unchanged.
# ============================================================================

# ----------------------------------------------------------------------------
# Sub-Assembly 5.1: Content-Preserving Atomic Write & Temporary File Cleanup
# ----------------------------------------------------------------------------
get_filename_component(_out_dir "${OUTPUT_FILE}" DIRECTORY)
file(MAKE_DIRECTORY "${_out_dir}")

set(_tmp_file "${OUTPUT_FILE}.tmp")
file(WRITE "${_tmp_file}" "${_out_header}")
execute_process(
```

```
    COMMAND "${CMAKE_COMMAND}" -E copy_if_different "${_tmp_file}" "${OUTPUT_FILE}"
    RESULT_VARIABLE _copy_rc
)
file(REMOVE "${_tmp_file}")

if(NOT _copy_rc EQUAL 0)
    message(FATAL_ERROR
        "GenerateConfigRom.cmake: Failed copying generated header to '${OUTPUT_FILE}' (rc=${_copy_rc})")
endif()
```

A.4. Tier-0 I/O Safe-Mode BIOS Boot, Floppy Floor & Downgrade Latch

Snapshot ID: snap_io_fallback_bios_group | **Mode**: Group | **File**: engine/src/io/io_fallback.c (L2908–L3332 of 3355 lines) | Jump back to chapter text

```
/**
 * @brief Downgrade the BIOS I/O subsystem into Safe Mode (`FLOPPY_RO` or `ROM_ONLY`).
 *
 *   ### For Game Designers & Non-Programmers:
 *   Safely protects storage whenever external folders fail or policies break: locks out
 *   unsafe writes if requested, continues serving factory ROM data, and logs the event.
 *
 *   ### For Systems Programmers & Engine Architects:
 *   Arms BIOS, increments `s_io_fb.safe_downgrades`, switches `active_sink` to
 *   `IO_FB_SINK_FLOPPY_RO` or `IO_FB_SINK_ROM_ONLY`, and forwards telemetry into both
 *   `io_fallback_record_fault()` and `debug_fallback_record_fault()`.
 *
 * @param[in] reason         Concise Safe-Mode transition reason, or NULL for generic default.
 * @param[in] lock_read_only `true` to engage read-only floppy latch; `false` for `ROM_ONLY`.
 * @return `true` after Safe-Mode transition and telemetry recording complete.
 * @triggers Increments `safe_downgrades`, updates `active_sink`, and notifies debug BIOS.
 */
bool io_fallback_downgrade_safe_mode(const char *reason, bool lock_read_only)
{
    /* Step 1: Arm BIOS, count Safe-Mode downgrade, and transition active sink */
    io_fb_auto_ensure_booted();
    s_io_fb.safe_downgrades++;
    if (lock_read_only) {
        s_io_fb.config.read_only_mode = true;
        s_io_fb.active_sink = IO_FB_SINK_FLOPPY_RO;
    } else if (s_io_fb.active_sink == IO_FB_SINK_OFFLINE) {
        s_io_fb.active_sink = IO_FB_SINK_ROM_ONLY;
    }

    /* Step 2: Emit dual fault telemetry across I/O BIOS ring and debug BIOS ring */
    const char *msg = (reason != NULL) ? reason : "Engaged Safe-Mode I/O BIOS";
    io_fallback_record_fault("io_fallback", "downgrade_safe_mode", msg, IO_ERR_FILE_NOT_FOUND);
    (void)debug_fallback_record_fault("io_fallback", msg, __FILE__, __LINE__,
                                      "TW_ENG_IO_ROM_INTEGRITY");
    return true;
}
```

```
bool io_fallback_boot(const io_fallback_config_t *opt_cfg)
{
    io_fallback_config_t selected = s_io_fallback_template;
    bool hydrated = false;

    if (opt_cfg != NULL && io_fallback_validate_policy_contract(opt_cfg)) {
        selected = *opt_cfg;
        selected.loaded_from_data = false;
    } else if (io_fallback_load_rom_config("default", &selected)) {
        hydrated = true;
    }

    s_io_fb.config = selected;
    s_io_fb.rom_hydrated = hydrated;
    io_fb_hydrate_subpolicies();
    str_safe_copy(s_io_fb.active_root,
                  selected.default_root[0] ? selected.default_root : IO_FB_DEFAULT_ROOT,
                  sizeof(s_io_fb.active_root));

```

```
    if (selected.read_only_mode) {
        s_io_fb.active_sink = IO_FB_SINK_FLOPPY_RO;
    } else {
        s_io_fb.active_sink = IO_FB_SINK_DISK_ACTIVE;
    }

    s_io_fb.booted = true;
    io_fallback_register_debug_probes();
    return true;
}

bool io_fallback_boot_floppy(void)
{
    io_fallback_config_t floppy = s_io_floppy_template;
    if (io_fallback_load_rom_config("floppy_safe", &floppy) ||
        io_fallback_load_rom_config("default", &floppy)) {
        s_io_fb.rom_hydrated = true;
    } else {
        s_io_fb.rom_hydrated = false;
    }
    floppy.read_only_mode = true;
    floppy.loaded_from_data = false;
    if (floppy.profile_name[0] == '\0' || strcmp(floppy.profile_name, "default") == 0) {
        str_safe_copy(floppy.profile_name, IO_FALLBACK_FLOPPY_PROFILE_NAME,
                      sizeof(floppy.profile_name));
    }

    s_io_fb.config = floppy;
    io_fb_hydrate_subpolicies();
    s_io_fb.active_sink = IO_FB_SINK_FLOPPY_RO;
    str_safe_copy(s_io_fb.active_root, IO_FB_DEFAULT_ROOT, sizeof(s_io_fb.active_root));
    s_io_fb.booted = true;
    io_fallback_register_debug_probes();
    return true;
}
```

A.5. Autonomous Validation Probes & Tripwire Trap Chain

Snapshot ID: snap_debug_probe_group | **Mode**: Group | **File**: engine/src/debug/debug.c (L1918–L2438 of 3221 lines) | Jump back to chapter text

```
/**
 * @brief Register or idempotently update a Block 4 subsystem validation probe.
 *
 *   ### For Game Designers & Non-Programmers:
 *   Plugs a subsystem's automated health monitor into the debugger so anomalies
 *   immediately report exact failing checks and test names.
 *
 *   ### For Systems Programmers & Engine Architects:
 *   Updates an existing `probe_id` slot in place while preserving its historical
 *   `trigger_count`, or claims the next free slot in `s_debug.probes[]`. Logs a
 *   guarded `LOG_WARN` if `DEBUG_MAX_VALIDATION_PROBES` capacity is exhausted.
 *
 * @param[in] probe Descriptor defining probe ID, subsystem tag, callback, and reaction.
 * @return Slot index `[0, DEBUG_MAX_VALIDATION_PROBES - 1]` on success; `-1` if null,
 *         empty `probe_id`, or probe registry is saturated.
 * @triggers Validates input guard and emits guarded `LOG_WARN` on registry saturation.
 */
int debug_register_validation_probe(const debug_validation_probe_t *probe)
{
    /* Step 1: Reject null descriptors or empty probe identifiers */
    if (probe == NULL || probe->probe_id[0] == '\0') {
        return -1;
    }

    /* Step 2: Update existing slot idempotently if probe_id already registered */
    for (size_t i = 0U; i < s_debug.probe_count; i++) {
        if (strcmp(s_debug.probes[i].probe_id, probe->probe_id) == 0) {
            uint32_t prev_count = s_debug.probes[i].trigger_count;
            s_debug.probes[i] = *probe;
            s_debug.probes[i].is_active = true;
            s_debug.probes[i].trigger_count = prev_count;
```

```
            return (int)i;
        }
    }

    /* Step 3: Guard static probe registry ceiling and log saturation telemetry */
    if (s_debug.probe_count >= DEBUG_MAX_VALIDATION_PROBES) {
        if (s_debug_active && !s_debug.in_audit && !s_debug.in_probe) {
            s_debug.in_probe = true;
            LOG_WARN(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, -0.4f,
                     "Probe registry saturated (%zu/%d); rejected '%s'",
                     s_debug.probe_count, DEBUG_MAX_VALIDATION_PROBES, probe->probe_id);
            s_debug.in_probe = false;
        }
        return -1;
    }

    /* Step 4: Claim next free slot in `.bss` and activate probe */
    size_t slot = s_debug.probe_count++;
    s_debug.probes[slot] = *probe;
    s_debug.probes[slot].is_active = true;
    return (int)slot;
}
```

```
/**
 * @brief Trigger a validation probe tripwire, downgrade safely, and enter DEBUG_STEP_AWAIT.
 *
 *   ### For Game Designers & Non-Programmers:
 *   Handles a live safety tripwire cleanly: confirms which check failed, lets
 *   Safe Mode preserve critical data, freezes the offending frame, and prints an
 *   actionable diagnostic packet rather than crashing abruptly.
 *
 *   ### For Systems Programmers & Engine Architects:
 *   Non-reentrant (`in_probe`) pipeline: resolves matched probe defaults, runs optional
 *   `validate_fn()`, increments `traps_triggered`/`probe_failures`, notifies BIOS via
 *   `debug_fallback_record_fault()` and `debug_fallback_finish_with_fallback()`, locks the
 *   latest snapshot if present, arms `DEBUG_STEP_AWAIT` (when active and non-audit),
 *   populates `s_debug.last_report`, and emits a guarded `LOG_ERROR`.
 *
 * @param[in] probe_id  Registered probe identifier, or NULL for ad-hoc tripwire.
 * @param[in] subsystem Originating module identifier override, or NULL to inherit probe/module default.
 * @param[in] condition Human-readable invariant failure description, or NULL for callback/default message.
 * @param[in] file      Source file name captured at trap call site (`__FILE__`).
 * @param[in] line      Source line number captured at trap call site (`__LINE__`).
 * @param[in] func      Enclosing C function name (`__func__`).
 * @return `true` when latched and Safe-Mode downgrade completed; `false` if suppressed due
 *         to active `in_probe` recursion guard.
 * @triggers Enforces reentrancy guard, Safe-Mode downgrade, snapshot pinning, and AWAIT latch.
 */
bool debug_trigger_probe(
    const char *probe_id,
    const char *subsystem,
    const char *condition,
    const char *file,
    int         line,
    const char *func
) {
    /* Step 1: Prevent reentrant tripwire loops while handling an active probe */
    if (s_debug.in_probe) {
        return false;
    }

    s_debug.in_probe = true;

    /* Step 2: Locate registered probe metadata if present and merge fallback overrides */
    debug_validation_probe_t *matched = NULL;
    if (probe_id != NULL && probe_id[0] != '\0') {
        for (size_t i = 0U; i < s_debug.probe_count; i++) {
            if (s_debug.probes[i].is_active &&
                strcmp(s_debug.probes[i].probe_id, probe_id) == 0) {
                matched = &s_debug.probes[i];
                matched->trigger_count++;
                break;
            }
        }
```

```
    }

    const char *eff_sub = (subsystem != NULL && subsystem[0] != '\0')
                              ? subsystem
                              : ((matched != NULL && matched->subsystem[0] != '\0')
                                     ? matched->subsystem
                                     : MODULE_ID);
    const char *eff_cond = (condition != NULL && condition[0] != '\0')
                               ? condition
                               : "Validation tripwire raised";
    debug_tripwire_t eff_wire = (matched != NULL && matched->tripwire_code != DEBUG_TRIPWIRE_CLEAR)
                                    ? matched->tripwire_code
                                    : DEBUG_TRIPWIRE_TEST_FAILED;
    debug_tripwire_reaction_t eff_react = (matched != NULL)
                                              ? matched->reaction
                                              : dbg_default_reaction_for_wire(eff_wire);
    debug_test_tier_t eff_tier = (matched != NULL) ? matched->tier : DEBUG_TEST_TIER_AUDIT;
    const char *eff_sec = (matched != NULL && matched->source_section[0] != '\0')
                              ? matched->source_section
                              : ((func != NULL) ? func : "unknown_section");
    const char *eff_test = (matched != NULL && matched->test_suite_ref[0] != '\0')
                               ? matched->test_suite_ref
                               : "BLOCK4_PREFLIGHT";
    bool want_await = (matched != NULL) ? matched->auto_await : s_debug.config.auto_await_on_tripwire;

    /* Step 3: Invoke Block 4 validation callback when registered to confirm health */
    char probe_detail_buf[128];
    if (matched != NULL && matched->validate_fn != NULL) {
        diag_result_t probe_res = matched->validate_fn();
        if (!probe_res.passed && probe_res.details != NULL && condition == NULL) {
            eff_cond = probe_res.details;
        }
        snprintf(probe_detail_buf, sizeof(probe_detail_buf), "%s [probe=%s]",
                 eff_cond, probe_res.passed ? "SELF_OK_BY_FALLBACK" : "FAIL_CONFIRMED");
    } else {
        str_safe_copy(probe_detail_buf, eff_cond, sizeof(probe_detail_buf));
    }

    /* Step 4: Latch primary debugger tripwire status and forensic failure counters */
    s_debug.tripwire_status = eff_wire;
    s_debug.traps_triggered++;
    s_debug.probe_failures++;

    snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
             "[%s:%s] %s (%s:%d | %s)",
             eff_sub,
             debug_test_tier_name(eff_tier),
             probe_detail_buf,
             (file != NULL) ? file : "unknown",
             line,
             eff_test);

    /* Step 5: Record into Tier-0 BIOS & downgrade to complete safely in fallback */
    (void)debug_fallback_record_fault(eff_sub, probe_detail_buf, file, line, eff_test);

    char fb_action[96];
    snprintf(fb_action, sizeof(fb_action),
             "Reaction=%s | Safe-Mode completed execution -> AWAIT",
             debug_tripwire_reaction_name(eff_react));
    (void)debug_fallback_finish_with_fallback(eff_sub, fb_action);

    s_debug.safe_mode = true;
    s_debug.fallback_completed = true;

    /* Step 6: Pin critical forensic snapshot in circular ring if snapshots exist */
    uint64_t pinned_id = 0ULL;
    bool snap_locked = false;
    if (s_debug_active && s_debug.snapshot_count > 0U &&
        debug_lock_snapshot(0ULL, "TRIGGER_TRIPWIRE_LATCH")) {
        const debug_snapshot_t *latest = debug_get_latest_snapshot();
        if (latest != NULL) {
            pinned_id = latest->snapshot_id;
            snap_locked = true;
        }
    }
```

```
2258
2259     /* Step 7: Enter DEBUG_STEP_AWAIT when debugger active, non-audit, and armed */
2260     bool engaged_await = false;
2261     if (s_debug_active && !s_debug.in_audit &&
2262         s_debug.config.break_on_tripwire && want_await) {
2263         s_debug.step_mode = DEBUG_STEP_AWAIT;
2264         s_debug.await_active = true;
2265         engaged_await = true;
2266     }
2267
2268     /* Step 8: Populate structured forensic report and emit guarded error telemetry */
2269     memset(&s_debug.last_report, 0, sizeof(s_debug.last_report));
2270     s_debug.last_report.tripwire_status = eff_wire;
2271     s_debug.last_report.reaction = eff_react;
2272     s_debug.last_report.tier = eff_tier;
2273     s_debug.last_report.fallback_completed = true;
2274     s_debug.last_report.await_active = engaged_await;
2275     s_debug.last_report.snapshot_locked = snap_locked;
2276     s_debug.last_report.locked_snapshot_id = pinned_id;
2277     str_safe_copy(s_debug.last_report.probe_id,
2278                   (probe_id != NULL) ? probe_id : "ad_hoc.tripwire",
2279                   sizeof(s_debug.last_report.probe_id));
2280     str_safe_copy(s_debug.last_report.subsystem, eff_sub, sizeof(s_debug.last_report.subsystem));
2281     str_safe_copy(s_debug.last_report.condition, probe_detail_buf,
2282                   sizeof(s_debug.last_report.condition));
2283     snprintf(s_debug.last_report.source_location, sizeof(s_debug.last_report.source_location),
2284              "%s:%d", (file != NULL) ? file : "unknown", line);
2285     str_safe_copy(s_debug.last_report.source_section, eff_sec,
2286                   sizeof(s_debug.last_report.source_section));
2287     str_safe_copy(s_debug.last_report.test_suite_ref, eff_test,
2288                   sizeof(s_debug.last_report.test_suite_ref));
2289     str_safe_copy(s_debug.last_report.fallback_action, fb_action,
2290                   sizeof(s_debug.last_report.fallback_action));
2291
2292     if (s_debug_active && !s_debug.in_audit) {
2293         LOG_ERROR(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, -0.75f,
2294                   "Probe tripwire latched: %s", s_debug.tripwire_details);
2295     }
2296
2297     s_debug.in_probe = false;
2298     return true;
2299 }
```

```
2367 /**
2368  * @brief Legacy compatibility helper that latches a state-corruption tripwire trap.
2369  *
2370  * @deprecated Prefer `debug_trigger_probe()` or `DEBUG_TRIGGER_PROBE()` so every trap
2371  *             carries a structured probe ID, tier classification, and forensic report.
2372  *
2373  *   ### For Game Designers & Non-Programmers:
2374  *   Preserved safety brake for older modules that report state errors without
2375  *   naming a specific modern probe identifier.
2376  *
2377  *   ### For Systems Programmers & Engine Architects:
2378  *   Latches `DEBUG_TRIPWIRE_STATE_CORRUPT`, increments `traps_triggered`, notifies
2379  *   BIOS via `debug_fallback_record_fault()`, performs guarded snapshot lock and
2380  *   `DEBUG_STEP_AWAIT` entry when active and non-audit, and emits `LOG_WARN`.
2381  *
2382  * @param[in] subsystem Subsystem tag reporting the fault, or NULL for `"unknown"`.
2383  * @param[in] condition Description of the violated condition, or NULL for `"anomaly"`.
2384  * @param[in] file      Originating source file (`__FILE__`).
2385  * @param[in] line      Originating source line (`__LINE__`).
2386  * @triggers Latches tripwire state, records BIOS fault, and warns via guarded log output.
2387  */
2388 void debug_trap_tripwire(const char *subsystem, const char *condition,
2389                          const char *file, int line)
2390 {
2391     /* Step 1: Latch classic STATE_CORRUPT code and forensic detail string verbatim */
2392     s_debug.tripwire_status = DEBUG_TRIPWIRE_STATE_CORRUPT;
2393     s_debug.traps_triggered++;
2394
2395     snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
2396              "[%s] %s (%s:%d)",
2397              (subsystem != NULL) ? subsystem : "unknown",
```

```
2398             (condition != NULL) ? condition : "anomaly",
2399             (file != NULL) ? file : "unknown", line);
2400
2401     /* Step 2: Record fault in Tier-0 BIOS and complete via Safe-Mode fallback */
2402     (void)debug_fallback_record_fault(
2403         subsystem,
2404         (condition != NULL) ? condition : "anomaly",
2405         file,
2406         line,
2407         "debug_trap_tripwire"
2408     );
2409
2410     if (s_debug_active && !s_debug.in_probe && !s_debug.in_audit) {
2411         (void)debug_fallback_finish_with_fallback(
2412             (subsystem != NULL) ? subsystem : MODULE_ID,
2413             "Completed trap safely via Safe-Mode fallback -> AWAIT"
2414         );
2415         s_debug.safe_mode = true;
2416         s_debug.fallback_completed = true;
2417
2418         if (s_debug.snapshot_count > 0U) {
2419             (void)debug_lock_snapshot(0ULL, "TRIGGER_TRIPWIRE_LATCH");
2420         }
2421         if (s_debug.config.break_on_tripwire && s_debug.config.auto_await_on_tripwire) {
2422             s_debug.step_mode = DEBUG_STEP_AWAIT;
2423             s_debug.await_active = true;
2424         }
2425     }
2426
2427     /* Step 3: Emit guarded telemetry warning without inducing recursive log traps */
2428     if (!s_debug.in_probe && !s_debug.in_audit) {
2429         s_debug.in_probe = true;
2430         LOG_WARN(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, -0.5f,
2431                  "Tripwire trapped: %s", s_debug.tripwire_details);
2432         s_debug.in_probe = false;
2433     }
2434
2435     if (s_debug_active && s_debug.config.auto_dump_on_tripwire && !s_debug.in_audit) {
2436         fprintf(stderr, "\n[DEBUG TRIPWIRE ALERT] %s\n", s_debug.tripwire_details);
2437     }
2438 }
```

A.6. Flight Log Subsystem Probe Registration & Validation Guards (Sub-Assembly 4.1)

Snapshot ID: snap_log_sub4_1_probes | **Mode**: Subassembly | **File**: engine/src/log/log.c (L1934–L2028 of 2833 lines) | Jump back to chapter text

```
1934 /*
1935 ------------------------------------------------------------------------------
1936 Sub-Assembly 4.1: Contract, Input & Cog Verification Cogs
1937 Mission: Verify received parameters, configuration contracts, and data provenance;
1938 verify that declared subsystem cogs run properly as standalone units.
1939 ------------------------------------------------------------------------------
1940 */
1941
1942 /**
1943  * @brief Returns whether the active logging subsystem was configured from data sources.
1944  *
1945  * ### For Game Designers & Non-Programmers:
1946  * Checks if settings were loaded from actual JSON files rather than emergency defaults.
1947  *
1948  * ### For Systems Programmers & Engine Architects:
1949  * Queries provenance boolean s_log.config.loaded_from_data.
1950  *
1951  * @return true if loaded from JSON data; false if using fallback code page.
1952  */
1953 bool log_is_data_driven(void)
1954 {
1955     return s_log.config.loaded_from_data;
1956 }
```

```

/**
 * @brief Queries whether the logging engine is currently operating in Safe Mode.
 *
 * ### For Game Designers & Non-Programmers:
 * Returns true if the engine booted using compiled-in ROM backups or emergency
 * defaults instead of live external JSON directories on disk.
 *
 * ### For Systems Programmers & Engine Architects:
 * Checks initialization state alongside s_log.safe_mode (!loaded_from_data).
 *
 * @return true if active and running in Safe Mode; false otherwise.
 */
bool log_is_safe_mode(void)
{
    return s_log.initialized && s_log.safe_mode;
}

/**
 * @brief Returns the current state of the logging data pipeline tripwire.
 *
 * ### For Game Designers & Non-Programmers:
 * Checks whether the data-driven logging pipeline encountered an error or is clean.
 *
 * ### For Systems Programmers & Engine Architects:
 * Queries s_log.tripwire_status latch.
 *
 * @return Active log_tripwire_t enum status code.
 */
log_tripwire_t log_get_tripwire_status(void)
{
    return s_log.tripwire_status;
}

/**
 * @brief Returns a detailed message explaining why a tripwire was tripped.
 *
 * ### For Game Designers & Non-Programmers:
 * Gives a clear, readable explanation of what went wrong if a configuration fault occurred.
 *
 * ### For Systems Programmers & Engine Architects:
 * Returns pointer to internal diagnostic message buffer s_log.tripwire_details.
 *
 * @return Null-terminated string describing the latched tripwire condition.
 */
const char *log_get_tripwire_details(void)
{
    if (s_log.tripwire_status == LOG_TRIPWIRE_CLEAR &&
        s_log.tripwire_details[0] == '\0') {
        return "All data pipeline tripwires clear.";
    }
    return s_log.tripwire_details;
}

/**
 * @brief Resets the tripwire state to CLEAR (used in diagnostic test suites).
 *
 * ### For Game Designers & Non-Programmers:
 * Clears any tripwire warning once the error has been noted or handled.
 *
 * ### For Systems Programmers & Engine Architects:
 * Restores s_log.tripwire_status to LOG_TRIPWIRE_CLEAR and reinitializes details.
 */
void log_reset_tripwire(void)
{
    s_log.tripwire_status = LOG_TRIPWIRE_CLEAR;
    str_safe_copy(
        s_log.tripwire_details,
        "All data pipeline tripwires clear.",
        sizeof(s_log.tripwire_details)
    );
}
```

A.7. Complete 5-Block VFS Precedence & Policy Configuration (vfs_routes.json)

Snapshot ID: snap_vfs_routes_cfg_whole | **Mode**: Whole | **File**: engine/config/io/vfs_routes.json (L1–L159 of 159 lines) |
Jump back to chapter text

```
{
  "block_1_metadata": {
    "project": "Tenebris and Lux",
    "file": "engine/config/io/vfs_routes.json",
    "id": "engine.config.io.vfs_routes",
    "layer": "Layer 0: Core Utilities / VFS Pathfinding",
    "version": "0.1.1-dev",
    "interface_rev": "Rev 5",
    "date": "2026-11-11",
    "author": "CreativeWorkzStudio LLC / ProfessorSeanEX",
    "repository": "https://github.com/Novaking34/tenebris-lux",
    "role": "Layered Virtual Filesystem routing specification defining 3-tier config precedence, candidate roots, and domain search directories.",
    "where_the_data_lives": {
      "topology": "Engine VFS Routing Table Specification",
      "path": "engine/config/io/vfs_routes.json",
      "header_binding": "engine/include/io/path_finder.h"
    },
    "what_it_enables": [
      "Deterministic 3-tier resolution: game overrides -> engine defaults -> presentation linkers",
      "Multi-root workspace probing (repo root, build/ subdir, or mounted mod directory)",
      "Zero-disk Safe-Mode virtual rom:// routing when disk files are absent",
      "Clean alignment to Rev 5 runtime/, diagnostics/, io/, and standards/ subtrees"
    ],
    "invariants": [
      "Zero dynamic heap allocation during VFS path probes (ALLOC_ZERO_HEAP)",
      "Resolved path output strictly < 512 bytes (PATH_FINDER_MAX_PATH)",
      "All resolved strings unconditionally null-terminated"
    ]
  },
  "block_2_opening": {
    "pre_conditions": {
      "path_finder_header": "io/path_finder.h",
      "bios_header": "io/io_fallback.h"
    },
    "static_declarations": {
      "subdomain_id": "io",
      "schema_id": "tenlux.data.io_vfs_routes.v1",
      "target_subsystem": "cog.path_finder",
      "max_root_bytes": 256,
      "max_path_bytes": 512
    },
    "generic_helper_contracts": {
      "mount_root": "path_finder_set_data_root",
      "load_routes": "path_finder_load_routes",
      "resolve": "path_finder_resolve",
      "resolve_asset": "path_finder_resolve_asset"
    },
    "capacities": {
      "config_tier_count": 3,
      "candidate_root_count": 6,
      "asset_root_count": 4,
      "domain_search_count": 36
    }
  },
  "block_3_body": {
    "default_data_root": "game/data",
    "env_override_var": "TL_DATA_ROOT",
    "block_path_traversal": true,
    "max_symlink_depth": 4,
    "allow_mod_mounts": true,
    "rom_fallback_enabled": true,
    "probe_await_on_corrupt": true,
    "config_precedence": [
      "game/config/",
      "engine/config/",
      "game/data/presentation/config/"
    ],
```

```
    "fallback_candidate_roots": [
      "game/data",
      "../game/data",
      "engine/config",
      "../engine/config",
      "data",
      "../data"
    ],
    "asset_roots": [
      "game/assets",
      "../game/assets",
      "assets",
      "../assets"
    ],
    "domain_search_paths": [
      "",
      "runtime/",
      "runtime/math/",
      "runtime/string/",
      "runtime/time/",
      "diagnostics/",
      "diagnostics/telemetry/",
      "diagnostics/debugger/",
      "diagnostics/taxonomy/",
      "diagnostics/formats/",
      "io/",
      "audio/",
      "viewport/",
      "input/",
      "standards/",
      "standards/blocks/",
      "mechanics/",
      "mechanics/items/",
      "mechanics/combat/",
      "mechanics/tactics/",
      "mechanics/campaign/",
      "mechanics/campaign/dispatch/",
      "world/",
      "world/geography/",
      "world/battlefields/",
      "world/terrains/",
      "entities/",
      "entities/classes/",
      "entities/roster/",
      "presentation/",
      "presentation/config/",
      "presentation/audio/",
      "presentation/ui/",
      "presentation/ui/fonts/",
      "presentation/visuals/",
      "presentation/visuals/palettes/"
    ],
    "diagnostic_aliases": {
      "log_prefix": "diagnostics/telemetry/",
      "debug_prefix": "diagnostics/debugger/",
      "io_prefix": "io/"
    }
  },
  "block_4_audit": {
    "audit_metrics": {
      "config_tier_count": 3,
      "candidate_root_count": 6,
      "asset_root_count": 4,
      "domain_search_count": 36,
      "zero_heap_compliant": true
    },
    "verification_tripwires": [
      {
        "name": "CONFIG_PRECEDENCE_3TIER",
        "rule": "config_precedence order preserves game overrides before engine defaults",
        "status": "VERIFIED_PASS"
      },
      {
        "name": "NO_STALE_SYSTEM_PREFIX",
        "rule": "diagnostic_aliases map cleanly to diagnostics/telemetry and diagnostics/debugger",
```

```
        "status": "VERIFIED_PASS"
      }
    ],
    "self_audit_target": "./build/tenebris-lux --audit"
  },
  "block_5_closing": {
    "execution_handshake": {
      "unit_test": "unit_path_finder_test",
      "rom_fallback_asset": "io/vfs_routes.json"
    },
    "navigation": {
      "parent_menu": "manifest.json",
      "root_manifest": "../manifest.json"
    }
  }
}
```

A.8. JSON Block 3 Payload Isolation (block_3_body of vfs_routes.json)

Snapshot ID: snap_vfs_routes_body_block | **Mode**: Block | **File**: engine/config/io/vfs_routes.json (L55–L125 of 159 lines) | Jump back to chapter text

```
  "block_3_body": {
    "default_data_root": "game/data",
    "env_override_var": "TL_DATA_ROOT",
    "block_path_traversal": true,
    "max_symlink_depth": 4,
    "allow_mod_mounts": true,
    "rom_fallback_enabled": true,
    "probe_await_on_corrupt": true,
    "config_precedence": [
      "game/config/",
      "engine/config/",
      "game/data/presentation/config/"
    ],
    "fallback_candidate_roots": [
      "game/data",
      "../game/data",
      "engine/config",
      "../engine/config",
      "data",
      "../data"
    ],
    "asset_roots": [
      "game/assets",
      "../game/assets",
      "assets",
      "../assets"
    ],
    "domain_search_paths": [
      "",
      "runtime/",
      "runtime/math/",
      "runtime/string/",
      "runtime/time/",
      "diagnostics/",
      "diagnostics/telemetry/",
      "diagnostics/debugger/",
      "diagnostics/taxonomy/",
      "diagnostics/formats/",
      "io/",
      "audio/",
      "viewport/",
      "input/",
      "standards/",
      "standards/blocks/",
      "mechanics/",
      "mechanics/items/",
      "mechanics/combat/",
      "mechanics/tactics/",
      "mechanics/campaign/",
      "mechanics/campaign/dispatch/",
```

```
      "world/",
      "world/geography/",
      "world/battlefields/",
      "world/terrains/",
      "entities/",
      "entities/classes/",
      "entities/roster/",
      "presentation/",
      "presentation/config/",
      "presentation/audio/",
      "presentation/ui/",
      "presentation/ui/fonts/",
      "presentation/visuals/",
      "presentation/visuals/palettes/"
    ],
    "diagnostic_aliases": {
      "log_prefix": "diagnostics/telemetry/",
      "debug_prefix": "diagnostics/debugger/",
      "io_prefix": "io/"
    }
  },
```

A.9. Scoped block_3_body JSON Accessor Contract (master_loader.h Sub-Assembly 3.2)

Snapshot ID: snap_master_loader_sub3_2 | **Mode**: Subassembly | **File**: engine/include/io/master_loader.h (L436–L608 of 987 lines) | Jump back to chapter text

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 3.2: Scoped `block_3_body` Accessors
Mission: Restrict all payload getters and array locators strictly inside the
         verified `[body_start, body_start + body_len)` span (or named `sub_obj`).

  ### For Game Designers & Non-Programmers:
  Reads gameplay properties cleanly from inside `block_3_body` only, preventing
  file headers or audit sections from ever colliding with gameplay field names.

  ### For Systems Programmers & Engine Architects:
  Delegates scope isolation and typed extraction exclusively to span-bounded
  `json_*_span` APIs with unconditional NUL termination on extracted strings.
--------------------------------------------------------------------------------
*/

/**
 * @brief Extracts a string value strictly within `block_3_body` (or named sub-object).
 *
 * ### For Game Designers & Non-Programmers:
 * Safely reads text properties (character names, formulas, IDs) from the data body
 * without accidentally matching header text outside Block 3.
 *
 * ### For Systems Programmers & Engine Architects:
 * Uses span-bounded `json_find_object_span()` + `json_get_string_span()` bounded to
 * `loader->body_len`. Unconditionally NUL-terminates `out_val` when `out_size > 0`.
 * @triggers `io.json.depth_bounds` if scan breaches `max_json_depth`.
 *
 * @param[in]  loader   Ready `master_loader_t` instance.
 * @param[in]  sub_obj  Optional sub-object key inside `block_3_body` (or NULL for root body).
 * @param[in]  key      Target property key inside resolved body/sub-object span.
 * @param[out] out_val  Caller string buffer.
 * @param[in]  out_size Capacity of `out_val` in bytes.
 * @return true on clean extraction; false if missing, wrong type, or invalid params.
 */
bool master_loader_get_string(const master_loader_t *loader,
                              const char *sub_obj,
                              const char *key,
                              char *out_val,
                              size_t out_size);

/**
 * @brief Extracts a signed integer strictly within `block_3_body` (or named sub-object).
```

```
 *
 * ### For Game Designers & Non-Programmers:
 * Reads whole-number gameplay values (HP caps, percentages, cooldown turns) cleanly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Span-bounded extraction via `json_get_int_span()` with overflow/syntax guards.
 * @triggers None on expected miss or parse rejection.
 *
 * @param[in]  loader  Ready `master_loader_t` instance.
 * @param[in]  sub_obj Optional sub-object key inside `block_3_body` (or NULL).
 * @param[in]  key     Target integer key.
 * @param[out] out_val Destination `int`.
 * @return true on valid integer parse; false otherwise.
 */
bool master_loader_get_int(const master_loader_t *loader,
                           const char *sub_obj,
                           const char *key,
                           int *out_val);

/**
 * @brief Extracts a non-negative `size_t` strictly within `block_3_body` (or `sub_obj`).
 *
 * ### For Game Designers & Non-Programmers:
 * Reads positive capacity counts (ring buffer slots, item stack limits) cleanly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Rejects negative JSON integers safely via `json_get_size_t_span()`.
 * @triggers None on expected miss or parse rejection.
 *
 * @param[in]  loader  Ready `master_loader_t` instance.
 * @param[in]  sub_obj Optional sub-object key inside `block_3_body` (or NULL).
 * @param[in]  key     Target non-negative integer key.
 * @param[out] out_val Destination `size_t`.
 * @return true on valid unsigned parse; false otherwise.
 */
bool master_loader_get_size_t(const master_loader_t *loader,
                              const char *sub_obj,
                              const char *key,
                              size_t *out_val);

/**
 * @brief Extracts a floating-point number strictly within `block_3_body` (or `sub_obj`).
 *
 * ### For Game Designers & Non-Programmers:
 * Reads decimal multipliers (damage ratios, audio volume dB, difficulty scales).
 *
 * ### For Systems Programmers & Engine Architects:
 * Bounded extraction via `json_get_float_span()`, rejecting NaN/Inf tokens cleanly.
 * @triggers None on expected miss or parse rejection.
 *
 * @param[in]  loader  Ready `master_loader_t` instance.
 * @param[in]  sub_obj Optional sub-object key inside `block_3_body` (or NULL).
 * @param[in]  key     Target float key.
 * @param[out] out_val Destination `float`.
 * @return true on valid finite float parse; false otherwise.
 */
bool master_loader_get_float(const master_loader_t *loader,
                             const char *sub_obj,
                             const char *key,
                             float *out_val);

/**
 * @brief Extracts a JSON boolean strictly within `block_3_body` (or `sub_obj`).
 *
 * ### For Game Designers & Non-Programmers:
 * Reads true/false feature switches from data manifests cleanly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Accepts strict JSON `true`/`false` literals only via `json_get_bool_span()`.
 * @triggers None on expected miss or parse rejection.
 *
 * @param[in]  loader  Ready `master_loader_t` instance.
 * @param[in]  sub_obj Optional sub-object key inside `block_3_body` (or NULL).
 * @param[in]  key     Target boolean key.
 * @param[out] out_val Destination `bool`.
```

```
 * @return true on valid boolean literal parse; false otherwise.
 */
bool master_loader_get_bool(const master_loader_t *loader,
                            const char *sub_obj,
                            const char *key,
                            bool *out_val);

/**
 * @brief Locates a JSON array span strictly within `block_3_body` (or `sub_obj`).
 *
 * ### For Game Designers & Non-Programmers:
 * Finds lists inside a file (such as enemy waves, settings menus, or item tables)
 * and counts how many entries exist.
 *
 * ### For Systems Programmers & Engine Architects:
 * Zero-copy slice via `json_find_array_span()` + `json_array_get_count()`.
 * @triggers `io.json.depth_bounds` if array bracket scan exceeds `max_json_depth`.
 *
 * @param[in]  loader    Ready `master_loader_t` instance.
 * @param[in]  sub_obj   Optional sub-object key inside `block_3_body` (or NULL).
 * @param[in]  array_key Target JSON array key.
 * @param[out] out_start Pointer to `[` delimiter inside loaded JSON buffer.
 * @param[out] out_len   Exact byte length from `[` through matching `]`.
 * @param[out] out_count Optional element count (may be NULL).
 * @return true if array found and well-formed; false otherwise.
 */
bool master_loader_find_array(const master_loader_t *loader,
                              const char *sub_obj,
                              const char *array_key,
                              const char **out_start,
                              size_t *out_len,
                              size_t *out_count);

/**
 * @brief Copies the `index`-th object element out of a located JSON array span.
 *
 * ### For Game Designers & Non-Programmers:
 * Steps through list entries one item at a time so loaders can import catalogs cleanly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Bounded copy via `json_array_get_object()` guaranteeing NUL-termination.
 * @triggers `io.json.depth_bounds` if element object exceeds `max_json_depth`.
 *
 * @param[in]  arr_start Pointer to `[` array boundary from `master_loader_find_array()`.
 * @param[in]  arr_len   Exact byte span of the array.
 * @param[in]  index     Zero-based element index.
 * @param[out] out_obj   Caller destination buffer for the extracted JSON object slice.
 * @param[in]  out_size  Byte capacity of `out_obj`.
 * @return true on valid extraction; false on out-of-bounds index or overflow.
 */
bool master_loader_get_array_object(const char *arr_start,
                                    size_t arr_len,
                                    size_t index,
                                    char *out_obj,
                                    size_t out_size);
```

A.10. Shared Zero-Heap JSON Block Resolver & VFS Parser Group

Snapshot ID: snap_io_fb_json_group | **Mode**: Group | **File**: engine/src/io/io_fallback.c (L779–L1194 of 3355 lines) | Jump back to chapter text

```
static bool io_fb_resolve_body_span(const char *json_text,
                                    size_t json_len,
                                    bool allow_strlen,
                                    const char **out_body_start,
                                    const char **out_body_end)
{
    if (json_text == NULL || out_body_start == NULL || out_body_end == NULL) {
        return false;
    }

    size_t effective_len = (json_len > 0U) ? json_len : (allow_strlen ? strlen(json_text) : 0U);
```

```
    if (effective_len == 0U || effective_len > IO_FB_JSON_MAX_BYTES) {
        return false;
    }

    const char *json_end = json_text + effective_len;
    const char *body_start = NULL;
    size_t body_len = 0U;
    if (!io_fb_find_block_span(json_text, json_end, "block_3_body", '{', '}',
                               &body_start, &body_len) ||
        body_start == NULL || body_len == 0U) {
        return false;
    }

    *out_body_start = body_start;
    *out_body_end = body_start + body_len;
    return true;
}
```

```
/**
 * @brief Hydrate VFS routing roots, 36+ domain paths, and symlink/traversal rules from JSON.
 *
 *   ### For Game Designers & Non-Programmers:
 *   Loads `vfs_routes.json` so asset folders, mod override precedence tiers, and
 *   diagnostic aliases stay configurable in data files rather than hardcoded C arrays.
 *
 *   ### For Systems Programmers & Engine Architects:
 *   Resolves `block_3_body` via `io_fb_resolve_body_span()`, seeds defaults with
 *   `io_fb_seed_default_vfs_routes()`, extracts precedence/root/domain arrays within
 *   fixed `.bss` caps, and forces `out_routes->loaded_from_data = false`.
 *
 * @param[in]  json_text  Raw `vfs_routes.json` text buffer.
 * @param[in]  json_len   Byte length of `json_text` (`0U` for `strlen()`).
 * @param[out] out_routes Hydrated output VFS routing descriptor.
 * @return `true` when body block parsed cleanly with non-empty root and domain lists.
 * @triggers Forces `loaded_from_data = false` so caller proves external disk provenance explicitly.
 */
bool io_fallback_parse_vfs_routes_json(const char *json_text,
                                       size_t json_len,
                                       io_fb_vfs_routes_t *out_routes)
{
    /* Step 1: Resolve `block_3_body` span and seed safe default routes */
    const char *body_start = NULL;
    const char *body_end = NULL;
    if (out_routes == NULL ||
        !io_fb_resolve_body_span(json_text, json_len, true, &body_start, &body_end)) {
        return false;
    }

    io_fb_seed_default_vfs_routes(out_routes);

    /* Step 2: Hydrate string identifiers and precedence/search directory arrays */
    char val_str[64];
    int val_int = 0;
    bool val_bool = false;

    if (io_fb_get_string_span(body_start, body_end, "default_data_root",
                              val_str, sizeof(val_str)) && val_str[0] != '\0') {
        str_safe_copy(out_routes->default_data_root, val_str,
                      sizeof(out_routes->default_data_root));
    }
    if (io_fb_get_string_span(body_start, body_end, "env_override_var",
                              val_str, sizeof(val_str)) && val_str[0] != '\0') {
        str_safe_copy(out_routes->env_override_var, val_str,
                      sizeof(out_routes->env_override_var));
    }

    (void)io_fb_get_string_array_span(body_start, body_end, "config_precedence",
                                      out_routes->config_precedence,
                                      IO_FB_MAX_PRECEDENCE_TIERS,
                                      &out_routes->config_precedence_count);
    (void)io_fb_get_string_array_span(body_start, body_end, "fallback_candidate_roots",
                                      out_routes->fallback_candidate_roots,
                                      IO_FB_MAX_VFS_CANDIDATES,
                                      &out_routes->candidate_root_count);
```

```
    (void)io_fb_get_string_array_span(body_start, body_end, "asset_roots",
                                      out_routes->asset_roots,
                                      IO_FB_MAX_ASSET_ROOTS,
                                      &out_routes->asset_root_count);
    (void)io_fb_get_string_array_span(body_start, body_end, "domain_search_paths",
                                      out_routes->domain_search_paths,
                                      IO_FB_MAX_VFS_DOMAIN_DIRS,
                                      &out_routes->domain_search_count);

    /* Step 3: Hydrate nested diagnostic domain alias prefixes if supplied */
    const char *alias_start = NULL;
    size_t alias_len = 0U;
    if (io_fb_find_block_span(body_start, body_end, "diagnostic_aliases", '{', '}',
                              &alias_start, &alias_len) &&
        alias_start != NULL && alias_len > 2U) {
        const char *alias_end = alias_start + alias_len;
        if (io_fb_get_string_span(alias_start, alias_end, "log_prefix",
                                  val_str, sizeof(val_str)) && val_str[0] != '\0') {
            str_safe_copy(out_routes->diag_log_prefix, val_str,
                          sizeof(out_routes->diag_log_prefix));
        }
        if (io_fb_get_string_span(alias_start, alias_end, "debug_prefix",
                                  val_str, sizeof(val_str)) && val_str[0] != '\0') {
            str_safe_copy(out_routes->diag_debug_prefix, val_str,
                          sizeof(out_routes->diag_debug_prefix));
        }
        if (io_fb_get_string_span(alias_start, alias_end, "io_prefix",
                                  val_str, sizeof(val_str)) && val_str[0] != '\0') {
            str_safe_copy(out_routes->diag_io_prefix, val_str,
                          sizeof(out_routes->diag_io_prefix));
        }
    }

    /* Step 4: Extract boolean security switches and verify non-empty route tables */
    if (io_fb_get_bool_span(body_start, body_end, "block_path_traversal", &val_bool)) {
        out_routes->block_path_traversal = val_bool;
    }
    if (io_fb_get_int_span(body_start, body_end, "max_symlink_depth", &val_int) &&
        val_int >= 0 && val_int <= 16) {
        out_routes->max_symlink_depth = val_int;
    }
    if (io_fb_get_bool_span(body_start, body_end, "allow_mod_mounts", &val_bool)) {
        out_routes->allow_mod_mounts = val_bool;
    }
    if (io_fb_get_bool_span(body_start, body_end, "rom_fallback_enabled", &val_bool)) {
        out_routes->rom_fallback_enabled = val_bool;
    }
    if (io_fb_get_bool_span(body_start, body_end, "probe_await_on_corrupt", &val_bool)) {
        out_routes->probe_await_on_corrupt = val_bool;
    }

    out_routes->loaded_from_data = false;
    return (out_routes->domain_search_count > 0U && out_routes->candidate_root_count > 0U);
}
```

A.11. Central Manifest Metadata & Registry Ontology (manifest.h Sub-Assembly 2.3)

Snapshot ID: snap_manifest_types_sub2_3 | **Mode**: Subassembly | **File**: engine/include/core/manifest.h (L207–L370 of 707 lines) | Jump back to chapter text

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 2.3: Structural Entity & Object Definitions
--------------------------------------------------------------------------------
Ontological types for architectural strata, memory models, and diagnostic audits.
--------------------------------------------------------------------------------
*/

/**
 * @enum module_layer
```

```
 * @brief Architectural layer strata within the hierarchical engine topology.
 *
 * For Designers: The 5 vertical tiers of the game engine, from deep bedrock
 * utilities (Layer 0) up to the game rules (Layer 1) and graphical display (Layer 2).
 *
 * For Programmers: Strict downward dependency hierarchy enforced at compile time.
 */
typedef enum module_layer {
    LAYER_0_CORE_UTIL = 0, /**< Layer 0: Foundational zero-dependency utils. */
    LAYER_1_SIMULATION,    /**< Layer 1: Headless game simulation engine. */
    LAYER_2_PRESENTATION,  /**< Layer 2: Graphical and text viewports. */
    LAYER_3_APPLICATION,   /**< Layer 3: Application entry and routing. */
    LAYER_4_TESTS,         /**< Layer 4: Verification test harness suites. */
    LAYER_COUNT
} module_layer_t;

/**
 * @enum alloc_policy
 * @brief Memory allocation contract governing a module's execution model.
 *
 * For Designers: Rules defining how a system manages memory, ensuring bedrock
 * systems never allocate heap memory to prevent crashes and stutter.
 *
 * For Programmers: Invariant memory policy verified during code audit.
 */
typedef enum alloc_policy {
    ALLOC_ZERO_HEAP = 0, /**< 100% stack or static layout; zero dynamic heap. */
    ALLOC_FIXED_POOL,    /**< Pre-allocated fixed-capacity buffer pools. */
    ALLOC_ARENA,         /**< Frame/scene temporary arena allocation. */
    ALLOC_DYNAMIC        /**< Managed dynamic heap allocation. */
} alloc_policy_t;

/**
 * @enum concurrency_policy
 * @brief Concurrency and reentrancy guarantees declared by a module.
 *
 * For Designers: Rules specifying whether a system can run simultaneously across
 * multiple CPU threads or must remain tied to the main display loop.
 *
 * For Programmers: Thread-safety model declared for scheduler evaluation.
 */
typedef enum concurrency_policy {
    CONCURRENCY_REENTRANT = 0, /**< Thread-safe, stateless pure functions. */
    CONCURRENCY_THREAD_LOCAL,  /**< Thread-local state isolation. */
    CONCURRENCY_MAIN_THREAD    /**< Main-thread execution affinity (GUI). */
} concurrency_policy_t;

/**
 * @enum diag_severity
 * @brief Five-tier diagnostic fault severity taxonomy for pre-flight machine audits.
 *
 * For Designers: A traffic light system for engine health:
 * PASS (green), INFO (blue notice), WARNING (yellow alert),
 * CRITICAL (orange, trigger safe fallback), FATAL (red, stop game safely).
 *
 * For Programmers: Strict escalation hierarchy: FATAL > CRITICAL > WARNING > PASS.
 */
typedef enum diag_severity {
    SEVERITY_PASS = 0, /**< Invariant holds cleanly; full readiness. */
    SEVERITY_INFO,     /**< Informational notice; no operational effect. */
    SEVERITY_WARNING,  /**< Non-fatal anomaly logged; execution proceeds. */
    SEVERITY_CRITICAL, /**< Subsystem offline; eligible for safe downgrade. */
    SEVERITY_FATAL     /**< Loose bolt detected; unrecoverable hard halt. */
} diag_severity_t;

/**
 * @enum preflight_status
 * @brief Aggregate pre-flight engine readiness state.
 *
 * For Designers: The overall health verdict after all engine systems test themselves.
 *
 * For Programmers: Composite state determined by highest severity finding across DAG.
 */
typedef enum preflight_status {
    PREFLIGHT_STATUS_OK = 0,    /**< Subsystems passed pre-flight audits. */
```

```
    PREFLIGHT_STATUS_WARNINGS,   /**< Non-fatal notices logged; system ready. */
    PREFLIGHT_STATUS_DOWNGRADED, /**< Critical fault downgraded to safe mode. */
    PREFLIGHT_STATUS_HALT_FATAL  /**< Loose bolt detected; engine halt. */
} preflight_status_t;

/**
 * @struct diag_result
 * @brief Individual diagnostic check outcome record.
 *
 * For Designers: A single test result card explaining if a test passed and any notes.
 *
 * For Programmers: Reentrant stack-allocated diagnostic result structure.
 */
typedef struct diag_result {
    bool            passed;     /**< Boolean pass/fail indicator. */
    diag_severity_t severity;   /**< Severity classification of the finding. */
    const char     *subsystem;  /**< Identifier of the audited subsystem. */
    const char     *check_name; /**< Descriptive name of the specific audit. */
    const char     *details;    /**< Human-readable diagnostic details. */
} diag_result_t;

/**
 * @struct module_metadata
 * @brief Compiled module metadata node binding identity, contracts, and tripwires.
 *
 * For Designers: The formal identity card of an engine subsystem, listing its name,
 * version, layer, who built it, what other systems it needs, and how to test it.
 *
 * For Programmers: Static immutable structure registered in central registry.
 */
typedef struct module_metadata {
    /* 1. Identity & Provenance */
    const char    *module_id;          /**< Canonical module ID. */
    const char    *file_name;          /**< Header file name. */
    module_layer_t layer;              /**< Architectural stratum. */
    struct {
        uint8_t major;
        uint8_t minor;
        uint8_t patch;
    } version;                         /**< Semantic version triplet. */
    uint8_t        interface_rev;      /**< Contract ABI revision counter. */
    const char    *last_updated;       /**< Full ISO 8601 date: YYYY-MM-DD. */
    const char    *description;        /**< Plain-English mission statement. */

    /* 2. Architectural Dependencies */
    size_t         dependency_count;   /**< Number of declared prerequisites. */
    const char    *dependencies[MANIFEST_MAX_DEPENDENCIES]; /**< Upstream IDs. */

    /* 3. Resource & Execution Contracts */
    alloc_policy_t       alloc_policy; /**< Memory allocation guarantee. */
    concurrency_policy_t concurrency;  /**< Concurrency / thread safety model. */
    uint32_t             max_budget_us;/**< Max latency budget (microseconds). */

    /* 4. Invariant Contracts */
    size_t         invariant_count;    /**< Number of documented invariants. */
    const char    *invariants[MANIFEST_MAX_INVARIANTS]; /**< Invariant rules. */

    /* 5. Pre-Flight Machine Self-Audit */
    diag_result_t (*preflight_check)(void); /**< Diagnostic callback. */
} module_metadata_t;

/**
 * @struct manifest_registry
 * @brief Central registry tracking registered modules, audit results, and status.
 *
 * For Designers: The master scoreboard storing every system's health report.
 *
 * For Programmers: Zero-heap fixed-capacity structure populated at engine boot.
 */
typedef struct manifest_registry {
    const module_metadata_t *modules[MANIFEST_MAX_MODULES]; /**< Pointers to registered module metadata nodes. */
    diag_result_t            results[MANIFEST_MAX_MODULES]; /**< Pre-flight check outcome records. */
    size_t                   count;                         /**< Number of currently registered modules. */
    size_t                   pass_count;                    /**< Total audits yielding SEVERITY_PASS. */
    size_t                   warn_count;                    /**< Total audits yielding SEVERITY_WARNING. */
```

```
    size_t                  critical_count;             /**< Total audits yielding SEVERITY_CRITICAL. */
    size_t                  fatal_count;                /**< Total audits yielding SEVERITY_FATAL. */
    preflight_status_t      status;                     /**< Aggregate pre-flight system readiness status. */
} manifest_registry_t;
```

A.12. Pre-Flight Loose-Bolts Engine Walk (manifest_run_preflight)

Snapshot ID: snap_manifest_fn_preflight | **Mode**: Function | **File**: engine/src/core/manifest.c (L412–L509 of 669 lines) | Jump back to chapter text

```c
/**
 * @brief Executes comprehensive pre-flight diagnostics across all registered modules.
 */
preflight_status_t manifest_run_preflight(manifest_registry_t *registry)
{
    if (registry == NULL) {
        debug_trap_tripwire("manifest", "NULL registry passed to manifest_run_preflight", __FILE__, __LINE__);
        return PREFLIGHT_STATUS_HALT_FATAL;
    }

    registry->pass_count = 0;
    registry->warn_count = 0;
    registry->critical_count = 0;
    registry->fatal_count = 0;

    /* Step 1: Active DAG Dependency Graph Verification */
    for (size_t i = 0; i < registry->count; i++) {
        const module_metadata_t *mod = registry->modules[i];
        if (mod == NULL) continue;

        for (size_t d = 0; d < mod->dependency_count; d++) {
            const char *req_dep = mod->dependencies[d];
            if (req_dep == NULL) continue;

            bool found = false;
            for (size_t j = 0; j < registry->count; j++) {
                if (registry->modules[j] != NULL &&
                    strcmp(registry->modules[j]->module_id, req_dep) == 0) {
                    found = true;
                    break;
                }
            }

            if (!found) {
                diag_result_t dep_res;
                dep_res.passed = false;
                dep_res.severity = SEVERITY_FATAL;
                dep_res.subsystem = mod->module_id;
                dep_res.check_name = "Dependency DAG";
                dep_res.details =
                    "Fatal: Unregistered upstream dependency declared.";
                registry->results[i] = dep_res;
                registry->fatal_count++;
                registry->status = PREFLIGHT_STATUS_HALT_FATAL;
                debug_trap_tripwire("manifest", "Unregistered upstream DAG dependency detected", __FILE__, __LINE__);
                return registry->status;
            }
        }
    }

    /* Step 2: Individual Module Pre-Flight Verification */
    for (size_t i = 0; i < registry->count; i++) {
        const module_metadata_t *mod = registry->modules[i];
        if (mod == NULL) {
            continue;
        }

        diag_result_t res;
        if (mod->preflight_check != NULL) {
            res = mod->preflight_check();
        } else {
            res.passed = true;
```

```
            res.severity = SEVERITY_PASS;
            res.subsystem = mod->module_id;
            res.check_name = "Presence";
            res.details = "No active preflight check defined.";
        }

        registry->results[i] = res;

        bool is_fatal = (res.severity == SEVERITY_FATAL) ||
            (!res.passed && res.severity != SEVERITY_WARNING &&
             res.severity != SEVERITY_CRITICAL);

        if (is_fatal) {
            registry->fatal_count++;
            registry->status = PREFLIGHT_STATUS_HALT_FATAL;
            return registry->status;
        } else if (res.severity == SEVERITY_CRITICAL) {
            registry->critical_count++;
        } else if (res.severity == SEVERITY_WARNING) {
            registry->warn_count++;
        } else {
            registry->pass_count++;
        }
    }

    /* Step 3: Determine aggregate system status */
    if (registry->critical_count > 0) {
        registry->status = PREFLIGHT_STATUS_DOWNGRADED;
    } else if (registry->warn_count > 0) {
        registry->status = PREFLIGHT_STATUS_WARNINGS;
    } else {
        registry->status = PREFLIGHT_STATUS_OK;
    }

    return registry->status;
}
```

A.13. Application Audit Integration (game/src/main.c BLOCK 4)

Snapshot ID: snap_main_preflight_block4 | **Mode**: Block | **File**: game/src/main.c (L193–L334 of 524 lines) | Jump back to chapter text

```
// ============================================================================
// BLOCK 4: AUDIT PROCEDURES BLOCK
// Stratum: Layer 3 (Application Entry) | Machine: Presentation Router
// ============================================================================

// ----------------------------------------------------------------------------
// [ SUB-ASSEMBLY 4.1: COMPREHENSIVE SUBSYSTEM SELF-AUDIT SUITE ]
// Mission: Execute pre-flight manifest checks, sim invariants, and GUI bounds.
// ----------------------------------------------------------------------------

/**
 * @brief Executes the comprehensive engine self-audit suite and manifests.
 *
 * ### Machine Role & Mission
 * Serves as the `--audit` diagnostic tripwire. Executes:
 * 1. Pre-flight verification across all registered modules with DAG checks.
 * 2. Layer 1 simulation invariant audit (`game_run_self_audit()`).
 * 3. Layer 2 GUI viewport layout and collision audit (`gui_audit_layout()`).
 * Returns non-zero if any invariant check or pre-flight test fails.
 *
 * ### Ontological Classification
 * - Stratum: Layer 3 (Application Entry & Presentation Router)
 * - Machine: app.entry.main (Executable Entry & Presentation Router)
 * - Assembly: Diagnostic Self-Audit & Pre-Flight Verification
 * - Cog: Full Engine Diagnostics & Invariant Self-Audit Suite
 *
 * ### Execution & Resource Contract
 * - Memory Model: ALLOC_ZERO_HEAP (Stack allocation only).
 * - Concurrency: CONCURRENCY_MAIN_THREAD (Synchronous diagnostic sweep).
 * - Latency Budget: < 5000 microseconds execution time.
```

```
 *
 * @return 0 if all invariant checks pass; non-zero if any check fails.
 */
static int run_full_subsystem_audit(void)
{
    log_init(NULL);
    LOG_INFO(LOG_CAT_LOG, "Audit", 0.0f, "Starting comprehensive diagnostic self-audit");

    /* Step 1: Initialize central manifest and run pre-flight diagnostics */
    manifest_registry_t registry;
    init_engine_manifest(&registry);
    manifest_run_preflight(&registry);
    manifest_print_report(&registry);

    /* Step 2: Evaluate pre-flight outcome for fatal loose-bolts */
    if (registry.status == PREFLIGHT_STATUS_HALT_FATAL) {
        LOG_FATAL(LOG_CAT_LOG, "Audit", -1.0f, "Pre-flight failed due to loose bolts!");
        fprintf(stderr, "[FATAL] Pre-flight failed due to loose bolts!\n");
        log_shutdown();
        return 1;
    }

    /* Step 2b: Run telemetry subsystem diagnostic audit */
    diag_result_t log_diag = log_audit(false);
    if (!log_diag.passed) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Core telemetry audit failed: %s", log_diag.details);
        fprintf(stderr, "[FAIL] Core telemetry subsystem audit failed: %s\n", log_diag.details);
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0a: Telemetry subsystem & grade invariants verified.\n");

    /* Step 2c: Run debugger subsystem diagnostic audit */
    diag_result_t debug_diag = debug_audit(false);
    if (!debug_diag.passed) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Core debugger audit failed: %s", debug_diag.details);
        fprintf(stderr, "[FAIL] Core debugger subsystem audit failed: %s\n", debug_diag.details);
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0b: Debugger subsystem & state rewind invariants verified.\n");

    /* Step 2d: Run frame pacing orchestrator diagnostic audit */
    if (!clock_pacer_audit()) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Clock pacer orchestrator audit failed");
        fprintf(stderr, "[FAIL] Clock pacer orchestrator audit failed.\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0c: Frame pacing & accumulator invariants verified.\n");

    /* Step 2e: Run battle formula orchestrator diagnostic audit */
    if (!formula_eval_audit()) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Formula evaluator orchestrator audit failed");
        fprintf(stderr, "[FAIL] Formula evaluator orchestrator audit failed.\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0d: Battle formulas & affinity invariants verified.\n");

    /* Step 2f: Run audio bus orchestrator diagnostic audit */
    if (!audio_bus_preflight_audit()) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Audio event bus audit failed");
        fprintf(stderr, "[FAIL] Audio event bus orchestrator audit failed.\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0e: Audio event bus & multi-channel mixer invariants verified.\n");

    /* Step 2g: Run Movie Reel presentation orchestrator diagnostic audit */
    if (!reel_audit()) {
```

```
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Movie Reel orchestrator audit failed");
        fprintf(stderr, "[FAIL] Movie Reel orchestrator audit failed.\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 0f: Movie Reel scene strip & depth kinematics invariants verified.\n");

    /* Step 3: Run simulation engine invariant audit */
    if (game_run_self_audit() != 0) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "Core game simulation audit failed");
        fprintf(stderr, "[FAIL] Core game simulation audit failed.\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }

    /* Step 4: Run GUI layout bounds and collision audit */
    if (!gui_audit_layout()) {
        LOG_CRITICAL(LOG_CAT_LOG, "Audit", -0.8f, "GUI viewport layout bounds exceeded");
        fprintf(stderr, "[FAIL] GUI viewport layout bounds exceeded!\n");
        debug_shutdown();
        log_shutdown();
        return 1;
    }
    printf("[PASS] Test 5: GUI viewport layout invariants verified.\n");

    /* Step 5: Report comprehensive pass summary */
    printf("============================================================\n");
    printf("Audit Result: ALL INVARIANT CHECKS PASSED (9/9 + Pre-Flight)\n");
    printf("============================================================\n");

    LOG_INFO(LOG_CAT_LOG, "Audit", 1.0f, "All invariant checks passed successfully (9/9 + Pre-Flight)");
    debug_shutdown();
    log_shutdown();
    return 0;
}
```

A.14. Layer 1 Deterministic Simulation State Audit (sim_state_audit)

Snapshot ID: snap_game_fn_audit | **Mode**: Function | **File**: game/src/hero/sim_state.c (L315–L376 of 420 lines) | Jump back to chapter text

```
game_audit_report_t sim_state_audit(const player_state_t *state)
{
    game_audit_report_t report;
    report.is_valid = true;
    report.error_count = 0;
    report.error_log[0] = '\0';

    if (state == NULL) {
        report.is_valid = false;
        report.error_count = 1;
        snprintf(report.error_log, sizeof(report.error_log),
                 "[ERROR] Player state pointer is NULL.");
        return report;
    }

    if (state->shield_power < 0 || state->shield_power > MAX_RESONANCE) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Shield power (%d) outside bounds [0, %d]. ",
            state->shield_power, MAX_RESONANCE);
    }

    if (state->sword_edge < 0 || state->sword_edge > MAX_RESONANCE) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
```

```
            "[ERROR] Sword edge (%d) outside bounds [0, %d]. ",
            state->sword_edge, MAX_RESONANCE);
    }

    if (state->moral_sway < -100 || state->moral_sway > 100) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Moral sway (%d) outside bounds [-100, 100]. ",
            state->moral_sway);
    }

    if (state->stance < 0 || state->stance >= STANCE_COUNT) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Stance enum (%d) outside valid range [0, %d). ",
            state->stance, STANCE_COUNT);
    }

    if (state->location < 0 || state->location >= LOC_COUNT) {
        report.is_valid = false;
        report.error_count++;
        snprintf(report.error_log + strlen(report.error_log),
            sizeof(report.error_log) - strlen(report.error_log),
            "[ERROR] Location enum (%d) outside valid range [0, %d). ",
            state->location, LOC_COUNT);
    }

    return report;
}
```

A.15. Layer 2 640x360 Viewport & 2.5D Movie Reel Data Structures (gui_types.h Sub-Assembly 2.3)

Snapshot ID: snap_gui_types_sub2_3 | **Mode**: Subassembly | **File**: engine/include/viewport/raster/gui_types.h (L138–L448 of 531 lines) | Jump back to chapter text

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 2.3: Structural Entity & Object Definitions
--------------------------------------------------------------------------------
Core types, layout bounding rectangles, color models, view modes, and widget
structures for the desktop presentation engine.
--------------------------------------------------------------------------------
*/

/**
 * @enum gui_screen_mode
 * @brief Presentation screen views and modal navigation states.
 *
 * For Designers:
 * Represents the 5 primary game screens accessible via the top navigation tabs:
 * Tactical Grid, Hero Dossier, Inventory, Provincial Map, and Chronicle Codex.
 *
 * For Programmers:
 * Bounded enumerator indexed [0, 4] matching tab descriptors.
 */
typedef enum gui_screen_mode {
    GUI_SCREEN_TACTICAL = 0,    /**< 2.5D / 2D Tactical battlefield & combat */
    GUI_SCREEN_STATUS,          /**< Full protagonist dossier & attributes */
    GUI_SCREEN_INVENTORY,       /**< Bag, key items, Grandma's Watch */
    GUI_SCREEN_WORLD_MAP,       /**< Provincial realm topology & travel */
    GUI_SCREEN_CHRONICLE        /**< Complete lore codex & history of deeds */
} gui_screen_mode_t;

/**
 * @struct gui_tab
 * @brief Top navigation tab descriptor.
```

```
 *
 * For Designers:
 * A clickable header tab bar button linking directly to a primary screen.
 *
 * For Programmers:
 * Bounded rectangular button descriptor with single hotkey accelerator.
 */
typedef struct gui_tab {
    int x;
    int y;
    int width;
    int height;
    const char *label;
    gui_screen_mode_t screen;
    char hotkey;
} gui_tab_t;

/**
 * @enum gui_view_mode
 * @brief Display view mode for the tactical world viewport.
 *
 * For Designers:
 * Switches between classic top-down orthographic grid and dimetric isometric 2.5D.
 *
 * For Programmers:
 * Pure discrete rendering mode toggle.
 */
typedef enum gui_view_mode {
    GUI_VIEW_2D_FLAT = 0,         /**< Classic orthographic top-down grid */
    GUI_VIEW_25D_ISOMETRIC = 1    /**< Tactical dimetric grid with Z-height */
} gui_view_mode_t;

/**
 * @struct gui_palette
 * @brief Hardware and software color palette representation.
 *
 * For Designers:
 * The complete thematic color palette defining background, panel, text,
 * relic, morale, and tactical tile colors.
 *
 * For Programmers:
 * Pixel color values packed for X11 TrueColor (0xRRGGBB).
 */
typedef struct gui_palette {
    unsigned long bg;             /**< Obsidian canvas */
    unsigned long panel;          /**< Granite panel background */
    unsigned long border;         /**< Carved stone rim */
    unsigned long text_main;      /**< Ivory text */
    unsigned long text_muted;     /**< Silver secondary text */
    unsigned long shield_gold;    /**< Radiant Lux gold */
    unsigned long gold_bright;    /**< Lux edge highlight */
    unsigned long sword_violet;   /**< Tenebrous violet */
    unsigned long violet_bright;  /**< Tenebrous edge highlight */
    unsigned long virtue_green;   /**< Emerald virtue */
    unsigned long malice_red;     /**< Crimson malice */
    unsigned long btn_bg;         /**< Button slate */
    unsigned long btn_border;     /**< Button rim */
    unsigned long btn_hover;      /**< Button hover highlight */
    unsigned long tile_grid;      /**< Tactical grid lines */
    unsigned long tile_cobble;    /**< Cobblestone fill */
    unsigned long tile_grass;     /**< Meadow grass fill */
    unsigned long tile_water;     /**< Abyssal water fill */
    unsigned long tile_wall;      /**< Rampart stone wall */
    unsigned long tactical_move;  /**< Move range highlight */
    unsigned long tactical_atk;   /**< Attack range highlight */
    unsigned long shadow;         /**< Drop shadow tint */
} gui_palette_t;

/**
 * @struct gui_panel_rect
 * @brief Named bounding box representing a major desktop presentation zone.
 *
 * For Designers:
 * A positioned rectangular section of the screen with a label title.
 *
```

```
 * For Programmers:
 * Simple bounding rectangle with non-owning title string pointer.
 */
typedef struct gui_panel_rect {
    int x;
    int y;
    int width;
    int height;
    const char *title;
} gui_panel_rect_t;

/**
 * @struct gui_layout
 * @brief Unified desktop presentation layout geometry.
 *
 * For Designers:
 * Specifies the arrangement of header, 2D viewport, hero status, chronicle log,
 * and command dock across the window.
 *
 * For Programmers:
 * Fixed non-overlapping panel partitions filling window dimensions.
 */
typedef struct gui_layout {
    int window_width;
    int window_height;
    gui_panel_rect_t header;       /**< Top title banner and cycle indicator */
    gui_panel_rect_t viewport_2d;  /**< 2D / 2.5D tactical map viewport */
    gui_panel_rect_t status;       /**< Right hero vitals & relics panel */
    gui_panel_rect_t chronicle;    /**< Bottom left narrative log */
    gui_panel_rect_t command_dock; /**< Bottom right tactical action dock */
} gui_layout_t;

/**
 * @struct gui_button
 * @brief Interactive button widget definition in screen coordinates.
 *
 * For Designers:
 * A button in the tactical command dock with label, hotkey, and active/hover states.
 *
 * For Programmers:
 * Screen-coordinate widget descriptor with hover and active state flags.
 */
typedef struct gui_button {
    int x;
    int y;
    int width;
    int height;
    const char *label;
    const char *command;
    char hotkey;
    bool is_hovered;
    bool is_active;
} gui_button_t;

/**
 * @struct gui_log
 * @brief Scrollable ring-buffer storing recent chronicle narrative events.
 *
 * For Designers:
 * Stores the running history of recent battle logs and story narration.
 *
 * For Programmers:
 * Fixed 2D character matrix with count index, zero dynamic heap.
 */
typedef struct gui_log {
    char lines[GUI_MAX_LOG_LINES][GUI_LOG_ENTRY_LEN];
    int count;
} gui_log_t;

/**
 * @enum gui_asset_id
 * @brief Canonical asset identifier for characters, monsters, terrain, and relics.
 *
 * For Designers:
 * Defines generic engine visual slots (hero, minion, boss, tile, relic, vfx)
```

```
 * mapped cleanly to game domain narrative aliases (Elijah, Dream Eater, Shield of Lux).
 *
 * For Programmers:
 * Tagged enumeration representing integer asset slot keys in the asset registry.
 */
typedef enum gui_asset_id {
    ASSET_NONE = 0,
    ASSET_ENTITY_PROTAGONIST_1 = 1,  /**< Primary hero entity slot */
    ASSET_ENTITY_PROTAGONIST_2 = 2,  /**< Secondary companion entity slot */
    ASSET_ENTITY_PROTAGONIST_3 = 3,  /**< Tertiary companion entity slot */
    ASSET_ENTITY_ENEMY_MINION_1 = 10, /**< Standard enemy minion slot 1 */
    ASSET_ENTITY_ENEMY_MINION_2 = 11, /**< Standard enemy minion slot 2 */
    ASSET_ENTITY_BOSS_1        = 12, /**< Major boss entity slot */
    ASSET_TILE_PATH            = 20, /**< Standard walkable terrain tile */
    ASSET_TILE_PLAINS          = 21, /**< Vegetative / meadow terrain tile */
    ASSET_TILE_STRUCTURE       = 22, /**< Solid wall / obstacle terrain tile */
    ASSET_TILE_HAZARD          = 23, /**< Void / chasm terrain tile */
    ASSET_TILE_LIQUID          = 24, /**< Water / fluid terrain tile */
    ASSET_RELIC_DEFENSIVE      = 30, /**< Primary defensive ward / shield icon */
    ASSET_RELIC_OFFENSIVE      = 31, /**< Primary offensive blade icon */
    ASSET_VFX_BARRIER          = 40, /**< Radiant defensive arc visual effect */
    ASSET_VFX_SLASH            = 41, /**< Striking slash visual effect */

    /* Game Domain Aliases for Backward Compatibility during v0.1.1 iteration */
    ASSET_HERO_ELIJAH          = ASSET_ENTITY_PROTAGONIST_1,
    ASSET_HERO_KATRINA         = ASSET_ENTITY_PROTAGONIST_2,
    ASSET_HERO_CASATTI         = ASSET_ENTITY_PROTAGONIST_3,
    ASSET_ENEMY_DREAM_EATER    = ASSET_ENTITY_ENEMY_MINION_1,
    ASSET_ENEMY_SHADOW_STALKER = ASSET_ENTITY_ENEMY_MINION_2,
    ASSET_ENEMY_DARK_LORD      = ASSET_ENTITY_BOSS_1,
    ASSET_TILE_COBBLESTONE     = ASSET_TILE_PATH,
    ASSET_TILE_GRASS           = ASSET_TILE_PLAINS,
    ASSET_TILE_STONE_WALL      = ASSET_TILE_STRUCTURE,
    ASSET_TILE_VOID_CRAG       = ASSET_TILE_HAZARD,
    ASSET_TILE_WATER           = ASSET_TILE_LIQUID,
    ASSET_RELIC_SHIELD_LUX     = ASSET_RELIC_DEFENSIVE,
    ASSET_RELIC_SWORD_TENEBRIS = ASSET_RELIC_OFFENSIVE,
    ASSET_VFX_LUX_BARRIO       = ASSET_VFX_BARRIER,
    ASSET_VFX_CUTTATIO_SLASH   = ASSET_VFX_SLASH
} gui_asset_id_t;

/**
 * @enum gui_asset_kind
 * @brief Asset pipeline origin and representation kind.
 *
 * For Designers:
 * Distinguishes procedural vector art drawn directly by code from artist bitmaps
 * loaded from disk, ensuring the game is 100% playable with zero external art.
 *
 * For Programmers:
 * Discrete rendering mode indicator governing raster dispatch.
 */
typedef enum gui_asset_kind {
    ASSET_KIND_PROCEDURAL = 0,    /**< Synthesized geometric vector art */
    ASSET_KIND_BITMAP = 1         /**< External PNG/BMP artist spritesheet */
} gui_asset_kind_t;

/**
 * @struct gui_asset
 * @brief Asset specification in the Asset Abstraction Layer.
 *
 * For Designers:
 * Describes an individual graphic asset: its slot ID, display name, kind (procedural vs bitmap),
 * pixel dimensions, and external artist file path.
 *
 * For Programmers:
 * Lightweight asset metadata record with non-owning path pointers.
 */
typedef struct gui_asset {
    gui_asset_id_t id;
    const char *name;
    gui_asset_kind_t kind;
    int width;
    int height;
    const char *artist_file_path; /**< Future artist spritesheet path */
```

```
} gui_asset_t;

/**
 * @struct gui_iso_point
 * @brief Isometric screen point with elevation depth.
 *
 * For Designers:
 * Represents a point in 2.5D space projected onto the screen with vertical elevation.
 *
 * For Programmers:
 * Triplet of screen coordinates (screen_x, screen_y) and discrete elevation_z.
 */
typedef struct gui_iso_point {
    int screen_x;
    int screen_y;
    int elevation_z;
} gui_iso_point_t;

/**
 * @struct gui_grid_coord
 * @brief Discrete grid coordinate.
 *
 * For Designers:
 * Specifies a specific square tile on the battlefield grid (gx, gy).
 *
 * For Programmers:
 * Integer grid pair with zero floating point inaccuracy.
 */
typedef struct gui_grid_coord {
    int gx;
    int gy;
} gui_grid_coord_t;

/**
 * @struct gui_depth_entity
 * @brief Depth-sorted entity render packet for the Painter's Algorithm.
 *
 * For Designers:
 * An entity (hero, monster, or boss) queued for drawing with proper depth sorting
 * so entities in front overlap entities behind them correctly.
 *
 * For Programmers:
 * Lightweight packet containing calculated depth sorting key, screen coordinates,
 * asset identifier, facing direction, and HP indicators.
 */
typedef struct gui_depth_entity {
    int depth_key;               /**< (gx + gy) * 1000 + gz * 10 */
    int screen_x;
    int screen_y;
    gui_asset_id_t asset_id;
    int facing;                  /**< 0: South, 1: West, 2: East, 3: North */
    int hp;
    int max_hp;
    bool is_hero;
} gui_depth_entity_t;
```

A.16. Flight Log Leaf Vocabulary & Entry Layout (log_types.h Sub-Assembly 2.3)

Snapshot ID: snap_log_types_sub2_3 | **Mode**: Subassembly | **File**: engine/include/log/log_types.h (L153–L303 of 421 lines) | Jump back to chapter text

```
/*
-------------------------------------------------------------------------------
Sub-Assembly 2.3: Shared Structural Entity & Telemetry Value Definitions
-------------------------------------------------------------------------------
Canonical non-anonymous enums and POD structures shared across operational and
quarantined BIOS telemetry cogs.
-------------------------------------------------------------------------------
*/

/**
 * @enum log_severity
```

```
 * @brief Graded severity taxonomy for engine diagnostics and telemetry events.
 *
 * Ordered from highest severity (FATAL) down to finest detail (TRACE).
 *
 * ### For Game Designers & Non-Programmers:
 * Defines how critical an event is:
 *   - FATAL/CRITICAL: Showstoppers that threaten gameplay continuity.
 *   - ERROR/WARNING: Problems handled gracefully with automatic fallbacks.
 *   - INFO: Normal story, quest, and milestone announcements.
 *   - DEBUG/TRACE: Granular technical values for combat balance and timing.
 *
 * ### For Systems Programmers & Engine Architects:
 * Strict integer ordering enables threshold filtering and weight-table lookup.
 */
typedef enum log_severity {
    LOG_SEV_FATAL = 0,    /**< Unrecoverable failure or fatal state corruption. */
    LOG_SEV_CRITICAL,     /**< Severe failure threatening gameplay loop. */
    LOG_SEV_ERROR,        /**< Discrete operation failure with fallback. */
    LOG_SEV_WARNING,      /**< Suspicious condition or degraded state. */
    LOG_SEV_INFO,         /**< Major milestone or narrative state shift. */
    LOG_SEV_DEBUG,        /**< Granular spatial, coordinate, or stance telemetry. */
    LOG_SEV_TRACE,        /**< High-frequency per-tick calculation telemetry. */
    LOG_SEV_COUNT         /**< Enumerator bound sentinel (not a valid severity). */
} log_severity_t;

/**
 * @enum log_category
 * @brief Functional categorization of logging events for focused channel filtering.
 *
 * ### For Game Designers & Non-Programmers:
 * Divides events into distinct thematic channels (`CHARACTER`, `MESSAGE`,
 * `EVENT`, `IO`, `LOG`, `DEBUG`) so designers can inspect one lane at a time.
 *
 * ### For Systems Programmers & Engine Architects:
 * Fixed enum indexes into static descriptor rows and categorical archive streams.
 */
typedef enum log_category {
    LOG_CAT_DEBUG = 0,    /**< Low-level developer engine trace. */
    LOG_CAT_LOG,          /**< Standard system lifecycle and startup logs. */
    LOG_CAT_EVENT,        /**< Significant world, combat, or quest trigger. */
    LOG_CAT_MESSAGE,      /**< Narrative dialogue or NPC communicative prompt. */
    LOG_CAT_CHARACTER,    /**< Actor/enemy spatial and equipment stance action. */
    LOG_CAT_IO,           /**< Filesystem, VFS, JSON & BIOS ROM I/O telemetry. */
    LOG_CAT_COUNT         /**< Enumerator bound sentinel (not a valid category). */
} log_category_t;

/**
 * @struct log_entry
 * @brief Static telemetry record stored in memory ring buffer and emitted to disk.
 *
 * ### For Game Designers & Non-Programmers:
 * A discrete "black box" flight recorder entry capturing when an event occurred,
 * which actor or module triggered it, and the normalized scalar grade [-1.0..+1.0].
 *
 * ### For Systems Programmers & Engine Architects:
 * 100% stack/static fixed-layout record. Source file/func pointers must point to
 * static `.rodata` string literals (`__FILE__`, `__func__`) rather than stack storage.
 */
typedef struct log_entry {
    uint64_t       sequence_id;                 /**< Monotonic sequence identifier. */
    uint64_t       monotonic_us;                /**< Monotonic clock delta in microseconds. */
    char           iso_timestamp[32];           /**< UTC ISO 8601 timestamp string. */
    log_severity_t severity;                    /**< Graded severity rank. */
    log_category_t category;                    /**< Domain channel category. */
    char           entity[LOG_MAX_ENTITY_LEN + 1];   /**< Actor or system entity identifier. */
    float          grade;                       /**< Normalized continuum [-1.0f .. +1.0f]. */
    int            grade_percent;               /**< Scaled integer readout [-100 .. +100]. */
    const char    *file;                        /**< Source file anchor (__FILE__). */
    int            line;                        /**< Source line anchor (__LINE__). */
    const char    *func;                        /**< Source function anchor (__func__). */
    char           message[LOG_MAX_MESSAGE_LEN + 1]; /**< NUL-terminated message payload. */
} log_entry_t;

/**
 * @enum log_tripwire
```

```
 * @brief Diagnostic tripwire state for the data-driven logging pipeline.
 *
 * ### For Game Designers & Non-Programmers:
 * Circuit breakers catching corrupt telemetry JSON files so Safe Mode activates
 * automatically without crashing the session.
 *
 * ### For Systems Programmers & Engine Architects:
 * Latching status enum reported across shared `log_fallback_parse_config_json()`
 * and verified during Block 4 self-audits.
 */
typedef enum log_tripwire {
    LOG_TRIPWIRE_CLEAR = 0,        /**< No tripwires active; data pipeline healthy. */
    LOG_TRIPWIRE_MANIFEST_MISSING, /**< Telemetry manifest could not be found. */
    LOG_TRIPWIRE_MANIFEST_INVALID, /**< Manifest unparseable or lacks menu entries. */
    LOG_TRIPWIRE_SETTING_MISSING,  /**< Designated setting JSON profile not found. */
    LOG_TRIPWIRE_SETTING_INVALID,  /**< Setting file syntax or attributes corrupted. */
    LOG_TRIPWIRE_BOUNDS_VIOLATION  /**< Configuration violated byte/file limits. */
} log_tripwire_t;

/**
 * @struct log_config
 * @brief Runtime configuration for the logging subsystem.
 *
 * ### For Game Designers & Non-Programmers:
 * Configurable profile selecting log folder paths, max archive size, minimum
 * severity cutoff, and console color output.
 *
 * ### For Systems Programmers & Engine Architects:
 * Hydrated cleanly from `engine/config/diagnostics/telemetry/` or `.rodata` ROM;
 * `loaded_from_data` stays strictly `false` whenever hydrated from BIOS/ROM.
 */
typedef struct log_config {
    char           log_dir[64];            /**< Target directory for log files. */
    char           base_filename[64];      /**< Primary active log filename. */
    size_t         max_file_size_bytes;    /**< Byte threshold triggering file rotation. */
    size_t         max_rotated_files;      /**< Maximum archive files retained (1..25). */
    log_severity_t min_severity;           /**< Cutoff severity (looser levels filtered). */
    bool           save_by_category;       /**< Write per-category archives under log_dir. */
    bool           echo_to_console;        /**< Mirror formatted logs to console streams. */
    bool           use_colors;             /**< Emit ANSI color escapes on console. */
    char           profile_name[32];       /**< Active configuration preset identifier. */
    bool           loaded_from_data;       /**< True only when hydrated from live JSON VFS. */
} log_config_t;

/**
 * @struct log_stats
 * @brief Aggregate runtime telemetry report constructed by inspection procedures.
 *
 * ### For Game Designers & Non-Programmers:
 * Summary scorecard showing total logged events, active file byte count,
 * archive rotations, and whether disk or Safe-Mode presets are active.
 *
 * ### For Systems Programmers & Engine Architects:
 * Populated atomically by `log_execute_inspect()` with zero heap allocations.
 */
typedef struct log_stats {
    uint64_t       total_entries;       /**< Total log entries recorded since init. */
    size_t         ring_buffer_count;   /**< Active entries retained in circular ring. */
    size_t         rotation_count;      /**< Number of archive rotations performed. */
    size_t         active_file_bytes;   /**< Current size in bytes of primary log file. */
    log_severity_t min_severity;        /**< Active minimum severity cutoff. */
    bool           is_active;           /**< True when logging engine is initialized. */
    bool           loaded_from_data;    /**< True if active config originated from disk JSON. */
    char           active_profile[32];  /**< Active telemetry preset identifier. */
} log_stats_t;
```

A.17. Flight Recorder Formatted Entry Dispatch API (log.h Sub-Assembly 3.3)

Snapshot ID: snap_log_ring_api_sub3_3 | **Mode**: Subassembly | **File**: engine/include/log/log.h (L406–L484 of 954 lines) | Jump back to chapter text

```
406 /*
```

```
/* ------------------------------------------------------------------------------
Sub-Assembly 3.3: Formatted Entry Dispatch Cogs
Mission: Format, serialize, and emit structured flight recorder entries.
@triggers
  - Tripwires      : LOG_TRIPWIRE_SETTING_INVALID / BOUNDS_VIOLATION -> probe "log.write_telemetry"
  - Block 4 Audits : log_validate_telemetry_integrity(), log_preflight()
  - Layer 4 Tests  :
      * tests/unit/log/test_log_unit.c
        [Section 3: INVARIANTS & CONSTRAINTS / test_log_unit_assertions_and_guarantees]
        [Section 4: RING BUFFER FIFO OVERWRITE & FILE ROTATION / test_log_unit_emergence]
      * tests/functional/test_log_functional.c
        [Section 3: INVARIANTS & SATURATION / test_log_functional_assertions_and_guarantees]
      * tests/integrated/test_log_integrated.c
        [Section 4: CATEGORICAL DISK ROUTING & HUD RING BUFFER / test_log_integrated_emergence]
      * tests/system/test_log_system.c
        [Section 1: NOMINAL HIGH-THROUGHPUT BURST / test_log_system_always_pass]
        [Section 3: FIXED-WIDTH REGEX DETERMINISM / test_log_system_assertions_and_guarantees]
------------------------------------------------------------------------------ */

/**
 * @brief Emits a structured log entry into the ring buffer, file, and console.
 *
 * ### For Game Designers & Non-Programmers:
 * Writes an event to the game's flight recorder journal, capturing the character involved,
 * their current moral balance percentage, and what action took place.
 *
 * ### For Systems Programmers & Engine Architects:
 * Formats a fixed-column log record with microsecond monotonic timestamp and ISO 8601 UTC time,
 * pushes it to the static circular ring buffer, and writes to disk archives with zero heap allocation.
 *
 * @param[in] severity Graded severity level.
 * @param[in] category Target functional category.
 * @param[in] entity   Entity/Actor label (e.g. "Player", "Companion", "System").
 * @param[in] grade    Normalized grade [-1.0f .. +1.0f] (e.g. alignment sway or vitals).
 * @param[in] file     Source file name (__FILE__).
 * @param[in] line     Source line number (__LINE__).
 * @param[in] func     Source function name (__func__).
 * @param[in] fmt      Printf-style format string.
 * @return Monotonic sequence ID assigned to the entry, or 0 on drop/failure.
 */
uint64_t log_write_entry(log_severity_t severity,
                         log_category_t category,
                         const char *entity,
                         float grade,
                         const char *file,
                         int line,
                         const char *func,
                         const char *fmt,
                         ...);

/**
 * @brief Converts a normalized float grade [-1.0f, +1.0f] to integer percent [-100, +100].
 *
 * ### For Game Designers & Non-Programmers:
 * Converts an internal moral sway decimal into a clean -100% to +100% readout.
 *
 * ### For Systems Programmers & Engine Architects:
 * Clamps `grade` to [-1.0f, 1.0f] and scales symmetrically into a signed 32-bit integer percentage.
 *
 * @param[in] grade Normalized floating-point balance ratio in `[-1.0f .. +1.0f]`.
 * @return Integer percentage clamped to `[-100 .. +100]` (`0` on NaN).
 */
int log_grade_to_percent(float grade);

/**
 * @brief Converts an integer percent [-100, +100] to a normalized float grade [-1.0f, +1.0f].
 *
 * ### For Game Designers & Non-Programmers:
 * Converts a whole-number percentage into an internal decimal balance ratio.
 *
 * ### For Systems Programmers & Engine Architects:
 * Clamps `percent` to [-100, 100] and maps to floating-point representation in [-1.0f, 1.0f].
 *
 * @param[in] percent Integer percentage value (`-100 .. +100`).
```

```
 * @return Normalized float clamped inside `[-1.0f .. +1.0f]`.
 */
float log_percent_to_grade(int percent);
```

A.18. Tier-0 Flight Log BIOS Boot & Early-Entry Replay Drain

Snapshot ID: snap_log_fb_bios_group | **Mode**: Group | **File**: engine/src/log/log_fallback.c (L1256–L1801 of 1865 lines) | Jump back to chapter text

```
/**
 * @brief Drains buffered Tier-0 Safe-Mode entries in FIFO order for primary handoff.
 *
 * ### For Game Designers & Non-Programmers:
 * Hands off early boot messages stored in Safe Mode to the main log file cleanly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Unrolls oldest-first entries via `log_fb_copy_ring_entries()` and decrements
 * `s_fb.ring_count` while incrementing `s_fb.total_replayed` for `log_init()` replay.
 *
 * @param[out] out_entries Caller array receiving entries (or NULL to discard cleanly).
 * @param[in]  max_entries Maximum capacity of `out_entries`.
 * @return Number of entries drained.
 */
size_t log_fallback_drain_entries(log_entry_t *out_entries, size_t max_entries)
{
    size_t count = log_fb_copy_ring_entries(out_entries, max_entries);
    if (count > 0U) {
        s_fb.ring_count -= count;
        s_fb.total_replayed += (uint64_t)count;
    }

    return count;
}
```

```
/**
 * @brief Arms the Tier-0 BIOS Safe-Mode logger before CLI flags or disk VFS boot.
 *
 * ### For Game Designers & Non-Programmers:
 * Turns on emergency logging immediately when the program starts so early boot
 * bugs or missing folders are never lost, while leaving repo folders clean.
 *
 * ### For Systems Programmers & Engine Architects:
 * When opt_cfg is NULL, arms purely in .bss ring + stderr alert mode with zero
 * filesystem I/O. When opt_cfg is supplied, opens bounded Safe-Mode disk output
 * (applying build/logs sandbox protection) and degrades gracefully to RAM Ring +
 * stderr if mkdir/fopen fails on read-only or full media.
 *
 * @param[in] opt_cfg Optional Safe-Mode config (NULL for clean RAM BIOS boot).
 * @return true on clean BIOS Safe-Mode arming (survives disk failures).
 */
bool log_fallback_boot(const log_config_t *opt_cfg)
{
    if (s_fb.file != NULL) {
        fflush(s_fb.file);
        fclose(s_fb.file);
        s_fb.file = NULL;
    }

    s_fb.disk_fault = false;
    s_fb.current_bytes = 0;

    if (opt_cfg == NULL) {
        s_fb.config = s_log_floppy_template;
        s_fb.config.loaded_from_data = false;
        if (io_fallback_is_disk_read_only()) {
            s_fb.disk_fault = true;
        }
        s_fb.active_sink = LOG_FALLBACK_SINK_RING_AND_STDERR;
        s_fb.booted = true;
        return true;
    }
```

```
1738
1739    s_fb.config = *opt_cfg;
1740    s_fb.config.loaded_from_data = false;
1741
1742    if (s_fb.config.max_file_size_bytes == 0U) {
1743        s_fb.config.max_file_size_bytes = LOG_FALLBACK_FLOPPY_MAX_BYTES;
1744    }
1745    if (s_fb.config.max_rotated_files == 0U ||
1746        s_fb.config.max_rotated_files > LOG_FB_MAX_ROTATION_CAP) {
1747        s_fb.config.max_rotated_files = LOG_FALLBACK_FLOPPY_MAX_FILES;
1748    }
1749    if (s_fb.config.log_dir[0] == '\0' || s_fb.config.base_filename[0] == '\0') {
1750        s_fb.disk_fault = true;
1751        s_fb.active_sink = s_fb.config.echo_to_console
1752            ? LOG_FALLBACK_SINK_RING_AND_STDERR
1753            : LOG_FALLBACK_SINK_RING_ONLY;
1754        s_fb.booted = true;
1755        return true;
1756    }
1757
1758    if (io_fallback_is_disk_read_only()) {
1759        s_fb.disk_fault = true;
1760        s_fb.active_sink = s_fb.config.echo_to_console
1761            ? LOG_FALLBACK_SINK_RING_AND_STDERR
1762            : LOG_FALLBACK_SINK_RING_ONLY;
1763        s_fb.booted = true;
1764        return true;
1765    }
1766
1767    log_fb_resolve_safe_dir(&s_fb.config);
1768
1769    if (!fs_ensure_directory(s_fb.config.log_dir)) {
1770        s_fb.disk_fault = true;
1771        s_fb.active_sink = s_fb.config.echo_to_console
1772            ? LOG_FALLBACK_SINK_RING_AND_STDERR
1773            : LOG_FALLBACK_SINK_RING_ONLY;
1774        s_fb.booted = true;
1775        return true;
1776    }
1777
1778    char full_path[LOG_FB_PATH_CAP];
1779    str_path_join(full_path, sizeof(full_path), s_fb.config.log_dir, s_fb.config.base_filename);
1780
1781    s_fb.file = fopen(full_path, "a");
1782    if (s_fb.file == NULL) {
1783        s_fb.disk_fault = true;
1784        s_fb.active_sink = s_fb.config.echo_to_console
1785            ? LOG_FALLBACK_SINK_RING_AND_STDERR
1786            : LOG_FALLBACK_SINK_RING_ONLY;
1787        s_fb.booted = true;
1788        return true;
1789    }
1790
1791    struct stat st;
1792    if (stat(full_path, &st) == 0) {
1793        s_fb.current_bytes = (size_t)st.st_size;
1794    } else {
1795        s_fb.current_bytes = 0;
1796    }
1797
1798    s_fb.active_sink = LOG_FALLBACK_SINK_DISK_AND_RING;
1799    s_fb.booted = true;
1800    return true;
1801 }
```

A.19. Flight Log System Emergence Verification (test_log_system_emergence)

Snapshot ID: snap_log_test_system | **Mode**: Function | **File**: tests/system/test_log_system.c (L283–L385 of 475 lines) | Jump back to chapter text

```
283 /**
284  * @brief Executes 10,000-entry chaos load, floppy BIOS endurance burst, and probe checks.
```

```
285 */
286 static void test_log_system_emergence(void)
287 {
288     printf("  [PASS-4] Running Section 4: Emergence (10,000-Entry Chaos, Floppy BIOS & Probes)...\n");
289 
290     TEST_ASSERT(debug_init(NULL), "Debugger online to monitor log system endurance");
291 
292     log_config_t cfg = log_get_default_config();
293     cfg.echo_to_console = false;
294     cfg.min_severity = LOG_SEV_TRACE;
295     cfg.save_by_category = true;
296     cfg.max_file_size_bytes = 16384U;
297     cfg.max_rotated_files = 10U;
298     str_safe_copy(cfg.log_dir, "build/logs/system_chaos", sizeof(cfg.log_dir));
299     str_safe_copy(cfg.base_filename, "chaos.log", sizeof(cfg.base_filename));
300 
301     TEST_ASSERT(log_init(&cfg), "Initialized for chaos stress");
302 
303     for (int i = 1; i <= TEST_LOG_SYS_CHAOS_COUNT; i++) {
304         log_severity_t sev = (log_severity_t)(i % LOG_SEV_COUNT);
305         log_category_t cat = (log_category_t)(i % LOG_CAT_COUNT);
306         float grade = ((float)(i % 200) - 100.0f) / 100.0f;
307 
308         LOG_RECORD(sev, cat, "ChaosAgent", grade, "High-stress event step #%06d", i);
309 
310         if (i % 2000 == 0) {
311             log_entry_t sample[10];
312             size_t fetched = log_get_top_entries(sample, 10U);
313             TEST_ASSERT_INT_EQ((int)fetched, 10,
314                                "Ring buffer top entries query succeeds during live write");
315         }
316     }
317 
318     log_stats_t final_stats;
319     TEST_ASSERT(log_execute_inspect(&final_stats), "Inspected final chaos stats");
320     TEST_ASSERT(final_stats.total_entries >= (uint64_t)TEST_LOG_SYS_CHAOS_COUNT,
321                 "Recorded at least 10,000 entries");
322     TEST_ASSERT(final_stats.rotation_count > 0U,
323                 "Multiple rotations executed under stress");
324 
325     /* Verify live telemetry probes remain healthy immediately after 10,000-entry stress */
326     cornerstone_test_probe_check_t p_rot = cornerstone_test_check_probe("log.rotation_io_bounds", LOG_MODULE_ID);
327     cornerstone_test_probe_check_t p_wri = cornerstone_test_check_probe("log.write_telemetry", LOG_MODULE_ID);
328     TEST_ASSERT(p_rot.found && p_rot.passed,
329                 "log.rotation_io_bounds probe remains PASS after 10,000-entry chaos load");
330     TEST_ASSERT(p_wri.found && p_wri.passed,
331                 "log.write_telemetry probe remains PASS after 10,000-entry chaos load");
332 
333     log_shutdown();
334 
335     /* Compact 1.44 MB floppy BIOS disk burst & tight 2-archive rotation cap */
336     remove("build/logs/system_floppy/floppy_stress.log");
337     remove("build/logs/system_floppy/floppy_stress.log.1");
338     remove("build/logs/system_floppy/floppy_stress.log.2");
339     remove("build/logs/system_floppy/floppy_stress.log.3");
340 
341     log_config_t floppy_cfg = log_fallback_get_floppy_config();
342     snprintf(floppy_cfg.log_dir, sizeof(floppy_cfg.log_dir), "build/logs/system_floppy");
343     snprintf(floppy_cfg.base_filename, sizeof(floppy_cfg.base_filename), "floppy_stress.log");
344     floppy_cfg.max_file_size_bytes = 2048U;
345     floppy_cfg.max_rotated_files = 2U;
346     floppy_cfg.echo_to_console = false;
347 
348     TEST_ASSERT(log_fallback_boot(&floppy_cfg),
349                 "Armed compact floppy BIOS disk sink for rotation stress");
350     for (unsigned int i = 0U; i < TEST_LOG_SYS_FLOPPY_FRAMES; ++i) {
351         uint64_t seq = log_fallback_record(
352             LOG_SEV_WARNING, LOG_CAT_LOG, "FloppyStress", 0.5f, __FILE__, __LINE__, __func__,
353             "Compact floppy rotation burst frame #%03u carrying fixed payload padding [0123456789ABCDEF0123456789ABCDEF]",
354             i);
355         TEST_ASSERT(seq > 0U, "Floppy BIOS stress entry recorded");
356     }
357 
358     log_fallback_stats_t fb_stats;
359     TEST_ASSERT(log_fallback_get_stats(&fb_stats),
```

```
                "Inspected floppy BIOS stats after 80-entry burst");
    TEST_ASSERT(!fb_stats.disk_fault && fb_stats.active_sink == LOG_FALLBACK_SINK_DISK_AND_RING,
                "Floppy BIOS sink sustained disk writes without fault");
    TEST_ASSERT(fb_stats.disk_bytes_written > 0U,
                "Floppy BIOS recorded non-zero disk writes");
    log_fallback_shutdown();
    debug_shutdown();

    FILE *f_rot1 = fopen("build/logs/system_floppy/floppy_stress.log.1", "r");
    FILE *f_rot2 = fopen("build/logs/system_floppy/floppy_stress.log.2", "r");
    FILE *f_rot3 = fopen("build/logs/system_floppy/floppy_stress.log.3", "r");
    TEST_ASSERT(f_rot1 != NULL, "Floppy BIOS rotation archive .1 exists");
    TEST_ASSERT(f_rot2 != NULL, "Floppy BIOS rotation archive .2 exists");
    TEST_ASSERT(f_rot3 == NULL, "Floppy BIOS rotation archive .3 suppressed by 2-file cap");
    if (f_rot1 != NULL) {
        fclose(f_rot1);
    }
    if (f_rot2 != NULL) {
        fclose(f_rot2);
    }
    if (f_rot3 != NULL) {
        fclose(f_rot3);
    }

    printf("  [PASS-4] Section 4 passed: 10,000-entry chaos, concurrent ring, probes & floppy BIOS verified.\n");
}
```

A.20. State Debugger Leaf Vocabulary & Snapshot Structs (debug_types.h Sub-Assembly 2.3)

Snapshot ID: snap_debug_types_sub2_3 | **Mode**: Subassembly | **File**: engine/include/debug/debug_types.h (L147–L453 of 652 lines) | Jump back to chapter text

```
/*
-------------------------------------------------------------------------------
Sub-Assembly 2.3: Shared Structural Ontology & POD Debugger Types
-------------------------------------------------------------------------------
Canonical non-anonymous enums and structures shared identically between
`cog.debug` and quarantined `cog.debug.fallback`.
-------------------------------------------------------------------------------
*/

/**
 * @enum debug_tripwire
 * @brief Latched diagnostic tripwire states for the debugger and validation pipeline.
 *
 * ### For Game Designers & Non-Programmers:
 * Alarm switches indicating whether configuration data was missing or damaged,
 * an automated safety test fired, or Safe Mode engaged after a fault.
 *
 * ### For Systems Programmers & Engine Architects:
 * Latching error enum preserving `0..4` legacy discriminants and `5..7` active
 * validation probe, audit breach, and Safe-Mode downgrade discriminants.
 */
typedef enum debug_tripwire {
    DEBUG_TRIPWIRE_CLEAR = 0,         /**< All tripwires clear; simulation healthy. */
    DEBUG_TRIPWIRE_MANIFEST_MISSING,  /**< Debugger/subsystem manifest unavailable. */
    DEBUG_TRIPWIRE_SETTING_MISSING,   /**< Designated JSON setting profile missing. */
    DEBUG_TRIPWIRE_STATE_CORRUPT,     /**< Simulation state breached domain invariants. */
    DEBUG_TRIPWIRE_HASH_MISMATCH,     /**< Snapshot checksum diverged from recorded state. */
    DEBUG_TRIPWIRE_TEST_FAILED,       /**< Registered validation probe / fail-test fired. */
    DEBUG_TRIPWIRE_AUDIT_BREACH,      /**< Block 4 preflight or audit invariant failed. */
    DEBUG_TRIPWIRE_FALLBACK_ENGAGED   /**< Downgraded to Safe Mode to complete execution. */
} debug_tripwire_t;

/**
 * @enum debug_tripwire_reaction
 * @brief Recovery action taxonomy ingested from taxonomy/tripwire_matrix.json.
 *
 * ### For Game Designers & Non-Programmers:
 * Selects how the engine reacts when an alarm trips: logging warnings, clamping
```

```
 * bad numbers, dropping into Safe Mode, rewinding time, or halting in AWAIT.
 *
 * ### For Systems Programmers & Engine Architects:
 * Mapped 1:1 to `tripwire_matrix.json` canonical reaction identifier strings.
 */
typedef enum debug_tripwire_reaction {
    DEBUG_REACTION_LOG_AND_CONTINUE = 0, /**< Emit telemetry note and continue. */
    DEBUG_REACTION_SATURATE_CLAMP,       /**< Clamp out-of-range value to safe limits. */
    DEBUG_REACTION_FALLBACK_MODE,        /**< Downgrade step cleanly via fallback code. */
    DEBUG_REACTION_AUTO_REWIND,          /**< Restore last locked healthy snapshot. */
    DEBUG_REACTION_PANIC_HALT            /**< Complete via fallback then halt in AWAIT. */
} debug_tripwire_reaction_t;

/**
 * @enum debug_test_tier
 * @brief Ontological verification tier associated with a tripwire or probe.
 *
 * ### For Game Designers & Non-Programmers:
 * Identifies which inspection level (in-file self-check or one of the four
 * external verification suites) caught an anomaly.
 *
 * ### For Systems Programmers & Engine Architects:
 * Binds registered `debug_validation_probe_t` entries cleanly to Block 4 audits
 * and Layer 4 `Unit`, `Functional`, `Integrated`, and `System` suites.
 */
typedef enum debug_test_tier {
    DEBUG_TEST_TIER_AUDIT = 0,     /**< In-file Block 4 preflight / validation cog. */
    DEBUG_TEST_TIER_UNIT = 1,      /**< Layer 4 Unit isolation test suite. */
    DEBUG_TEST_TIER_FUNCTIONAL,    /**< Layer 4 Functional cog verification suite. */
    DEBUG_TEST_TIER_INTEGRATED,    /**< Layer 4 Cross-subsystem integrated suite. */
    DEBUG_TEST_TIER_SYSTEM         /**< Layer 4 End-to-end system stress & soak suite. */
} debug_test_tier_t;

/**
 * @enum debug_step_mode
 * @brief Execution stepping mode of the engine debugger.
 *
 * ### For Game Designers & Non-Programmers:
 * Controls how simulation steps advance (`RUN`, `OVER_TURN`, `INTO_COG`,
 * `PAUSED`, or emergency `AWAIT` wait state after Safe-Mode recovery).
 *
 * ### For Systems Programmers & Engine Architects:
 * Execution cadence discriminator driving loop gating and tripwire freezes.
 */
typedef enum debug_step_mode {
    DEBUG_STEP_RUN = 0,        /**< Continuous execution until breakpoint/tripwire. */
    DEBUG_STEP_OVER_TURN = 1,  /**< Pause cleanly at turn boundaries. */
    DEBUG_STEP_INTO_COG = 2,   /**< Micro-step individual cog sub-assemblies. */
    DEBUG_STEP_PAUSED = 3,     /**< Simulation paused for interactive inspection. */
    DEBUG_STEP_AWAIT = 4       /**< Halted idle awaiting repair and healthy restore. */
} debug_step_mode_t;

/**
 * @enum debug_breakpoint_type
 * @brief Watchpoint trigger condition taxonomy for simulation breaks.
 *
 * ### For Game Designers & Non-Programmers:
 * Chooses what pauses gameplay automatically: specific turns, stance shifts,
 * moral sway swings, weapon resonance limits, low HP, locations, or tripwires.
 *
 * ### For Systems Programmers & Engine Architects:
 * Evaluated in O(1) per breakpoint slot at each simulation step boundary.
 */
typedef enum debug_breakpoint_type {
    DEBUG_BREAK_NONE = 0,          /**< Disabled / unused breakpoint slot. */
    DEBUG_BREAK_TURN,              /**< Break when turn counter matches target value. */
    DEBUG_BREAK_STANCE,            /**< Break when actor enters target stance ID. */
    DEBUG_BREAK_SWAY_RANGE,        /**< Break when |moral_sway| exceeds threshold. */
    DEBUG_BREAK_RESONANCE_BREACH,  /**< Break when primary gauge exceeds threshold. */
    DEBUG_BREAK_HP_THRESHOLD,      /**< Break when HP falls at or below target. */
    DEBUG_BREAK_LOCATION,          /**< Break upon entering target zone/tile ID. */
    DEBUG_BREAK_TRIPWIRE           /**< Break whenever any diagnostic tripwire trips. */
} debug_breakpoint_type_t;

/**
```

```
 * @struct debug_state_fields
 * @brief Decoupled, generic state representation for Layer 0 snapshots.
 *
 * ### For Game Designers & Non-Programmers:
 * Captures an actor's generic vitals, posture, scalar balance, location index,
 * active turn timers, and threat readout inside the time-machine filmstrip.
 *
 * ### For Systems Programmers & Engine Architects:
 * Pure integer/char POD slot layout guaranteeing zero Layer 1 simulation headers
 * ever leak into `engine/`.
 */
typedef struct debug_state_fields {
    int  turn;                          /**< Active simulation turn index. */
    char hero_name[DEBUG_MAX_NAME_LEN];   /**< Generic active actor label slot. */
    int  hp;                            /**< Current health points slot. */
    int  max_hp;                        /**< Maximum health points capacity slot. */
    int  mp;                            /**< Secondary energy pool slot. */
    int  tp;                            /**< Tactical technique points slot. */
    int  shield_power;                  /**< Defensive artifact charge gauge slot. */
    int  sword_edge;                    /**< Offensive artifact charge gauge slot. */
    int  stance;                        /**< Active numerical posture/stance slot. */
    int  moral_sway;                    /**< Signed alignment/balance scalar slot. */
    int  location;                      /**< Discrete world zone/location identifier. */
    int  lux_barrio_turns;              /**< Protective ward duration counter slot. */
    int  charm_turns;                   /**< Status modifier duration counter slot. */
    int  threat_hp;                     /**< Opposing active target HP readout slot. */
    int  foes_vanquished;               /**< Cumulative defeated opposition counter. */
} debug_state_fields_t;

/**
 * @struct debug_snapshot
 * @brief Frozen state snapshot stored in the debugger circular ring buffer.
 *
 * ### For Game Designers & Non-Programmers:
 * One time-machine frame recording the exact turn values, command typed, log
 * line link, and whether the frame is pinned so later turns cannot overwrite it.
 *
 * ### For Systems Programmers & Engine Architects:
 * Fixed POD record verified by 32-bit FNV-1a checksum (`state_hash`) across
 * rewind and determinism comparisons.
 */
typedef struct debug_snapshot {
    uint64_t             snapshot_id;      /**< Monotonic snapshot sequence number. */
    uint64_t             timestamp_ms;     /**< Capture timestamp in milliseconds. */
    debug_state_fields_t state;            /**< Decoupled state scalar payload. */
    char                 last_command[64]; /**< Canonical command verb executed. */
    char                 last_arg[64];     /**< Optional command argument string. */
    uint32_t             state_hash;       /**< 32-bit FNV-1a deterministic hash. */
    uint64_t             log_sequence_id;  /**< Correlated flight recorder sequence. */
    bool                 locked;           /**< Eviction guard enforced up to quota. */
    char                 lock_reason[48];  /**< Human/probe lock tag (e.g. TRIPWIRE_LATCH). */
} debug_snapshot_t;

/**
 * @struct debug_breakpoint
 * @brief Dynamic watchpoint registration descriptor.
 *
 * ### For Game Designers & Non-Programmers:
 * Holds one active debug watchpoint rule with its trigger type, target value,
 * and readable designer label.
 *
 * ### For Systems Programmers & Engine Architects:
 * Fixed-width slot entry stored inside `debug_internal_state_t.breakpoints[]`.
 */
typedef struct debug_breakpoint {
    int                      id;           /**< Assigned positive breakpoint handle ID. */
    debug_breakpoint_type_t type;          /**< Watchpoint trigger discriminator. */
    int                      target_value; /**< Numeric threshold or target ID. */
    bool                     is_active;    /**< True while watchpoint is armed. */
    char                     label[64];    /**< Descriptive breakpoint tag. */
} debug_breakpoint_t;

/**
 * @struct debug_config
 * @brief Debugger configuration parameters loaded from data or BIOS fallback.
```

```
 *
 * ### For Game Designers & Non-Programmers:
 * Settings controlling rewind buffer length, watchpoint triggers, inspection
 * card auto-dumps, interactive console behavior, and Safe-Mode AWAIT pauses.
 *
 * ### For Systems Programmers & Engine Architects:
 * Shared cleanly between `cog.debug` and `cog.debug.fallback` with explicit
 * `loaded_from_data` provenance tracking (`false` for ROM/monolith defaults).
 */
typedef struct debug_config {
    bool   enabled;                         /**< Master debugger enable switch. */
    bool   break_on_error;                  /**< Break upon logged ERROR severity. */
    bool   break_on_tripwire;               /**< Break upon diagnostic tripwire latch. */
    bool   break_on_sway_breach;            /**< Break upon alignment bound breach. */
    bool   break_on_resonance_breach;       /**< Break upon artifact gauge overflow. */
    bool   break_on_hp_zero;                /**< Break when primary HP reaches zero. */
    bool   break_on_stance_change;          /**< Break upon actor posture transitions. */
    size_t max_snapshots;                   /**< Active rewind cap (<= DEBUG_MAX_SNAPSHOTS). */
    int    snapshot_interval_turns;         /**< Automatic snapshot cadence in turns. */
    bool   auto_dump_on_tripwire;           /**< Auto-format state card on tripwire. */
    bool   interactive_repl;                /**< Enable interactive terminal pause prompt. */
    bool   ansi_color;                      /**< Enable ANSI color styling on cards. */
    char   trace_channels[DEBUG_MAX_CHANNELS_LEN]; /**< Enabled channel filter CSV. */
    char   profile_name[32];                /**< Active configuration preset identifier. */
    bool   loaded_from_data;                /**< True only when loaded from disk VFS JSON. */
    bool   engine_traps_active;             /**< Gate for Layer 0 infrastructure traps. */
    bool   game_traps_active;               /**< Gate for Layer 1 simulation traps. */
    size_t locked_slot_quota;               /**< Reserved pinned critical slots in ring. */
    bool   verify_on_step;                  /**< Evaluate probes on each step check. */
    bool   auto_await_on_tripwire;          /**< Enter DEBUG_STEP_AWAIT after fallback step. */
} debug_config_t;

/**
 * @brief Validation probe callback signature returning a Block 4 `diag_result_t`.
 */
typedef diag_result_t (*debug_validation_probe_fn)(void);

/**
 * @struct debug_validation_probe
 * @brief Registered Block 4 / Layer 4 fail-test trigger linked into tripwire monitoring.
 *
 * ### For Game Designers & Non-Programmers:
 * Connects an automated safety check or verification rule directly to the
 * debugger so any violation stops execution in AWAIT instead of silently drifting.
 *
 * ### For Systems Programmers & Engine Architects:
 * Binds source location tags, optional callback function pointers, and Layer 4
 * test suite references inside a 32-slot zero-heap table.
 */
typedef struct debug_validation_probe {
    char                       probe_id[48];       /**< Unique probe ID (e.g. "io.loader.envelope"). */
    char                       subsystem[32];      /**< Owning subsystem ID (e.g. "core.io.master_loader"). */
    char                       source_section[64]; /**< File/block/function trigger anchor. */
    char                       test_suite_ref[64]; /**< Mapped Layer 4 test suite & section. */
    debug_test_tier_t          tier;               /**< Verification tier (AUDIT..SYSTEM). */
    debug_tripwire_t           tripwire_code;      /**< Categorized tripwire latched on fail. */
    debug_tripwire_reaction_t reaction;           /**< Configured recovery reaction policy. */
    debug_validation_probe_fn validate_fn;        /**< Optional Block 4 validation probe callback. */
    bool                       auto_await;         /**< True to pause in AWAIT after Safe-Mode step. */
    bool                       is_active;          /**< True if probe registration slot is live. */
    uint32_t                   trigger_count;      /**< Lifetime count of triggered firings. */
} debug_validation_probe_t;

/**
 * @struct debug_tripwire_report
 * @brief Forensic report capturing active tripwire, fallback completion, and AWAIT state.
 *
 * ### For Game Designers & Non-Programmers:
 * Full emergency incident ticket explaining which probe fired, which snapshot
 * got locked, what Safe Mode did, and whether the engine is paused in AWAIT.
 *
 * ### For Systems Programmers & Engine Architects:
 * Fixed-layout diagnostic report populated cleanly by `debug_trigger_probe()`.
 */
typedef struct debug_tripwire_report {
```

```
    debug_tripwire_t          tripwire_status;    /**< Categorized latched tripwire code. */
    debug_tripwire_reaction_t reaction;           /**< Selected reaction policy. */
    debug_test_tier_t         tier;               /**< Verification tier of firing probe. */
    bool                      fallback_completed; /**< True once Safe Mode finished step. */
    bool                      await_active;       /**< True while halted in DEBUG_STEP_AWAIT. */
    bool                      snapshot_locked;    /**< True if incident snapshot stayed pinned. */
    uint64_t                  locked_snapshot_id; /**< ID of pinned forensic snapshot. */
    char                      probe_id[48];       /**< Probe identifier that raised alarm. */
    char                      subsystem[32];      /**< Owning subsystem identifier. */
    char                      condition[128];     /**< Failure condition summary text. */
    char                      source_location[96];/**< Formatted `file:line` anchor. */
    char                      source_section[64]; /**< Source Block/function section anchor. */
    char                      test_suite_ref[64]; /**< Linked Layer 4 test reference. */
    char                      fallback_action[96];/**< Summary of Safe-Mode recovery action. */
} debug_tripwire_report_t;

/**
 * @struct debug_stats
 * @brief Aggregate runtime telemetry report constructed by Block 5 inspection procedures.
 *
 * ### For Game Designers & Non-Programmers:
 * Status scorecard showing rewind frame usage, locked emergency slots, probe
 * failure counts, Safe-Mode status, and whether playback is paused in AWAIT.
 *
 * ### For Systems Programmers & Engine Architects:
 * Zero-heap runtime snapshot copied from internal debugger state cleanly.
 */
typedef struct debug_stats {
    size_t            snapshot_count;         /**< Active snapshots stored in ring buffer. */
    size_t            max_snapshots;          /**< Configured snapshot ring capacity. */
    size_t            breakpoint_count;       /**< Count of armed watchpoint slots. */
    uint32_t          traps_triggered;        /**< Lifetime debugger trap activations. */
    debug_step_mode_t step_mode;              /**< Active stepping execution mode. */
    bool              is_active;              /**< True if primary debugger is initialized. */
    bool              loaded_from_data;       /**< True if active config came from disk JSON. */
    char              active_profile[32];     /**< Active configuration preset identifier. */
    size_t            locked_snapshot_count;  /**< Pinned critical snapshots in ring. */
    size_t            registered_probe_count; /**< Active registered validation probes. */
    uint32_t          probe_failures;         /**< Total validation probe fail triggers. */
    bool              safe_mode;              /**< True if operating via Safe-Mode fallback. */
    bool              await_active;           /**< True while halted in DEBUG_STEP_AWAIT. */
    bool              fallback_completed;     /**< True if Safe Mode completed halted step. */
    debug_tripwire_t  tripwire_status;        /**< Active debugger tripwire latch code. */
} debug_stats_t;
```

A.21. Deterministic State Capture & Time-Travel Rewind (debug.c)

Snapshot ID: snap_debug_rewind_group | **Mode**: Group | **File**: engine/src/debug/debug.c (L1264–L1567 of 3221 lines) | Jump back to chapter text

```
/**
 * @brief Captures an actor/world snapshot into `.bss` circular ring storage with lock safety.
 *
 * ### For Game Designers & Non-Programmers:
 * Records the complete state of a simulation step in rewind memory while protecting pinned
 * forensic evidence slots from being erased when history wraps around.
 *
 * ### For Systems Programmers & Engine Architects:
 * Selects destination index under `CIRCULAR_FIFO_WITH_LOCKED_SLOTS` (invoking
 * `dbg_normalize_ring_for_locks()` and `memmove()` when preserving locked frames), hashes
 * `fields`, stamps monotonic `next_snapshot_id` + `time_get_monotonic_ms()`, and updates
 * watermark telemetry.
 *
 * @triggers
 * - Layer 4 Tests : `tests/unit/debug/test_debug_unit.c:Section1`,
 *                   `tests/functional/test_debug_functional.c`
 *
 * @param[in] fields Simulation state fields to capture.
 * @param[in] cmd    Associated command verb string (may be NULL).
 * @param[in] arg    Associated command argument string (may be NULL).
 * @return Assigned monotonic snapshot ID (`>= 1`), or `0ULL` if inactive or `fields == NULL`.
 */
```

```
uint64_t debug_record_snapshot(const debug_state_fields_t *fields,
                               const char *cmd, const char *arg)
{
    if (!s_debug_active || fields == NULL) {
        return 0ULL;
    }

    size_t cap = s_debug.config.max_snapshots;
    if (cap == 0U || cap > DEBUG_MAX_SNAPSHOTS) {
        cap = DEBUG_MAX_SNAPSHOTS;
    }

    size_t write_idx = 0U;

    /* Step 1: Select destination slot respecting CIRCULAR_FIFO_WITH_LOCKED_SLOTS */
    if (s_debug.snapshot_count < cap) {
        write_idx = s_debug.snapshot_head;
        s_debug.snapshot_head = (s_debug.snapshot_head + 1U) % cap;
        s_debug.snapshot_count++;
    } else if (s_debug.locked_count == 0U) {
        write_idx = s_debug.snapshot_head;
        s_debug.snapshot_head = (s_debug.snapshot_head + 1U) % cap;
    } else {
        /* Evict oldest UNLOCKED snapshot while preserving chronological order */
        dbg_normalize_ring_for_locks();
        size_t evict_idx = cap;
        for (size_t i = 0U; i < cap; i++) {
            if (!s_debug.snapshots[i].locked) {
                evict_idx = i;
                break;
            }
        }
        if (evict_idx == cap) {
            evict_idx = 0U;
            if (s_debug.locked_count > 0U) {
                s_debug.locked_count--;
            }
            if (!s_debug.in_audit) {
                LOG_WARN(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, -0.25f,
                         "All snapshot ring slots locked; forcefully retired oldest slot.");
            }
        }
        if (evict_idx < cap - 1U) {
            memmove(
                &s_debug.snapshots[evict_idx],
                &s_debug.snapshots[evict_idx + 1U],
                (cap - 1U - evict_idx) * sizeof(debug_snapshot_t)
            );
        }
        write_idx = cap - 1U;
        s_debug.snapshot_head = 0U;
    }

    /* Step 2: Populate immutable snapshot frame */
    debug_snapshot_t *snap = &s_debug.snapshots[write_idx];
    memset(snap, 0, sizeof(*snap));
    snap->snapshot_id = s_debug.next_snapshot_id++;
    snap->timestamp_ms = time_get_monotonic_ms();
    snap->state = *fields;
    str_safe_copy(snap->last_command, (cmd != NULL) ? cmd : "", sizeof(snap->last_command));
    str_safe_copy(snap->last_arg, (arg != NULL) ? arg : "", sizeof(snap->last_arg));
    snap->state_hash = debug_compute_state_hash(fields);
    snap->log_sequence_id = log_get_total_entries();
    snap->locked = false;

    /* Step 3: Track ring buffer telemetry */
    dbg_update_snapshot_watermark(s_debug.snapshot_count);
    s_debug.total_snapshots_captured++;

    return snap->snapshot_id;
}
```

```
/**
 * @brief Rewinds state fields back to snapshot `index` after cryptographic checksum check.
```

```
 *
 * ### For Game Designers & Non-Programmers:
 * Restores actor stats from an earlier saved frame, pausing safely in AWAIT if memory
 * corruption altered the snapshot's stored fingerprint.
 *
 * ### For Systems Programmers & Engine Architects:
 * Fetches snapshot via `debug_get_snapshot()`, verifies `debug_compute_state_hash(&snap.state)`
 * matches `snap.state_hash` (engaging `DEBUG_TRIPWIRE_HASH_MISMATCH` + Safe-Mode AWAIT +
 * `LOG_ERROR` on mismatch), and emits `LOG_INFO` on verified restore.
 *
 * @triggers
 * - Layer 4 Tests : `tests/unit/debug/test_debug_unit.c:Section1`
 *
 * @param[in]  index      Chronological target snapshot offset.
 * @param[out] out_fields Restored simulation state fields on verified success.
 * @return true if checksum passed and `out_fields` populated; false on error or mismatch.
 */
bool debug_rewind_to_snapshot(size_t index, debug_state_fields_t *out_fields)
{
    /* Step 1: Validate preconditions and fetch chronological snapshot copy */
    if (!s_debug_active || out_fields == NULL) {
        return false;
    }

    debug_snapshot_t snap;
    if (!debug_get_snapshot(index, &snap)) {
        return false;
    }

    /* Step 2: Verify determinism checksum against tampering before restoring */
    uint32_t verify_hash = debug_compute_state_hash(&snap.state);
    if (verify_hash != snap.state_hash) {
        s_debug.tripwire_status = DEBUG_TRIPWIRE_HASH_MISMATCH;
        snprintf(s_debug.tripwire_details, sizeof(s_debug.tripwire_details),
                 "Rewind checksum mismatch on snap #%lu (0x%08X != 0x%08X)",
                 (unsigned long)snap.snapshot_id, verify_hash, snap.state_hash);
        s_debug.traps_triggered++;
        (void)debug_fallback_record_fault(
            MODULE_ID, s_debug.tripwire_details, __FILE__, __LINE__,
            "debug_rewind_to_snapshot"
        );
        (void)debug_fallback_finish_with_fallback(
            MODULE_ID, "Aborted corrupted rewind target and entered AWAIT"
        );
        if (!s_debug.in_audit) {
            LOG_ERROR(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, -0.8f,
                      "%s", s_debug.tripwire_details);
        }
        s_debug.await_active = true;
        s_debug.step_mode = DEBUG_STEP_AWAIT;
        return false;
    }

    /* Step 3: Commit verified snapshot state into caller out_fields */
    *out_fields = snap.state;
    if (!s_debug.in_audit) {
        LOG_INFO(LOCAL_LOG_CAT, LOCAL_MODULE_TAG, 0.25f,
                 "Rewound to Snapshot #%lu (Turn %d, Hash 0x%08X%s)",
                 (unsigned long)snap.snapshot_id, snap.state.turn,
                 snap.state_hash, snap.locked ? ", LOCKED" : "");
    }
    return true;
}
```

A.22. Tier-0 Debugger BIOS Boot & ROM Profile Hydration (debug_fallback.c)

Snapshot ID: `snap_debug_fb_bios_group` | **Mode**: `Group` | **File**: `engine/src/debug/debug_fallback.c` (`L1008`–`L1766` of `1852 lines`) | Jump back to chapter text

```
/**
 * @brief Hydrates a complete Safe-Mode debugger config from `.rodata` ROM JSON.
 *
```

```
 * ### For Game Designers & Non-Programmers:
 * Loads a factory debugger preset (`"default"`, `"strict"`, `"trace"`) plus snapshot locking
 * rules straight from executable ROM when disk folders are unreadable.
 *
 * ### For Systems Programmers & Engine Architects:
 * Parses `.rodata` 5-block envelope in memory via `master_loader_load_from_memory()` +
 * `debug_fallback_parse_config_json()`, stamps `loaded_from_data = false`, overlays ROM
 * `snapshot_policy`, and emits guarded BIOS telemetry on corruption or hydration.
 *
 * @triggers
 * - Primary Callers: `debug_get_default_config()`, `debug_init_emergency_monolith()`
 * - Layer 4 Tests  : `tests/unit/debug/test_debug_fallback_unit.c:Section3`,
 *                    `tests/integrated/test_debug_integrated.c:Section1`
 *
 * @param[in]  profile_id  Target profile stem (defaults cleanly to `"default"` if NULL).
 * @param[out] out_config  Caller `debug_config_t` destination buffer.
 * @param[in]  config_size Must equal `sizeof(debug_config_t)`.
 * @return true if ROM profile hydrated cleanly; false on missing/corrupt asset or bad args.
 */
bool debug_fallback_load_rom_profile(
    const char     *profile_id,
    debug_config_t *out_config,
    size_t          config_size
) {
    /* Step 1: Validate destination struct size and resolve target profile stem */
    if (out_config == NULL || config_size != sizeof(debug_config_t)) {
        return false;
    }

    const char *target = (profile_id != NULL && profile_id[0] != '\0')
                             ? profile_id
                             : "default";

    /* Step 2: Locate qualified debugger/<id> ROM payload or fallback leaf */
    char qualified_id[64];
    snprintf(qualified_id, sizeof(qualified_id), "debugger/%s", target);

    size_t raw_size = 0U;
    const char *raw_json = debug_fallback_find_rom_asset(qualified_id, &raw_size);
    if (raw_json == NULL) {
        raw_json = debug_fallback_find_rom_asset(target, &raw_size);
    }
    if (raw_json == NULL || raw_size == 0U) {
        return false;
    }

    /* Step 3: Unwrap 5-block envelope directly from immutable .rodata memory */
    master_loader_t loader;
    if (!master_loader_load_from_memory(&loader, raw_json, raw_size) ||
        loader.body_start == NULL) {
        fb_emit_bios_log(
            LOG_SEV_ERROR, __FILE__, __LINE__, __func__, MODULE_ID,
            "Corrupt 5-block envelope inside diagnostics ROM profile asset."
        );
        return false;
    }

    /* Step 4: Parse shared profile schema and overlay ROM snapshot retention quotas */
    debug_tripwire_t tw = DEBUG_TRIPWIRE_CLEAR;
    char err_msg[128];
    if (!debug_fallback_parse_config_json(
            loader.body_start,
            target,
            false,
            out_config,
            config_size,
            &tw,
            err_msg,
            sizeof(err_msg))) {
        fb_emit_bios_log(
            LOG_SEV_ERROR, __FILE__, __LINE__, __func__, MODULE_ID,
            err_msg
        );
        return false;
    }
```

```
1086
1087     (void)debug_fallback_load_rom_snapshot_policy(out_config, config_size);
1088
1089     /* Step 5: Guarantee non-disk provenance and log Safe-Mode ROM hydration */
1090     out_config->loaded_from_data = false;
1091
1092     s_fb.rom_hydrated = true;
1093     str_safe_copy(s_fb.active_rom_profile,
1094                   out_config->profile_name,
1095                   sizeof(s_fb.active_rom_profile));
1096
1097     fb_emit_bios_log(
1098         LOG_SEV_INFO,
1099         __FILE__,
1100         __LINE__,
1101         __func__,
1102         MODULE_ID,
1103         s_fb.active_rom_profile
1104     );
1105     return true;
1106 }
```

```
1715 /**
1716  * @brief Arms Tier-0 Safe-Mode BIOS state and seeds active ROM profile telemetry.
1717  *
1718  * ### For Game Designers & Non-Programmers:
1719  * Turns on emergency fault recording right at startup so any failure before external
1720  * configs finish loading is still captured and recoverable.
1721  *
1722  * ### For Systems Programmers & Engine Architects:
1723  * Copies caller profile name into `s_fb.active_rom_profile`, or hydrates `"default"`
1724  * via `debug_fallback_load_rom_profile()` with monolith fallback, sets `s_fb.booted = true`,
1725  * and logs an isolated `LOG_SEV_INFO` boot record into `cog.log.fallback`.
1726  *
1727  * @triggers
1728  * - Layer 4 Tests : `tests/unit/debug/test_debug_fallback_unit.c:Section4`
1729  *
1730  * @param[in] opt_config  Optional pre-populated `debug_config_t` (or NULL for ROM default).
1731  * @param[in] config_size Must equal `sizeof(debug_config_t)` when `opt_config != NULL`.
1732  * @return true once Tier-0 Safe-Mode debugger BIOS is booted.
1733  */
1734 bool debug_fallback_boot(const debug_config_t *opt_config, size_t config_size)
1735 {
1736     /* Step 1: Resolve active Safe-Mode profile string from caller, ROM, or monolith */
1737     debug_config_t scratch;
1738     if (opt_config != NULL && config_size == sizeof(debug_config_t)) {
1739         str_safe_copy(s_fb.active_rom_profile, opt_config->profile_name,
1740                       sizeof(s_fb.active_rom_profile));
1741     } else if (!debug_fallback_load_rom_profile("default", &scratch, sizeof(scratch))) {
1742         debug_get_monolith_fallback_config_raw(&scratch, sizeof(scratch));
1743         str_safe_copy(s_fb.active_rom_profile, scratch.profile_name,
1744                       sizeof(s_fb.active_rom_profile));
1745         fb_emit_bios_log(
1746             LOG_SEV_WARNING,
1747             __FILE__,
1748             __LINE__,
1749             __func__,
1750             MODULE_ID,
1751             "ROM default profile unavailable; seeded Safe-Mode BIOS from monolith template."
1752         );
1753     }
1754
1755     /* Step 2: Arm Tier-0 BIOS active flag and emit guarded boot log */
1756     s_fb.booted = true;
1757     fb_emit_bios_log(
1758         LOG_SEV_INFO,
1759         __FILE__,
1760         __LINE__,
1761         __func__,
1762         MODULE_ID,
1763         "Tier-0 Debugger Safe-Mode BIOS armed."
1764     );
1765     return true;
1766 }
```

A.23. Debugger Unit Emergence & Tripwire Verification (test_debug_unit_emergence)

Snapshot ID: snap_debug_test_unit | **Mode**: Function | **File**: tests/unit/debug/test_debug_unit.c (L410–L510 of 563 lines)

| Jump back to chapter text

```
410 static void test_debug_unit_emergence(void)
411 {
412     cornerstone_test_section(4, "Emergence (1000-Turn Wrap, 4-Slot Lock Quota & AWAIT Recovery)");
413 
414     TEST_ASSERT(debug_init(NULL), "debug_init succeeds for emergence section");
415 
416     /* 4.1 1000-turn ring wrap-around */
417     for (int t = 1; t <= 1000; t++) {
418         debug_state_fields_t s = cornerstone_test_make_state(
419             t, "Elijah", 500 - (t % 200), t % 100, (t * 2) % 100, t % 4, (t % 200) - 100);
420         uint64_t sid = debug_record_snapshot(&s, "sim_turn", "step");
421         TEST_ASSERT_U64_EQ(sid, (uint64_t)t, "Monotonic ID strictly tracked across 1000 turns");
422     }
423 
424     TEST_ASSERT_U64_EQ(debug_get_snapshot_count(), (uint64_t)DEBUG_MAX_SNAPSHOTS,
425                        "Ring capacity capped at 64");
426 
427     debug_snapshot_t oldest;
428     TEST_ASSERT(debug_get_snapshot(0U, &oldest) && (oldest.state.turn == 937),
429                 "Oldest turn after 1000 pushes is 937");
430     const debug_snapshot_t *latest = debug_get_latest_snapshot();
431     TEST_ASSERT((latest != NULL) && (latest->state.turn == 1000), "Latest turn is 1000");
432 
433     debug_state_fields_t restored;
434     TEST_ASSERT(debug_rewind_to_snapshot(30U, &restored) &&
435                     (restored.turn == 967) &&
436                     (restored.hp == (500 - (967 % 200))),
437                 "Rewind index 30 restores turn 967 and exact HP formula");
438 
439     debug_shutdown();
440 
441     /* 4.2 Locked critical snapshot quota (4) + DEBUG_TRIGGER_PROBE -> AWAIT recovery */
442     TEST_ASSERT(debug_init(NULL), "Re-init debugger for lock quota & AWAIT check");
443     TEST_ASSERT(debug_can_step() && !debug_is_awaiting(), "Stepping armed initially");
444 
445     uint64_t lock_ids[5] = { 0U };
446     for (int t = 1; t <= 6; t++) {
447         debug_state_fields_t s = cornerstone_test_make_state(t, "Elijah", 450 + t, 60, 40, t % 4, t);
448         uint64_t sid = debug_record_snapshot(&s, "lock_setup", "critical");
449         if (t <= 5) {
450             lock_ids[t - 1] = sid;
451         }
452     }
453 
454     for (size_t i = 0U; i < DEBUG_FALLBACK_LOCKED_SLOT_QUOTA; i++) {
455         TEST_ASSERT(debug_lock_snapshot(lock_ids[i], "UNIT_CRITICAL_LOCK"),
456                     "Lock snapshot within 4-slot quota succeeds");
457     }
458     TEST_ASSERT(debug_lock_snapshot(lock_ids[4], "QUOTA_ROTATE"),
459                 "Locking 5th snapshot rotates oldest lock cleanly");
460 
461     debug_stats_t lock_stats;
462     TEST_ASSERT(debug_execute_inspect(&lock_stats) &&
463                     (lock_stats.locked_snapshot_count == DEBUG_FALLBACK_LOCKED_SLOT_QUOTA),
464                 "Active locked_snapshot_count clamped at quota == 4");
465 
466     for (int t = 7; t <= 126; t++) {
467         debug_state_fields_t s = cornerstone_test_make_state(
468             t, "Elijah", 400 + (t % 90), 50, 50, t % 4, 0);
469         (void)debug_record_snapshot(&s, "advance", "fifo");
470     }
471     TEST_ASSERT_U64_EQ(debug_get_snapshot_count(), (uint64_t)DEBUG_MAX_SNAPSHOTS,
472                        "Ring count remains clamped at 64 with locked slots");
473 
474     for (size_t idx = 0U; idx < DEBUG_FALLBACK_LOCKED_SLOT_QUOTA; idx++) {
475         debug_snapshot_t locked_snap;
476         TEST_ASSERT(debug_get_snapshot(idx, &locked_snap) &&
```

```
                    locked_snap.locked &&
                    (locked_snap.state.turn == (int)(idx + 2U)),
                "Locked snapshots 2..5 survive 120-turn ring wrap");
    }
    debug_snapshot_t sliding_head;
    TEST_ASSERT(debug_get_snapshot(DEBUG_FALLBACK_LOCKED_SLOT_QUOTA, &sliding_head) &&
                    (sliding_head.state.turn == 67),
                "First evictable slot after 4 locks reflects turn 67");

    TEST_ASSERT(DEBUG_TRIGGER_PROBE("unit.state_guard", "unit.sim", "Synthetic fail-test trap"),
                "DEBUG_TRIGGER_PROBE latches unit validation probe");
    TEST_ASSERT(debug_is_awaiting() &&
                    (debug_get_step_mode() == DEBUG_STEP_AWAIT) &&
                    !debug_can_step(),
                "Debugger latches DEBUG_STEP_AWAIT and blocks stepping");

    debug_set_step_mode(DEBUG_STEP_RUN);
    TEST_ASSERT(debug_get_step_mode() == DEBUG_STEP_AWAIT,
                "debug_set_step_mode cannot bypass active DEBUG_STEP_AWAIT");

    debug_state_fields_t corrupt = cornerstone_test_make_state(126, "Elijah", -25, 50, 50, 0, 250);
    TEST_ASSERT(!debug_resolve_await(&corrupt, DEBUG_STEP_RUN, "Corrupt reject check") &&
                    debug_is_awaiting(),
                "debug_resolve_await rejects out-of-bounds HP/sway and holds AWAIT");

    debug_state_fields_t repaired = cornerstone_test_make_state(126, "Elijah", 450, 50, 50, 0, 10);
    TEST_ASSERT(debug_resolve_await(&repaired, DEBUG_STEP_RUN, "Unit state repaired cleanly") &&
                    !debug_is_awaiting() &&
                    debug_can_step() &&
                    (debug_get_step_mode() == DEBUG_STEP_RUN),
                "debug_resolve_await restores execution once valid state verified");

    debug_shutdown();
}
```

A.24. Bounded POSIX Filesystem & Read-Only Guard API (fs_utils.h Sub-Assembly 3.2)

Snapshot ID: snap_io_fs_utils_sub3_2 | **Mode**: Subassembly | **File**: engine/include/io/fs_utils.h (L359–L455 of 838 lines) | Jump back to chapter text

```
/*
-------------------------------------------------------------------------------
Sub-Assembly 3.2: Bounded Buffered I/O & Directory Management Cogs
Mission: Bounded text reading, safe file writing, and parent directory synthesis.

  ### For Game Designers & Non-Programmers:
  Loads configuration and story files safely within memory caps and saves outputs
  atomically while blocking mutations during Read-Only Safe Mode.

  ### For Systems Programmers & Engine Architects:
  Enforces active policy ceilings (`max_config_bytes`, `path_max_bytes`), NUL
  termination, Safe-Mode RO protection, and shared Tier-0 BIOS atomic write/mkdir.
-------------------------------------------------------------------------------
*/

/**
 * @brief Reads entire text file contents into a bounded, caller-supplied buffer.
 *
 * Guarantees unconditional null termination and prevents buffer overruns.
 * If the file exceeds `buf_size - 1` or `max_config_bytes`, the read is safely rejected.
 *
 * ### For Game Designers & Non-Programmers:
 * Loads an entire text file safely into memory, guaranteeing it never overflows or crashes the engine.
 *
 * ### For Systems Programmers & Engine Architects:
 * Convenience wrapper delegating cleanly into `fs_read_file_bounded()` without length output.
 *
 * @param[in]  filepath Null-terminated path or `"rom://..."` URI to target file.
 * @param[out] buf      Caller-supplied memory destination buffer.
 * @param[in]  buf_size Byte capacity of destination buffer (including null terminator).
```

```
 * @return true if file opened and completely read within buffer bounds; false otherwise.
 * @triggers Inherits truncation fault and warning log behavior from `fs_read_file_bounded()`.
 */
bool fs_read_file_contents(const char *filepath, char *buf, size_t buf_size);

/**
 * @brief Reads file or `"rom://..."` text and reports exact count of bytes read.
 *
 * Similar to `fs_read_file_contents()`, and writes the exact number of bytes read
 * (excluding NUL terminator) into optional `out_bytes_read`.
 *
 * ### For Game Designers & Non-Programmers:
 * Reads a text file and tells callers the exact character byte length loaded.
 *
 * ### For Systems Programmers & Engine Architects:
 * Enforces `min(buf_size, max_config_bytes)` cap, increments `read_ops`/`truncation_rejections`,
 * and records BIOS faults (`IO_ERR_FILE_NOT_FOUND`/`IO_ERR_TRUNCATED_READ`).
 *
 * @param[in]  filepath       Null-terminated path or `"rom://..."` URI to target file.
 * @param[out] buf            Caller-supplied destination buffer.
 * @param[in]  buf_size       Capacity of destination buffer in bytes.
 * @param[out] out_bytes_read Optional pointer receiving bytes read (excluding null).
 * @return true if read completed without truncation; false otherwise.
 * @triggers Emits `LOG_WARN` and records BIOS fault on open error or policy truncation.
 */
bool fs_read_file_bounded(const char *filepath, char *buf, size_t buf_size,
                          size_t *out_bytes_read);

/**
 * @brief Safely writes or overwrites bounded data to a file on disk.
 *
 * Automatically creates parent directories recursively using configured permissions
 * and flushes/renames atomically when enabled in active policy.
 *
 * ### For Game Designers & Non-Programmers:
 * Saves files safely with automatic folder creation and read-only lock protection.
 *
 * ### For Systems Programmers & Engine Architects:
 * Validates path cap and Floppy RO lock (`io.fs.readonly_guard`), then delegates POSIX
 * write/flush/atomic rename to `io_fallback_fs_write_with_flags()`.
 *
 * @param[in] filepath Null-terminated target filepath.
 * @param[in] data     Buffer of data bytes to write.
 * @param[in] data_len Number of bytes to write.
 * @return true if all bytes were written successfully; false on guard block or error.
 * @triggers Triggers `io.fs.readonly_guard` when write attempted under read-only Safe Mode.
 */
bool fs_write_file_contents(const char *filepath, const char *data,
                            size_t data_len);

/**
 * @brief Ensures that a directory path exists, recursively creating missing parents.
 *
 * Mimics POSIX `mkdir -p` behavior using bounded stack buffers with zero heap allocations.
 *
 * ### For Game Designers & Non-Programmers:
 * Automatically creates folders needed for logs or outputs unless write-locked.
 *
 * ### For Systems Programmers & Engine Architects:
 * Validates `FS_PATH_MAX` cap and RO latch (`io.fs.readonly_guard`), then delegates to
 * `io_fallback_fs_ensure_dir_mode()`.
 *
 * @param[in] dir_path Null-terminated directory path to create.
 * @return true if directory exists or was created; false on RO lock, overflow, or error.
 * @triggers Triggers `io.fs.readonly_guard` when directory creation is blocked under RO latch.
 */
bool fs_ensure_directory(const char *dir_path);
```

A.25. Multi-Tier VFS Path & Asset Resolution API (path_finder.h Sub-Assembly 3.2)

Snapshot ID: snap_io_path_finder_sub3_2 | **Mode**: Subassembly | **File**: engine/include/io/path_finder.h (L396–L510 of 719 lines) | Jump back to chapter text

```
/*
--------------------------------------------------------------------------------
Sub-Assembly 3.2: Multi-Tier VFS Path & Asset Resolution Cogs
Mission: Execute layered precedence search, diagnostic alias expansion, symlink
         depth bounds, traversal rejection, and `.rodata` ROM fallback cleanly.

  ### For Game Designers & Non-Programmers:
  Finds configuration manifests, game data files, fonts, and artwork across game
  overrides, engine baselines, asset folders, and zero-disk ROM fallbacks.

  ### For Systems Programmers & Engine Architects:
  Bounded-stack candidate resolution enforcing `max_symlink_depth`, `..` traversal
  containment, destination `out_size` overflow guards, and `io.vfs.traversal_guard`.
--------------------------------------------------------------------------------
*/

/**
 * @brief Resolves a logical domain and relative path to a concrete disk file or `rom://` URI.
 *
 * Search sequence (all tables driven by `io/vfs_routes.json`):
 *  1. `..` parent traversal sandbox guard (`block_path_traversal`) & explicit `rom://` check.
 *  2. Direct accessible file probe (enforcing `max_symlink_depth`).
 *  3. Mounted `active_root` (`<root>/<domain>/<clean>`, `<root>/<clean>`) and `env_override_var`.
 *  4. Layered `config_precedence` tiers (`game/config/`, `engine/config/`, linkers) + `diagnostic_aliases`.
 *  5. Hydrated `fallback_candidate_roots` x `domain_search_paths` matrix.
 *  6. Compiled `.rodata` ROM vault fallback (`rom://<clean>`) when disk targets are absent.
 *
 * ### For Game Designers & Non-Programmers:
 * Finds any configuration file, story file, or manifest automatically—checking game overrides
 * first, engine baselines second, and built-in ROM backups last.
 *
 * ### For Systems Programmers & Engine Architects:
 * Bounded stack probing (zero heap), strict `out_size` overflow rejection, `lstat`/`readlink`
 * symlink depth verification, and `LOG_CAT_IO` telemetry tracking.
 * @triggers `io.vfs.traversal_guard` (on `..`/symlink rejection), `LOG_CAT_IO`.
 *
 * @param[in]  domain   Optional logical domain hint (`"config"`, `"mechanics"`, `"io"`, etc.) or NULL.
 * @param[in]  rel_path Relative asset/config path or explicit `"rom://..."` URI.
 * @param[out] out_path Caller-supplied output path buffer (`out_path[0] = '\0'` on failure).
 * @param[in]  out_size Capacity of `out_path` in bytes.
 * @return true if resolved cleanly onto disk or ROM within `out_size`; false otherwise.
 */
bool path_finder_resolve(const char *domain,
                         const char *rel_path,
                         char *out_path,
                         size_t out_size);

/**
 * @brief Resolves a subsystem directory string to its `manifest.json` anchor file.
 *
 * ### For Game Designers & Non-Programmers:
 * Locates the master menu file (`manifest.json`) for any folder such as `"config"`,
 * `"config/diagnostics/telemetry"`, or `"mechanics"`.
 *
 * ### For Systems Programmers & Engine Architects:
 * Normalizes trailing slashes, enforces traversal guards before formatting
 * `<subsystem>/manifest.json`, and delegates to `path_finder_resolve()`. Zero heap.
 * @triggers `io.vfs.traversal_guard`, `LOG_CAT_IO`.
 *
 * @param[in]  subsystem_path Directory fragment naming the subsystem subtree.
 * @param[out] out_path       Caller output buffer receiving resolved manifest path.
 * @param[in]  out_size       Capacity of `out_path` in bytes.
 * @return true if a readable `manifest.json` was located; false otherwise.
 */
bool path_finder_find_manifest(const char *subsystem_path,
                               char *out_path,
                               size_t out_size);

/**
 * @brief Resolves a media or artwork asset under configured `asset_roots` and category folders.
 *
 * ### For Game Designers & Non-Programmers:
 * Finds fonts (`Cinzel-Bold.ttf`), sprites (`elijah_32x32.png`), audio tracks, and UI textures
 * across `game/assets/` or active mod asset folders.
 *
```

```
471  * ### For Systems Programmers & Engine Architects:
472  * Iterates hydrated `routes->asset_roots[]` (`game/assets`, `../game/assets`, etc.) plus active
473  * mount overlays with strict traversal and symlink checks. Zero heap.
474  * @triggers `io.vfs.traversal_guard`, `LOG_CAT_IO`.
475  *
476  * @param[in]  asset_category Category subfolder (`"fonts"`, `"sprites"`, `"audio"`, etc.).
477  * @param[in]  asset_name     Asset filename (`"Cinzel-Bold.ttf"`, etc.).
478  * @param[out] out_path       Caller destination path buffer.
479  * @param[in]  out_size       Capacity of `out_path` in bytes.
480  * @return true if asset file was located safely; false otherwise.
481  */
482 bool path_finder_resolve_asset(const char *asset_category,
483                                const char *asset_name,
484                                char *out_path,
485                                size_t out_size);
486
487 /**
488  * @brief Resolves an asset using an explicit artist file path hint before category fallbacks.
489  *
490  * ### For Game Designers & Non-Programmers:
491  * Checks an artist's exact custom artwork path first; if unavailable, seamlessly falls back to
492  * standard category folders (`sprites/`, `fonts/`) or data directories.
493  *
494  * ### For Systems Programmers & Engine Architects:
495  * Validates `direct_path_hint` against traversal guards, probes direct/VFS path first, and falls
496  * back to `path_finder_resolve_asset()` (when `asset_name != NULL`) or `path_finder_resolve()`.
497  * @triggers `io.vfs.traversal_guard`, `LOG_CAT_IO`.
498  *
499  * @param[in]  direct_path_hint Preferred direct or relative artist path (nullable).
500  * @param[in]  asset_category   Fallback category subfolder (`"sprites"`, `"fonts"`, etc., nullable).
501  * @param[in]  asset_name       Fallback filename (nullable when only probing hint).
502  * @param[out] out_path         Caller destination path buffer.
503  * @param[in]  out_size         Capacity of `out_path` in bytes.
504  * @return true if any valid target resolved safely; false otherwise.
505  */
506 bool path_finder_resolve_with_hint(const char *direct_path_hint,
507                                    const char *asset_category,
508                                    const char *asset_name,
509                                    char *out_path,
510                                    size_t out_size);
```

A.26. Zero-Heap JSON Token Span Locator API (json_utils.h Sub-Assembly 3.1)

Snapshot ID: snap_io_json_utils_sub3_1 | **Mode**: Subassembly | **File**: engine/include/io/json_utils.h (L357–L461 of 1205 lines) | Jump back to chapter text

```
357 /*
358 ------------------------------------------------------------------------------
359 Sub-Assembly 3.1: JSON Token Locator Cogs
360 Mission: Scan flat or span-bounded JSON payloads for matching keys, objects, and
361          arrays while respecting escape sequences and nesting limits cleanly.
362
363   ### For Game Designers & Non-Programmers:
364   Finds named sections, tables, and lists inside JSON configuration files cleanly
365   without mistaking words inside text descriptions for structural headings.
366
367   ### For Systems Programmers & Engine Architects:
368   Exposes zero-copy, zero-heap token and balanced-container locators constrained
369   to `[json_start, json_start + json_len)` and active `max_json_depth` policy.
370 ------------------------------------------------------------------------------
371 */
372
373 /**
374  * @brief Scans a JSON buffer for a specific top-level or scoped key token.
375  *
376  * Core search primitive that locates occurrences of "key" followed by a colon
377  * (:), ignoring intervening whitespace. Prevents false positive matches on
378  * substring prefixes or keys embedded inside quoted strings.
379  *
380  * ### For Game Designers & Non-Programmers:
381  * Searches through a data file to locate where a property begins (e.g. finding "attack_power:").
382  *
```

```
 * ### For Systems Programmers & Engine Architects:
 * Reentrant zero-heap token locator respecting quote escapes and whitespace.
 * @triggers On nesting limit exceeded -> `IO_ERR_JSON_DEPTH_EXCEEDED` via `io.json.depth_bounds`.
 *
 * @param[in] json Null-terminated JSON buffer.
 * @param[in] key  Null-terminated key name to search for (without quotes).
 * @return Pointer to value position immediately following the colon (:),
 *         or NULL if key is not found or parameters are invalid.
 */
const char *json_find_key(const char *json, const char *key);

/**
 * @brief Span-bounded key locator preventing sibling overrun across sliced buffers.
 *
 * ### For Game Designers & Non-Programmers:
 * Searches strictly inside one specific section of a JSON file so properties from
 * neighboring sections can never bleed in accidentally.
 *
 * ### For Systems Programmers & Engine Architects:
 * Zero-copy locator bounded strictly to `[json_start, json_start + json_len)`.
 * Properly skips quoted string values (respecting `\\` and `\"` escapes) and
 * enforces active `max_json_depth`.
 *
 * @triggers On nesting overflow -> `IO_ERR_JSON_DEPTH_EXCEEDED` fault + `io.json.depth_bounds`.
 *
 * @param[in] json_start Pointer to JSON slice start.
 * @param[in] json_len   Exact byte length of slice (`0` measures NUL-terminated buffer).
 * @param[in] key        Key name to match.
 * @return Pointer to trimmed value token following ':', or NULL if absent/invalid.
 */
const char *json_find_key_span(const char *json_start, size_t json_len, const char *key);

/**
 * @brief Span-bounded nested JSON object (`{...}`) locator.
 *
 * ### For Game Designers & Non-Programmers:
 * Finds a sub-table inside a JSON file (such as `metrics` or `stats`) and
 * fences its boundaries cleanly so fields outside that table cannot leak in.
 *
 * ### For Systems Programmers & Engine Architects:
 * Delegates to `json_find_delimited_span_internal()` inside `[json_start, json_len)`,
 * matching balanced `{}` braces up to `max_json_depth` with zero allocation.
 *
 * @triggers On depth overflow -> `IO_ERR_JSON_DEPTH_EXCEEDED` fault + `io.json.depth_bounds`.
 *
 * @param[in]  json_start    Pointer to JSON slice start.
 * @param[in]  json_len      Exact byte length of slice (`0` measures NUL-terminated).
 * @param[in]  object_key    Target key whose value must be a JSON object.
 * @param[out] out_obj_start Receives pointer to opening ''{'`.
 * @param[out] out_obj_len   Receives byte length including closing ''}'`.
 * @return true if bounded object slice located cleanly; false otherwise.
 */
bool json_find_object_span(const char *json_start, size_t json_len,
                           const char *object_key,
                           const char **out_obj_start, size_t *out_obj_len);

/**
 * @brief Span-bounded nested JSON array (`[...]`) locator.
 *
 * ### For Game Designers & Non-Programmers:
 * Finds a JSON list (such as `"tags"` or `"skills"`) and fences its exact
 * start and end positions so adjacent lists are never read by mistake.
 *
 * ### For Systems Programmers & Engine Architects:
 * Bounded zero-copy bracket matcher operating strictly within `[json_start, json_len)`.
 * Enforces `max_json_depth` without touching caller memory beyond `json_len`.
 *
 * @triggers On depth overflow -> `IO_ERR_JSON_DEPTH_EXCEEDED` fault + `io.json.depth_bounds`.
 *
 * @param[in]  json_start    Pointer to JSON slice start.
 * @param[in]  json_len      Exact byte length of slice (`0` measures NUL-terminated).
 * @param[in]  array_key     Target key whose value must be a JSON array.
 * @param[out] out_arr_start Receives pointer to opening ''['`.
 * @param[out] out_arr_len   Receives byte length including closing '']'`.
 * @return true if bounded array slice located cleanly; false otherwise.
```

```
 */
bool json_find_array_span(const char *json_start, size_t json_len,
                          const char *array_key,
                          const char **out_arr_start, size_t *out_arr_len);
```

A.27. I/O Pipeline System Emergence & ROM Downgrade Verification (test_io_system_emergence)

Snapshot ID: snap_io_test_system | **Mode**: Function | **File**: tests/system/test_io_system.c (L328–L399 of 477 lines) | Jump back to chapter text

```
/**
 * @brief Verifies sandboxed mount redirection and live v1 -> v2 JSON hot-patch reflection.
 */
static void test_io_system_emergence(void)
{
    printf("  [EMERGE-4] Running Section 4: Emergence (Dynamic Mount & Live Hot-Patching)...\n");

    const char *hotpatch_dir = "build/system_test_patch/system";
    const char *hotpatch_file = "build/system_test_patch/system/runtime_patch.json";

    TEST_ASSERT(fs_ensure_directory(hotpatch_dir), "Synthesized hot-patch directory");

    static const char patch_v1[] =
        "{\n"
        "  \"block_1_metadata\": {\"project\": \"Tenebris and Lux\", \"file\": \"runtime_patch.json\"},\n"
        "  \"block_2_types\": {\"schema_id\": \"tenlux.data.patch.v1\"},\n"
        "  \"block_3_body\": {\"difficulty_mult\": 1.0, \"active_boss\": \"Usuru\"}\n"
        "}\n";
    TEST_ASSERT(fs_write_file_contents(hotpatch_file, patch_v1, strlen(patch_v1)),
                "Wrote patch v1");

    path_finder_init();
    master_loader_init();
    path_finder_set_data_root("build/system_test_patch");

    master_loader_t loader;
    bool l1 = master_loader_execute_load_verified("system", "runtime_patch.json",
                                                  "tenlux.data.patch.v1",
                                                  s_io_sys_load_buf,
                                                  sizeof(s_io_sys_load_buf),
                                                  &loader);
    TEST_ASSERT(l1 && loader.status == LOADER_STATUS_OK, "Loaded patch v1");
    float diff1 = 0.0f;
    char boss1[64];
    TEST_ASSERT(master_loader_get_float(&loader, NULL, "difficulty_mult", &diff1) &&
                diff1 >= 0.999f && diff1 <= 1.001f,
                "Patch v1 difficulty_mult == 1.0");
    TEST_ASSERT(master_loader_get_string(&loader, NULL, "active_boss", boss1, sizeof(boss1)),
                "Patch v1 boss extracted");
    TEST_ASSERT_STR_EQ(boss1, "Usuru", "Boss is authentic Dark Lord Usuru");

    static const char patch_v2[] =
        "{\n"
        "  \"block_1_metadata\": {\"project\": \"Tenebris and Lux\", \"file\": \"runtime_patch.json\"},\n"
        "  \"block_2_types\": {\"schema_id\": \"tenlux.data.patch.v1\"},\n"
        "  \"block_3_body\": {\"difficulty_mult\": 2.5, \"active_boss\": \"The Dark Lord Usuru\"}\n"
        "}\n";
    TEST_ASSERT(fs_write_file_contents(hotpatch_file, patch_v2, strlen(patch_v2)),
                "Wrote patch v2");

    bool l2 = master_loader_execute_load_verified("system", "runtime_patch.json",
                                                  "tenlux.data.patch.v1",
                                                  s_io_sys_load_buf,
                                                  sizeof(s_io_sys_load_buf),
                                                  &loader);
    TEST_ASSERT(l2 && loader.status == LOADER_STATUS_OK, "Loaded patch v2");
    float diff2 = 0.0f;
    char boss2[64];
    TEST_ASSERT(master_loader_get_float(&loader, NULL, "difficulty_mult", &diff2) &&
                diff2 >= 2.499f && diff2 <= 2.501f,
```

```
388                 "Hot-patched difficulty_mult == 2.5");
389     TEST_ASSERT(master_loader_get_string(&loader, NULL, "active_boss", boss2, sizeof(boss2)),
390                 "Hot-patched boss extracted");
391     TEST_ASSERT_STR_EQ(boss2, "The Dark Lord Usuru",
392                        "Boss updated to The Dark Lord Usuru");
393 
394     path_finder_set_data_root(NULL);
395     master_loader_shutdown();
396     path_finder_shutdown();
397 
398     printf("  [EMERGE-4] Section 4 passed: Dynamic mount reconfiguration and hot-patching succeeded.\n");
399 }
```